

MARINHA DO BRASIL

CENTRO DE INSTRUÇÃO ALMIRANTE ALEXANDRINO



INTRODUÇÃO AOS MICROCONTROLADORES 3ª REVISÃO – 2020

**INTRODUÇÃO AOS
MICROCONTROLADORES**

MARINHA DO BRASIL

CENTRO DE INSTRUÇÃO ALMIRANTE ALEXANDRINO

2020

FINALIDADE: DIDÁTICA

ATO DE APROVAÇÃO

APROVO, para emprego no Centro de Instrução Almirante Alexandrino, para as turmas do Quadro Técnico de Praças do CPA, dos Cursos de Aperfeiçoamento em Eletrônica, Eletricidade e Comunicações Interiores, a apostila da disciplina ELETRÔNICA IV.

Rio de Janeiro, RJ.

Em 09 de setembro de 2020.

JOSE AFONSO BARBOZA LOBIANCO

Capitão-de-Fragata (RM-1)

Coordenador de Cursos da Escola de Eletricidade e Eletrônica

ÍNDICE

PÁGINAS

Folha de Rosto	I
Ato de Aprovação	II
Índice.....	III
Introdução	VI

CAPÍTULO 1 – INTRODUÇÃO AOS MICROCONTROLADORES.

1.1 – Introdução.....	1.1
1.2 – Família de microcontroladores.....	1.1
1.3 – Funções internas comuns à maioria dos modelos de PIC.....	1.2
1.4 – Arquitetura e instruções.....	1.6

CAPÍTULO 2 – VISÃO GERAL DO MICROCONTROLADOR MICROCHIP PIC16F877.

2.1 – Introdução.....	2.1
2.2 – Características.....	2.1
2.3 – Arquitetura interna do PIC16F87x.....	2.3
2.4 – Ciclo de instrução do PIC.....	2.5
2.5 – Diagrama de pinagem.....	2.7

CAPÍTULO 3 – ORGANIZAÇÃO DA MEMÓRIA DO PIC.

3.1 – Introdução.....	3.1
3.2 – Organização da memória de programa (Flash).....	3.1
3.3 – Organização da memória de dados (RAM).....	3.6
3.4 – Classificação dos Registros.....	3.14

CAPÍTULO 4 – PORTAS DE ENTRADA E SAÍDA.

4.1 – Introdução.....	4.1
4.2 – Porta A e registro TrisA.....	4.1
4.3 – Porta B e registro TrisB.....	4.4
4.4 – Porta C e registro TrisC.....	4.8

4.5 – Porta D e registro TrisC.....	4.9
4.6 – Porta E e registro TrisE.....	4.11
4.7 – Porta Paralela Escrava (PSP).....	4.13

CAPÍTULO 5 – MEMÓRIA DE DADOS (EEPROM) E DE PROGRAMA (FLASH).

5.1 – Introdução.....	5.1
5.2 – Registros EECON1, EECON2, EEDATA, EEDATH, EEADR e EEADRH.....	5.3
5.3 – Instruções do PIC16F877 – Introdução.....	12.1
5.4 – Conjunto de instruções do PIC16F877.....	12.3

CAPÍTULO 6 – ESTUDO DOS TIMERS.

6.1 – Módulo Timer0.....	6.1
6.2 – Módulo Timer1.....	6.7
6.3 – Módulo Timer2.....	6.15

CAPÍTULO 7 – MÓDULOS CAPTURE/COMPARE/PWM.

7.1 – Funções básicas dos módulos CCP.....	7.1
7.2 – Modo Capture.....	7.3
7.3 – Modo Compare.....	7.8
7.4 – Modo PWM.....	7.12

CAPÍTULO 8 – MÓDULO MSSP (Porta Série Síncrona Mestra).

8.1 – Introdução.....	8.1
8.2 – Operação da porta MSSP no Modo SPI.....	8.7
8.3 – Operação da Porta MSSP no Modo I2C.....	8.16
8.4 – Configurando o MCU como dispositivo Mestre.....	8.31
8.5 – Configurando o MCU como dispositivo Escravo.....	8.60

CAPÍTULO 9 – UNIVERSAL SYNCHRONOUS ASYNCHRONOUS RECEIVE TRANSMITTER (USART).

9.1 – Introdução.....	9.1
9.2 – Gerador de Baud Rate da USART (BRG).....	9.1
9.3 – USART no modo Assíncrono.....	9.6
9.4 – USART no modo Síncrono Mestre.....	9.16
9.5 USART no modo Síncrono Escravo.....	9.20

CAPÍTULO 10 – MÓDULO CONVERSOR ANALÓGICO-DIGITAL (ADC).

10.1 – Introdução.....	10.1
10.2 – Registros ADCON0 e ADCON1.....	10.3
10.3 – Operação durante o Sleep do conversor A/D.....	10.8
10.4 – Efeito de Reset.....	10.9

CAPÍTULO 11 – CARACTERÍSTICAS ESPECIAIS DA CPU.

11.1 – Introdução.....	11.1
11.2 – Bits de configuração do registro CONFIG do PIC16F877.....	11.2
11.3 – Configuração do oscilador.....	11.6
11.4 – Tipos de Reset.....	11.7
11.5 – Interrupções.....	11.15
11.6 – Sleep (Power Down Mode).....	11.22
11.7 - Temporizador Watchdog (WDT).....	11.26
11.8 – Sistema de emulação IN-CIRCUIT (Debugger Mode).....	11.29
11.9 – Sistema de código de proteção (Code Protection).....	11.29
11.10 – Registros de identificação (IDS).....	11.30
11.11 – Gravação em série no próprio circuito (ICSP).....	11.30
11.12 – Gravação em baixa tensão – LVP.....	11.31

CAPÍTULO 12 – INSTRUÇÕES DO PIC16F877.

12.1 – Introdução.....	12.1
12.2 – Conjunto de instruções do PIC16F877.....	12.3

INTRODUÇÃO

1 – PROPÓSITO.

Esta publicação foi elaborada para dar orientação sobre o Curso de Introdução aos Microcontroladores e seus circuitos.

Os assuntos nela contidos foram extraídos de publicações de fácil compreensão, preenchidos pelas exigências dos currículos, com o propósito de facilitar a aprendizagem por parte dos alunos. Entretanto, os complementos dos assuntos aqui elaborados serão melhores absorvidos pelos alunos especializados, pela prática em locais que trabalhem com Microcontroladores e Microprocessadores.

2 – DESCRIÇÃO.

Esta publicação está dividida em 12 (doze) Capítulos, a saber. Capítulo 1 - Introdução aos microcontroladores; Capítulo 2 – Microcontrolador Microchip PIC16F877; Capítulo 3 – Organização da memória do PIC16F877; Capítulo 4 – Portas de entrada e saída; Capítulo 5 – Memórias de dados (EEPROM) e programa (FLASH); Capítulo 6 – Estudo dos Timers; Capítulo 7 – Módulos Capture/Compare/PWM; Capítulo 8 - Módulo MSSP (Porta Série Síncrona Mestre); Capítulo 9 – Addressable Universal Synchronous Asynchronous Receiver Transmitter; Capítulo 10 – Módulo conversor analógico-digital (ADC); Capítulo 11 – Características especiais da CPU e Capítulo 12 – Instruções do PIC16F87X.

3 - AUTORIA E EDIÇÃO.

Esta publicação é de autoria do SO–ET– SB (RM1) – MONROE e foi elaborada e editada, no CENTRO DE INSTRUÇÃO ALMIRANTE ALEXANDRINO (CIAA).

4 - DIREITOS DE EDIÇÃO.

Reservados para o CENTRO DE INSTRUÇÃO ALMIRANTE ALEXANDRINO. Proibida a reprodução total ou parcial, sob qualquer forma ou meio.

5 – CLASSIFICAÇÃO.

Esta publicação é classificada, de acordo com o EMA-411 (Manual de Publicações da Marinha) em: Publicação da Marinha do Brasil, não controlada, ostensiva, didática e manual.

CAPÍTULO 1.

INTRODUÇÃO AOS MICROCONTROLADORES.

1.1 – INTRODUÇÃO.

Um sistema computacional é composto por uma unidade de processamento, memória e portas de entrada/saída (I/O).

A maioria das pessoas limita o conceito de sistema computacional ao computador que temos em casa, vulgo PC (Personal Computer). No entanto todo sistema que a partir de dados de entrada, executa algum processamento mediante um programa armazenado em uma memória gerando uma saída, é chamado de Sistema computacional.

Podemos definir microcontroladores como sendo um sistema computacional integrado, que se caracterizam por incorporarem internamente em um único chip, CPU, memórias de programa e dados e vários periféricos como Timers, Watchdog, Comunicação serial, Conversor analógico/digital, Gerador de PWM, etc. Fazendo com que a aplicação final fique extremamente compacta.

1.2 - FAMÍLIAS DE MICROCONTROLADORES.

1.2.1 - Família 8051.

Um dos micros mais usados em aplicações industriais. Sua base inicial foi lançada no começo da década de 80 pela INTEL. Vários fabricantes compraram sua patente e começaram a produzir componentes que tem como base esta família. Dentre eles, temos: ATMEL, National, Motorola, Philips, AMD, Fujitsu, Oki, etc.

1.2.2 - Família PIC (Peripheral Interface Controller) Interface Controladora de Periféricos.

Fabricado pela Microchip e todos os modelos foram desenvolvidos a partir da mesma filosofia de produto. Assim, os códigos das instruções são extremamente compatíveis entre todos os modelos, permitindo a migração de um modelo de micro para outro sem necessidade de grandes alterações no programa que estava sendo utilizado.

Esses micros começaram a ser comercializados na década de 70 e vem sendo aprimorados e criadas novas famílias, que possuem recursos que atendem plenamente às necessidades de qualquer projeto desde as simples até as mais complexas aplicações e o que é mais importante, com custo baixo.

Podemos classificar todos os modelos de PIC em três grandes grupos: Pequenas, médias e grandes aplicações. Os modelos para pequenas aplicações possuem uma ou duas portas de entrada/saída e memória de programação Flash de pequena capacidade. Eles estão disponíveis em encapsulamentos DIP de 8, 16, 18 e 20 pinos. Já os modelos para aplicações médias e grandes possuem cinco ou mais portas de entrada ou saídas, possuem internamente comparadores analógicos, conversores A/D, portas seriais e vários temporizadores, alguns possuem memória EEPROM de fácil programação. Estão disponíveis em encapsulamentos DIP de 28 e 40 pinos.

Todos os modelos da família PIC possuem um conjunto de instruções e características semelhantes.

Nosso curso terá essa família como base, em especial o PIC16F877.

1.3 – FUNÇÕES INTERNAS COMUNS Á MAIORIA DOS MODELOS DE PIC.

Todos os modelos de microcontroladores PIC possuem uma arquitetura similar, embora suas características sejam bem diferentes, pois depende do modelo que se está trabalhando. Uma descrição geral pode ser perfeitamente aplicável a qualquer um dos tipos existentes.

13.1 - Unidade de memória.

Basicamente são de dois tipos:

a) ROM (memória de programa) - armazena instruções do programa e pode ser do tipo EPROM, EEPROM ou FLASH.

b) RAM - memória de dados (usada como registros).

Obs: Alguns modelos possuem uma EPROM adicional como memória de dados.

1.3.2 - Unidade Central de Processamento (CPU).

Coordena o trabalho dos outros blocos e executa o programa do utilizador. Possui memória interna chamada de registros. Estes registros são usados pela CPU para armazenar dados temporários.

1.3.3 – Barramentos.

É usado na interligação entre a CPU e outros blocos. Fisicamente corresponde a um grupo de 8, 16 ou mais condutores. Existem os barramentos de dados e de endereços. O número de linhas do barramento de dados depende da largura da palavra de dados e

normalmente se liga a todos os blocos dentro do micro. O número de linhas do barramento de endereço depende da quantidade de memória que se deseja endereçar, normalmente, serve para transmitir endereços da CPU para a memória.

1.3.4 - Unidade de entrada/saída.

São ligações físicas, comumente chamadas de Portas, que liga o microcontrolador ao mundo exterior. Existem portas de entrada, saída e entrada/saída. Quando se trabalha com essas portas, é necessário selecionar qual a porta em que vamos enviar ou receber dados. A porta funciona como um local de memória armazenando o dado que chega ou o dado que sai.

1.3.5 - Comunicação série.

Na comunicação com o mundo exterior, um dos inconvenientes é o número de linhas que é necessário para transferir dados, principalmente em longas distâncias onde o número de linhas vezes o número de quilômetros afeta a economia do projeto. Assim, podemos usar o barramento série com uma linha para receber, outra linha para transmitir e uma terceira usada como referência das duas anteriores. Além disso, precisamos definir as regras usadas na troca de dados, a isto chamamos de protocolo.

Na comunicação paralela, os dados são transmitidos de uma só vez enquanto na comunicação série a transmissão é bit a bit.

Depois que os dados são recebidos eles são lidos e armazenados na memória. Na transmissão, os dados vêm da memória através do barramento para o local de transmissão e daí para a unidade de recepção de acordo com o protocolo.

1.3.6 - Unidade de temporização.

A unidade básica do temporizador é um contador que é na realidade um registro cujo conteúdo aumenta de uma unidade num intervalo fixo de tempo, assim, anotando o seu valor durante os instantes de tempos T1 e T2 e calculando sua diferença, saberemos o tempo decorrido.

1.3.7 – Watchdog.

O watchdog é um recurso disponível no PIC que parte do princípio que todo sistema é passível de falha. Se todo sistema pode falhar, cabe ao mesmo ter recursos para que, em ocorrendo uma falha, algo seja feito de modo a tornar o sistema novamente operacional. No nosso computador, de um modo geral, existe um botão de reset que

deve ser usado sempre que o sistema travar por qualquer motivo, no entanto, este botão não existe no micro.

Dentro do PIC existe um contador incrementado por um oscilador (RC) independente. Toda vez que este contador extrapola o seu valor máximo retornando a zero, é provocado a reinicialização do sistema (reset).

Se o sistema estiver funcionando da maneira correta, de tempos em tempos uma instrução denominada Clear Watchdog Timer (CLRWDT) zera o valor deste contador, impedindo que o mesmo chegue ao valor máximo. Desta maneira o Watchdog somente irá "estourar" quando algo de errado ocorrer (travando o sistema). O período normal de estouro do Watchdog Timer é de aproximadamente 18 ms. No entanto, algumas vezes este tempo é insuficiente para que o programa seja normalmente executado. A saída neste caso é alocar o recurso de um Postscaler de modo a aumentar este período.

1.3.8 - Conversor analógico – digital.

Como os sinais dos periféricos são analógicos é preciso convertê-los para o formato digital a fim de que possam ser compreendido pelo micro, o que é feito por um conversor A/D.

1.3.9 – Programa.

Programação é o ato de escrever um programa.

Programa é um conjunto completo de *instruções* que descreve exatamente o que fazer a cada passo de sua operação. Este conjunto de instruções é preparado por uma ou mais pessoas, *programadores*, para cada tarefa que o computador deva fazer. Os programas são colocados na memória na forma binária, tendo cada instrução um código único. O micro toma estes códigos de instruções da memória um por vez e realiza a operação associada ao código.

A tarefa de programação pode ser executada em várias linguagens tais como o Assembler, C e Basic que são as mais usadas. O Assembler pertence ao grupo das linguagens de baixo nível que implicam em um trabalho de programação lento, no entanto, ocupam menos espaço em memória e uma maior rapidez na sua execução. Programas na linguagem C são mais fáceis de escrever e compreender, no entanto, são mais lentos na execução que o Assembler. Basic é a mais fácil de todas para se

aprender e suas instruções são semelhantes à maneira de um ser humano se exprimir, mais similar à C, é também de execução mais lenta.

Depois que o programa foi escrito é introduzido no micro que recebe alimentação a fim de que possa trabalhar corretamente.

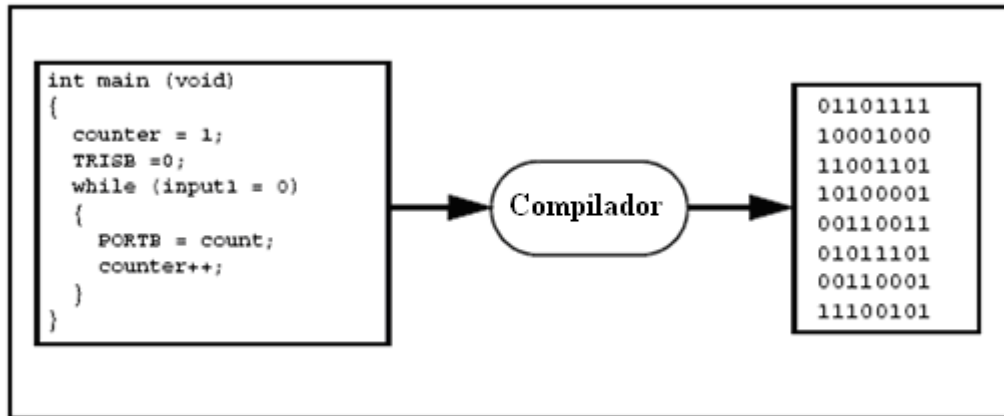


Figura 1.1 - Compilador transforma o programa fonte em instruções de máquina.

1.3.10 - Contador de Programa (PC).

Estabelece o fluxo de controle dos direcionamentos da memória que o código do programa contém.

Na maioria das instruções, o PC incrementa-se automaticamente para apontar a instrução seguinte, no entanto, quando a instrução é de salto do tipo direto, o valor que é carregado no PC provém de uma parte dos bits do código OP (operando) da própria instrução.

Nos saltos relativos, a ULA soma o valor que o PC contém o do salto, e volta a ser armazenado no PC; por conseguinte esse será o novo endereço do salto. Disto deduz-se que o PIC possui 3 tipos de endereçamento (direcionamento):

- a) Imediato** - o valor do dado está contido no OP da instrução
- b) Direto** - o endereço da área de memória de dados onde se encontra o operando está contido no OP da instrução.
- c) Indireto** - endereço de dados que o operando guarda está contido em um registro.

1.4 – ARQUITETURAS E INSTRUÇÕES.

Dois tipos de arquiteturas são as mais utilizadas em microcontroladores e microprocessadores:

1.4.1 - Arquitetura Von-Neumann.

Na arquitetura Von-Neumann, os barramentos de dados e endereços são compartilhados entre memórias de programas e de dados na comunicação com a CPU. Nesse tipo de arquitetura, quando a CPU está acessando a memória de programa não pode acessar a memória de dados, porque usa os mesmos barramentos para as duas memórias. Veja **Figura 1.2**.

Isto faz com seja necessária uma quantidade maior de ciclos de máquina para executar uma instrução.

Nesta arquitetura, para fazer uma operação de soma de dois números, precisamos de um microcódigo. O microcódigo nada mais é do que uma seqüência de operações que fará com que a ULA faça a operação de soma. Assim, para somarmos dois números que serão inseridos na entrada do microcontrolador e obtermos a resposta em sua saída, teremos que fazer várias operações.

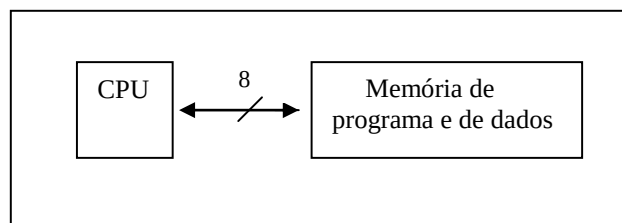


Figura 1.2 – Arquitetura Von-Neumann

1.4.2 - Arquitetura Harvard.

Sua principal característica é possuir dois barramentos um para dados e outro para instruções, como visto na **Figura 1.3**.

Nesta arquitetura, para fazer uma operação de soma de dois números, é necessário apenas um ciclo de máquina, uma vez que não são usados microcódigos, pois as

operações são feitas diretamente em cada um dos blocos e através dos barramentos de dados e de instruções.

A vantagem disto é que, enquanto uma instrução é processada, outra já pode estar executando o seu ciclo de busca, carregando assim, a próxima instrução. Isto faz com seja necessária uma quantidade menor de ciclos de máquina para executar uma instrução.

Esta arquitetura permite o processamento, ao mesmo tempo, de um ciclo de busca e um ciclo de execução. Assim, a cada ciclo de máquina existirá uma instrução pronta para ser executada. Este sistema de busca/execução é conhecido como Segmentação ou Pipeline.

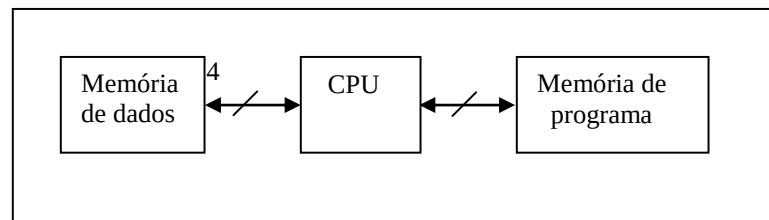


Figura 1.3 – Arquitetura Harvard.

1.4.3 – Instruções.

As instruções estão ligadas ao tipo de arquitetura utilizada no microcontrolador, assim temos:

a) Instruções CISC (Complex Instruction Set Computing) - Computação com Conjunto Complexo de Instruções.

Para a arquitetura Von-Neumann, usam-se instruções CISC. São complexas internamente, pois um simples comando de mover uma informação da porta de entrada para a porta de saída necessitará de vários clocks, diversos ciclos de máquina e precisará de um microcódigo para controlar a execução de cada uma das instruções.

b) Instruções RISC (Reduced Instruction Set Computing) - Computação com Conjunto de Instruções Reduzidas.

Para a arquitetura Harvard, usam-se instruções RISC. São reduzidas porque o próprio hardware interno do microcontrolador já está montado de maneira a, com um único ciclo de máquina, fazer uma busca de nova instrução e executar a instrução que foi buscada anteriormente, não necessitando de microcódigo para controlar a execução das instruções.

c) Comparação CISC/RISC.

- I)** O RISC possui menor quantidade de instruções que o CISC com a mesma capacidade de processamento.
- II)** Instruções fáceis de serem memorizadas no RISC.
- III)** Os programas se tornam maiores no RISC, pois nem todas as instruções disponíveis para o CISC estão disponíveis para o RISC.
- IV)** Instruções que faltam no RISC devem ser criadas pelo programador, via software, que deve ter bastante experiência em termos de programação.

Obs: A arquitetura do PIC segue o modelo Harvard, onde a CPU usa barramentos diferentes com as memórias de instruções (programa) e de dados, e se tem acesso simultaneamente a estas duas memórias. Além de maior velocidade e usar um número reduzidos de instruções (RISC) é possível usar instruções com mais de 8 bits. Em geral, uma instrução é executada em um único ciclo de máquina (corresponde a 4 ciclos de clock). Usa a técnica de Segmentação ou Pipeline e instruções Ortogonais (contêm a fonte e o destino dos dados)

d) Entendendo instruções CISC/RISC.

Quando um novo microprocessador é fabricado, aumenta-se o seu conjunto de instruções para que ele se torne mais poderoso.

Na construção de um novo processador, uma nova instrução substitui um grupo inteiro de instruções que antes eram necessárias para a execução de uma determinada tarefa. Para escrever um programa fica mais fácil, pois não há a necessidade de escrever um pequeno programa (sub-rotinas) para executar o que a nova instrução fazia sozinha. Na verdade, o pequeno programa continua sendo

necessário só que ele já está escrito dentro do microprocessador e que executará a ação quando uma determinada instrução for acionada. Estes programas são armazenados em uma área do microprocessador chamada de Microcódigo.

Um novo processador fabricado possuirá um maior conjunto de instruções, maior área de microcódigo e fisicamente serão maiores.

Quando uma instrução é dada ao microprocessador, seu decodificador de instruções verifica, no conjunto de instruções, se ela é válida e assim, executa ou não o subprograma referente à instrução dentro do microcódigo. Quanto maior o conjunto de instruções, mais lento ficará o microprocessador na procura da validade e, portanto, na execução efetiva da instrução. Isto vai de encontro ao que imaginamos que um processador mais avançado será muito mais rápido.

Para compensar a demora real um dos recursos usado para aumentar o desempenho é a técnica de Pipeline. Enquanto uma área dentro do microprocessador está terminando de executar uma instrução, a outra já estará lendo e decodificando a próxima instrução, pois a execução e a decodificação da instrução ocorrem em áreas distintas do microprocessador.

Outras técnicas também usadas são o cachê de memória, integração de circuitos de apoio dentro do microprocessador, arquitetura superescalar, previsão de desvio, etc. O aumento cada vez maior do conjunto de instruções dos novos microprocessadores necessita também de novos recursos para aumentar seu desempenho, além disso, chegou-se a conclusão de que só 20% das instruções, no geral, são realmente usadas pelos programadores.

Com isso, surgiu a idéia de construção de microprocessadores com um conjunto reduzido de instruções (RISC) e ainda a eliminação do microcódigo.

Para que isso aconteça devemos lembrar que instruções CISC não são padronizadas. Apesar de existir, por exemplo, instruções MOV diferentes, cada uma é decodificada de modo diferente, interpretada pelo microcódigo de maneira diferente, demora tempos diferentes dentro do microprocessador e requer uma quantidade de parâmetros que variará conforme o tipo de instrução (exemplo, MOV AL, precisa de dados de 8 bits, enquanto MOV AX precisa de dados de 16 bits). A solução encontrada foi a completa padronização.

A programação de um microprocessador RISC tende a ser mais complexa, por conter menos instruções em comparação com o CISC equivalente. Quando uma instrução mais complexa é necessária, o programador deverá criar um pequeno programa (sub-rotina) com a tarefa que ele pretende executar, como acontecia com os microprocessadores mais rudimentares. Mesmo com essa sub-rotina sendo executada fora do microprocessador, ele será mais rápido do que se fosse executada dentro do microprocessador, no microcódigo, isto graças ao “enxugamento” que foi feito no processador.

Como essas arquiteturas são incompatíveis entre si (precisam de decodificador CISC ou RISC), alguns microprocessadores podem usar uma arquitetura híbrida CISC/RISC.

Na execução das instruções RISC, é usada a técnica de Segmentação (Pipeline) o que permite ao processador realizar duas funções ao mesmo tempo (execução da instrução e localização do código da instrução seguinte em um único ciclo). De um modo geral, essas instruções são Ortogonais onde qualquer uma delas pode controlar a fonte e o destino dos dados.

CAPÍTULO 2

VISÃO GERAL DO MICROCONTROLADOR MICROCHIP PIC16F877.

2.1 - INTRODUÇÃO.

O PIC16F877 da Microchip pertence ao tipo de processador RISC que é um processador com instruções reduzidas e todas possuem o mesmo tempo de processamento, exceto em instruções de salto. Este tipo de processador possui arquitetura Harvard o que significa trabalhar com as memórias de programa e de dados de forma separadas.

2.2 – CARACTERÍSTICAS.

2.2.1 - As principais características deste microcontrolador são:

- a) **Processador com arquitetura Harvard e Instruções RISC avançada.**
- b) **Jogo de 35 instruções com palavra de 14 bits.** Todas elas executadas em um ciclo de instrução exceto instruções de salto que demoram 2 ciclos.
- c) **Frequência externa máxima de operação de 20 MHz.**
- d) **Ciclo de instrução de 200ns.**
- e) **Memória Flash de programa com capacidade de 8K x 14.**
- f) **Memória RAM de dados de 512 x 8 (368 x 8 para o programador).**
- g) **Memória EEPROM de dados de 256 bytes (256 x 8).**
- h) **Possui 14 fontes de interrupções internas e externas.**
- i) **Pilha com 8 níveis.**
- j) **Modos de endereçamento:** direto, indireto e imediato.
- k) **Resets:** Power-On Reset (POR), Power-up Timer (PWRT), Oscilador Start-upTimer (OST) e Brown-out Reset (BOR)
- l) **Watchdog Timer (WDT)** com seu próprio oscilador RC confiável.
- m) **Código de proteção programável.**
- n) **Modo Sleep de baixo consumo.**
- o) **Opções de seleção do oscilador.**
- p) **Tecnologia CMOS de alta velocidade e baixo consumo** para as memórias Flash/EEPROM.
- q) **Programação série no circuito** via 2 pinos (In-Circuit Serial Programming (ICSP)).
- r) **Capacidade de Programação série no próprio circuito com 5V.**
- s) **In-Circuit Debugging via dois pinos.**

- t) Larga faixa de voltagem de alimentação:** 2V a 5,5V.
- u)** High Sink/Source current: 25mA.
- v) Baixo consumo de potência** (< 0,6 mA a 3 V e 4 MHz).
- x) Portas I/O:** A, B, C, D e E.

2.2.2 - Características dos periféricos:

a) Três (3) Timers:

- I)** Timer0: contador/temporizador de 8 bits e prescaler com 8 bits
- II)** Timer1: contador/temporizador (16 bits = 2 x 8 bits) com prescaler (divide os impulsos de entrada por 1, 2, 4 ou 8), que pode ser incrementado durante o SLEEP via clock externo a cristal.
- III)** Timer2: temporizador de 8 bits com Registrador de período, prescaler (divide 1, 4, ou 16) e postscaler programável de 1 a 16 e que pode ser usado para gerar a base de tempo do PWM.

b) Dois (2) módulos CCP (Capture/Compare/PWM).

- I)** Capture 16 bits e máxima resolução de 12,5ns.
- II)** Compare 16 bits e resolução máxima de 200ns.
- III)** PWM máxima resolução é 10 bits.

c) Um Conversor Analógico/Digital de 8 canais e resolução de 10 bits.

d) Porta Serial Síncrona (SSP), modo SPI (Mestre/Escravo) e modo I2C (Mestre/Escravo).

e) Universal Synchronous Asynchronous Receiver Transmitter (USART/SCI com detecção de endereço de 9 bits).

f) Porta Paralela Escrava (PSP) de 8 bits, com controles externos de /RD, /WR e /CS.

g) Circuito Brown-out detection para Brown-out Reset (BOR)

2.3 - ARQUITETURA INTERNA DO PIC16F87X.

Observamos que o micro possui vários blocos internos: processador, memória de programa, memória de dados, periféricos, contadores, etc.

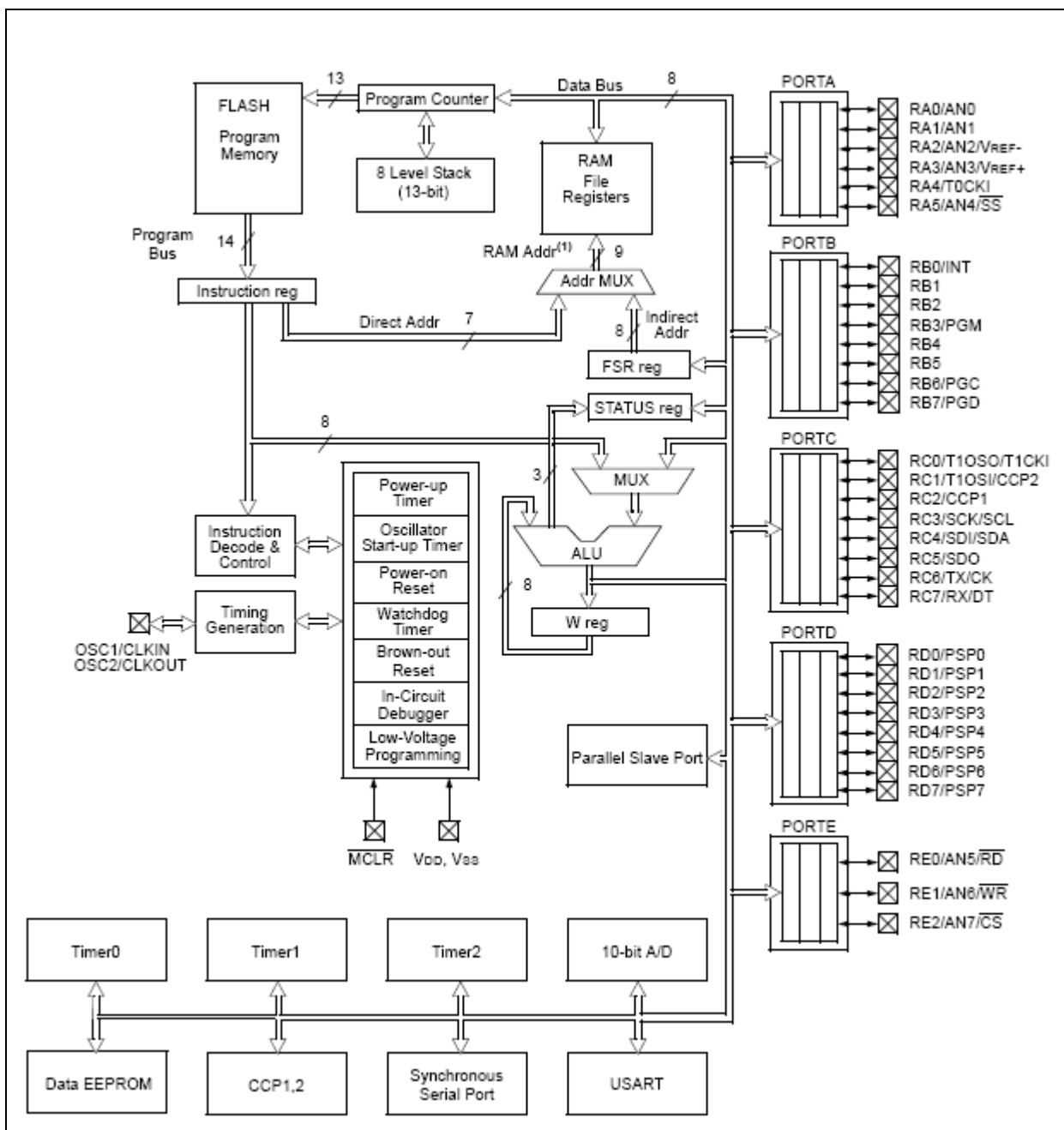


Figura 2.1: Diagrama de blocos do PIC16F877.

2.3.1 - No diagrama podemos identificar:

- Memória de Programa, (FLASH)** de 8K x 14 (8192 posições ou endereços de 14 bits cada).
- Memória de Dados, (RAM)** de 512 x 8 (512 posições ou endereços de 8 bits).

- c) **Memória EEPROM**, 256 posições e 8 bits.
- d) **Processador propriamente dito** é formado pela ALU (Unidade Lógica e Aritmética) e o Registro de trabalho (W).
- e) **Periféricos I/O**: Portas A, B, C, D, E.
- f) **Contador de Programa (PC)** de 13 bits capaz de direcionar até 8K de memória Flash de programa (14 bits). Este registro aponta sempre a próxima instrução a ser executada o que nem sempre é seqüencial.
- g) **Pilha ou Stack** possui 8 níveis e não pode ser lida ou escrita diretamente. Funciona como um buffer temporal onde se guarda o Contador de programa cada vez que se solicita uma chamada a um procedimento I/O (incluindo interrupções). Quando de uma instrução CALL, o endereço do PC é armazenado na Pilha para posterior recuperação quando da primeira instrução RETURN, RETLW ou RETFIE que encontrar.
- h) **Registro de Instruções**, registrador de 14 bits que se carrega cada vez que a ALU solicita uma nova instrução a ser processada.
- i) **Decodificador de instruções**: identifica que operação será realizada; recebe a instrução da memória via RI e gera os códigos do microprograma.
- j) **Status Reg.** (Registro STATUS) que é um registro encarregado de anotar o estado atual do sistema, todas as vezes em que uma instrução for executada. Cada um desses eventos está associado a um bit desse registro.
- k) **FSR reg.** (Registro FSR) que cumpre uma função similar a do Contador de programa, direcionando, neste caso, para a RAM. O FSR é um ponteiro direcionado para a RAM.
- l) **MUX**, a existência de MUX se deve às diferentes fontes de dados encontradas. Mas adiante explicaremos sobre eles.
- m) **TMR0** (temporizador/contador de eventos, de 8 bits e prescaler de 8 bits).
- n) **TMR1** de 16 bits (dois registros de 8 bits cada), que pode contar os impulsos de um oscilador externo quando o micro estiver no estado de SLEEP, também dispõe de prescaler que pode dividir os impulsos de entrada por 1, 2, 4 ou 8.

o) TMR2 de 8 bits e dispõe de prescaler que pode dividir por 1, 4 ou 16. Dispõe também de postscaler programável de 1 a 16. Pode ser usado para gerar a base de tempo do PWM.

p) In-Circuit Debugger (MPLAB-ICD), junto com os blocos detetores de alimentação, temos o In-Circuit Debugger (MPLAB-ICD) que é controlado pelo MPLAB. Juntamente com o hardware que o fabricante desenvolveu, permite rodar e depurar os programas em tempo real e com o micro instalado no circuito sob teste.

No modo ICD, alguns recursos são reduzidos, como:

I) A Pilha (Stack) que se reduz para 7 níveis.

II) Para uso interno, usa seis endereços de memória de dados (70h e de 1EBh a 1EFh).

III) É preciso deixar livre as linhas RB6 e RB7, pois são as linhas que comunicam o micro com o circuito de controle.

2.4 - CICLO DE INSTRUÇÃO NO PIC.

O clock interno deste micro é equivalente ao clock externo dividido por 4.

$$CK_{int} = CK_{ext}/4$$

Com um cristal de 4MHz a frequência interna é de 1MHz, com isso, o ciclo de máquina (CM ou Tcy) será:

$$T_{CY} = 1/CK_{int} = 1\mu s.$$

Isto se deve ao fato de que para a execução de uma única instrução pelo micro, várias operações precisam ser realizadas. Como não existe capacidade de processamento paralelo, essas operações são executadas em subciclos do ciclo de máquina originados pela divisão do clock interno. Esses subciclos formam as fases Q1, Q2, Q3 e Q4.

O Contador de programa (PC) é incrementado automaticamente na fase Q1 do ciclo de máquina e a instrução seguinte é buscada da memória de programa e armazenada no registrador de instruções no ciclo Q4. Ela é decodificada e executada no próximo ciclo, no intervalo de Q1 e Q4, sendo trocadas informações com a memória de dados e o registro W (Work) sempre que necessário. Essa característica de buscar a informação em um ciclo de máquina e executá-la no próximo é conhecida como Pipeline (seguimentação). Ela permite que quase todas as instruções sejam executadas em apenas

um ciclo, gastando assim $1\mu s$ para a frequência de 4MHz e tornando o sistema muito mais rápido. As únicas exceções são para as instruções que geram saltos no programa, como chamadas de sub-rotinas e retornos. Ao executar essas instruções o Pipeline deve ser primeiramente limpo para depois poder ser carregado novamente com o endereço correto, consumindo para isso dois ciclos de máquina.

De um modo geral, as instruções são Ortogonais onde qualquer uma delas pode controlar a fonte e o destino dos dados.

Normalmente um ciclo de instrução consome 4 ciclos de clock. Isto não se aplica às instruções de salto, pois não se conhece o endereço da instrução seguinte até completar a bifurcação, pelo que esta operação necessita de 2 ciclos de instrução.

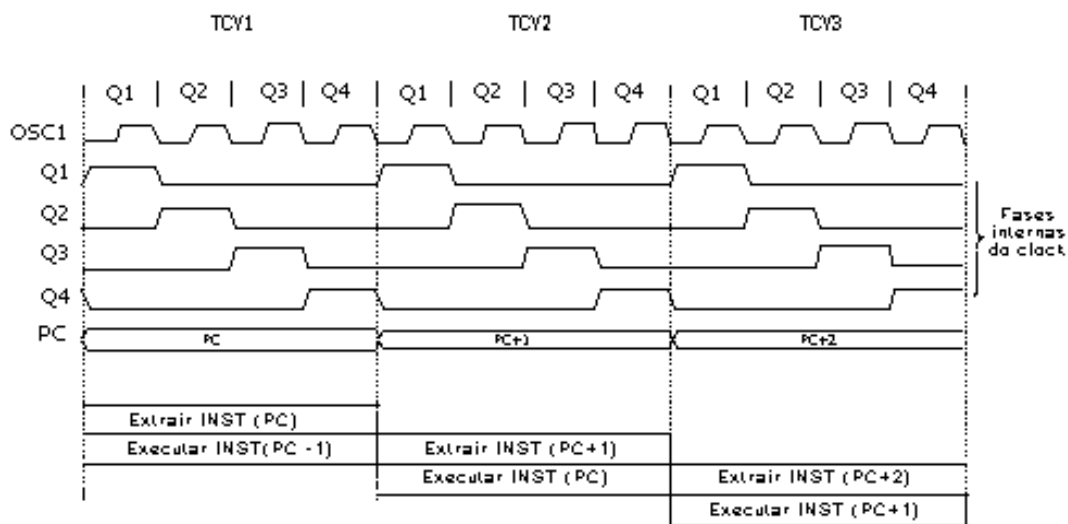


Figura 2.2: Diagrama temporal do ciclo de máquina.

Ao ciclo de instrução abaixo, foram adicionadas 5 instruções como exemplo: Carregar W com 55h, mover este valor para PORTB, chamar uma subrotina e cumprir essa subrotina. A instrução 3 não será cumprida, pois a execução seguinte é uma subrotina.

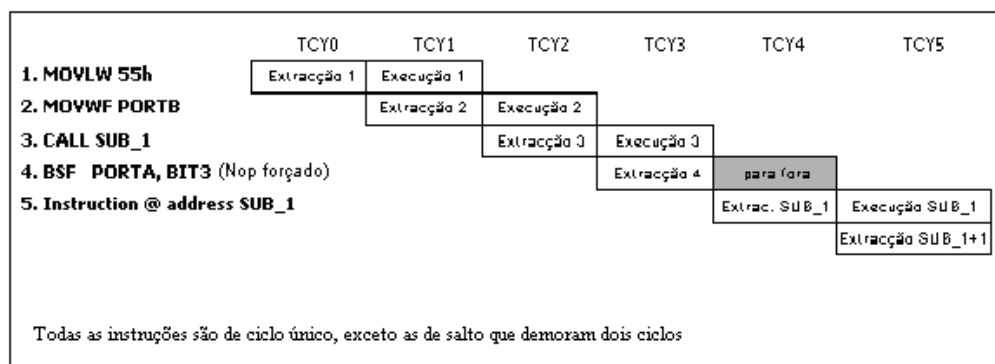


Figura 2.3: Programa com os ciclos de extração e execução.

2.5 - DIAGRAMA DE PINAGEM.

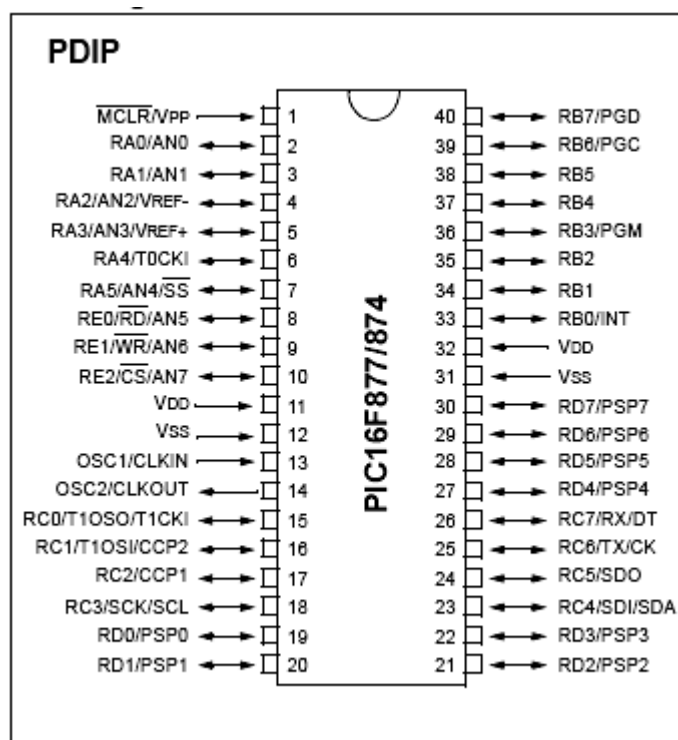


Figura 2.4: Diagrama de pinagem.

2.5.1 - Significado das siglas.

RA0-5 : Input/Output port A RB0-7 : Input/Output port B RC0-7 : Input/Output port C RD0-7 : Input/Output port D RE0-2 : Input/Output port E AN0-7 : Analog input port RX : USART Asynchronous Receive TX : USART Asynchronous Transmit SCK : Synchronous serial clock input SCL : Output for both SPI and I ² C modes DT : Synchronous Data CK : Synchronous Clock SDO : SPI Data Out (SPI mode) SDI : SPI Data In (SPI mode) SDA : Data I/O (I ² C mode) CCP1,2 : Capture In/Compare Out/PWM Out OSC1/CLKIN : Oscillator In/External Clock In OSC2/CLKOUT : Oscillator Out/Clock Out	MCLR : Master Clear (Active low Reset) Vpp : Programming voltage input THV : High voltage test mode control VREF+/- : Reference voltage SS : Slave select for the synchronous serial port T0CKI : Clock input to Timer0 T1OSO : Timer1 oscillator output T1OSI : Timer1 oscillator input T1CKI : Clock input to Timer1 PGD : Serial programming data PGC : Serial programming clock PGM : Low voltage programming input INT : External interrupt RD : Read control for the parallel slave port WR : Write control for the parallel slave port CS : Select control for the parallel slave PSP0-7 : Parallel slave port VDD : Positive supply for logic and I/O pins Vss : Ground reference for logic and I/O pins
--	--

2.5.2 - Funções associadas aos pinos.

Pino	Nome	Descrição
13	OSC1/CLKIN :	Entrada para cristal/ Entrada para osc. externos (híbridos ou RC).
14	OSC2/CLKOUT :	Saída do osc. a cristal. Cristal ou ressonador no modo osc. a cristal. No modo RC, a saída OSC2 é ¼ da freq. de OSC1 (ciclo de instrução)
1	MCLR(barrado)/Vpp/THV :	Reset externo / Entrada de tensão de programação (13V) / Controle de modo de teste para alta tensão (13V).
12, 31	Vss :	Referência de terra para pinos de E/S lógicos.
11, 32	Vdd :	Alimentação para pinos de E/S lógicos.

Pino	Nome (PORTA A)	Descrição
2	RA0/AN0:	E/S digital / Entrada analógica
3	RA1/AN1 :	E/S digital / Entrada analógica
4	RA2/AN2/Vref-:	E/S digital / Entrada analógica / Tensão negativa de referência
5	RA3/AN3/Vref+:	E/S digital / Entrada analógica / Tensão positiva de referência
6	RA4/T0CK1:	E/S digital / Entrada de relógio para o Timer0
7	RA5/SS(barrado)/AN4:	E/S digital / Entrada analógica / Habilitação externa para porta escrava serial síncrona.
(PORTA B)		
33	RB0/INT:	E/S digital / Interrupção externa porta B
34	RB1:	E/S digital porta B
35	RB2:	E/S digital porta B
36	RB3/PGM:	E/S digital / Entrada de programação de baixa tensão (5V) porta B
37	RB4:	E/S digital / Pino de interrupção por mudança de estado no pino da porta B
38	RB5:	E/S digital / Pino de interrupção por mudança de estado no pino da porta B
39	RB6/PGC:	E/S digital / Pino de interrupção por mudança de estado / Pino de clock da programação serial do In-Circuit Debugger porta B
40	RB7/PGD:	E/S digital / Pino de interrupção por mudança de estado / Pino de dados da programação serial do In-Circuit Debugger porta B
(PORTA C)		
15	RC0/T1OSO/T1CKI:	E/S digital / Saída do osc. Timer1 / Entrada de clock do Timer1
16	RC1/T1OSI/CCP2:	E/S digital / Entrada do osc. Timer1 / Entrada do Capture2 / Saída do Compare2 / Saída do PWM2.
17	RC2/CCP1:	E/S digital / Entrada do Capture1 / Saída do Compare1 / Saída do PWM1.
18	RC3/SCK/SCL:	E/S digital / Entrada/Saída do relógio síncrono para os modos SPI e I ² C.
23	RC4/SDI/SDA:	E/S digital / Entrada de dados SPI / E/S de dados I ² C.
24	RC5/SDO:	E/S digital / Saída de dados SPI.
25	RC6/TX/CK:	E/S digital / Transmissão assíncrona da USART / Clock síncrono.
26	RC7/RX/DT:	E/S digital / Recepção assíncrona da USART / Dados síncrono.
(PORTA D)		
19	RD0/ PSPO:	E/S digital / Dado 0 (comunicação paralela).
20	RD1/PSP1:	E/S digital / Dado 1 (comunicação paralela)
21	RD2/PSP2:	E/S digital / Dado 2 (comunicação paralela)
22	RD3/ PSP3:	E/S digital / Dado 3 (comunicação paralela).
27	RD4/PSP4:	E/S digital / Dado 4 (comunicação paralela)
28	RD5/PSP5:	E/S digital / Dado 5 (comunicação paralela).
29	RD6/PSP6:	E/S digital / Dado 6 (comunicação paralela).
30	RD7/PSP7:	E/S digital / Dado 7 (comunicação paralela).
(PORTA E)		
8	RE0/RD(barrado)/AN5:	E/S digital / Controle de leitura da porta paralela / Entrada analógica 5.
9	RE1/WR/AN6:	E/S digital / Controle de escrita da porta paralela / Entrada analógica 6.
10	RE2/CS/AN7:	E/S digital / Controle de seleção da porta paralela / Entrada analógica 7.

CAPÍTULO 3.

ORGANIZAÇÃO DA MEMÓRIA DO PIC.

3.1 – INTRODUÇÃO.

O PIC16F877 possui dois blocos de memórias separados (dados e programa), agrupados em três tipos de memórias, que são:

a) Memória de Programa Flash – armazena o programa que se escreveu.

b) Memória de Dados:

I) RAM - parte é usada como registros SFR é parte é de dados como os registros GPR.

II) Memória de Dados EEPROM – armazena dados que não podem ser perdidos quando o micro fica sem alimentação. Esta memória de dados possui capacidade de 256 x 8 e será analisada no Capítulo 5.

Obs: A memória EEPROM e os Registros de Uso Geral (GPR) na RAM, constituem o bloco para dados.

3.2 - Organização da Memória de Programa (Flash).

A Memória de Programa é uma memória regravável eletronicamente do tipo Flash e capacidade de 8K x 14 bits. Ela pode ser acessada por software, possibilitando que o programa seja reescrito dinamicamente, ou ainda, que ela seja usada como expansão da memória EEPROM de dados.

3.2.1 - Vetor de Reset.

Usa o primeiro endereço da Memória de programa (0x0000) que será executado após um Start-up ou Reset.

3.2.2 - O Vetor de Interrupção.

Este micro possui 14 tipos de interrupções diferentes. Entretanto, quando qualquer uma delas acontece (se habilitadas), o programa será desviado para o vetor de interrupção endereço 0x0004.

3.2.3 - Mapa da Memória de Programa.

Para dispositivos com mais de 512 bytes de memória de programa usa-se um esquema de páginas

Esta memória possui capacidade de 8K x 14 (8192 posições ou endereços com 14 bits por posição) e está dividida em quatro Páginas de 2K (2048 posições) cada e seus endereços, são:

- a) Página 0** (0005h a 07FFh) 5 a 2047.
- b) Página 1** (0800h a 0FFFh) 2048 a 4095.
- c) Página 2** (1000h a 17FFh) 4096 a 6143.
- d) Página 3** (1800h a 1FFFh) 6144 a 8191.

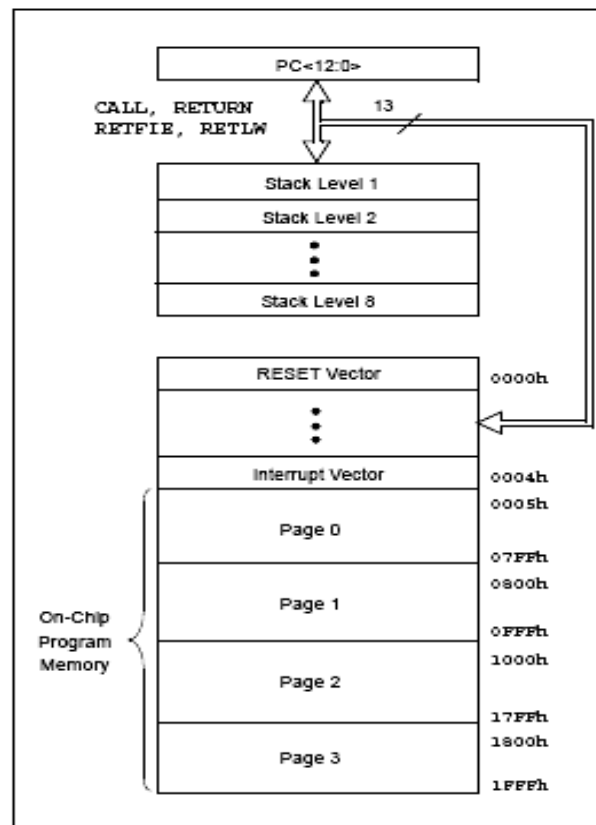


Figura 3.1: Mapa da memória de Programa.

3.2.4 - O porquê desta paginação tem uma explicação:

Cada instrução é carregada através do endereço armazenado no PC (Contador de programa), assim sendo, para podermos acessar os 8192 endereços diferentes necessitaríamos de um PC com 13 bits. Isto é resolvido por meio do desmembramento do PC em dois registros (FSRs): PCL (parte baixa com 8 bits) e PCH (parte alta com 5

bits). Em parte o problema foi resolvido, no entanto, quando usamos as instruções CALL e GOTO estas vinculam ao seu Opcode de 14 bits o endereço para onde o programa deve ser desviado.

Exemplo:

CALL: Opcode (100 kkk kkkk kkkk) e **GOTO:** Opcode (101 kkk kkkk kkkk); o termo (kkk kkkk kkkk) equivale ao endereço de destino que possui 11 bits.

Como só existem 11 bits para o endereço de destino da instrução só conseguimos gerenciar 2K ($2^{11} = 2048$ posições) de memória. Com isso, não podemos usar essas instruções com toda a memória de programa. Para resolver esse problema foi criado o conceito de paginação, isto é, a memória de 8K foi dividida em 4 páginas de 2K cada uma. Dentro de cada página as instruções CALL e GOTO funcionam perfeitamente, pois possuem capacidade de gerenciamento para esse tamanho. O problema passa a existir somente quando quisermos utilizar essas instruções com um endereço de destino em outra página. A solução é configurar manualmente os dois bits que estão faltando localizados nos bits 3 e 4 do registro PCLATH. Esses bits passam a operar como chaves seletoras da página de programa do momento.

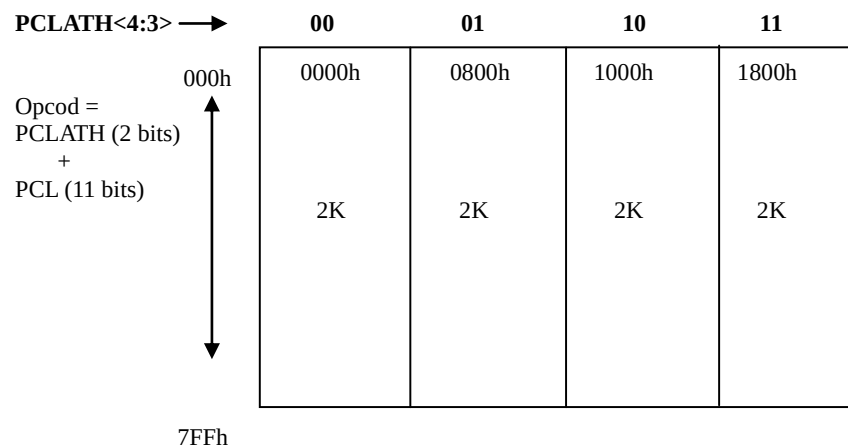


Figura 3.2: Bits fornecidos pelo Registro PCLATH nas instruções CALL e GOTO.

Se for executado o retorno de uma instrução CALL ou interrupção, os 13 bits completos do PC estão na Pilha (Stack). Por isso, não é necessário acionar os bits

PCLATH<4:3> para instruções de retorno, pois o endereço do retorno é fornecido pela Pilha (Stack).

Obs: Não há mudança no conteúdo do registro PCLATH após a execução das instruções RETURN ou RETFIE. O usuário deve reescrever o conteúdo do registro PCLATH para qualquer subsequente chamada de sub-rotina ou instruções GOTO.

O exemplo abaixo mostra a chamada de uma sub-rotina na página 1 da memória de Programa. Este exemplo assume que PCLATH foi salvo e restaurado pela Interrupt Service Routine (ISR), se a interrupção for usada.

Chamada de uma sub-rotina na página 1 da página 0.

```
org 0x500
bcf PCLATH,4
bsf PCLATH,3 ;Seleção de página 1 (800h-FFFh)
call SUB1_P1 ;chama sub-rotina SUB_P1na página 1 (800h-FFFh)
:
org 0x900 ;pagina 1 (800h-FFFh)
SUB1_P1
: ;chamada de sub-rotina pagina 1 (800h-FFFh)
:
return ;retorna a página 0 após executar sub-rotina (000h-7FFh)
```

Para podermos usar as instruções CALL e GOTO entre páginas, primeiro devemos selecionar a página correta do endereço destino através dos bits PCLATH<4:3>. A diretiva PAGESEL pode ser usada para selecionar a página correta para um destino qualquer.

Obs1: No acesso seqüencial do programa não é necessário usar o registro PCLATH. Adiante veremos mais detalhes do Registro PC.

3.2.5 - Pilha (Stack).

É um local, fisicamente separado da memória de programa, onde serão armazenados os endereços de retorno quando utilizamos instruções de desvio para rotinas de chamadas. Quando o programa é desviado para o começo de uma rotina através da instrução correta (CALL ou Interrupção), o endereço seguinte ao ponto que estava sendo

executado é armazenado na Pilha para que, ao final da sub-rotina, o programa possa retornar novamente ao ponto em que estava. Portanto, todas as instruções CALL antes de serem executadas, salvam na Pilha o PC inicial incrementado de uma unidade. Quando posteriormente encontrar e executar uma instrução RETURN (RETLW ou RETFIE), o valor da Pilha é devolvido ao PC para que prossiga a execução normal do programa.

Os PIC16F877 possuem oito níveis de Pilha com endereços de 13 bits, assim é possível armazenar oito endereços de retorno, possibilitando oito desvios consecutivos. A pilha não pode ser lida ou escrita diretamente. Se houver chamada para mais de oito sub-rotinas, o programa apresentará erro, pois o primeiro endereço armazenado será substituído pelo último e assim, haverá perda do endereço de retorno.

Como já visto, a função básica da Pilha é guardar o valor do PC quando ocorrem saltos do programa principal para o endereço de um subprograma a ser executado. Depois de ter executado o subprograma, para que o micro possa continuar com o programa principal a partir do ponto em que deixou, ele tem que ir buscar na Pilha esse endereço e carregá-lo no PC. Quando nos movemos de um programa para um subprograma, o conteúdo do PC é empurrado para o interior da Pilha (exemplo é a instrução CALL). Quando são executadas instruções tais como RETURN, RETLW ou RETFIE no fim de um subprograma, o PC é retirado da Pilha, de modo a que o programa possa continuar do ponto em que a sequência foi interrompida. Estas operações de colocar e extrair da Pilha o PC, são designadas por PUSH (meter na pilha) e POP (tirar da pilha), estes nomes provêm de instruções de micros de maior porte.

O PCLATH não é afetado pelas operações PUSH ou POP.

A Pilha opera como um buffer circular do tipo LIFO (Last In First Out), caracterizada por apresentar um único ponto de acesso denominado Topo da Pilha (TOS – Top of Stack). Isto significa que após ter usado os 8 níveis da Pilha (PUSH) o nono push escreverá sobre o valor que estava armazenado no primeiro push. O décimo push escreverá sobre o segundo push, e assim por diante.

Obs1: Não há bits de status para indicar condições de stack overflow (pilha cheia) ou stack underflow (pilha vazia ou incompleta).

3.3 - ORGANIZAÇÃO DA MEMÓRIA DE DADOS (RAM).

A memória de dados está composta por Registros ou bytes de memória RAM, de modo que cada endereço de memória pode ser especificado pelo nome do Registro. Serve para guardar as variáveis e os registros usados pelo programa, é de 8 bits e é volátil.

Esta memória, como veremos adiante, é dividida em múltiplos Bancos os quais contém:

- a)** General Purpose Registers – GPR (Registros de Propósitos Gerais)
- b)** Special Function Registers – SFR (Registros Especiais ou de Funções Especiais).

3.3.1 - General Purpose Register File (GPR).

Os registros de arquivos podem ser acessados direta e indiretamente.

Os Registros de Arquivos de Propósitos Gerais, são endereços livres de RAM para que o programador disponha deles, podem ser lidos ou escritos.

File Address	File Address	File Address	File Address
Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾	Indirect addr. ⁽¹⁾
TMR0	OPTION_REG	TMR0	OPTION_REG
PCL	PCL	PCL	PCL
STATUS	STATUS	STATUS	STATUS
FSR	FSR	FSR	FSR
PORTA	TRISA		
PORTB	TRISB	PORTB	TRISB
PORTC	TRISC		
PORTD ⁽¹⁾	TRISD ⁽¹⁾		
PORTE ⁽¹⁾	TRISE ⁽¹⁾		
PCLATH	PCLATH	PCLATH	PCLATH
INTCON	INTCON	INTCON	INTCON
PIR1	PIE1	EEDATA	EECON1
PIR2	PIE2	EEADR	EECON2
TMR1L	PCON	EEDATH	Reserved ⁽²⁾
TMR1H		EEADRH	Reserved ⁽²⁾
T1CON			
TMR2	SSPCON2		
T2CON	PR2		
SSPBUF	SSPADD		
SSPCON	SSPSTAT		
CCPR1L			
CCPR1H			
CCP1CON			
RCSTA	TXSTA		
TXREG	SPBRG		
RCREG			
CCPR2L			
CCPR2H			
CCP2CON			
ADRESH	ADRESL		
ADCON0	ADCON1		
General Purpose Register 96 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes
	accesses 70h-7Fh	accesses 70h-7Fh	accesses 70h - 7Fh
Bank 0	Bank 1	Bank 2	Bank 3

■ Sem implementar, em leitura é '0'.

* Não é um registro físico.

Nota 1: Estes registros não são implementados nos dispositivos de 28 terminais.

2: Estes registros estão reservados, manter sempre apagados.

Figura 3.3: Mapa dos registros de arquivos do PIC16F877.

3.3.2 - Registros de Funções Especiais (SFR).

São registros usados pela CPU e módulos periféricos para controlar a operação desejada do dispositivo. Estes registros são implementados como SRAM e podem ser lidos ou escritos tanto pelo usuário como pelo hardware e servem também para a configuração de muitas funções e para a utilização de todos os periféricos.

a) Os SFR podem ser classificados em dois grupos:

I) Relacionados ao Núcleo (CPU), serão descritos a seguir.

II) Relacionados aos Periféricos, são relacionados à operação de periféricos e serão vistos no trabalho com o periférico.

A lista desses registros é mostrada na. **Figura 3.3.**

3.3.3 - Bancos de memória de dados

A arquitetura do PIC está preparada para operar com uma RAM de 512 bytes. Entretanto, verificamos que nem toda a memória está disponível ao usuário. São 368 bytes para uso geral, 77 bytes para registros especiais (alguns espelhados), 19 bytes indisponíveis e 48 bytes (não fisicamente implementados) equivalem a posições espelhadas de 16 endereços válidos (Banco 0). O espelhamento facilita ao programador evitando trocas constantes de bancos.

3.3.4 - O porquê do uso do termo Banco, tem uma explicação:

O mesmo tipo de problema de gerenciamento de endereços da memória de programa surge novamente para a memória de dados.

Exemplos:

MOVF, Opcode (00 1000 dfff ffff); d = destino.

MOVWF, Opcode (00 0000 1fff ffff); fffffff = endereço na RAM.

Verificamos que só existem 7 bits para endereçamento de todos os registros, com isso, só podemos gerenciar 128 endereços diferentes ($2^7 = 128$). Por este motivo a Microchip dividiu a memória de dados em grupos de 128 bytes, aos quais chamou de Banco.

A seleção do banco é feita através dos bits RP1e RP0 (STATUS <6:5>) que funcionam como chave seletora dos bancos. O acesso deste modo é chamado de Acesso Direto e é visto na **Figura 3.5**

RP1 : RP0	Banco	Endereço do banco
00	0	00h a 7Fh (0 a 127)
01	1	80h a FFh (128 a 255)
10	2	100h a 17Fh (256 a 383)
11	3	180h a 1FFh (384 a 511)

Figura 3.4: Bits usados na seleção de Banco.

Cada Banco se estende até 7Fh (128 bytes). As localizações mais baixas de cada banco são reservadas para os SFR e as mais altas para os GPR, implementados como SRAM. Todos os bancos implementados possuem SFR. Alguns SFR freqüentemente usados de um banco, podem ser espelhados em outro banco para redução de código e acesso rápido.

Devido a essa estruturação, antes de utilizarmos qualquer comando que trabalhe com os registros da RAM, devemos comutar para o banco onde o mesmo se encontra.

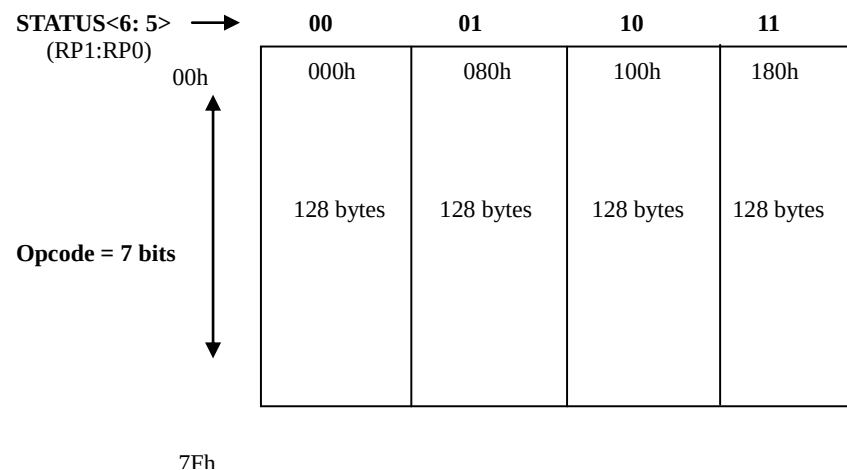


Figura 3.5: Acesso direto

Para a seleção do banco de memória, no acesso direto, existe também a diretiva do compilador BANKSEL e BANK.

Existe um caso onde a seleção do banco é feita de maneira diferente. Trata-se do Acesso Indireto através dos registros FSR e INDF. Quando acessamos a memória dessa forma, o endereço do registro desejado será colocado em FSR que possui 8 bits,

por isso, só falta um bit para completar o gerenciamento. Neste caso, a chave seletora é o bit IRP (STATUS<7>). Adiante veremos este acesso com mais detalhes.

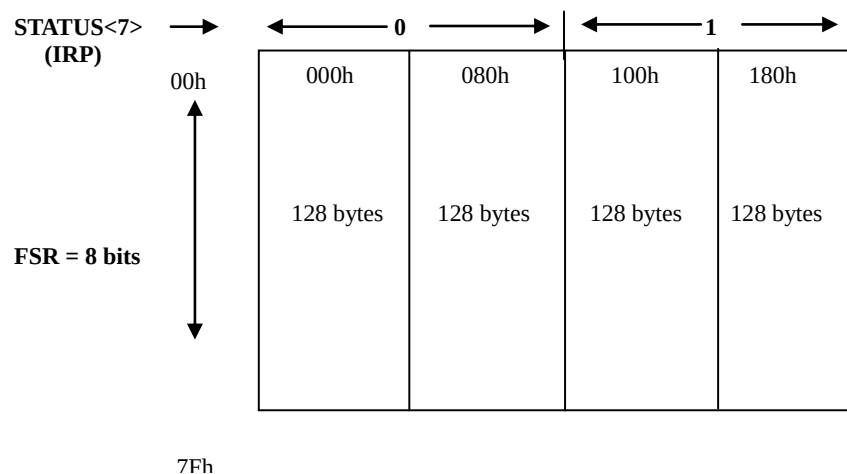


Figura 3.6: Acesso indireto.

a) **Bancos de Registros Especiais** e funções que desempenham e valores que terão na inicialização.

I) Banco 0.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Details on page:
Bank 0											
00h ⁽³⁾	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								0000 0000	27
01h	TMR0	Timer0 Module Register								xxxxx xxxxx	47
02h ⁽³⁾	PCL	Program Counter (PC) Least Significant Byte								0000 0000	26
03h ⁽³⁾	STATUS	IRP	RP1	RP0	T0	PD	Z	DC	C	0001 1xxxx	18
04h ⁽³⁾	FSR	Indirect Data Memory Address Pointer								xxxxx xxxxx	27
05h	PORTA	—	—	PORTA Data Latch when written: PORTA pins when read						--0x 0000	29
06h	PORTB	PORTB Data Latch when written: PORTB pins when read								xxxxx xxxxx	31
07h	PORTC	PORTC Data Latch when written: PORTC pins when read								xxxxx xxxxx	33
08h ⁽⁴⁾	PORTD	PORTD Data Latch when written: PORTD pins when read								xxxxx xxxxx	35
09h ⁽⁴⁾	PORTE	—	—	—	—	—	RE2	RE1	RE0	--- -xxxx	36
0Ah ^(1,3)	PCLATH	—	—	—	Write Buffer for the upper 5 bits of the Program Counter					--0 0000	26
0Bh ⁽³⁾	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	20
0Ch	PIR1	PSPIF ⁽³⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	22
0Dh	PIR2	—	(5)	—	EEIF	BCLIF	—	—	CCP2IF	-x-0 0--0	24
0Eh	TMR1L	Holding register for the Least Significant Byte of the 16-bit TMR1 Register								xxxxx xxxxx	52
0Fh	TMR1H	Holding register for the Most Significant Byte of the 16-bit TMR1 Register								xxxxx xxxxx	52
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	51
11h	TMR2	Timer2 Module Register								0000 0000	55
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	55
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								xxxxx xxxxx	70, 73
14h	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000	67
15h	CCPR1L	Capture/Compare/PWM Register1 (LSB)								xxxxx xxxxx	57
16h	CCPR1H	Capture/Compare/PWM Register1 (MSB)								xxxxx xxxxx	57
17h	CCP1CON	—	—	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0	--00 0000	58
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	96
19h	TXREG	USART Transmit Data Register								0000 0000	99
1Ah	RCREG	USART Receive Data Register								0000 0000	101
1Bh	CCPR2L	Capture/Compare/PWM Register2 (LSB)								xxxxx xxxxx	57
1Ch	CCPR2H	Capture/Compare/PWM Register2 (MSB)								xxxxx xxxxx	57
1Dh	CCP2CON	—	—	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000	58
1Eh	ADRESH	A/D Result Register High Byte								xxxxx xxxxx	116
1Fh	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON	0000 00-0	111

II) Banco 1.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Details on page:	
Bank 1												
80h ⁽³⁾	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								0000 0000	27	
81h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	19	
82h ⁽³⁾	PCL	Program Counter (PC) Least Significant Byte								0000 0000	26	
83h ⁽³⁾	STATUS	IRP	RP1	RP0	T0	PD	Z	DC	C	0001 1xxxx	18	
84h ⁽³⁾	FSR	Indirect Data Memory Address Pointer								xxxxxx xxxxxx	27	
85h	TRISA	—	—	PORTA Data Direction Register						--11 1111	29	
86h	TRISB	PORTB Data Direction Register								1111 1111	31	
87h	TRISC	PORTC Data Direction Register								1111 1111	33	
88h ⁽⁴⁾	TRISD	PORTD Data Direction Register								1111 1111	35	
89h ⁽⁴⁾	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	37	
8Ah ^(1,3)	PCLATH	—	—	—	Write Buffer for the upper 5 bits of the Program Counter						-- -0 0000	26
8Bh ⁽³⁾	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBF	0000 000x	20	
8Ch	PIE1	PSPIE ⁽²⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	21	
8Dh	PIE2	—	(5)	—	EEIE	BCLIE	—	—	CCP2IE	-x-0 0--0	23	
8Eh	PCON	—	—	—	—	—	—	POR	BOR	---- --gg	25	
8Fh	—	Unimplemented								—	—	
90h	—	Unimplemented								—	—	
91h	SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN	0000 0000	68	
92h	PR2	Timer2 Period Register								1111 1111	55	
93h	SSPADD	Synchronous Serial Port (I ² C mode) Address Register								0000 0000	73, 74	
94h	SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF	0000 0000	66	
95h	—	Unimplemented								—	—	
96h	—	Unimplemented								—	—	
97h	—	Unimplemented								—	—	
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	95	
99h	SPBRG	Baud Rate Generator Register								0000 0000	97	
9Ah	—	Unimplemented								—	—	
9Bh	—	Unimplemented								—	—	
9Ch	—	Unimplemented								—	—	
9Dh	—	Unimplemented								—	—	
9Eh	ADRESL	A/D Result Register Low Byte								xxxxxx xxxxxx	116	
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	0--- 0000	112	

III) Bancos 2 e 3.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Details on page:	
Bank 2												
100h ⁽³⁾	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								0000 0000	27	
101h	TMR0	Timer0 Module Register								x0x0x x0x0x	47	
102h ⁽³⁾	PCL	Program Counter's (PC) Least Significant Byte								0000 0000	26	
103h ⁽³⁾	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C	0001 1xx0x	18	
104h ⁽³⁾	FSR	Indirect Data Memory Address Pointer								x0x0x x0x0x	27	
105h	—	Unimplemented								—	—	
106h	PORTB	PORTB Data Latch when written: PORTB pins when read								x0x0x x0x0x	31	
107h	—	Unimplemented								—	—	
108h	—	Unimplemented								—	—	
109h	—	Unimplemented								—	—	
10Ah ^(1,3)	PCLATH	—	—	—	Write Buffer for the upper 5 bits of the Program Counter				---0 0000	26		
10Bh ⁽³⁾	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	20	
10Ch	EEDATA	EEPROM Data Register Low Byte								x0x0x x0x0x	41	
10Dh	EEADR	EEPROM Address Register Low Byte								x0x0x x0x0x	41	
10Eh	EEDATH	—	—	EEPROM Data Register High Byte						x0x0x x0x0x	41	
10Fh	EEADRH	—	—	—	EEPROM Address Register High Byte						x0x0x x0x0x	41
Bank 3												
180h ⁽³⁾	INDF	Addressing this location uses contents of FSR to address data memory (not a physical register)								0000 0000	27	
181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	19	
182h ⁽³⁾	PCL	Program Counter (PC) Least Significant Byte								0000 0000	26	
183h ⁽³⁾	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C	0001 1xx0x	18	
184h ⁽³⁾	FSR	Indirect Data Memory Address Pointer								x0x0x x0x0x	27	
185h	—	Unimplemented								—	—	
186h	TRISB	PORTB Data Direction Register								1111 1111	31	
187h	—	Unimplemented								—	—	
188h	—	Unimplemented								—	—	
189h	—	Unimplemented								—	—	
18Ah ^(1,3)	PCLATH	—	—	—	Write Buffer for the upper 5 bits of the Program Counter				---0 0000	26		
18Bh ⁽³⁾	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	20	
18Ch	EECON1	EEPGD	—	—	—	WRERR	WREN	WR	RD	x- - - x000	41, 42	
18Dh	EECON2	EEPROM Control Register2 (not a physical register)								---- ----	41	
18Eh	—	Reserved maintain clear								0000 0000	—	
18Fh	—	Reserved maintain clear								0000 0000	—	

Legenda: x = desconhecido/indefinido, u = não muda/estado anterior, q = depende das condições, - = sem implementar, em leitura é zero (0), r = reservados.

Localizações sombreadas não estão implementadas e são lidas como zero (0).

Nota 1: O byte superior do PC não é acessível diretamente e sim através do registro PCLATH.

2: Bits PSPIE e PSPIF são reservados no PIC16F873/876; bits sempre mantidos em zero (0).

3: Estes registros podem ser acessados de qualquer banco.

4: PORTD, PORTE, TRISD, e TRISE não são fisicamente implementados no PIC16F873/876; lidos como zero (0).

5: PIR2<6> e PIE2<6> são reservados nesses dispositivos; bits são mantidos em zero (0).

3.4 - CLASSIFICAÇÃO DOS REGISTROS.

Aqui, faremos uma apresentação geral dos Registros de acordo com a função que exercem dentro do micro. Na sequência, apresentaremos detalhes de alguns deles enquanto outros serão visto dentro do Capítulo referente ao Módulo junto com o qual trabalham.

Cada bit dentro desses registros recebe um nome, sendo também especificado se o bit pode ser lido (R) ou escrito (W). Caso não esteja implementado será lido como zero (0).

3.4.1 – Registro W (WORK).

O Registro W é um registro especial usado com características únicas:

- a) Pode ser usado como destino de diversas operações aritméticas e lógicas;
- b) Não pode ser usado como fonte de dados em diversas operações internas da CPU (rotação, incremento, decremento, etc.);
- c) Não é mapeado na memória RAM dos PIC como os registros SFR e GPR;
- d) É principalmente usado como ponte entre os registros “f”, pois não é possível trocar informações diretamente entre esses registros.

3.4.2 – Outros registros.

a) Registros gerais:

- I) STATUS
- II) OPTION_REG
- III) PCON
- IV) PCL e PCLATH
- V) FSR e INDF

b) Registros das Portas:

- I) PORTA e TRISA
- II) PORTB e TRISB
- III) PORTC e TRISC
- IV) PORTD e TRISD
- V) PORTE e TRISE

c) Registros dos Temporizadores (Timer):

- I) TMR0
- II) T1CON, TMR1L e TMR1H
- III) T2CON, TMR2 e PR2

d) Registros das Interrupções:

- I) INTCON – controle de interrupções.
- II) PIE1 – permissão de interrupções.
- III) PIR1 – sinalização de interrupções.
- IV) PIE2 – permissão de interrupções.
- V) PIR2 – sinalização de interrupções.

e) Registros do Conversor A/D:

- I) ADCON0
- II) ADCON1
- III) ADRESL e ADRESH

f) Registros Compare/Capture/PWM:

- I) CCP1CON
- II) CCPR1L e CCPR1H
- III) CCP2CON
- IV) CCPR2L e CCPR2H

g) Registros da EEPROM:

- I) EECON1 e EECON2
- II) EEADR e EEADRH
- III) EEDATA e EEDATH

h) Registros do SSP:

- I) SSPCON
- II) SSPCON2
- III) SSPSTAT
- IV) SSPADD
- V) SSPBUF

i) Registros da USART:**I) TXSTA****II) RCSTA****III) TXREG e RCREG****IV) SPBRG****j) Comparadores:****I) CMCON****II) CVRCON****3.4.3 - Registro STATUS (endereços 03h, 83h, 103h e 183h).**

É um registro de arquivo usado para verificação do estado geral do micro. Este registro contém o estado aritmético da ULA (<2:0>), estado do RESET (<4:3>) e os bits de seleção do Banco para a Memória RAM de Dados (<7:5>). Os bits 5 e 6 são usados no endereçamento direto e o bit 7 no endereçamento indireto. Este registro pode ser também o destino para qualquer instrução, como qualquer outro registro. Está presente em todos os bancos.

Quando este registro for o destino de uma instrução que afeta os bits Z, DC ou C, então a escrita nestes três bits fica desabilitada. Estes bits são setados ou resetados de acordo com a lógica do dispositivo. Além disso, não se pode escrever nos bits /TO e /PD (só leitura), por isso, o resultado de uma instrução com o Registro STATUS como destino, pode ser diferente do esperado.

Por exemplo, a instrução CLRF STATUS, teoricamente, deveria colocar a zero (0) todos os bits do registro e setar o bit Z. No entanto, temos:

a) Zera os 3 bits MSB (seleção de banco de memória, bits IRP:RP1:RP0);

b) Os bits /TO e /PD são somente de leitura, não se alteram;

c) Bits DC e C não são alterados;

d) Bit Z é setado.

Isto deixa o registro de STATUS com o valor 000u u1uu (u = bits não afetados). É recomendado que somente as instruções BCF, BSF, SWAPF e MOVWF sejam usadas para alterar o registro de STATUS, porque estas instruções não afetam os bits Z, C e DC do registro de STATUS a não ser que estas instruções sejam dirigidas a eles, como abaixo:

BSF STATUS, C ; seta o bit C.

R/W-0	R/W-0	R/W-0	R-1	R-1	R/W-x	R/W-x	R/W-x
IRP	RP1	RP0	$\overline{TO}$	$\overline{PD}$	Z	DC	C
bit 7							bit 0

bit 7 IRP: Register Bank Select bit (Seleção do banco de memória de dados no endereçamento indireto).

0 = Banco 0 e 1 (00h - FFh)

1 = Banco 2 e 3 (100h - 1FFh)

bit 6-5 RP1:RP0: Register Bank Select bits (Seleção do banco de memória de dados no endereçamento direto)

00 = Banco 0 (00h - 7Fh)

01 = Banco 1 (80h - FFh)

10 = Banco 2 (100h - 17Fh)

11 = Banco 3 (180h - 1FFh)

Cada Banco é de 128 posições.

bit 4 /T0: Time-out bit (estouro de contagem de tempo do WDT)

1 = Ocorreu Power-up ou foram executadas instruções CLRWDWT ou SLEEP.

0 = Ocorreu um estouro do Watchdog Timer e provocou um reset no micro.

bit 3/PD: Power-down bit (desligamento ou Sleep – indica o estado do controle de energia do micro).

1 = Ocorreu Power-up (só alimentação) ou foi executada a instrução CLRWDWT.

0 = Indica que a instrução SLEEP (baixo consumo) foi executada..

bit 2 Z: Zero Bit (Bit zero)

1 = Quando o resultado de uma operação lógica ou aritmética for zero (0).

0 = Quando o resultado de operação lógica ou aritmética não for zero (0).

bit 1 DC: Digit carry/borrow bit (Transporte de dígito/Empréstimo de dígito). Usado para as instruções ADDWF, ADDLW, SUBWF, SUBLW, na indicação de transporte ou empréstimo no quarto bit - Nibble. Para borrow a polaridade é reversa.

ADDWF:

1 = Se ocorrer transporte do 4º bit do LSB.

0 = Se não ocorrer transporte do 4º bit do LSB.

SUBWF:

1 = Se não ocorrer empréstimo do 4º bit do LSB.

0 = Se ocorrer empréstimo do 4º bit do LSB.

bit 0 C: Carry/borrow bit (Transporte/empréstimo em instruções ADDWF, ADDLW, SUBWF e SUBLW).

1 = Se ocorrer transporte no MSB.

0 = Se não ocorrer transporte no MSB.

ADDWF (C = 1, produz-se transporte no bit 7 e C = 0, não se produz transporte).

SUBWF (C = 1, não ocorreu empréstimo no bit 7 e C = 0, ocorreu empréstimo).

RRF/RLF (carrega bit C com LSB ou MSB respectivamente).

Legenda: R = só leitura, W = só escrita, U = sem implementar é lido como '0', -n = valor ao reiniciar (POR) e x = desconhecido.

Obs1: Os bits C e DC operam como transportes quando realizamos operações de subtração.

Obs2: Para borrow a polaridade do flag C é invertida. Sempre antes de uma subtração, o chip irá setar o flag C, isto é, cria uma reserva de 256 para eventual empréstimo na subtração. A subtração é executada pela adição a complemento a dois do segundo operando. Para instruções de deslocamento (RRF e RLF), o bit deslocado no byte vai para o bit C e o bit que estava em C retorna para o byte. O bit C ou é MSb ou LSb.

3.4.4 - Registro OPTION_REG (endereço 81h e 181h).

É um registro de leitura e escrita que contém vários bits de controle para configurar o prescaler TMR0/WDT, resistores de pull-up na porta B, borda da interrupção externa, borda que incrementa o TMR0, etc.

Os bits 2:0 (PS2:PS0) configuram o fator de divisão do prescaler e que é diferente conforme seja atribuído ao TMR0 (Timer0) ou ao WDT (Watchdog). Os outros bits agem diretamente no comportamento e não na configuração de alguns terminais. Como exemplo, se colocarmos o bit RBPU a zero, habilitamos os resistores de pull-up internos que são conectados entre os terminais da porta B e o positivo, mas não servem para nada se a porta for colocada como saída (desconecta-se automaticamente o resistor de cada uma das linhas que for configurada como saída). Se a interrupção externa no pino RB0/INT estiver desabilitada, o estado do bit INTEDG, é irrelevante.

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBP	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

bit 7 RBP: PORTB Pull-up Enable bit (Habilita o resistor de pull-up internos para a PORTB)

1 = Resistores de pull-ups desabilitados

0 = Resistores de pull-ups habilitados para os pinos do PORTB configurados como entrada).

bit 6 INTEDG: Interrupt Edge Select bit (Bit de seleção da borda que gera a interrupção externa no RB0/INT)

1 = Interrupção ocorrerá na borda de subida no pino RB0/INT.

0 = Interrupção ocorrerá na borda de descida no pino RB0/INT.

bit 5 T0CS: TMR0 Clock Source Select bit (Bit de seleção de clock para TMR0 e que determina se o Timer0 será um contador ou um temporizador).

1 = TMR0 funcionará como um contador e será incrementado externamente pela mudança no pino RA4/T0CK1).

0 = TMR0 funcionará como um temporizador e será incrementado pelo oscilador interno – CLKOUT = Fosc/4)

bit 4 T0SE: TMR0 Source Edge Select bit (Configuração da borda que incrementará o TMR0, quando ajustado para um contador de eventos, no pino RA4/T0CKI, isto é, T0CS = 1).

1 = É incrementado na transição de alto para baixo do pino RA4/T0CKI.

0 = É incrementado na transição de baixo para alto do pino RA4/T0CKI.

bit 3 PSA: Prescaler Assignment bit (Bit de atribuição do Prescaler)

1 = Prescaler é atribuído ao Watchdog.

0 = Prescaler é atribuído ao modulo Timer0 (TMR0)

bit 2-0 PS2:PS0: Prescaler Rate Select bits (Bits de seleção do fator de divisão do prescaler)

Bit Value	TMR0 Rate	WDT Rate
000	1 : 2	1 : 1
001	1 : 4	1 : 2
010	1 : 8	1 : 4
011	1 : 16	1 : 8
100	1 : 32	1 : 16
101	1 : 64	1 : 32
110	1 : 128	1 : 64
111	1 : 256	1 : 128

O valor inicial da contagem pode ser ajustado diretamente no registro TMR0, e quando estourar a contagem, isto é, passe de 255 para 0, irá acionar o flag T0IF (INTCON<2>). Enquanto não houver estouro da contagem, o bit T0IF permanece apagado (0), se houver estouro esse mesmo bit será setado e permanece assim até que

seja zerado pelo usuário. O estouro deste contador pode gerar uma interrupção no micro dependendo se o bit T0IE (INTCON <5>) foi previamente setado.

Obs: Quando usando Low Voltage ICSP Programming (LVP) e pull-ups na PORTB habilitados, o bit 3 no registro TRISB deve ser limpo para desabilitar o pull-up em RB3 e assegurar a operação adequada do dispositivo.

3.4.5 - Registro PCON (endereço 8Eh).

O Registro de Controle de Alimentação (Power Control - PCON), contém os bits de flags para permitir a diferenciação entre um reset durante a alimentação (Power-on Reset - POR), reset durante uma queda de energia (Brown-out Reset – BOR), reset do Watchdog (WDT) ou um reset externo no pino /MCLR. O estado do bit de flag BOR não possui nenhum efeito se bit BODEN da palavra de configuração estiver zerado (isto será visto no Capítulo 11).

U-0	U-0	U-0	U-0	U-0	U-0	R/W-0	R/W-1
—	—	—	—	—	—	POR	BOR
bit 7							bit 0

bit 7-2 Não implementado, lido como '0'

bit 1 /POR: Power-on Reset Status bit (Estado do bit Power-on Reset)

1 = Não ocorreu um Power-On Reset

0 = Ocorreu um Power-on Reset (deve ser setado pelo software).

bit 0 /BOR: Brown-out Reset Status bit (Bit de estado se houver queda de energia)

1 = Não ocorreu um Brown-out Reset.

0 = Ocorreu um Brown-out Reset (deve ser setado pelo software).

3.4.6 - PCL (endereços 02h, 82h, 102h e 182h) e PCLATH (endereços 0Ah, 8Ah, 10Ah e 18Ah).

O Contador de Programa (PC) é o responsável pelo controle da sequência da execução das instruções no PIC. O registro PC aponta sempre para a próxima instrução a ser executada pela CPU, desta forma, se desejarmos alterar o fluxo do programa, devemos alterar o conteúdo do PC.

O PC é um registro de 13 bits dividido em dois outros registros:

a) PCL é um registro de leitura e escrita usado para acesso aos 8 bits menos significativos do PC (bits 7:0).

b) PCH, registro para armazenar os 5 bits mais significativos do PC (bits 12:8).

Este registro não pode ser lido pelo usuário, mas pode ser alterado por intermédio de um outro registro PCLATH. Portanto, o registro PCLATH (bits 4:0) é um registro usado para acesso aos 5 bits mais significativos do PC.

Com qualquer reset, os bits mais significativos serão apagados (zerados). O PCLATH não é modificado por chamadas CALL, RETURN, etc. Nas instruções CALL e GOTO, estas só fornecem 11 bits do endereço de salto (2K) e o PCLATH fornece mais 2 bits (PCLATH <4:3>). Quando se realiza uma chamada a uma sub-rotina (CALL) ou um salto (GOTO), o usuário tem de proporcionar estes bits (PCLATH<4:3>).

Se for executado o retorno de uma instrução CALL ou interrupção, os 13 bits completos do PC estão na Pilha (Stack). Por isso, não é necessário acionar os bits PCLATH<4:3> para instruções de retorno, pois o endereço do retorno é fornecido pela Pilha.

c) A interação entre os registros PC, PCL, PCH e PCLATH pode ocorrer de quatro formas distintas:

I) Execução de uma instrução que tem como destino o registro PCL.

Neste caso, o registro PC será carregado, em sua parte baixa com o valor escrito no registro PCL e em sua parte alta (PCH) com os cinco bits menos significativos do registro PCLATH.

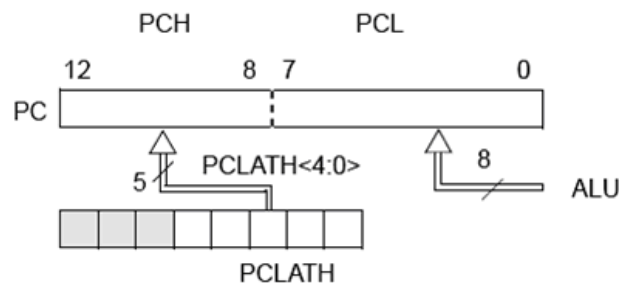


Figura 3.7: Instruções que tem como destino o registro PCL.

II) Execução de uma instrução GOTO.

Quando uma instrução GOTO (desvio incondicional) é executada, o conteúdo do PC é carregado com os 11 bits do endereço incluso no opcode da instrução. Estes 11 bits são carregados nos bits inferiores do PC. Os dois bits superiores do PC são carregados com os bits 4 e 3 do registro PCLATH.

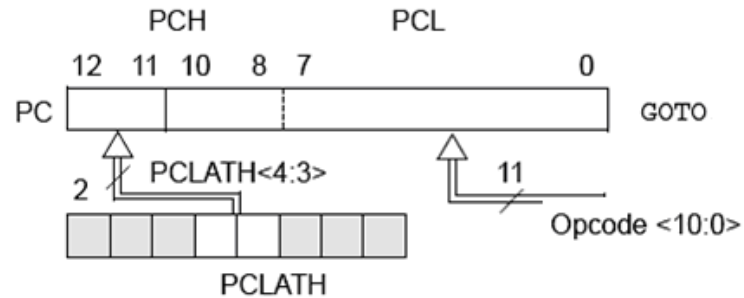


Figura 3.8: Instrução do tipo GOTO.

III) Execução de uma instrução CALL.

A execução de uma instrução CALL (chamada incondicional de sub-rotina) faz com que o conteúdo atual do PC (que está apontando para a próxima instrução a ser executada após a instrução CALL) seja armazenado na Pilha.

O PC é então, carregado com os 11 bits do endereço incluso no opcode da instrução. Estes 11 bits são carregados nos bits inferiores do PC.

Os dois bits superiores do PC são carregados com os bits 4 e 3 do registro PCLATH.

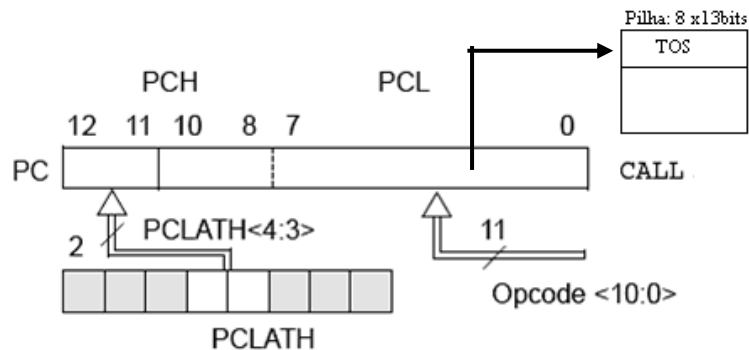


Figura 3.9: Instrução do tipo CALL.

Obs: As instruções GOTO e CALL permitem saltos dentro de 2048 posições de memória. Para saltos fora da página atual, será necessário programar o registro PCLATH com o valor correto. Nos PICs com menos de 2K de memória de programa não é necessário utilizar o registro PCLATH em qualquer desvio

IV) Execução das instruções RETURN, RETLW ou RETFIE.

Nestes casos, o PC é carregado com o valor que é puxado da Pilha. Este valor já é de 13 bits, preenchendo totalmente o PC. Portanto, nas instruções de retorno, não é necessário o uso do registro PCLATH em qualquer hipótese.

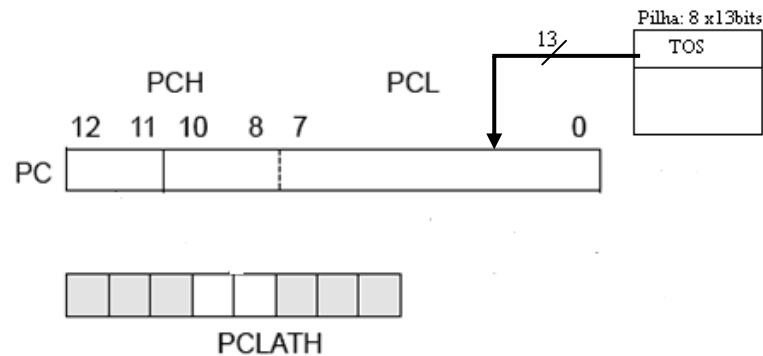


Figura 3.10: Carregando o PC nas instruções de retorno.

3.4.7 - Registros INDF (Indirect Addressing) e FSR (File Select Register – endereços 04h, 84h, 104h e 184h).

Registros usados para acesso indireto a uma posição da memória RAM de dados.

a) Modos de endereçamento:

- I) Direto, é feito por um endereço de nove bits sendo sete bits do endereço direto da instrução mais dois bits (RP1 e RP0) do registro STATUS. Qualquer acesso aos registros especiais (SFR), pode ser um exemplo de endereçamento direto. Veja **Figura 3.11**

Exemplo:

```
bsf STATUS,RP0 ;banco 1
movlw 0xFF      ;w = 0xFF
movwf TRISA     ;PORTA como entrada.
```

II) Indireto, não tira o endereço do código da instrução, mas sim com a ajuda do bit IRP (STATUS<7>) e do registro FSR. O local endereçado é acessado através do registro INDF e coincide com o endereço contido em FSR.

O Registro INDF não é um registro físico e ao ser acessado causará endereçamento indireto. Qualquer instrução usando o INDF como registro, na verdade acessa um dado cujo endereço está contido no registro FSR (File Select Register). Ler o registro INDF, quando o registro FSR = 0 resulta em 00h. Escrever no registro INDF, indiretamente resulta na instrução NOP (nenhuma operação), embora seja possível afetar bits de status.

FSR – usado para especificar o endereço de um registro a ser acessado indiretamente.

INDF – usado para escrita/leitura do conteúdo do endereço especificado pelo registro FSR.

No endereçamento indireto, os nove bits necessários são obtidos por oito bits do registro FSR e o bit IRP (STATUS<7>), como na **Figura 3.11**.

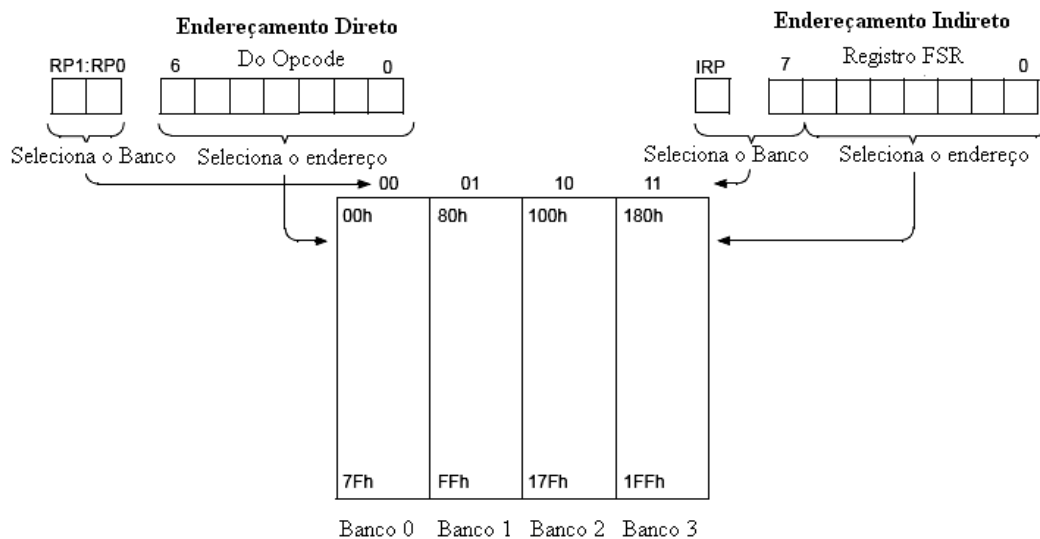


Figura 3.11: Endereçamento direto/indireto da memória RAM de dados.

b) Exemplos de endereçamento indireto.**Exemplo1:**

Registro 05 contém 10h
 Registro 06 contém 0Ah
 Carregue o valor 05 em FSR
 Lendo INDF temos 10h

 Incrementar FSR (06h)
 Lendo INDF temos 0Ah

Exemplo2: Limpa do endereço 20h a 2Fh da RAM

MOVLW 0x20
 MOVWF FSR
 NEXT CLR F INDF
 INC FSR,F ;limpa o valor que está
 ;no endereço 20h
 BTFSS FSR,4
 GOTO NEXT

CONTINUA?

3.4.8 - Registro INTCON (endereços 0Bh, 8Bh, 10Bh e 18Bh).

É um registro de leitura e escrita que possui bits de habilitações e bits de flags usados no controle das interrupções do micro. Basicamente se divide em:

a) Controle geral de interrupções;**b) Controles individuais de interrupções;****c) Flags sinalizadores da ocorrência da interrupção.**

É preciso zerar os bits flag antes de habilitar as interrupções para que possam ter efeito. Como exemplo, se o bit T0IF (transbordamento do Timer0) permanecer setado, nunca saberemos se voltou a transbordar.

GIE = apagado (0), não será atendida nenhuma interrupção, independentemente do modo em que se encontrem os diferentes bits de controle das interrupções individuais.

PEIE possui controle dos periféricos, pois quando em zero, não permite que os periféricos possam ocasionar uma interrupção. Há bits que habilitam ou não certas interrupções e bits que agem como bandeiras (flags) indicadoras de que um evento ocorreu em algum terminal ou módulo.

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-x
GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
bit 7							bit 0

bit 7 GIE: Global Interrupt Enable bit (Habilitação geral das interrupções – chave geral).

1 = Habilita todas as interrupções para controle individual.

0 = Desabilita todas as interrupções.

bit 6 PEIE: Peripheral Interrupt Enable bit (Habilitação das interrupções de periféricos – chave geral de periféricos).

1 = Habilita todas as interrupções de periféricos para controle individual.

0 = Desabilita todas as interrupções de periféricos.

bit 5 T0IE: TMR0 Overflow Interrupt Enable bit (Habilitação da interrupção de estouro de TMR0 – chave individual).

1 = Habilita a interrupção do TMR0.

0 = Desabilita a interrupção do TMR0.

bit 4 INTE: RB0/INT External Interrupt Enable bit (Habilitação de entrada externa de interrupção pino RB0/INT – chave individual).

1 = Habilita a interrupção externa no pino RB0/INT.

0 = Desabilita a interrupção externa.

bit 3 RBIE: RB Port Change Interrupt Enable bit (Habilitação da interrupção por mudança de estado nos pinos RB4 a RB7 – chave individual).

1 = Habilita a interrupção por mudança de estado

0 = Desabilita a interrupção por mudança de estado.

bit 2 T0IF: TMR0 Overflow Interrupt Flag bit (Identificação de estouro ou transbordamento do TMR0).

1 = Ocorreu transbordamento do TMR0 (este bit deve ser limpo por software).

0 = Não ocorreu transbordamento do TMR0.

bit 1 INTF: RB0/INT External Interrupt Flag bit (Flag de identificação de interrupção externa no pino RB0/INT).

1 = RB0/INT ocasionou uma interrupção (este bit deve ser limpo por software).

0 = RB0/INT não causou interrupção.

bit 0 RBIF: RB Port Change Interrupt Flag bit (Flag de identificação de interrupção por mudança de estado nos pinos RB4 a RB7).

1 = Algum dos bits RB7:RB4 mudou de estado (deve ser limpo por software).

0 = Nenhuma mudança de estado dos pinos RB7: RB4.

3.4.9 - Registro PIE1 (endereço 8Ch).

Registro de controle de interrupções de diferentes módulos e periféricos. As interrupções estão sempre condicionadas à forma em que se encontram os bits GIE e PEIE do registro INTCON (INTCON<7:6>). Também neste caso os conteúdos dos bits podem ser irrelevantes se habilitarmos a interrupção do módulo conversor A/D, bit ADIE (PIE1<6>) e tivermos apagado o bit ADON (ADCON0<0>) o que nunca poderá provocar uma interrupção. Assim, a atuação destes bits está condicionada ao fato de que, tanto as interrupções gerais como as de periféricos estejam habilitadas.

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
bit 7							bit 0

bit 7 PSPIE: Parallel Slave Port Read/Write Interrupt Enable bit (Bit de habilitação de interrupção leitura /escrita da porta paralela (chave individual)).

1 = Interrupção da porta paralela PSP habilitada (leitura/escrita)

0 = Desabilitada

bit 6 ADIE: A/D Converter Interrupt Enable bit (Bit de habilitação de interrupção do conversor A/D (chave individual)).

1 = Interrupção do conversor habilitada.

0 = Desabilitada.

bit 5 RCIE: USART Receive Interrupt Enable bit (Bit de habilitação de interrupção de recepção do USART (chave individual)).

1 = Interrupção de recepção da USART habilitada.

0 = Desabilitada.

bit 4 TXIE: USART Transmit Interrupt Enable bit (Bit de habilitação de interrupção de transmissão do USART (chave individual)).

1 = Interrupção de transmissão da USART habilitada.

0 = Desabilitada.

bit 3 SSPIE: Synchronous Serial Port Interrupt Enable bit (Bit de habilitação de interrupção da Porta Série Síncrona - SSP (chave individual)).

1 = Interrupção da porta serial SSP habilitada.

0 = Desabilitada.

bit 2 CCP1IE: CCP1 Interrupt Enable bit (Bit de habilitação de interrupção do módulo CCP1 (chave individual)).

1 = Interrupção de CCP1 habilitada.

0 = Desabilitada.

bit 1 TMR2IE: TMR2 to PR2 Match Interrupt Enable bit (Bit de habilitação da interrupção do TMR2/PR2 (chave individual)).

1 = Interrupção do TMR2/PR2 habilitada.

0 = Desabilitada.

bit 0 TMR1IE: TMR1 Overflow Interrupt Enable bit (Bit de habilitação de interrupção de estouro do TMR1 (chave individual)).

1 = Interrupção do Timer1 habilitada.

0 = Desabilitada.

3.4.10 - Registro PIR1 (endereço 0Ch).

O registro PIR1 contém flags para identificações individuais de interrupções de periféricos.

Por meio dos bits dos registros anteriores habilitávamos as interrupções de alguns dos periféricos, e com os bits deste registro saberemos qual periférico ocasionou a interrupção. Todos os bits são flags de controle de estado de certos periféricos nos quais, alguns deles têm dupla funcionalidade, conforme estejam configuradas as portas. Como exemplo, temos o bit SSPIF que, conforme esteja configurada a porta série, bem como SSP (Porta Serial Síncrona) ou bem como porta I²C, tem diferentes significados. É preciso apagar os bits flag antes de habilitar as interrupções para que possam ter efeito.

Com o bit GIE (INTCON<7>) apagado (0), não será atendida nenhuma interrupção, independentemente do modo em que estejam os diferentes bits de controle das interrupções individuais.

R/W-0	R/W-0	R-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0
PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
bit 7							bit 0

bit 7 PSPIF: Parallel Slave Port Read/Write Interrupt Flag bit (Flag de identificação de interrupção leitura/escrita da porta paralela, chave individual).

1 = Leitura/escrita na porta paralela foi completada (bit de ser apagado pelo software).

0 = Não ocorreu.

bit 6 ADIF: A/D Converter Interrupt Flag bit (Flag de identificação da interrupção do conversor A/D, chave individual).

1 = Conversão A/D foi completada.

0 = Conversão A/D não foi completada.

bit 5 RCIF: USART Receive Interrupt Flag bit (Flag de identificação da interrupção de recepção da USART, chave individual).

1 = Buffer de recepção da USART está cheio.

0 = Buffer de recepção da USART está vazio.

bit 4 TXIF: USART Transmit Interrupt Flag bit (Flag de identificação da interrupção de transmissão da USART, chave individual).

1 = Buffer de transmissão da USART está vazio (deve ser apagado pelo software).

0 = Buffer de transmissão da USART está cheio.

bit 3 SSPIF: Synchronous Serial Port (SSP) Interrupt Flag (Flag de identificação da interrupção da porta serial síncrona – SSP, chave individual).

1 = O SSP ocasionou uma interrupção, e deve ser apagado por software antes de chamar a subrotina do serviço de interrupções. Este bit será setado em alguma destas condições:

- **SPI**

- A transmissão/recepção foi completada.

- **I2C Slave**

- A transmissão/recepção foi completada.

- **I2C Master**

- A transmissão/recepção foi completada.

- A condição de START foi completada pelo modulo SSP.

- A condição de STOP foi completada pelo modulo SSP.

- A condição Restart foi completada pelo modulo SSP.

- A condição de Reconhecimento (ACK) foi completada pelo módulo SSP.

- Ocorreu uma condição de START enquanto o módulo SSP estava inativo (Sistema Multi-Master).

- Ocorreu uma condição de STOP enquanto o módulo SSP estava inativo (Sistema Multi-Master).

0 = Não ocorreu condição de interrupção no módulo SSP.

bit 2 CCP1IF: CCP1 Interrupt Flag bit (Flag de identificação da interrupção do módulo CCP1, chave individual).

Modo captura:

1 = Ocorreu uma captura no registro TMR1 (deve ser apagado pelo software)

0 = Não ocorreu uma captura.

Modo comparador:

1 = Comparação em registro TMR1 igual (deve ser apagado por pelo software)

0 = Comparação desigual.

Modo PWM: Não usado.

bit 1 TMR2IF: TMR2 to PR2 Match Interrupt Flag bit (Flag de identificação da interrupção de TMR2/PR2 , have individual).

1 = Ocorreu evento da interrupção do Timer2, chave individual (este bit deve ser apagado por software).

0 = Não ocorreu.

bit 0 TMR1IF: TMR1 Overflow Interrupt Flag bit (Flag de identificação da interrupção de estouro do TMR1, chave individual).

1 = Ocorreu estouro no registro TMR1 (deve ser apagado por software).

0 = Não ocorreu estouro.

3.4.11 - Registro PIE2 (endereço 8Dh).

Este registro contém bits de habilitações individuais para a interrupção do periférico CCP2, interrupção por colisão no barramento e interrupção de operação de escrita na EEPROM.

Os bits deste registro habilitam as interrupções de diferentes módulos e periféricos, embora estejam sempre condicionadas à forma em que se encontram os bits GIE e PEIE (INTCON<7:6>). Também neste caso os conteúdos dos bits podem ser irrelevantes se habilitarmos a interrupção do módulo conversor A/D, bit ADIE (PIE1<6>) e tivermos apagado o bit ADON (ADCON0<0>) o que nunca poderá provocar uma interrupção. Assim, a atuação destes bits está condicionada ao fato de que, tanto as interrupções gerais como as de periféricos estejam habilitadas.

U-0	R/W-0	U-0	R/W-0	R/W-0	U-0	U-0	R/W-0
—	Reserved	—	EEIE	BCLIE	—	—	CCP2IE
bit 7							bit 0

bit 7 Sem implementar: Em leitura é zero ('0')

bit 6 Reservada: Manter sempre apagado (0).

bit 5 Sem implementar: Lido como '0'

bit 4 EEIE: EEPROM Write Operation Interrupt Enable (Habilitação da interrupção de fim de escrita na EEPROM, chave individual.

1 = Interrupção de fim de escrita na EEPROM habilitada.

0 = Desabilitada

bit 3 BCLIE: Bus Collision Interrupt Enable (Habilitação da interrupção por colisão no barramento I²C, chave individual.

1 = Interrupção de colisão no barramento I²C habilitada.

0 = Desabilitada.

bit 2-1 Sem implementar: Em leitura é zero (0)

bit 0 CCP2IE: CCP2 Interrupt Enable bit (Bit de habilitação da interrupção do módulo CCP2, chave individual.

1 = Interrupção do CCP2 habilitada.

0 = Desabilitada.

3.4.12 - Registro PIR2 (endereço 0Dh).

Este registro possui flags para identificar interrupções do modulo CCP2; interrupção de colisão no barramento SSP e interrupção da operação de escrita em EEPROM.

Por meio dos bits dos registros anteriores habilitávamos as interrupções de alguns dos periféricos. E com os bits deste registro saberemos que periférico ocasionou a interrupção. Todos os bits são flags de controle de estado de certos periféricos nos quais, alguns deles têm dupla funcionalidade, conforme estejam configuradas as portas. Como exemplo, temos o bit SSPIF do registro PIR1 que, conforme esteja configurada a porta série, bem como SSP (porta serial síncrona) ou bem como porta I²C, tem diferentes significados.

É preciso apagar (zerar) os bits flag antes de habilitar as interrupções para que possam ter efeito.

Com o bit GIE (INTCON<7>) apagado (0), não será atendida nenhuma interrupção, independentemente do modo em que estejam os diferentes bits de controle das interrupções individuais.

U-0	R/W-0	U-0	R/W-0	R/W-0	U-0	U-0	R/W-0
—	Reserved	—	EEIF	BCLIF	—	—	CCP2IF
bit 7							bit 0

bit 7 Sem implementar: Em leitura é zero (0)

bit 6 Reservado: Manter sempre apagado (0).

bit 5 Sem implementar: Em leitura é zero (0)

bit 4 EEIF: EEPROM Write Operation Interrupt Flag bit (Flag de identificação da interrupção de fim de escrita na EEPROM, chave individual).

1 = Operação de escrita na EEPROM foi completada (bit deve ser apagado por software).

0 = Operação de escrita na EEPROM não foi completada e nova escrita não pode ser iniciada.

bit 3 BCLIF: Bus Collision Interrupt Flag bit (Flag de identificação da interrupção de colisão no barramento I²C, chave individual).

1 = Ocorreu colisão no barramento SSP quando configurado I²C Master mode

0 = Não ocorreu.

bit 2-1 Sem implementar: Em leitura é zero (0)

bit 0 CCP2IF: CCP2 Interrupt Flag bit (Flag de identificação da interrupção do módulo CCP2, chave individual).

Modo captura:

1 = O registro TMR1 fez uma captura (bit deve ser apagado por software)

0 = Não houve captura.

Modo comparador:

1 = O registro TMR1 foi comparado e é igual (bit deve ser apagado por software)

0 = A comparação não é igual.

Modo PWM: Não usado.

CAPÍTULO 4

PORTAS DE ENTRADA E SAÍDA.

4.1 – INTRODUÇÃO.

Alguns pinos das portas de entrada/saída são multiplexados com funções alternadas para características de periféricos no dispositivo. Em geral, quando um periférico está habilitado, este pino pode não ser usado como um pino de entrada/saída de uso geral.

a) O PIC16F877A possui suas portas divididas da seguinte forma:

- I) Uma (1) porta de 6 I/O (PORTA).**
- II) Três (3) portas de 8 I/O (PORTB, PORTC E PORTD)**
- III) Uma (1) porta de 3 I/O (PORTE).**

4.2 - PORTA (endereço 05H) e Registro TRISA (endereço 85H).

A porta A (PORTA) é uma porta bidirecional de 6 bits. O registro de direção de dados correspondente é o TRISA. Um bit do registro TRISA setado, fará com que o pino correspondente da porta A seja uma entrada (o driver correspondente da saída fica no estado de alta Z). Um bit do registro TRISA apagado (0), fará com que o pino correspondente da porta A seja uma saída (coloca o conteúdo do latch de saída no pino selecionado).

Lendo-se o registro PORTA, temos o estado dos pinos, visto que escrevendo-se na porta A será escrito no latch da mesma. Todas as operações de escrita são do tipo leitura-modificação-escrita. Portanto, a escrita na porta implica que os pinos são lidos, modificados e escritos novamente no latch de dados da porta.

A leitura é feita diretamente sobre o pino, enquanto a escrita passa por um Latch. Isto pode ocasionar um atraso entre o comando de escrita e a efetiva alteração da saída. É por este motivo que não é recomendável efetuar uma operação de leitura imediatamente após a alteração de uma saída. O certo é aguardar pelo menos um ciclo de máquina entre as duas operações. O latch possibilita também que uma operação de escrita seja executada mesmo que o pino esteja configurado como entrada, através do registro TRISA. O latch é alterado e, quando o estado do pino for modificado para saída, o valor atual do latch lhe será imposto. A configuração da porta como saída ou entrada é feita

pelo registro TRISA, enquanto a operação de escrita ou leitura é executada pelo registro PORTA.

Como visto no bloco da porta A, cinco pinos são do tipo TTL enquanto um é do tipo Schmitt Trigger. Quando estamos operando com um pino como entrada isto interfere diretamente no nível de tensão interpretado pelo PIC.

Os pinos RA0 a RA3 e RA5 se ligam também ao Conversor A/D. “Analog Input Mode” representa a seleção entre modo digital ou analógico para esses pinos (Registro ADCON1). Estes pinos possuem níveis TTL de entrada e drivers de saída CMOS.

O pino RA4 é multiplexado com a entrada do clock do módulo Timer0 tornando-se o pino RA4/T0CKI. Isto possibilita que um clock imposto a esse pino incremente automaticamente o TMR0, independentemente do clock real da máquina. Outra característica desta saída é não possuir o FET canal P, capaz de impor Vdd à saída, ficando uma saída dreno aberto. Para liberar Vdd em RA4 é necessário um resistor de pull-up externo.

O pino RA4/T0CKI é uma entrada Schmitt Trigger e uma saída dreno aberto.

Outros pinos da porta A são multiplexados com entradas analógicas e entrada de referência (Vref) analógica. A operação de cada pino é selecionada pelos bits de controle no registro ADCON1 (A/D Control Register1).

Obs: Na energização (Power-on Reset), esses pinos são configurados como entradas analógicas e em leitura são zero (0).

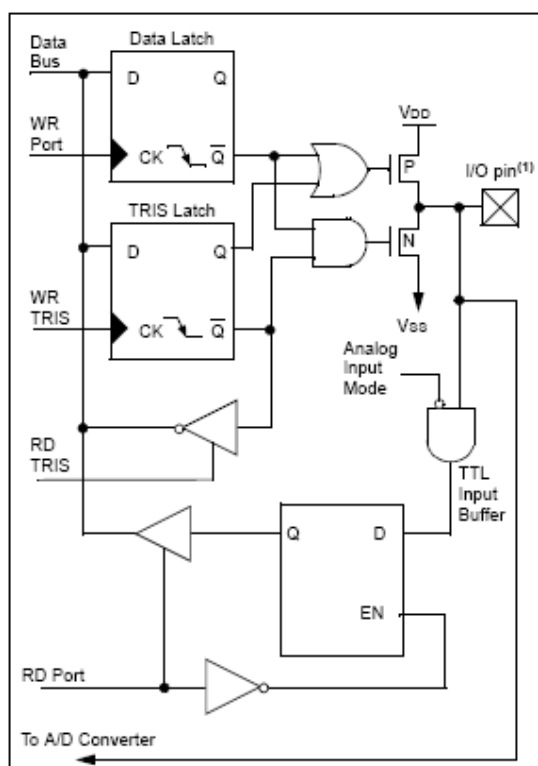
O Registro TRISA controla a direção dos pinos RA, mesmo que eles estejam sendo usados como entradas analógicas.

O usuário precisa garantir que os bits do registro TRISA sejam mantidos setados quando usados como entradas analógicas.

Exemplo de inicialização da PORTA A.

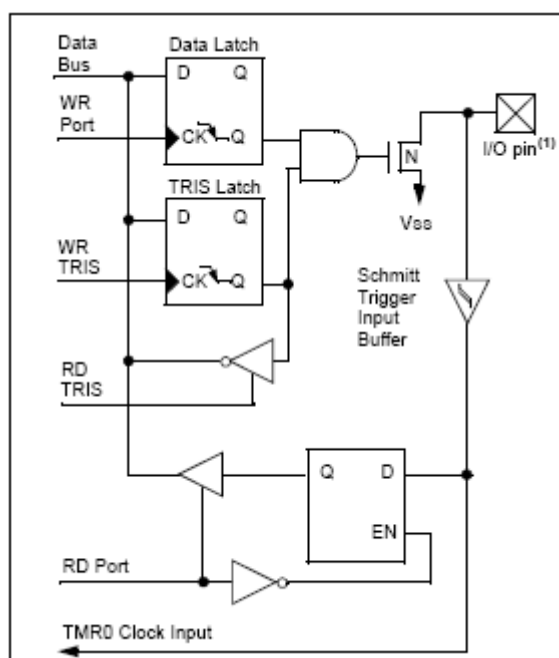
bcf	STATUS, RP0	
bcf	STATUS, RP1	;banco0
clrf	PORTA	;inicialização da porta A pela limpeza dos Latches de saída
bsf	STATUS, RP0	;seleciona o Banco1
movlw	0x06	;configura todos os pinos como entradas
movwf	ADCON1	;digitais
movlw	0xCF	;valor usado para inicializar a direção de dados
movwf	TRISA	;coloca RA0 a RA3 como entradas e RA4 e RA5 como
		;saídas, TRISA 6 e 7 são sempre lidos como zero (0).

Diagrama de bloco dos pinos RA0 a RA3 e RA5.



Pinos E/S são protegidos por diodos para Vdd e Vss.

Digrama de bloco do pino RA4/T0CKI.



Pinos E/S são protegidos por diodos para Vss.

Figura 4.1: Diagrama de blocos da PORTA A.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
05h	PORTA	—	—	RA5	RA4	RA3	RA2	RA1	RA0	--0x 0000	--0u 0000
85h	TRISA	—	—	PORTA Data Direction Register						--11 1111	--11 1111
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Option_Reg bits T0CS e T0SE também são usados.

Figura 4.2: Registros associados com PORTA.

Obs: Quando usando o modulo SSP no modo SPI Escravo e pino /SS habilitado, o conversor A/D deve ser setado para um dos seguintes modos, onde PCFG3:PCFG0 = 0100, 0101, 011x, 1101, 110, 1111.

4.3 - PORTA B (endereços 06h e 106h) e Registro TRISB (endereços 86h e 186h).

A porta B (PORTB) é bidirecional de 8 bits. O registro de direção de dados correspondente é o TRISB. Um bit do registro TRISB setado, fará com que o pino correspondente da porta B seja uma entrada (o driver correspondente da saída fica no estado de alta Z). Um bit do registro TRISB apagado (0), fará com que o pino correspondente da porta B seja uma saída (coloca o conteúdo do latch de saída no pino selecionado).

Três dos pinos da porta B são multiplexados com a função de Programação em Baixa Tensão (Low Voltage Programming function), são eles: RB3/PGM, RB6/PGC e RB7/PGD. As funções alternadas desses pinos são descritas mais adiante.

Uma característica elétrica da porta B é ligação de pull-up interno (FET). Esta ligação para Vdd pode ser feita ou não em todos os pinos ao mesmo tempo através do bit RBPU (OPTION_REG<7>). O pull-up é automaticamente desligado quando um pino da porta é configurado como saída e durante a energização (Power-on Reset).

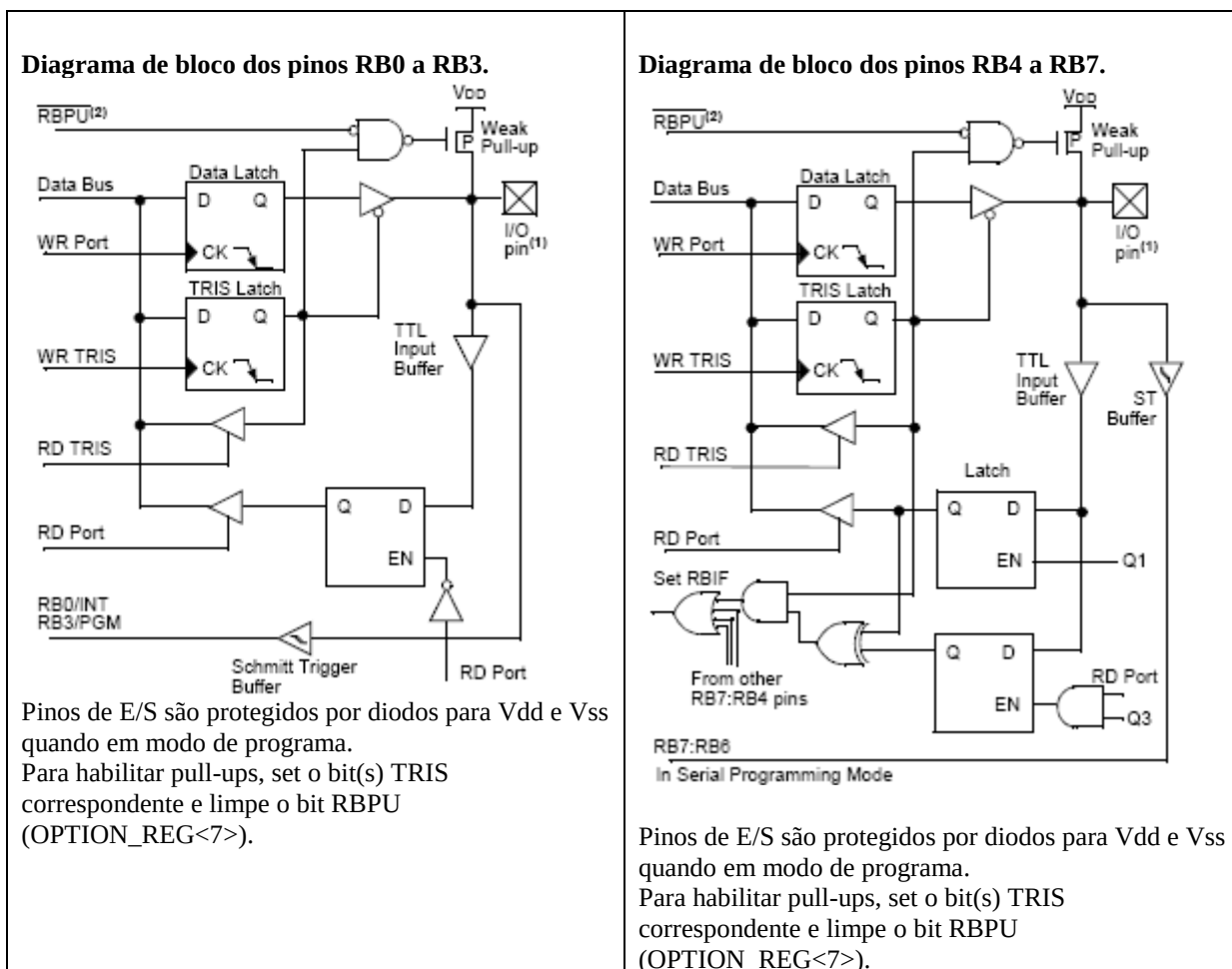


Figura 4.3: Diagrama de blocos da PORTA B.

Toda a porta B é entrada TTL, exceto o pino RB0 quando configurado para utilizar a interrupção externa e os pinos RB3, RB6 e RB7 quando em modo de programação série onde são do tipo ST.

Os pinos RB4 a RB7 são interligados à interrupção de mudança de estado quando configurados como entradas e o pino RB0 é associado à interrupção externa, desde que configurado como entrada. Habilitações e identificações encontram-se no registro INTCON e a configuração da borda que gera a interrupção externa é feita pelo bit INTEDG (OPTION_REG<6>).

Os pinos de entrada (RB7:RB4) são comparados com os valores antigos armazenados na última leitura da porta B. As saídas diferentes passam por uma operação “OU” para

gerar a interrupção por mudança na porta B (RB Port Change Interrupt) mudando a interrupção com o bit de flag RBIF (INTCON<0>).

Esta interrupção pode despertar o MCU do estado de SLEEP. O usuário, no Serviço da Rotina de Interrupção (Interrupt Service Routine) pode desabilitar a interrupção das seguintes maneiras:

a) Qualquer leitura ou escrita na porta B. Isto acabará com qualquer condição de diferença.

b) Zerar o bit de flag RBIF.

A característica interrupção mudança de estado junto com os Pull-ups configuráveis por software em quatro desses pinos, permite fácil interligação com teclado e torna possível o despertar pelo pressionamento da tecla.

RB0/INT é um pino de entrada de interrupção externa e é configurado usando o bit INTEDG (OPTION_REG<6>).

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
06h, 106h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	xxxxx xxxxx	uuuuu uuuuu
86h, 186h	TRISB	PORTB Data Direction Register								1111 1111	1111 1111
81h, 181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
0Bh...	INTCON	GIE				INTE	RBIE	INTF	RBIF		

Figura 4.4: Registros associados com a PORTA B.

;Lendo a porta B:

```
movf    PORTB,W    ;escreve o valor da PortaB em W
movwf   DADO        ;registro DADO recebe o registro PORTB
```

;Inicializando as variáveis:

```
clrf    PORTA       ;zero em todas as saídas da PortaA
clrf    PORTB       ;zero em todas as saídas da portaB
movlw   .10
movwf   Contador    ;inicia contador com 10 decimal
```

;Escrevendo na porta

```
Bank1                ;muda para o banco 1
bsf      TRISB,0      ;transforma RB0 em entrada
Bank0                ;retorna ao banco0
.
.                    ;vários comandos
.
bsf      PORTB,0      ;escreve '1' no latch do RB0 (ainda é entrada)
Bank1                ;muda para banco1
bcf      TRISB,0      ;transforma RB0 em saída. Neste momento o pino será inicializado em
                    ;'1' devido ao latch
Bank0                ;retorna ao banco0
```

Obs: Não se deve escrever na porta em um ciclo e efetuar a leitura no ciclo seguinte, deve-se ter pelo menos 1 ciclo de máquina entre escrita e leitura. Neste caso, usa-se a instrução NOP que demora 1 ciclo.

```
movlw   .10          ;escreve 10 em W
movwf   PORTB        ;transfere 10 para a portaB
NOP                        ;demora 1 ciclo para assegurar a escrita na portaB
movf    PORTB, W     ;lê o valor da portaB e coloca em W
movwf   DADO         ;transfere o valor de W para o registro DADO
```

4.4 - PORTA C (endereço 07h) e Registro TRISC (endereço 87h).

A porta C (PORTC) é bidirecional de 8 bits. O registro de direção de dados correspondente é o TRISC. Um bit do registro TRISC setado, fará com que o pino correspondente da porta C seja uma entrada (o driver correspondente da saída fica no estado de alta Z). Um bit do registro TRISC apagado (0), fará com que o pino correspondente da porta C seja uma saída (coloca o conteúdo do latch de saída no pino selecionado).

A porta C é multiplexada com funções de vários periféricos e seus pinos possuem buffers de entrada Schmit Trigger.

Quando o módulo I²C é habilitado, os pinos RC3 e RC4 podem ser configurados com níveis I²C normais ou com níveis SMBus (System Management Bus) pelo uso do bit CKE (SSPSTAT<6>).

Quando funções periféricas são habilitadas, cuidados devem ser tomados em definir os bits TRISC para cada pino da porta C. Alguns periféricos alteram (override) o bit TRISC para fazer do pino uma saída, enquanto outros alteram (override) o bit TRISC para fazer do pino uma entrada. Uma vez que a alteração do bit TRISC esteja em efeito enquanto o periférico está habilitado, instruções de leitura/modificação/escrita (BSF, BCF, XORWF) com o registro TRISC como destino, devem ser evitados.

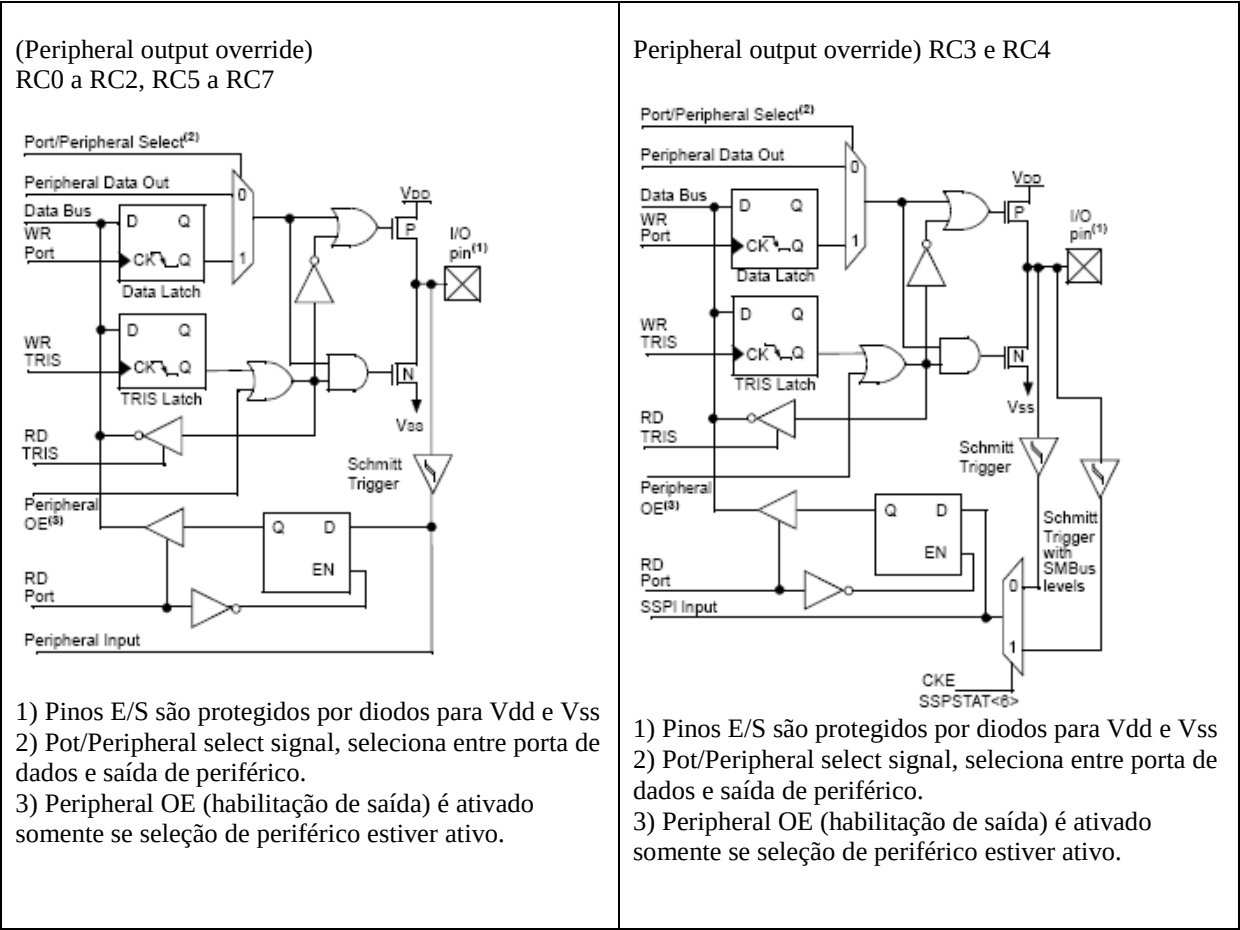


Figura 4.5: Diagrama de bloco da PORTA C

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
07h	PORTC	RC7	RC6	RC5	RC4	RC3	RC2	RC1	RC0	xxxx xxxx	uuuu uuuu
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111

Figura 4.6: Registros associados com PORTA C

4.5 - PORTA D (endereço 08h) e Registro TRISD (endereço 88h).

A porta D (PORTD) é uma porta de 8 bits com buffers de entrada Schmitt Trigger. Cada pino é configurado individualmente como entrada ou saída.

Esta porta pode ser configurada como uma Porta Paralela Escrava de 8 bits (PSP) do microprocessador, pela ativação do bit de controle PSPMODE (TRISE<4>). Neste modo, os buffers de entrada são TTL.

No modo escravo é assíncrona e pode ser lida com um sinal externo aplicado à entrada /RD (pino RE0//RD), ou escrita com o sinal de controle aplicado à entrada /WR (pino RE1//WR).

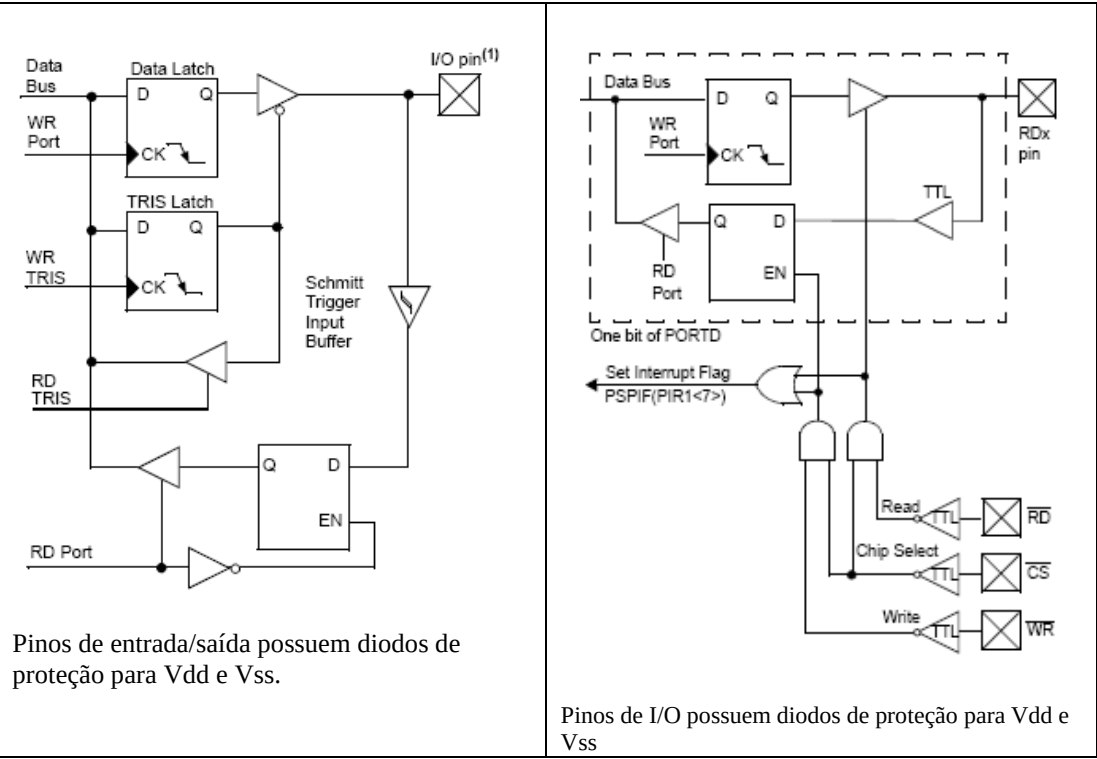


Figura 4.7: Diagrama de bloco da porta D modo E/S e Porta paralela escrava

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
08h	PORTD	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0	xxxx xxxx	uuuu uuuu
88h	TRISD	PORTD Data Direction Register								1111 1111	1111 1111
89h	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111

Figura 4.8: Registros associados com a Porta D.

4.6 - PORTA E (endereço 09h) e Registro TRISE (endereço 89h)

A porta E (PORTE) possui três pinos (RE0/RD/AN5, RE1/WR/AN6 e RE2/CS/AN7) os quais são individualmente configuráveis como entradas ou saídas. Estes pinos possuem buffers de entrada com Schmitt Trigger.

Seus pinos tornam-se entradas de controle de entradas/saídas para a porta do micro quando o bit PSPMODE (TRISE<4>) estiver setado. Neste modo, o usuário precisa ter certeza que os bits (TRISE<2:0>) estejam setados e que os pinos estejam configurados como entradas digitais. Deve garantir também que o registro ADCON1 esteja configurado para entradas/saídas digitais. Neste modo, os buffers de entradas são TTL.

Na **Figura 4.9** temos o registro TRISE que também controla a operação da Porta Paralela Escrava (PSP).

Os pinos da porta E são multiplexados com as entradas analógicas. Quando selecionadas para entradas analógicas, estes pinos são lidos como zeros (0). O registro TRISE controla a direção dos pinos da porta E, mesmo quando estão sendo usados como entradas analógicas. O usuário precisa manter os pinos configurados como entradas quando os mesmos estiverem sendo usados como entradas analógicas.

Obs: Na energização (Power-on Reset), estes pinos são configurados como entradas analógicas e lidos como zero (0).

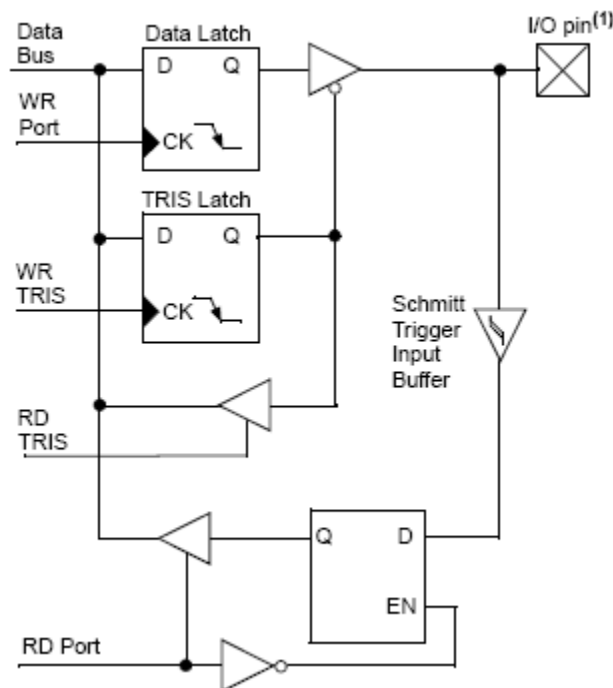


Figura 4.9: Digrama de blocos da PORTA E

4.6.1 - Registro TRISE endereço 89h.

R-0	R-0	R/W-0	R/W-0	U-0	R/W-1	R/W-1	R/W-1
IBF	OBF	IBOV	PSPMODE	—	Bit2	Bit1	Bit0
bit 7							bit 0

Status da Porta Paralela Escrava/Bits de Controle:

bit 7 IBF: Input Buffer Full Status bit (Estado do buffer de entrada da porta paralela – PSP).

1 = Uma palavra foi recebida e está esperando para ser lida pela CPU

0 = Nenhuma palavra foi recebida, o buffer de entrada está vazio.

bit 6 OBF: Output Buffer Full Status bit (Estado do buffer de saída da porta paralela - PSP).

1 = Existe uma palavra não lida no buffer de saída.

0 = O buffer de saída foi lido.

bit 5 IBOV: Input Buffer Overflow Detect bit (Indicação de estouro no buffer de entrada)

1 = Uma nova palavra chegou sem que a anterior tivesse sido lida do buffer (este bit deve ser limpo por software).

0 = Não houve estouro.

bit 4 PSPMODE: Parallel Slave Port Mode Select bit (Modo de operação dos pinos da porta paralela - PSP)

1 = Porta D funcionando no modo PSP, pinos controlados internamente pelo sistema.

0 = Porta D funcionando no modo I/O convencional.

bit 3 Sem implementar: Lido como zero (0).

Bits de direção de dados da Porta D:

bit 2 RE2: Bit Controle de Direção para o pino RE2//CS/AN7

1 = Entrada

0 = Saída

bit 1 RE1: Bit Controle de Direção para o pino RE1//WR/AN6.

1 = Entrada

0 = Saída

bit 0 RE0: Bit Controle de Direção para o pino RE0//RD/AN5.

1 = Entrada

0 = Saída

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
09h	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu
89h	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Figura 4.10: Registros associados com a PORTA E.

4.7 - PORTA PARALELA ESCRAVA (PSP).

A porta D (PORTD) pode ser configurada como Porta de Propósito Geral ou como Porta Paralela Escrava (Parallel Slave Port - PSP) de 8 bits, pela ativação do bit de controle PSPMODE (TRISE<4>). No modo PSP, os buffers de entrada são TTL.

Esta porta só trabalha no modo Escravo. Neste modo, é assíncrona e pode ser lida com um sinal de controle externo aplicado à entrada /RD pino RE0//RD ou escrita com o sinal de controle aplicado à entrada /WR pino RE1//WR.

Ela também pode ser usada como interface para conexão com outros microcontroladores de 8 bits. O microprocessador externo pode ler ou escrever nos latch da porta D como um latch de 8 bits.

Quando o bit PSPMODE e os bits de direção de dados do registro TRISE<2:0> estão setados, o pino RE0//RD converte-se em entrada de controle de leitura, o pino RE1//WR fica como entrada de controle de escrita e o pino RE2//CS fica como entrada de seleção (Chip select). Além disso, os bits de configuração da porta A/D, PCFG3:PCFG0 (ADCON1<3:0>), devem ser configuradas de modo que os pinos RE2:RE0 sejam entradas/saídas digitais.

Existem dois latches de 8 bits, um para dados de saída e outro para dados de entrada. O usuário escreve dados de 8 bits no latch de dados da porta D e lê dados vindos do pino do latch da porta (observe que eles possuem o mesmo endereço). Neste modo, o registro do TRISD é ignorado, visto que dispositivo externo está controlando a direção do fluxo de dados.

Escrita do exterior do micro na porta PSP, só é possível quando as linhas /CS e /WR são detectadas em nível baixo e a leitura ocorre quando as linhas /CS e /RD também estão em nível baixo.

Após uma escrita, as linhas /CS ou /WR assumem nível alto (nível de disparo), o flag bit de status IBF (TRISE<7>) e o flag de interrupção, bit PSPIF (PIR1<7>) são setados somente após completado o ciclo Q4 do clock. IBF somente pode ser limpo (zerado) pela leitura do latch de entrada da porta D.

O flag de status Input Buffer Overflow (IBOV) bit (TRISE<5>) é setado se uma segunda escrita para a porta PSP for tentada quando o byte anterior não tenha sido lido no buffer de saída (**Figure 4.12**).

Leitura na porta PSP, ocorre quando as linhas /CS e /RD forem primeiramente detectadas em nível baixo. Neste caso, o flag de status Output Buffer Full (OBF) bit (TRISE<6>) é limpo (zerado) imediatamente (**Figure 4.13**), indicando que o latch da porta D está esperando para ser lido pelo barramento externo. Quando os pinos /CS ou /RD assumem nível alto, o flag de interrupção bit PSPIF (PIR1<7>) será setado somente ao completar o ciclo Q4 do clock. OBF permanece em nível baixo até que dados sejam escritos para a porta D pelo firmware proprietário.

Quando não estiver no modo PSP, os bits IBF e OBF são mantidos apagados (0). Entretanto, se o bit de flag IBOV foi previamente setado, ele deve ser limpo no firmware. Uma interrupção é gerada e armazenada no flag, bit PSPIF (PIR1<7>) quando uma operação de leitura ou escrita for completada. PSPIF deve ser limpo (zerado) pelo usuário no firmware e a interrupção pode ser desabilitada pelo bit PSPIE (PIE1<7>).

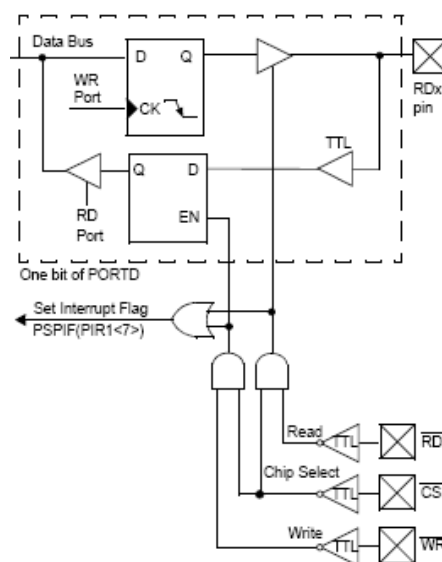


Figura 4.11: Diagrama de blocos da Porta D como Porta Paralela Escrava.

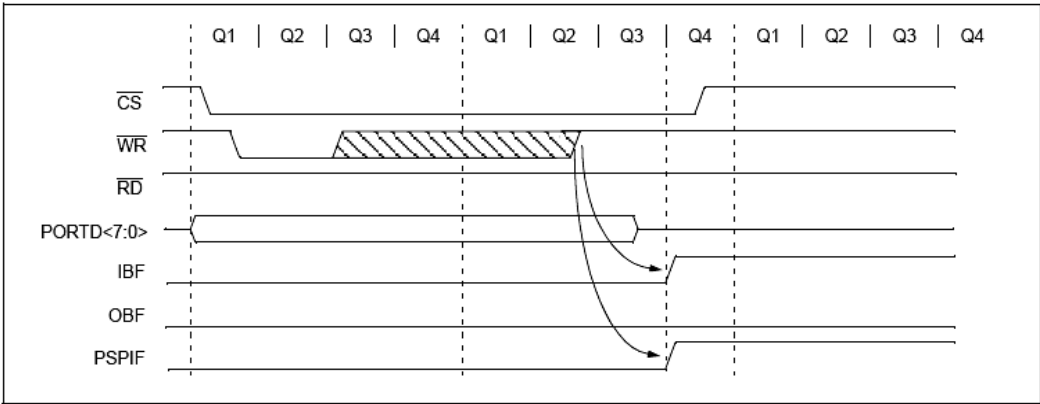


Figura 4.12: Formas de ondas da Porta Paralela Escrava na escrita

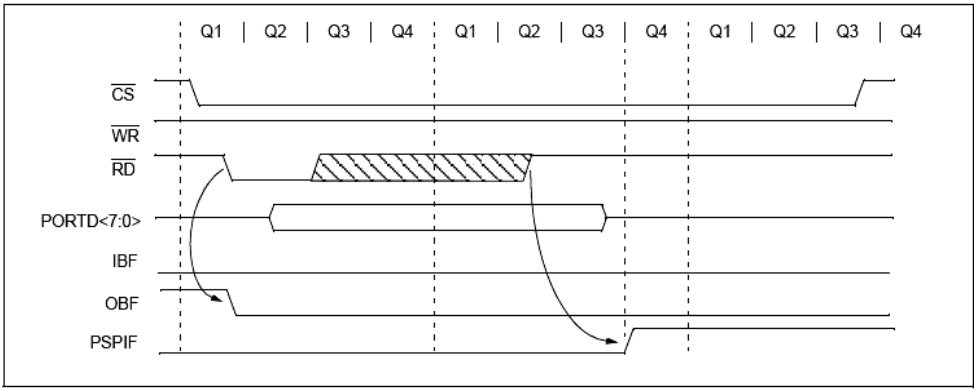


Figura 4.13: Formas de ondas da Porta Paralela Escrava na leitura

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
08h	PORTD	Port Data Latch when written: Port pins when read								xxxx xxxx	uuuu uuuu
09h	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu
89h	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Figura 4.14: Registros associados com a Porta Paralela Escrava

CAPÍTULO 5

MEMÓRIAS DE DADOS (EEPROM) E DE PROGRAMA (FLASH).

5.1 – INTRODUÇÃO.

Acesso para leitura e escrita em ambas as memórias depende de alguns Registros de Funções Especiais (SFR). Os seis SFRs usados são:

- a) EEDATA (endereço 10Ch).
- b) EEDATH (endereço 10Eh).
- c) EEADR (endereço 10Dh).
- d) EEADRH (endereço 10Fh).
- e) EECON1 (endereço 18Ch).
- f) EECON2 (endereço 18Dh).

O registro EECON2 não é um registro físico pelo que não é possível lê-lo. É usado exclusivamente nas sequências de controle de escrita das memórias EEPROM e Flash.

O registro EECON1 possui a tarefa de controlar o acesso às operações de leitura e escrita nas memórias EEPROM e Flash.

A escrita e a leitura em EEPROM não é tão simples como em RAM, por isso, usamos o registro EECON1 com seus cinco bits implementados, como visto no item 5.1.1.

- a) **EEPGD<7>** seleciona entre memória Flash de programa e EEPROM de dados.
- b) **WRERR<3> setado**, indica que a operação terminou prematuramente (um reset MCLR ou houve transbordamento do Watchdog).
- c) **WREN<2> setado**, habilita a escrita na memória, e com /WR setado é iniciado o ciclo de escrita.
- d) **Para a leitura, o bit RD será setado.** Para a escrita o bit WR será setado. Os bits WR e RD só podem ser setados e não podem ser apagados, pois é o próprio sistema interno que se encarrega de apagá-los quando finalizar o ciclo de escrita ou leitura.



u Flash.

5.1.1 - Registro EECON1 (endereço 18Ch).

R/W-x	U-0	U-0	U-0	R/W-x	R/W-0	R/S-0	R/S-0
EEPGD	—	—	—	WRERR	WREN	WR	RD
bit 7							bit 0

bit 7 EEGD: Program/Data EEPROM Select bit (Seleção de acesso, Flash/EEPROM)

1 = Seleciona memória de programa.

0 = Seleciona memória de dados.

Este bit não pode ser mudado enquanto uma operação de leitura ou escrita estiver em progresso

bit 6-4 Sem implementar: Em leitura é zero (0).

bit 3 WRERR: EEPROM Error Flag bit (Flag de indicação de erro durante a escrita na EEPROM)

1 = Ocorreu um erro por uma operação de escrita não completada (pode ter ocorrido um reset /MCLR ou um reset WDT durante operação normal)

0 = Não ocorreu erro, a escrita foi completada.

bit 2 WREN: EEPROM Write Enable bit (Habilitação de escrita EEPROM/Flash (bit de segurança))

1 = Habilita a escrita na EEPROM/Flash

0 = Desabilita a escrita na EEPROM/Flash

bit 1 WR: Write Control bit (Controle de início de escrita na EEPROM/Flash)

1 = Inicia o ciclo de escrita (É apagado pelo hardware quando finaliza a escrita e setado somente por software).

0 = Ciclo de escrita terminou (zerado somente pelo hardware).

bit 0 RD: Read Control bit (Controle de leitura da EEPROM/Flash).

1 = Inicia o ciclo de leitura. (É apagado por hardware quando finaliza a leitura e setado somente por software).

0 = Zerado somente por hardware no término do ciclo de leitura.

Legenda:

R = leitura; W = escrita; U = sem implementar, lido como zero (0); - n = valor durante POR;
x = desconhecido.

5.2 - NOS PROGRAMAS ABAIXO USAREMOS OS REGISTROS EECON1, EECON2, EEDATA, EEDATH, EEADR e EEADRH.

5.2.1 - Lendo EEPROM de dados.

A leitura é muito mais simples, pois não precisa de tanta proteção. Esta operação é também muito mais rápida. Antes, podemos verificar se há uma operação de leitura em andamento através do bit RD.

a) O roteiro para criação da rotina de leitura é o seguinte:

O endereço para leitura deve ser armazenado previamente no registro EEADR, selecione a memória a ser usada e em seguida o bit RD é setado. Instantes após (próximo ciclo), o dado lido da memória EEPROM é armazenado no registro EEDATA e o bit RD é apagado pelo hardware.

b) Programa para efetuar a leitura de um endereço na EEPROM de dados.

;Leitura na EEPROM de dados com o valor sendo movido para o registro W.

INCLUDE "P16F877.inc"

; definição de endereço inicial do programa

ORG 0x00

GOTO CONFIG

; corpo do programa

CONFIG

BSF STATUS, RP1 ;

BCF STATUS, RP0 ;seleciona banco2

MOVLW 0x30

MOVWF EEADR ;leitura do endereço 30h

BSF STATUS, RP0 ;seleciona banco3

BCF EECON1, EEPGD ;seleciona memória EEPROM de dados

BSF EECON1, RD ;inicia leitura

BCF STATUS, RP0 ;seleciona banco2

;após a leitura

```
MOVF    EEDATA, W    ;W = EEDATA
END
```

5.2.3 - Escrevendo na EEPROM de dados.

A complexibilidade da escrita é necessária para garantir a proteção do sistema evitando escritas acidentais e tornando o sistema robusto e seguro.

a) A escrita da EEPROM deve seguir o seguinte roteiro:

- I)** O endereço para a escrita deve ser colocado em EEADR. Como existem 256 bytes disponíveis, este endereço deve estar entre 0 e 255. Não se esqueça de alterar para Banco2 antes de atualizar esse registro.
- II)** O dado a ser escrito deve ser colocado em EEDATA. Só podemos escrever um byte de cada vez. Esse registro também encontra-se em Banco2.
- III)** Devemos ajustar a opção de trabalho para memória de dados (EEPROM) apagando (0) o bit EEPGD (EECON1<7>). Antes devemos alterar para Banco3.
- IV)** A escrita deve ser habilitada setando o bit WREN (EECON1<2>). Continue em Banco3. Para maior segurança, este bit deve ser desabilitado por software no final da escrita.
- V)** As interrupções devem ser desligadas para evitarmos conflitos, apagando (0) o bit GIE (INTCON<7>). Aqui não existe problema quanto ao banco de memória.
- VI)** O registro EECON2 deve ser carregado com os valores 0x55 e 0xAA, sequencialmente. Esse procedimento é obrigatório e utilizado para a proteção da escrita. Essas operações continuam sendo feitas no Banco3.
- VII)** Setar o bit WR (EECON1<1>) para iniciar a a escrita. Mais uma vez não se altera o Banco.
- VIII)** As interrupções podem ser novamente ligadas setando o bit GIE (INTCON<7>).
- IX)** A operação de escrita é um pouco demorada, e ela só terá terminado quando o bit WR (EECON1<1>) tiver sido apagado automaticamente pelo hardware. Por isso, ficamos esperando que essa ação aconteça. No caso de não podermos ficar esperando pelo fim da escrita, podemos ligar a interrupção relacionada a esse

evento através do bit EEIE (PIE2<4> e esperar que ela aconteça com o bit EEIF (PIR2<4>) setado, para considerarmos finalizada a escrita.

X) Apagar o bit WREN (EECON1<2>).

XI) Caso algum erro ocorra durante a operação de escrita, o bit WRERR (EECON1<3>) será setado. No caso de sucesso na operação esse bit será mantido apagado (0).

XII) Não se esqueça de mudar de banco de memória conforme suas necessidades, para a continuação da execução do programa.

b) Escrita na Memória EEPROM de Dados.

;Escrita na EEPROM de dados.

INCLUDE "P16F877.inc"

; definição de endereço inicial do programa

ORG 0x00

GOTO CONFIG

; corpo do programa

CONFIG

BSF STATUS, RP1

BSF STATUS, RP0 ;seleciona banco3

TESTA

BTFSC EECON1, WR ;verifica se há escrita em andamento

GOTO TESTA ;retorna a linha anterior e testa novamente

BCF STATUS, RP0 ;seleciona banco2

MOVLW 0x30

MOVWF EEADR ;carregada endereço 30h

MOVLW 0x20

MOVWF EEDATA ;carrega o valor 20h (dado)

BSF STATUS, RP0 ;seleciona banco3

BCF EECON1, EEPGD ;seleciona memória de dados

BSF EECON1, WREN ;habilita escrita

BCF INTCON, GIE ;desabilita interrupção

MOVLW 0x55

MOVWF EECON2 ;escreve 55h em EECON2

```

MOVLW 0xAA
MOVWF EECON2           ;escreve AAh em EECON2
BSF    EECON1, WR       ;inicia a escrita
BSF    INTCON, GIE       ;habilita interrupção
BCF    EECON1, WREN      ;por segurança, desabilita escrita
END

```

5.2.3 – Leitura na FLASH de Programa.

A leitura da memória de dados e a de programa são muito parecidas, a diferença é que agora temos que usar dois registros para endereço e outros dois para dado.

a) O roteiro para a criação da rotina de leitura fica sendo da seguinte maneira:

- I) O endereço para a leitura deve ser colocado em EEADRH e EEADR. Como existem 8 K disponíveis, este endereço deve estar entre 0 e 8.191 (1FFFh). Não se esqueça de alterar para Banco2 antes de atualizar estes registros.
- II) Devemos ajustar a opção de trabalho para memória de programa setando o bit EEPGD (EECON1<7>). Antes devemos alterar para Banco3.
- III) Setar o bit RD (EECON1<1>) para iniciar a leitura.
- IV) Duas instruções NOP devem ser colocadas no programa, obrigatoriamente, devido a lógica interna do sistema. Não podem ser substituídas por GOTO \$+1.
- V) O dado lido será colocado em EEDATH e EEDATA. Após armazenados, serão transferidos para os registros DATAL e DATAH. Para acessá-los não se esqueça de alterar antes para Banco2.

b) Leitura da Memória FLASH de Programa.

;Leitura na Flash de programa no endereço 02FFh e o valor sendo movido para os registros
;DATAL e DATAH.

```
INCLUDE "P16F877.inc"
```

```

;      atribuições de variáveis
ADDRL  EQU  0x20
ADDRH  EQU  0x21 (atenção para estes registros)
DATAL  EQU  0x90
DATAH  EQU  0x91
;      definição de endereço inicial do programa

```

```

                ORG    0x00
                GOTO CONFIG
;      corpo do programa
CONFIG
BSF    STATUS, RP1
BCF    STATUS, RP0    ;seleciona banco2
MOVLW  0xFF
MOVWF  EEADR          ;carrega byte menos significativo do endereço
MOVLW  02xx
MOVWF  EEADRH         ;carrega byte mais significativo do endereço
BSF    STATUS, RP0    ;seleciona banco3
BSF    EECON1, EEPGD  ;seleciona memória de programa
BSF    EECON1, RD     ;inicia leitura
NOP
NOP
BCF    STATUS, RP0    ;seleciona banco2
                        ;após leitura

MOVF   EEDATA, W
MOVWF  DATAL          DATAL = byte menos significativo do dado
MOVF   EEDATH, W
MOVWF  DATAH         DATAH = byte mais significativo do dado
END

```

5.2.4 – Escrita na FLASH de Programa.

A escrita na memória de programa é a única em que o micro não executa instruções enquanto esta operação esteja em andamento. Uma vez a operação de escrita tenha terminado, o micro reinicia executando o código de onde tenha parado. Verifique se esta operação pode ser executada, pois o bit WRT do Registro de configuração (em zero lógico) protege a memória contra a escrita.

a) O roteiro para esta operação fica então da seguinte maneira:

- I) O endereço para a escrita deve ser colocado em EEADRH e EEADR. Como existem 8K disponíveis, este endereço deve estar entre 0 e 8.191 (1FFFFh), mas deve ser utilizado obrigatoriamente um endereço terminado em 00₂. Não se esqueça de alterar para Banco2 antes de atualizar estes registros.

- II)** O primeiro dado a ser escrito deve ser colocado em EEDATH e EEDATA. Como só podemos escrever uma palavra de 14 bits de cada vez, os 2 bits mais significativos de EEDATH serão ignorados. Esses registros também encontram-se em Banco2.
- III)** Devemos ajustar a opção de trabalho para memória de programa setando o bit EEPGD (EECON1<7>). Antes devemos alterar para Banco3.
- IV)** A escrita deve ser habilitada através de EECON1<WREN>='1'. Continua em Banco3.
- V)** As interrupções devem ser desligadas para evitar conflitos, apagando o bit GIE (INTCON<7>). Aqui não existe problema quanto ao banco de memória.
- VI)** O registro EECON2 deve ser carregado com os valores 0x55 e 0xAA, sequencialmente. Este procedimento é obrigatório e utilizado para a proteção da escrita. Essas operações também continuam sendo feitas no Banco3.
- VII)** A escrita deve ser iniciada através do bit EECON1<WR>='1'. Mais uma vez não se altera o banco.
- VIII)** Duas instruções NOP devem ser colocadas no programa, obrigatoriamente, devido a lógica interna do sistema. Não podem ser por GOTO \$+1.
- IX)** As interrupções podem ser novamente ligadas setando o GIE.
- X)** A operação de escrita é um pouco demorada, e ela só terá terminado quando o bit EECON1<WR> tiver sido limpo automaticamente pelo hardware. Por isso, normalmente ficamos esperando que esta ação aconteça. No caso de não podermos ficar esperando pelo fim da escrita, podemos ligar a interrupção relacionada a este evento através do bit EEIE (PIE2<4> e esperar que ela aconteça para considerarmos finalizada a escrita.
- XI)** Apagar o bit WREN (EECON1<2>).
- XII)** Caso algum erro ocorra durante a operação de escrita, o bit WREER (EECON1<3> será setado. No caso de sucesso na operação este bit será mantido em zero.
- XIII)** Não se esqueça de ajustar o banco de memória novamente, conforme suas necessidades, para a continuação da execução do programa.

b) Escrita na Memória FLASH de Programa.

```

;Escrita na FLASH de programa.
        INCLUDE      "P16F877.inc"
;      definição de endereço inicial do programa
        ORG    0X00
        GOTO CONFIG
;      corpo do programa
CONFIG

BSF      STATUS, RP1
BCF      STATUS, RP0    ;seleciona banco2
MOVF     ADDRL, W        ;escreve endereço
MOVWF    EEADR           ;desejado
MOVF     ADDRH, W        ;da localização
MOVWF    EEADRH          ;da memória de programa
MOVF     VALUEL, W       ;escreve valor
MOVWF    EEDATA          ;desejado
MOVF     VALUEH, W       ;na localização
MOVWF    EEDATH          ;da memória de programa
BSF      STATUS, RP0    ;banco 3
BSF      EECON1, EEPGD   ;seleciona memória de Programa
BSF      EECON1, WREN    ;habilita a escrita
BCF      INTCON, GIE     ;desabilita interrupções se já estiver habilitada
MOVLW    0x55            ;escreve 55h em EECON2
MOVWF    EECON2
MOVLW    0xAA            ;escreve AAh em EECON2
MOVWF    EECON2
BSF      EECON1, WR      ;inicia operação de escrita
NOP
NOP
BSF      INTCON, GIE     ;somente se usando interrupção
BCF      EECON1, WREN    ;como proteção, desabilita escrita

```

5.2.5 - Tratando a interrupção de final de escrita na EEPROM e Flash.

a) Bit EEIF (PIR2<4>).

Esta interrupção deve ser usada em sistemas que não podem ficar parados esperando a operação de escrita terminar. Por isso, tão logo a operação de escrita seja iniciada, a rotina deve ser finalizada (não deve possuir o teste do bit WR (EECON1<1> como no exemplo anterior). Quando a escrita for terminada, uma interrupção irá ocorrer e o sistema poderá tomar as ações pertinentes.

Para que esta interrupção ocorra, a chave individual bit EEIE (PIE2<4>) deve estar setada. Dentro da rotina de tratamento devemos testar o flag EEIF (PIR2<4> para sabermos se foi esta interrupção que ocorreu. Antes de sairmos do tratamento devemos limpar este mesmo flag por software.

b) Proteção contra escritas espúrias.

Em determinadas condições, o dispositivo pode não querer escritas nessas memórias. Para proteger contra escritas espúrias o micro possui alguns mecanismos internos.

- I)** No Power-up, o bit WREN é limpo e o Power-up Timer (se habilitado) previne contra escritas.
- II)** A sequência iniciada da escrita junto com o bit WREN, ajudam a prevenir escritas acidentais durante o Brown-out, Power glitches ou malfuncionamento do firmware.

c) Operação enquanto protegida.

O PIC16F877 possui dois mecanismos de proteção:

- I)** um bit para a EEPROM
- II)** dois bits para a Flash.

O dado pode ser lido ou escrito na EEPROM independente do estado do código de proteção, bit CPD. Quando o código de proteção é habilitado e CPD limpo, acessos externos através ICSP é desabilitado independente do estado dos bits do código de proteção da Flash. Isto impede que o conteúdo da EEPROM seja lido por dispositivo externo.

O estado dos bits do código de proteção da Flash, CP0 e CP1, não afetam a execução de instruções fora da Flash. Os dispositivos PIC16F877 podem sempre ler os valores na Flash independente do estado dos bits do código de proteção.

Entretanto, o estado dos bits do código de proteção e o bit WRT, terão diferentes efeitos quando escrevendo na Flash. Veja **Figura 5.1**.

Uma vez o código de proteção tenha sido habilitado, para a EEPROM e Flash, somente um apagamento total de todo o dispositivo desabilitará o código de proteção.

d) Proteção de escrita na Flash.

A palavra de configuração contém o bit WRT, que protege a Flash contra a escrita. Este só pode ser acessado durante a programação do PIC via ICSP. Uma vez habilitada a proteção de escrita somente um apagamento de todo o dispositivo irá desabilitá-la. Quando habilitada, a proteção de escrita impede qualquer escrita na Flash. Proteção de escrita não afeta leituras na Flash.

Configuration Bits			Memory Location	Internal Read	Internal Write	ICSP Read	ICSP Write
CP1	CP0	WRT					
0	0	x	All program memory	Yes	No	No	No
0	1	0	Unprotected areas	Yes	No	Yes	No
0	1	0	Protected areas	Yes	No	No	No
0	1	1	Unprotected areas	Yes	Yes	Yes	No
0	1	1	Protected areas	Yes	No	No	No
1	0	0	Unprotected areas	Yes	No	Yes	No
1	0	0	Protected areas	Yes	No	No	No
1	0	1	Unprotected areas	Yes	Yes	Yes	No
1	0	1	Protected areas	Yes	No	No	No
1	1	0	All program memory	Yes	No	Yes	Yes
1	1	1	All program memory	Yes	Yes	Yes	Yes

Figura 5.1 – Comportamento interno para leitura/escrita da Flash.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
10Dh	EEADR	EEPROM Address Register, Low Byte								xxxx xxxx	uuuu uuuu
10Fh	EEADRH	—	—	—	EEPROM Address, High Byte					xxxx xxxx	uuuu uuuu
10Ch	EEDATA	EEPROM Data Register, Low Byte								xxxx xxxx	uuuu uuuu
10Eh	EEDATH	—	—	EEPROM Data Register, High Byte						xxxx xxxx	uuuu uuuu
18Ch	EECON1	EEPGD	—	—	—	WRERR	WREN	WR	RD	x--- x000	x--- u000
18Dh	EECON2	EEPROM Control Register2 (not a physical register)								—	—
8Dh	PIE2	—	(1)	—	EEIE	BCLIE	—	—	CCP2IE	-r-0 0--0	-r-0 0--0
0Dh	PIR2	—	(1)	—	EEIF	BCLIF	—	—	CCP2IF	-r-0 0--0	-r-0 0--0

Figura 5.2: Registros associados com as memórias EEPROM e FLASH.

CAPÍTULO 6.

ESTUDO DOS TIMERS.

O PIC16F87X possui três Timers que serão estudados separadamente.

6.1 - Módulo TIMER0.

É um contador de 8 bits, cujo valor é armazenado no registro TMR0 (01h e 101h), pode ser lido ou escrito, possibilitando a inicialização do contador com um valor diferente de zero, por software.

O conteúdo deste registro é sucessivamente incrementado usando um clock que tanto pode ser interno ($F_{osc}/4$) como externo (pino RA4/T0CK1). Pode usar um prescaler para poder ampliar a contagem.

6.1.1 – O módulo possui as seguintes características:

- a) Opera como Temporizador ou Contador de 8 bits
- b) Pode ser lido ou escrito
- c) Prescaler de 8 bits programável por software
- d) Seleção de clock interno ($F_{osc}/4$) ou externo (pino RA4/T0CK1)
- e) Interrupção por estouro ou transbordamento (overflow) quando a contagem passar de FFh para 00h (255 para 0), se habilitada.
- f) Seleção da borda de subida ou descida para o clock externo

A Figura 6.1 mostra o diagrama de blocos do módulo Timer0, Prescaler e WDT.

6.1.2 – O Timer0 pode operar de dois modos:

a) Modo Temporizador

O modo Temporizador (contagem de tempo) é selecionado no MUX pelo bit T0CS (OPTION_REG<5>). Neste modo, o módulo Timer0 incrementará livremente a cada ciclo de instrução, isto é, $F_{osc}/4$ (sem prescaler). Quando se escreve no registro TMR0, o incremento é inibido para os dois ciclos de instruções seguintes antes de começar a contar e isto deve ser levado em consideração pelo software.

b) Modo contador.

O modo Contador (contagem de eventos externos) é selecionado no MUX pelo bit T0CS (OPTION_REG<5>). Neste modo, o módulo Timer0 incrementará a cada pulso ou clock presente no pino RA4/T0CK1

T0CS = 0: incremento a cada ciclo de máquina ($F_{osc}/4$).

T0CS = 1: incremento a cada transição no pino RA4/T0CK1.

Quando selecionamos o incremento pelo pino T0CK1 (pulso ou clock externo) é possível ainda selecionar se o incremento acontecerá na borda de subida ou na de descida do sinal de entrada, através do bit T0SE (OPTION_REG<4>).

T0SE = 1: incremento na borda de descida.

T0SE = 0: incremento na borda de subida.

Para incremento externo ou interno, podemos aplicar um prescaler antes do registro TMR0 ser realmente alterado. Caso o prescaler esteja configurado 1:4, por exemplo, serão necessários quatro ciclos de máquina (ou pulsos externos) para que o TMR0 seja incrementado. O contador interno do prescaler é de 8 bits, mas não está disponível nem para a leitura e nem para a escrita. Este prescaler será visto adiante.

Para usar o prescaler é necessário primeiramente direcioná-lo ao Timer0 ou ao WDT, o que é feito através do bit PSA (OPTION_REG<3>). Depois é necessário ainda configurar o prescaler com o fator de divisão através dos bits PS2:PS0 (OPTION_REG<2:0>).

PSA = 1: prescaler aplicado ao WDT

PSA = 0: prescaler aplicado ao Timer0

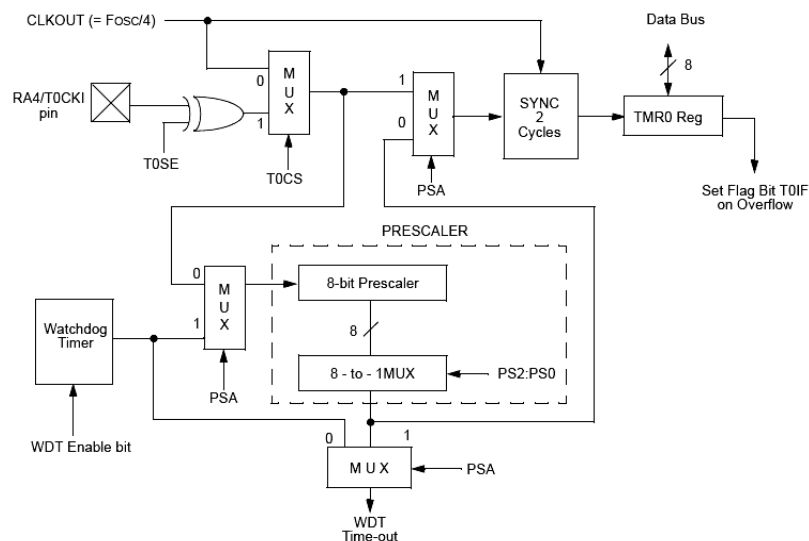


Figura 6.1: Diagrama de blocos do Timer0/WDT/Prescaler

6.1.3 - Interrupção no Timer0.

O temporizador pode gerar uma interrupção quando o registro TMR0 transborda na sua contagem de FFh para 00h (255 para 0), mas para isso, é preciso que as interrupções (bits GIE e T0IE (INTCON<7 e 5>), estejam habilitadas. Este transbordamento ou estouro seta o flag T0IF (INTCON<2>), gerando o pedido de interrupção (este bit deve ser limpo por software durante o tratamento da interrupção). Neste caso, o endereço PC + 1 será salvo na Pilha e o programa é desviado para o endereço 04h onde tem início a rotina de tratamento de interrupções. Esta interrupção pode ser mascarada pelo bit T0IE (INTCON<5>). A interrupção do TMR0 não pode despertar o micro de um SLEEP, visto que o temporizador está desligado durante um SLEEP.

6.1.4 - Parte do Registro INTCON usado com o Timer0.

bit 7 GIE: Habilitação geral das interrupções (chave geral).

1 = Habilita todas as interrupções para controle individual.

0 = Desabilita todas as interrupções.

bit 5 T0IE: Habilitação da interrupção de estouro de TMR0 (chave individual).

1 = Habilita a interrupção do TMR0.

0 = Desabilita a interrupção do TMR0.

bit 2 T0IF: Flag de identificação da interrupção por estouro do TMR0).

1 = Ocorreu estouro do registro TMR0 (este bit deve ser apagado por software).

0 = Não ocorreu estouro do TMR0.

6.1.5 - Usando o Timer0 com clock externo.

Quando o prescaler não está sendo usado, a entrada de clock externa é a mesma da saída do prescaler. A sincronização de T0CKI com a fase do clock interno é feita pela amostra da saída do prescaler nos ciclos Q2 e Q4 da fase do clock interno. Portanto, é necessário que T0CKI se mantenha em nível alto por 2Tosc (e um pequeno retardo RC de 20 ns) e baixo por 2Tosc (e um pequeno retardo RC de 20 ns).

6.1.6 - Prescaler.

O prescaler nada mais é do que um contador/divisor programável que é utilizado para dividir a frequência do sinal de clock aplicado ao Timer0 por um fator conhecido. Existe somente um prescaler, o qual é compartilhado entre o modulo Timer0 ou o Watchdog Timer (WDT). É um contador de 8 bits que não pode ser lido ou escrito e seu fator de divisão é feito pelos bits PS2:PS0 (OPTION_REG<2:0>). É sincronizado com clocks internos, e por isso podem ocorrer atrasos de até dois ciclos de clock até o

incremento do contador TMR0. Quando o prescaler for atribuído ao modulo Timer0 significa que o Watchdog Timer não poderá usá-lo, e vice-versa. O bit PSA (OPTION_REG<3>), determina para quem o prescaler é atribuído, isto é, Timer0 ou Watchdog.

Quando atribuído ao modulo Timer0, todas as instruções escritas no registro TMR0 (ex: clrf TMR0, movwf TMR0, bsf TMR0, etc.) limpam (zeram) o conteúdo do contador do divisor (prescaler) e não o fator de divisão. Quando atribuído ao WDT, o contador só pode ser apagado com a instrução CLRWD. Esta instrução apaga o prescaler e o Watchdog Timer.

Durante a execução de um programa é possível passar o prescaler do Tmer0 para o Watchdog e vice-versa, porém o conteúdo será apagado na transferência.

Obs: Escrevendo no TMR0, quando o prescaler estiver atribuído ao Timer0, zera o contador do prescaler, mas não mudará a atribuição do prescaler.

6.1.7 - Parte do registro OPTION_REG usado com o Timer0.

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
RBP	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
bit 7							bit 0

bit 7 /RBP

bit 6 INTEDG

bit 5 T0CS: TMR0 Clock Source Select bit (Bit de seleção de clock para TMR0 e que determina se o Timer0 será um contador ou um temporizador).

1 = TMR0 funcionará como um contador e será incrementado externamente pela mudança no pino RA4/T0CK1).

0 = TMR0 funcionará como um temporizador e será incrementado pelo oscilador interno (CLKOUT).

bit 4 T0SE: TMR0 Source Edge Select bit (Configuração da borda que incrementará o TMR0, quando ajustado para um contador de eventos, no pino RA4/T0CKI, isto é, T0CS = 1).

1 = É incrementado na transição de alto para baixo do pino RA4/T0CKI.

0 = É incrementado na transição de baixo para alto do pino RA4/T0CKI.

bit 3 PSA: Prescaler Assignment bit (Bit de atribuição do Prescaler)

1 = Prescaler é atribuído ao Watchdog.

0 = Prescaler é atribuído ao modulo Timer0 (TMR0)

bit 2-0 PS2:PS0: Prescaler Rate Select bits (Bits de seleção do fator de divisão do prescaler)

PS2:PS0	TMR0	WDT
---------	------	-----

000	1:2	1:1
001	1:4	1:2
010	1:8	1:4
011	1:16	1:8
100	1:32	1:16
101	1:64	1:32
110	1:128	1:64
111	1:256	1:128

Suponhamos que o módulo Timer0 funcione como contador de eventos e o prescaler está habilitado. O valor é 1:1, isso significa que cada pulso que entre incrementará o valor no registro TMR0 (o valor máximo da contagem é 256). Se o prescaler vale 1:8 significa que para cada 8 eventos que se sucedam só aumentará uma contagem no registro TMR0 (o valor máximo de contagem será $8 \times 256 = 2048$ eventos). O bit PSA (OPTION_REG<3>), define se o prescaler funciona com o Watchdog ou com o Timer0. Observamos que a única maneira de incrementar o TMR0 com uma relação de 1:1 é aplicar o prescaler ao WDT.

;Programa para mudar o prescaler do Timer0 para o Watchdog

```

bcf    STATUS,RP0    ; banco 0
clrf   TMR0          ; apaga o registro TMR0 e prescaler
bsf    STATUS,RP0    ; banco 1
clrwdt                ; apaga WDT
movlw  b'xxxx1xxx'   ; carrega W com este valor
movwf  OPTION_REG    ; prescaler é atribuído ao WDT
bcf    STATUS,RP0    ; banco 0

```

;Programa para mudar o prescaler do Watchdog para o Timer0

```

clrwdt                ; apaga WDT e prescaler
bsf    STATUS,RP0    ; banco 1
movlw  b'xxxx0xxx'   ; carrega W com este valor
movwf  OPTION_REG    ; prescaler é atribuído ao Timer0
bcf    STATUS,RP0    ; banco 0

```

Devemos ter em mente que, se as interrupções do temporizador estiverem habilitadas quando prescaler é atribuído ao WDT, seria conveniente desabilitar as mesmas e voltar a habilitá-las de novo quando este for transferido para o temporizador.

Cálculo da frequência com que vão ocorrer as interrupções, no modo temporizador, é feita pela seguinte fórmula:

$$\text{Fint} = \text{Clock} / \text{Prescaler} / (256 - \text{TMR0})$$

$$\text{Clock} = \text{Fosc} / 4 \text{ ou } \text{T0CK1}$$

Prescaler = fator de divisão

TMR0 = valor inicial do registro TMR0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
01h, 101h	TMR0	Timer0 Module's Register								xxxx xxxx	uuuu uuuu
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
81h, 181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
05h	PORTA	RA4									

Figura 6.3: Registros associados com o TIMER0.

6.2 - MÓDULO TIMER1.

O módulo Timer1 é um temporizador/contador de 16 bits consistindo de dois registradores de 8 bits, TMR1H (endereço 0Fh) e TMR1L (endereço 0Eh), os quais podem ser lidos ou escritos pelo programador.

6.2.1 – Este módulo possui as seguintes características:

- Opera como Temporizador ou Contador de 16 bits**
- Prescaler interno** independente capaz de dividir o sinal de entrada por 1, 2, 4 ou 8.
- Pode ligar/desligar** a contagem do módulo pelo bit TMR1ON.
- Sinal de clock interno** ($F_{osc}/4$).
- Sinal pulsado de clock externo** em RC0 ou clock a cristal em RC0 e RC1 (frequência máxima de 200KHz).
- Possibilidade de funcionamento no modo SLEEP.**

g) Capacidade de Reset gerado pelo módulo CCP.

h) Registro de controle específico T1CON.

i) Dois registros para armazenamento da contagem $TMR1 = TMR1H + TMR1L$

6.2.2 - O Timer1 pode operar de dois modos:

a) Modo Temporizador.

O modo Temporizador é selecionado pelo bit TMR1CS (T1CON<1>). Neste modo, o clock de entrada para o Temporizador é $F_{osc}/4$. O bit de controle de sincronização /T1SYNC (T1CON<2>) não possui nenhum efeito, visto que o clock interno está sempre em sincronismo.

b) Modo Contador.

Pode ser Síncrono ou Assíncrono e é selecionado pelo bit TMR1CS (T1CON<1>) e o incremento é na borda de subida do clock externo de entrada. Com isso, existem duas formas de fornecer sinais de clock ao Timer1 no modo contador. Uma delas é aplicar impulsos a contar no terminal RC0/T1OSO/T1CKI e a outra é gerar os impulsos com um oscilador externo conectando um cristal oscilador entre os terminais RC0/T1OSO/T1CKI e RC1/T1OSI/CCP2. Após o Timer1 ser habilitado no modo Contador, o módulo deve primeiro ter uma borda de descida antes do contador iniciar o incremento. Veja **Figura 6.5**.

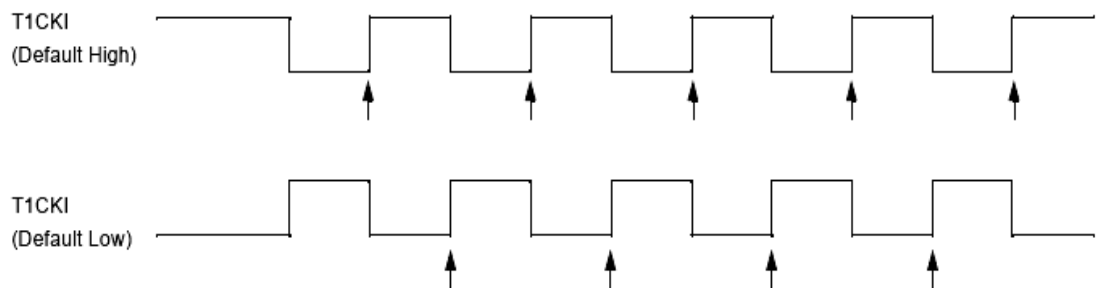


Figura 6.4: Borda de incremento do Timer1.

I) Modo contador sincronizado.

O modo Contador é selecionado pelo bit TMR1CS (T1CON<1>). Neste modo, o temporizador incrementa a cada borda de subida do clock de entrada no pino

RC1/T1OSI/CCP2, quando o bit T1OSCEN = 1 ou no pino RC0/T1OSO/T1CKI, quando o bit T1OSCEN = 0.

Se /T1SYNC = 0, então o clock externo de entrada é sincronizado com as fases dos clocks internos. A sincronização é feita após o estágio do prescaler. O estágio do prescaler é um contador de ripple assíncrono.

Nesta configuração, durante o modo SLEEP, o Timer1 não incrementará mesmo se o clock externo estiver presente, visto que o circuito de sincronização está desligado. O prescaler, entretanto, continuará a incrementar.

II) Modo Contador Assíncrono.

Com o bit de controle /T1SYNC (T1CON<2>) setado, o clock externo de entrada não está sincronizado. O temporizador mantém o incremento assíncrono para a fase interna do clock.

O Temporizador continuará funcionando durante o SLEEP e pode gerar uma interrupção por estouro (overflow), a qual despertará o micro. Entretanto, precauções especiais no software são necessárias para ler/escrever no temporizador.

No modo Contador Assíncrono, o Timer1 não pode ser usado como base de tempo para operações Compare ou Capture.

O Timer1 pode ser habilitado/desabilitado pelo bit de controle TMR1ON (T1CON<0>). O responsável pela habilitação do oscilador é o bit T1OSCEN (T1CON<3>).

T1OSCEN = 1: Oscilador ligado. RC0 e RC1 são configurados como entradas, independentemente do estado do registro TRISC. Estes pinos são perdidos como I/O.

T1OSCEN = 0: Oscilador desligado. RC0 opera como T1CKI (sinal externo para contagem) e RC1 opera como I/O normal.

No diagrama de blocos do TMR1 podemos ver que os dois terminais são conectados internamente a um buffer inversor que ao ser unido com um cristal, produzirá as oscilações. Este buffer é habilitado/desabilitado com o bit T1OSCEN (T1CON<3>). Na aplicação de sinal de clock externo e preciso desabilitar o buffer inversor.

O sinal gerado, independente do modo, é aplicado internamente a um divisor de frequência programável (prescaler do Timer1), que divide por 1, 2, 4 ou 8 o sinal de entrada ao temporizador de acordo com os bits T1CKPS1:T1CKPS0 (T1CON<5:4>).

Os contadores do prescaler são zerados quando se produz uma escrita nos registros do temporizador.

O Timer1 possui um circuito de sincronização para que o sinal externo possa sincronizar com a fase do clock principal, que depende do bit /T1SYNC (T1CON<2>). Com este sincronismo desligado podemos continuar operando durante um SLEEP se seu incremento for externo. Se o sinal for interno, a fase já estará sincronizada, visto que o sinal é o mesmo do ciclo de instrução.

Quando passamos o micro para o modo SLEEP e o Timer1 está configurado no modo síncrono, isto é, sincronizado internamente, o contador deixará de funcionar, pois desconecta-se o circuito de sincronismo; no entanto, no modo assíncrono seguirá contando durante o estado de repouso. Esta circunstância permite utilizá-lo como contador de tempo real e tirar o micro do modo Sleep ou utilizá-lo como base de tempos em operações de Captura e Comparação, por isso, o módulo CCP coloca o temporizador a zero quando se produz uma Captura ou Comparação.

6.2.3 - T1CON: Registro de Controle do Timer1 (Endereço 10h).

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	T1CKPS1	T1CKPS0	T1OSCEN	$\overline{T1SYNC}$	TMR1CS	TMR1ON
bit 7							bit 0

bit 7-6 Sem implementar: em leitura é zero (0).

bit 5-4 T1CKPS1:T1CKPS0: Timer1 Input Clock Prescale Select bits (Bits de Seleção do fator de divisão).

Prescaler para o clock do Timer1.

11 = divisão do prescaler de 1:8

10 = divisão do prescaler de 1:

01 = divisão do prescaler de 1:2

00 = divisão do prescaler de 1:1

bit 3 T1OSCEN: Timer1 Oscillator Enable Control bit (Bit de Controle de Habilitação do Oscilador externo do Timer1, pinos T1OSO e T1OSI).

1 = Oscilator externo habilitado

0 = Oscilador desligado (o inversor e o resistor estão desligados para reduzir o consumo)

bit 2 /T1SYNC: Timer1 External Clock Input Synchronization Control bit (Bit de controle de sincronização do clock externo).

Quando TMR1CS = 1:

1 = Não sincroniza o clock externo de entrada (modo assíncrono).

0 = Clock sincronizado com o ciclo de instrução interno.

Quando TMR1CS = 0:

Este bit é ignorado quando se usa o clock interno do Timer1.

bit 1 TMR1CS: Timer1 Clock Source Select bit (Bit de Seleção da origem do clock para o Timer1)

1 = Clock externo através do pino RC0/T1OSO/T1CKI (borda de subida). Modo contador.

0 = Clock interno ($F_{osc}/4$), (Modo temporizador do Timer1)

bit 0 TMR1ON: Timer1 On bit (Bit liga/desliga da contagem do Timer1).

1 = Timer1 habilitado.

0 = Timer1 inibido (contagem parada).

TMR1CS	/T1SYNC	Modos
0	0	Clock interno sincronizado
0	1	Clock interno sincronizado
1	0	Clock externo sincronizado
1	1	Clock externo não sincronizado

Quando o oscilador do Timer1 está habilitado pelo bit T1OSCEN (T1CON<3>), os pinos RC1/T1OSI/CCP2 e RC0/T1OSO/T1CKI tornam-se entradas. Isto é, o valor do TRISC<1:0> é ignorado, e estes pinos são lidos como zero (0).

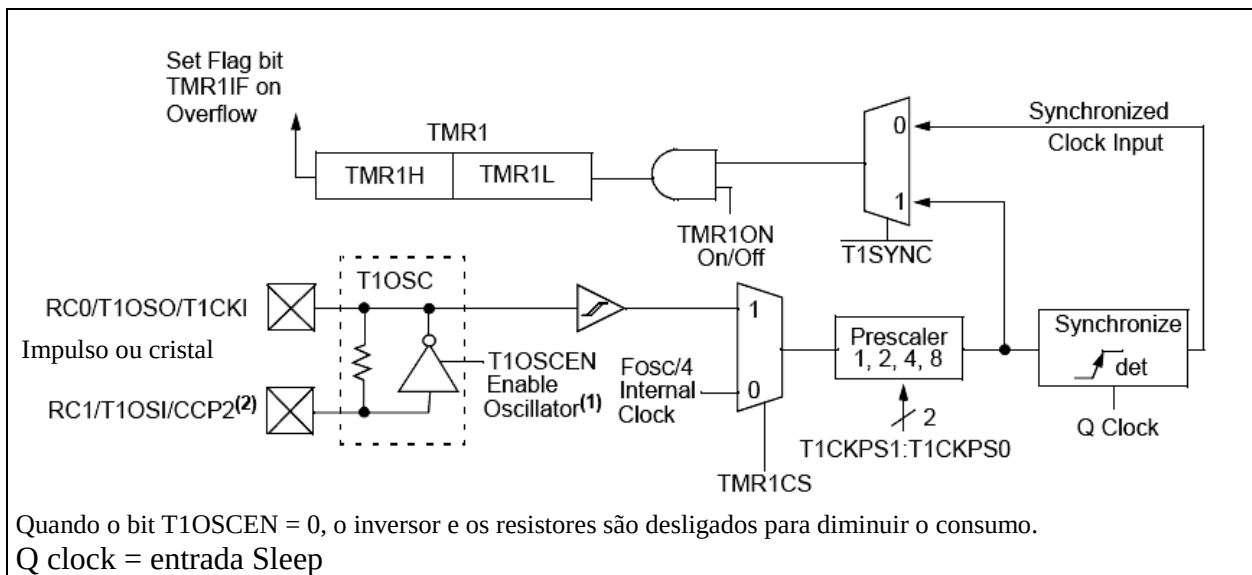


Figura 6.5: Diagrama de blocos do Timer1

6.2.4 - Interrupção do Timer1.

O par de registros TMR1 (TMR1H:TMR1L) incrementa de 0000h (0) a FFFFh (65535) e volta a 0000h. O transbordo do TMR1 ocorre se houver estouro do contador (contagem de FFFFh para 0000h) e indicada pelo flag de interrupção TMR1IF (PIR1<0>). A interrupção associada a este estouro depende dos bits TMR1IE (PIE1<0>), GIE e PEIE. (INTCON<7:6>).

6.2.5 - Lendo e escrevendo no Timer1 no modo contador assíncrono.

Escrever e ler o Timer1 pode ser complicado, pois como ele possui dois registros, enquanto trabalhamos com um o outro pode ser alterado. A maneira mais convencional seria desligarmos o timer antes de executarmos essas operações, mas nem sempre isso é possível no modo real. Outra forma é lermos os valores (primeiro TMR1H e depois TMR1L) guardando-os em variáveis temporárias. Depois da leitura, comparamos o valor de TMR1H. Caso ele tenha aumentado, lemos ambos os valores novamente. Para a escrita, primeiro devemos limpar TMR1L, depois carregamos TMR1H e TMR1L. Assim como no TMR0, a escrita nestes registros reseta o prescaler.

;Primeiro desabilitar todas as interrupções

```

movf    TMR1H,W    ; lê o byte alto
movwf   TEMPH
movf    TMR1L,W    ; lê o byte baixo
movwf   TEMPL
movf    TMR1H,W    ; lê de novo o byte alto
subwf   TMPH,W     ; subtrai a primeira leitura da segunda
btfsc   STATUS,Z   ; se o resultado for zero
goto    CONTINUA   ; a leitura é correta, senão, volta a ler
movf    TMR1H,W    ; lê novamente o byte alto
movwf   TEMPH
movf    TMR1L,W    ; lê de novo o byte baixo
movwf   TEMPL

                                ; reabilitar as interrupções
CONTINUA                                ;continua executando código

```

6.2.6 – Oscilador externo.

Há certas limitações na conexão do oscilador externo para o Timer1. Quando habilitamos o oscilador pelo bit T1OSCEN ($T1CON<3> = 1$), o cristal é conectado entre os pinos T1OSI (entrada) e T1OSO (saída do amplificador) e não deverá ser de frequência de ressonância superior a 200KHz, pois esta é a máxima frequência de trabalho para este oscilador que é de baixo consumo. O oscilador é do tipo LP (baixa frequência e baixo consumo) até 200 KHz.

Ele continuará a funcionar quando o micro está no modo SLEEP, pelo que, se a interrupção estiver habilitada por estouro do Timer1, $TMR1IE (PIE1<0>) = 1$, teremos a possibilidade de despertá-lo quando transbordar. A principal aplicação para usarmos o oscilador interno do Timer1 é em clock de tempo real e medição de tempo em que o tempo deve ser medido mesmo com o chip em modo SLEEP.

6.2.7 – Reset do Timer1 pelo uso do trigger de saída do CCP.

Se os módulos CCP1 ou CCP2 estiverem configurados no modo Compare para gerar um evento especial de disparo (special event trigger), registros CCP1CON ($CCP1M3:CCP1M0 = 1011$) e CCP2CON ($CCP2M3:CCP2M0 = 1011$), então este sinal resetará o Timer1.

Obs: O evento especial de disparo, de CCP1 e CCP2, não setará o flag de interrupção bit TMR1IF (PIR1<0>).

O Timer1 deve ser configurado para os modos Temporizador ou Contador Sincronizado para ter esta característica como vantagem. Se o Timer1 estiver funcionando no modo Contador Assíncrono, esta operação de RESET pode não funcionar.

No evento em que a escrita para o Timer1 coincide com o evento especial de disparo, de CCP1 ou CCP2, a escrita terá precedência. Neste modo de operação, o par de registros CCPRxH:CCPRxL (CCPxCON) efetivamente tornam-se o registrador de período para o Timer1.

6.2.8 - Reset do par de registros (TMR1H, TMR1L) do Timer1.

O par de registros TMR1H e TMR1L não é resetado para 00h no POR, ou em qualquer outro RESET, exceto pelo evento especial de disparo de CCP1 e CCP2.

O registro T1CON é resetado para 00h no Power-on Reset ou no Brown-out Reset, o que desliga o temporizador e coloca o prescaler em 1:1. Em todos os outros RESETs, o registro não é afetado.

6.2.9 - Prescaler do Timer1.

O contador prescaler é zerado na escrita para os registros TMR1H ou TMR1L.

Cálculo da frequência com que vão ocorrer as interrupções é feita como se segue:

$$\text{Fint} = \text{Clock} / \text{Prescaler} / (65536 - \text{TMR1})$$

$$\text{Clock} = \text{Fosc} / 4 \text{ ou } \text{T1CK1}$$

$$\text{Prescaler} = \text{fator de divisão}$$

$$\text{TMR1} = \text{valor inicial do par de registro TMR1H e TMR1L}$$

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	--uu uuuu

Figura 6.6: Registros associados com o TIMER1 como temporizador/contador

6.3 – MÓDULO TIMER2.

O Timer2 é um temporizador de 8 bits com um prescaler e um postscaler. Pode ser usado como base de tempo para os módulos CCP no modo PWM e geração de base de tempo de TX/RX do módulo SSP. O registro TMR2 pode ser lido e escrito, e é zerado em qualquer RESET de dispositivo.

6.3.1 - Características:

- a) Prescaler programável** de 1:1, 1:4 ou 1:16.
- b) Postscaler programável** de 1:1 a 1:16.
- c) Pode ligar/desligar a contagem do módulo** pelo bit TMR2ON.
- d) Registro de período PR2** que pode ser lido ou escrito.
- e) Funciona somente no modo temporizador**, pois o clock é interno ($F_{osc}/4$).
- f) Registro de controle específico T2CON.**

Não dispõe de entrada externa de clock pelo que se limita a contar ciclos internos de instrução ($F_{osc}/4$), com isso, não pode ser usado como contador de eventos. O clock de entrada ($F_{osc}/4$) tem opção de prescaler de 1:1, 1:4 ou 1:16, selecionada pelos bits de controle T2CKPS1:T2CKPS0 ($T2CON<1:0>$).

A saída do registro TMR2 também passa por um postscaler que pode dividi-la de 1:1 a 1:16 conforme os bits de controle TOUTPS3:TOUTPS0 ($T2CON<6:3>$), para gerar interrupção do TMR2. A função deste postscaler é de gerar a interrupção do Timer2 pelo bit TMR2IF ($PIR1<1>$).

O Timer2 pode ser desligado pelo bit de controle $TMR2ON = 0$ ($T2CON<2>$), para reduzir o consumo total de energia do micro.

O módulo Timer2 possui um registrador de período de 8 bits, PR2. Registro PR2 pode ser lido e escrito e é inicializado para FFh sob RESET.

O Timer2 não conta de zero até o limite imposto pelos 8 bits, mas sim, até o valor escrito no registro PR2. Sempre que $TMR2 = PR2$, o timer é resetado, voltando a zero. Neste mesmo momento, o contador do postscaler é incrementado. Quando o postscaler terminar, a interrupção associada ao Timer2 será gerada.

A função do postscaler está diretamente ligada à interrupção do Timer2. Isto porque ele nada mais é do que um contador de estouros do TMR2 (os estouros são uma comparação com PR2 e não no limite de 8 bits). Somente quando a quantidade de estouros for igual ao valor configurado no postscaler é que a interrupção irá acontecer.

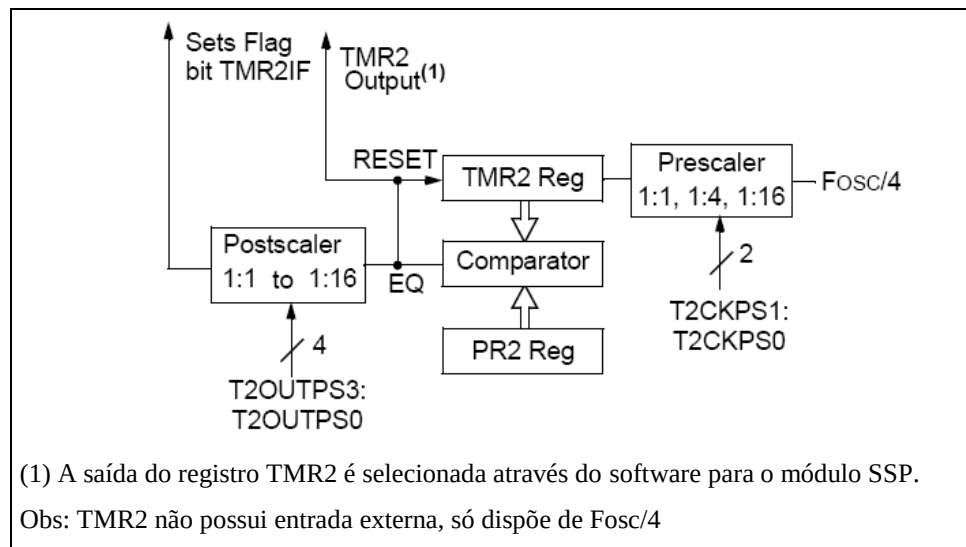


Figura 6.7: Diagrama de blocos do Timer2

6.3.2 - T2CON: Registro de Controle do TIMER2 (endereço 12h).

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
bit 7							bit 0

bit 7 Sem implementar: em leitura é zero (0).

bit 6-3 TOUTPS3:TOUTPS0: Timer2 Output Postscale Select bits (Bits de seleção do fator de divisão do Postscaler da saída do Timer2.

0000 = 1:1 Divisor do postscaler

0001 = 1:2 Divisor do postscaler

0010 = 1:3 Divisor do postscaler

•

•

1111 = 1:16 Divisor do postscaler

bit 2 TMR2ON: Timer2 On bit (Bit de Habilitação do Timer2.)

1 = Timer2 ligado

0 = Timer2 desligado

bit 1-0 T2CKPS1:T2CKPS0: Timer2 Clock Prescale Select bits (Bits de Seleção do fator de divisão do

Prescaler do Timer2.

00 = Prescaler de 1:1

01 = Prescaler de 1:4

1x = Prescaler de 1:16

6.3.3 - Prescaler e Postscaler.

a) Os Contadores prescaler e postscaler serão zerados quando ocorrer uma das seguintes situações:

I) Escrita para o registro TMR2

II) Escrita para o registro T2CON

III) Qualquer RESET de dispositivos (POR, /MCLR Reset, WDT Reset, ou BOR).

Obs: TMR2 não é zerado quando escrevendo no T2CON.

6.3.4 - Saída do TMR2.

A saída do TMR2 (antes do postscaler) é alimentada para o modulo SSP, que opcionalmente o usa para gerar um deslocamento do clock e também como base de tempo para o módulo CCP. A saída após o postscaler consiste no mesmo sinal anterior, porém dividido pelo fator programado no postscaler. Este sinal aciona o bit de flag de interrupção do Timer2 bit TMR2IF (PIR1<1>).

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
11h	TMR2	Timer2 Module's Register								0000 0000	0000 0000
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	-000 0000
92h	PR2	Timer2 Period Register								1111 1111	1111 1111

Figura 6.8: Registros associados com o TIMER2 como Temporizador

CAPÍTULO 7

MÓDULOS CAPTURE/COMPARE/PWM.

7.1 – FUNÇÕES BÁSICAS DO MÓDULO CCP

- a) Capturam a informação de 16 bits do Timer1.**
- b) Comparam o valor capturado com o armazenado em outro registro, e**
- c) Modulam ou controlam o intervalo de tempo que decorre entre a passagem de um para zero de um pino do micro.**

Cada módulo Capture/Compare/PWM (CCP) contém um registro de 16 bits os quais operam como Registros Capturadores de 16-bits, Registros Comparadores de 16-bits e Registros do ciclo de trabalho do PWM Master/Slave.

Este PIC dispõe de dois módulos CCP (CCP1 e CCP2), que são controlados com os registros CCP1CON e CCP2CON. Os módulos possuem funções idênticas, salvo em certas operações de disparo (special event trigger).

7.1.1 - Módulo CCP1.

Para realizar funções específicas, o módulo CCP1 possui o Registro1 Capture/Compare/PWM (CCPR1), é de leitura e escrita e formado por dois registros de 8 bits: CCPR1L (LSB) e CCPR1H (MSB). O registro CCP1CON controla as operações de CCP1. Um evento especial de disparo (special event trigger) é gerado quando houver igualdade na comparação e reseta o Timer1.

- a) Registro CCPR1L** (endereço 0x15), usado para armazenar os 8 bits menos significativos a serem usados na captura/comparação. No modo PWM, é usado para armazenar os 8 bits menos significativos usados na determinação do ciclo ativo do sinal.
- b) Registro CCPR1H** (endereço 0x16), usado para armazenar os 8 bits mais significativos a serem usados nos modos de captura e comparação de sinais. No modo de geração de sinais PWM, é usado como registro escravo, no qual é copiado o conteúdo do registro CCPR1L. Isto permite uma operação livre de ruídos e interferências que poderiam ser introduzidas no sinal caso fosse utilizado o registro CCPR1L diretamente.

7.1.2 - Módulo CCP2.

Para realizar funções específicas, o módulo CCP2 possui o Registro2 Capture/Compare/PWM (CCPR2), é de leitura e escrita e formado por dois registros de 8 bits: CCPR2L (LSB) e CCPR2H (MSB). O registro CCP2CON controla as operações de CCP2. Um evento especial de disparo (special event trigger) é gerado quando houver igualdade na comparação e reseta o Timer1 e inicia conversão A/D, se habilitado.

As **Figuras 7.1 e 7.2** mostram os recursos e a interação dos módulos CCP. Como os módulos compartilham como bases de tempo o Timer1 e o Timer2, podem existir algumas limitações conforme a combinação desejada.

Modo	Base de tempo
Capture	Timer1
Compare	Timer1
PWM	Timer2

Figura 7.1: Base de tempo para o CCP.

Modo desejado.		Influência.
Capture	Capture	Sem conflito, pois estão em sincronismo por usar a mesma base de tempo.
Capture	Compare	Se o Compare estiver configurado para zerar o Timer1, pode gerar conflito.
Compare	Compare	Se o Compare estiver configurado para zerar o Timer1, pode gerar conflito.
PWM	PWM	Estão em sincronismo, pois usam a mesma base de tempo.
PWM	Capture	Nenhuma, são independentes.
PWM	Compare	Nenhuma, são independentes.

Figura 7.2: Influência entre os módulos CCP.

7.1.3 - Registros CCP1CON/CCP2CON (endereços 17h e 1Dh).

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	CCPxX	CCPxY	CCPxM3	CCPxM2	CCPxM1	CCPxM0
bit 7							bit 0

bit 7-6 Não implementado: Lidos como zero (0).

bit 5-4 CCPxX:CCPxY: PWM Least Significant bits (Bits menos significativo do ciclo ativo do PWM). Não são significativos nos modos Capture/Compare.

Obs: Os oito MSB encontram-se em CCPRxL.

bit 3-0 CCPxM3:CCPxM0: CCPx Mode Select bits (Bit de seleção do modo de operação do módulo

CCPx – Compare/Capture/PWM)

0000 = Captura/Compara/PWM desabilitados (reset do módulo CCPx)

0100 = Modo Captura, captura a cada borda de descida com prescaler de 1:1.

0101 = Modo Captura, captura a cada borda de subida com prescaler de 1:1.

0110 = Modo Captura, captura na quarta borda de subida com prescaler de 1:4.

0111 = Modo Captura, captura na décima sexta borda de subida com prescaler de 1:16.

1000 = Modo Compare, seta a saída (CCPx) e o bit CCPxIF (PIR1/PIR2) quando houver igualdade na comparação.

1001 = Modo Compare, zera a saída (CCPx) e seta o bit CCPxIF (PIR1/PIR2) quando houver igualdade na comparação.

1010 = Modo Compare, a saída CCPx não é afetada mas seta o bit CCPxIF (PIR1/PIR2) quando houver igualdade.

1011 = Modo Compare, evento especial de disparo. CCPx não é afetado mas seta o bit CCPxIF (PIR1/PIR2). CCP1 reseta TMR1, CCP2 reseta TMR1 e inicia a conversão do módulo A/D se habilitado.

11xx = Modo PWM

Obs: Os registros CCP1CON e CCP2CON são idênticos, portanto a ilustração é válida para os dois e basta substituir o ‘x’ pelo número do módulo.

7.2 – MODO CAPTURE.

Este módulo tem como objetivo a contagem de tempo entre dois eventos ocorridos em um dos pinos CCPx. Assim, o período de um sinal aplicado à entrada CCPx é capturado (medido), sendo o resultado armazenado no par de registros CCPRxL e CCPRxH,

Figura 7.3.

Usa como base de tempo o Timer1 (modo síncrono tanto contador como temporizador), e no momento em que ocorre o evento nos pinos RC2/CCP1 ou RC1/CCP2 (conforme o modo de operação do módulo), é gerado um sinal de captura que copia a contagem atual do registro TMR1 (TMR1L e TMR1H) para o par de registros CCPRxL e CCPRxH. Por isso, configurar esses pinos como entradas através do registro TRISC.

A captura do valor do Timer1 não reseta este temporizador, por isso, para que se defina o tempo real entre duas leituras é necessário uma conta de subtração entre a última leitura e a anterior. Isto deve ser implementado por software.

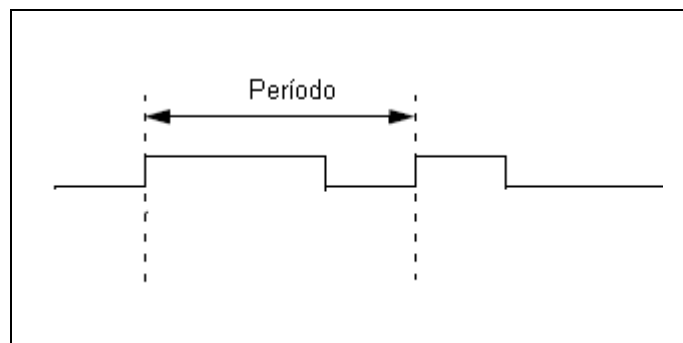


Figura 7.3: Período de um sinal.

7.2.1 - Eventos são definidos pelos bits de controle CCP1M3:CCP1M0 (CCPxCON<3:0>)

e são os seguintes:

- a) 0000** = Captura/Compara/PWM desabilitados (reset do módulo CCPx)
- b) 0100** = Captura a cada borda de descida do sinal aplicado em CCPx., com prescaler de 1:1.
- c) 0101** = Captura a cada borda de subida do sinal aplicado em CCPx, com prescaler de 1:1.
- d) 0110** = Captura na quarta borda de subida do sinal aplicado ao pino CCPx, com prescaler de 1:4.
- e) 0111** = Captura na décima sexta borda de subida do sinal aplicado ao pino CCPx, com prescaler de 1:16.

Quando termina uma captura, o flag de identificação de interrupção bit CCP1IF (PIR1<2>) para CCP1 ou CCP2IF (PIR2<0>) para CCP2, são setados para ocasionar uma interrupção. Para isso, é preciso habilitar as interrupções pelos bits CCP1IE (PIE1<2>) para CCP1 ou CCP2IE (PIE2<0>) para CCP2. Com esse flag setado, é possível ler os registros CCPRxH e CCPRxL.

Para evitar que se produzam falsas interrupções durante as modificações das condições nas quais se produzirá a captura, é necessário desabilitar a interrupção e/ou parar o módulo que voltará a ser ativada novamente ao final da condição.

O flag de interrupção precisa ser zerado por software. Se uma outra captura ocorrer antes que o valor dos registros CCPR1 ou CCPR2 sejam lidos, o valor capturado anteriormente será substituído pelo novo valor.

7.2.2 - Parte do Registro PIE1.

bit 2 CCP1IE: CCP1 Interrupt Enable bit (Bit de habilitação de interrupção do módulo CCP1, chave individual).

1 = Interrupção de CCP1 habilitada.

0 = Desabilitada.

bit 1 TMR2IE: TMR2 to PR2 Match Interrupt Enable bit (Bit de habilitação da interrupção do TMR2/PR2, chave individual).

1 = Enables the TMR2 to PR2 match interrupt (Interrupção do TMR2/PR2 habilitada).

0 = Desabilitada.

bit 0 TMR1IE: TMR1 Overflow Interrupt Enable bit (Bit de habilitação de interrupções do TMR1 por transbordamento, chave individual).

1 = Interrupção do Timer1 habilitada.

0 = Desabilitada.

7.2.3 - Parte do Registro PIR1.

bit 2 CCP1IF: CCP1 Interrupt Flag bit (Bit flag de identificação de interrupção do módulo CCP1, chave individual).

Modo captura:

1 = Ocorreu uma captura no registro TMR1 (deve ser apagado pelo software)

0 = Não ocorreu uma captura.

Modo comparador:

1 = Comparação em registro TMR1 igual (deve ser apagado por pelo software)

0 = Comparação desigual.

Modo PWM: Não usado.

bit 1 TMR2IF: TMR2 to PR2 Match Interrupt Flag bit (Bit flag de interrupção de TMR2/PR2, chave individual).

1 = TMR2 to PR2 match occurred (Ocorreu igualdade entre TMR2 e PR2, bit deve ser apagado pelo software).

0 = Não ocorreu.

bit 0 TMR1IF: TMR1 Overflow Interrupt Flag bit (Bit flag de interrupção do TMR1 por transbordamento, chave individual).

1 = O registro TMR1 transbordou (deve ser apagado pelo software).

0 = Não transbordou

7.2.4 - Parte do Registro PIE2.

bit 0 CCP2IE: CCP2 Interrupt Enable bit (Bit de habilitação de interrupção do módulo CCP2, chave individual).

1 = Interrupção do CCP2 habilitada.

0 = Desabilitada.

7.2.5 - Parte do Registro PIR2.

bit 0 CCP2IF: CCP2 Interrupt Flag bit (Bit flag de identificação de interrupção do módulo CCP2, chave individual).

Modo captura:

1 = O registro TMR1 fez uma captura (deve ser apagado pelo software)

0 = Não houve captura.

Modo comparador:

1 = O registro TMR1 foi comparado e é igual (deve ser apagado pelo software)

0 = A comparação não é igual.

Modo PWM: Não usado.

7.2.6 - Configuração do pino CCP.

No modo Capture, o pino RCx/CCPx precisa ser configurado como entrada pelos bits TRISC (TRISC<2 ou 1> setado).

Se o pino RCx/CCPx for configurado como saída, uma escrita para a porta pode ser considerada como mudança de borda, ativando a captura.

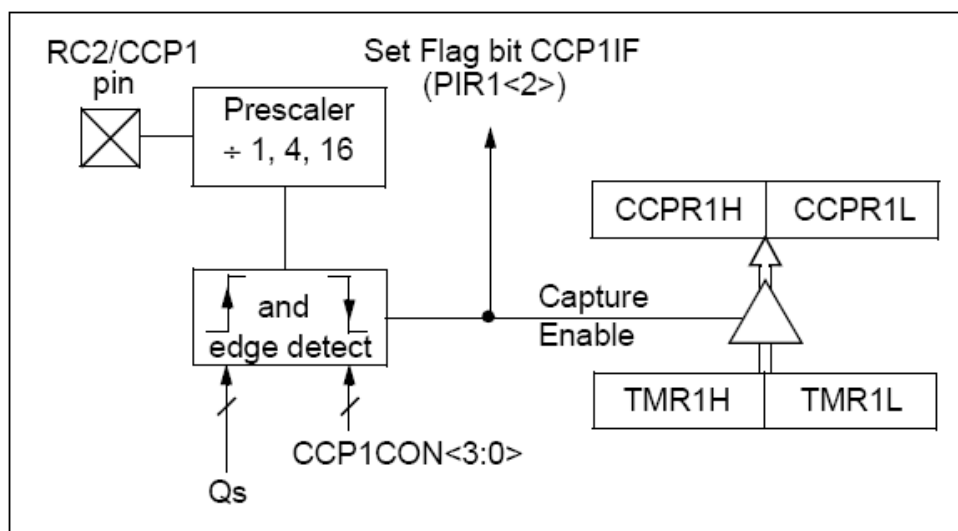


Figura 7.3 - Diagrama de blocos do modo Capture.

No diagrama de blocos podemos verificar que o módulo Capture dispõe de prescaler programável que é configurado com os bits CCPxM3:CCPxM0 (CCP1CON/CCP2CON<3:0>). Durante sua configuração podemos produzir falsas interrupções, uma maneira de evitá-las pode ser zerando seu conteúdo e reescrevendo um novo. Para apagá-los zeramos os bits CCPxM3:CCPxM0, isto é, desabilitamos o módulo.

7.2.7 - Timer1 – Seleção de modo.

O Timer1 precisa funcionar no modo temporizador ou no modo contador sincronizado (/T1SINC e TMR1CS do registro T1CON), para que o módulo CCP use suas características de captura. No modo contador assíncrono, a operação de captura pode não funcionar.

7.2.8 - Interrupção de software.

Quando o modo Capture é mudado, uma falsa interrupção de captura pode ser gerada. O usuário manterá o bit de habilitação de interrupção CCP1IE (PIE1<2>) = 0, para evitar falsas interrupções e limpará (zero) o bit de flag CCP1IF (PIR1<2>).

7.2.9 - Prescaler do CCP.

Há quatro configurações do prescaler, especificados pelos bits CCP1M3:CCP1M0. Sempre que o módulo CCP for desligado ou não esteja no modo Capture, o contador do prescaler é zerado. Qualquer reset limpará o contador do prescaler.

7.2.10 – Capture em modo SLEEP.

Nesta situação, ou o Timer1 não está funcionando ou está em modo assíncrono. A interrupção do CCP pode até acontecer, acordando o micro, mas os registros CCPRxH e CCPRxL não serão atualizados (não haverá captura).

Exemplo:

Clock externo do PIC= 4 MHz

Clock interno de 1 MHz o incremento do Timer 1 = 1us

Prescaler do Timer1 = 1:1

Após uma captura o valor armazenado no registro CCPRxL e CCPRxH = 1000

Período do sinal = 1000 . 1us = 1ms

7.3 - MODO COMPARE.

Enquanto no modo Capture o valor do registro TMR1 (TMR1H:TMR1L) era capturado e armazenado, aqui ele será constantemente comparado com os valores do registro CCPRx (CCPRxH e CCPRxL). Quando ocorrer igualdade o pino RCx/CCPx (RC1/CCP2 ou RC2/CCP1), que previamente foi configurado como saída pelo registro TRISC, será alterado de acordo com a seleção dos bits CCPxM3:CCPxM0 (CCPxCON <3:0>). Em qualquer caso, o flag CCPxIF será setado para ocasionar uma interrupção. De acordo com a seleção e ocorrendo uma igualdade, os pinos RC2/CCP1 ou RC1/CCP2 podem ser excitados em alto, excitados em baixo ou sem mudança.

7.3.1 - As opções para o modo Compare pelos bits CCPxM3:CCPxM0 (CCPxCON <3:0>), são:

- a) **0000** = Captura/Compara/PWM desabilitados (reset do módulo CCPx)
- b) **1000** = Seta a saída (CCPx) e o bit CCPxIF (PIR1/PIR2) quando houver igualdade na comparação.
- c) **1001** = Zera a saída (CCPx) e seta o bit CCPxIF (PIR1/PIR2) quando houver igualdade na comparação.
- d) **1010** = A saída CCPx não é afetada mas seta o bit CCPxIF (PIR1/PIR2) quando houver igualdade.

- e) **1011** = Evento especial de disparo. CCPx não é afetado mas seta o bit CCPxIF (PIR1/PIR2). CCP1 reseta TMR1, CCP2 reseta TMR1 e inicia a conversão do módulo A/D se habilitado.

O diagrama da **Figura 7.4** também é válido para o módulo CCP2.

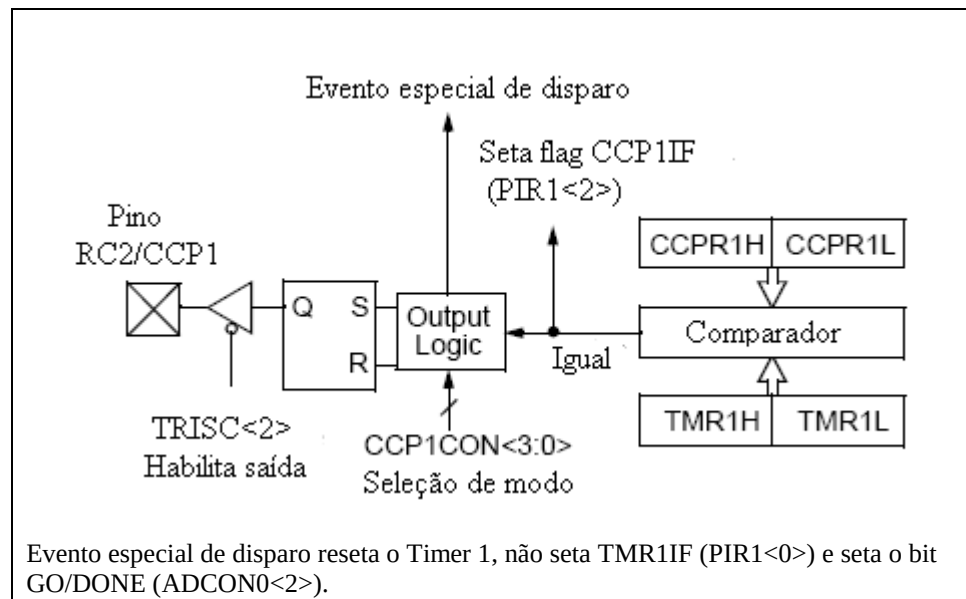


Figura 7.4: No modo Compare os circuitos associados com o pino RCx são diferentes.

7.3.2 - Configuração do pino CCP.

O usuário deve configurar o pino RC2/CCP1 como saída zerando o bit TRISC<2>. Zerando o registro CCP1CON forçará o RC2/CCP1 Compare Output Latch para o valor padrão de nível baixo (zero). Este não é o latch de dados de entrada/saída da Porta C.

7.3.3 - Timer1 – Seleção de modo.

O Timer1 precisa funcionar no modo síncrono, bit T1SYNC (T1CON<2>) = 0 (Temporizador ou Contador), se o módulo CCP estiver usando as características do modo Compare. No modo Contador Assíncrono, a operação compare pode não funcionar.

7.3.4 - Evento especial de disparo.

O módulo CCP, configurado no modo Compare, dispõe de uma função chamada de Função Especial de Disparo (Special Event Trigger), que será ativada quando programamos os bits CCPxM3:CCPxM0 (CCPxCON<3:0>) com o valor 1011, e que pode ser usado para iniciar uma ação.

O evento especial de disparo da saída de CCPx reseta o par de registros do TMR1, desta forma podemos usar o Timer1 de forma similar ao Timer2 com o registro de limite PR2 (CCPRx funcionaria como um registro de períodos capaz de ocasionar interrupções periódicas). A vantagem é que estamos trabalhando com registro de 16 bits e não de 8 bits.

Esta é a única diferença entre os módulos CCP1 e CCP2, pois CCP2 além de resetar o TMR1 e de ocasionar as interrupções (CCPxIF), inicia uma conversão no módulo conversor A/D (bit GO/DONE (ADCON0<2>)), se o módulo conversor estiver habilitado.

Obs: O evento especial de disparo dos módulos CCP1 e CCP2 não seta o flag de interrupção do Timer1 bit TMR1IF (PIR1<0>)

Para que ocorra periodicamente uma conversão de analógico para digital, podemos carregar nos registros CCPR2H e CCPR2L o número de impulsos nos quais, uma vez decorridos, desejamos que se realize uma conversão. Como estes registros são comparados continuamente com os registros do temporizador TMR1, quando ambos forem iguais começará a conversão. Esta operação não afeta o flag de interrupção do temporizador bit TMR1IF (PIR1<0>). Como a igualdade também coloca o temporizador a zero, assim o ciclo voltará a se repetir novamente.

Se só nos interessa a interrupção que há de ocasionar o módulo conversor quando finalizar a conversão, habilitaremos apenas a deste último, desabilitando a do comparador. É evidente que não desejamos saber quando tem lugar a igualdade entre os registros CCPR e TMR1 ou, o que é a mesma coisa, o início da conversão.

7.3.5 – Compare no modo SLEEP.

Neste modo, ou o Timer1 está em modo assíncrono ou está parado. Em nenhum dos dois casos o Compare poderá funcionar. Por isso, este modo não opera durante o

SLEEP. Só não se esqueça de que a saída CCPx é um pino como outro qualquer e, por isso, o nível de tensão da mesma será garantido mesmo no SLEEP.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
0Dh	PIR2	—	—	—	—	—	—	—	CCP2IF	---- --0	---- --0
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
8Dh	PIE2	—	—	—	—	—	—	—	CCP2IE	---- --0	---- --0
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	--uu uuuu
15h	CCPR1L	Capture/Compare/PWM Register1 (LSB)								xxxx xxxx	uuuu uuuu
16h	CCPR1H	Capture/Compare/PWM Register1 (MSB)								xxxx xxxx	uuuu uuuu
17h	CCP1CON	—	—	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0	--00 0000	--00 0000
1Bh	CCPR2L	Capture/Compare/PWM Register2 (LSB)								xxxx xxxx	uuuu uuuu
1Ch	CCPR2H	Capture/Compare/PWM Register2 (MSB)								xxxx xxxx	uuuu uuuu
1Dh	CCP2CON	—	—	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000	--00 0000

Figura 7.4: Registros associados com os módulos Capture, Compare e Timer1.

7.4 - MODO PWM.

Neste modo, o módulo CCP gera um sinal PWM. O sinal PWM consiste em um sinal de frequência fixa em que é variado o ciclo ativo do sinal da onda. Com ele, podemos obter uma tensão analógica a partir de um sinal digital. É particularmente importante, pois a partir dele é possível implementar um conversor D/A com um único pino do micro, uma vez que controlando a largura do pulso é possível obter uma tensão analógica variável. É muito usado no controle de potência (fonte chaveada, controles de velocidade, etc.) e em servocomandos.

O PIC16F877 possui dois canais PWMs (CCP1 e CCP2), cada um com resolução máxima de 10 bits. Portanto, o ciclo ativo poderá ser ajustado de 0 a 100% com uma resolução de 1024 pontos. Esta é a máxima resolução do modo PWM, no entanto, resoluções menores podem ser usadas em prol do aumento da frequência de saída.

7.4.1 - Controle do pulso e período do PWM.

No modo de Modulação de Largura de Pulso (PWM), temos os registros CCPR1H:CCPR1L ou CCPR2H:CCPR2L e PR2. Todas as operações são realizadas com o temporizador TMR2. Os pinos RC2/CCP1 ou RC1/T1OSI/CCP2 são configurados como saídas, para que oscilem entre 0 e 1 a intervalos variáveis de tempos.

Quando o valor armazenado no registro PR2 coincide com os oitos bits de maior peso do registro TMR2 ($TMR2 = PR2$), o flip-flop de saída é setado e o temporizador é resetado, reiniciando uma nova contagem. Neste instante, o conteúdo do registro CCPR1L concatenados com os dois bits $CCPxX:CCPxY$ ($CCPxCON<5:4>$) é lido e armazenado no registro escravo formado por CCPR1H e dois flip-flops internos. Quando o valor da contagem do Timer2 for igual ao valor armazenado em CCPR1H o flip-flop de saída será resetado até que ocorra um novo reinício no Timer2. Os valores carregados nos registros PR2 e CCPR1L correspondem ao tempo relativo em que o terminal permanecerá em 1 (um) ou em 0 (zero), respectivamente.

Os dois bits adicionais do Timer2 são obtidos internamente do prescaler do Timer2.

Zerando o registro CCP1CON forçará o CCP1 PWM Output Latch para o valor padrão de nível baixo (zero). Este não é o latch de dados de entrada/saída da Porta C.

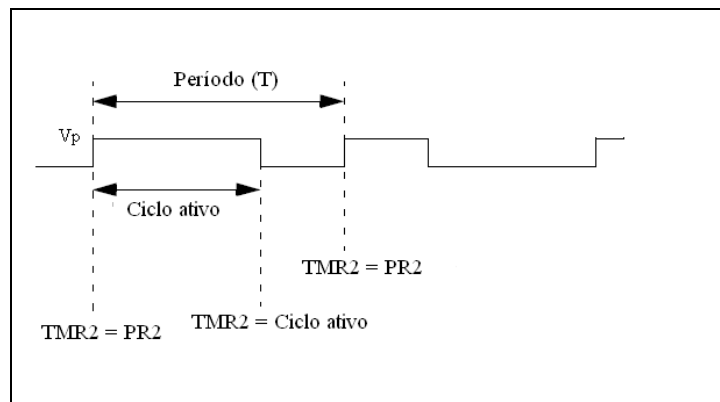


Figura 7.6: Saída do PWM.

a) Portanto, quando TMR2 for igual a PR2, os três eventos seguintes ocorrem no próximo ciclo de incremento:

I) TMR2 é resetado, iniciando o próximo período.

II) O pino CCPx é setado (exceto se ciclo ativo ou trabalho do PWM = 0%).

III) O ciclo de trabalho PWM é transferido de CCPR1L para CCPR1H atualizando a largura de pulso (caso tenha sido alterada durante o funcionamento).

O sistema passa, então, a monitorar o término do pulso, quando $TMR2 = CCPRxH$. Quando isto acontecer, a saída CCPx será resetada, até que um novo período comece. Caso o tamanho do pulso seja especificado como sendo maior do que o período total, a saída CCPx nunca será resetada, mas o sistema funcionará normalmente, como ajustado para 100% do PWM.

Para que a saída comece a operar, é necessário ainda ajustar o prescaler do Timer2 e ligá-lo através do registro T2CON. Por último, ligue o módulo PWM através dos bits CCPM3:CCPM0 ($CCPxCON<3:0>$).

0000 = Captura/Compara/PWM desabilitados (reset do módulo CCPx).

11xx = Modo PWM.

Obs1: O postscaler do Timer2 não é usado na determinação da frequência do PWM.

Obs2: No modo PWM, o registro CCPR1H é somente de leitura.

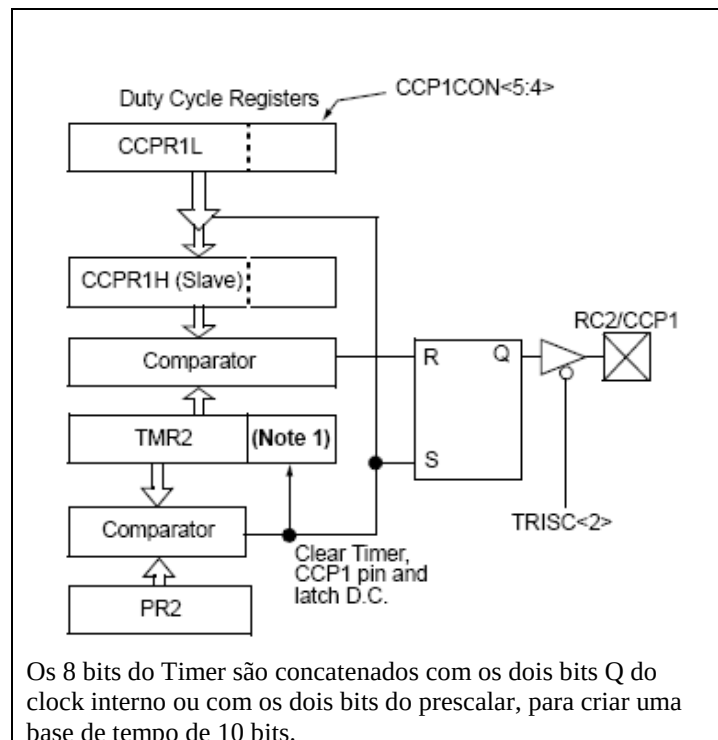


Figura 7.5: Diagrama de blocos do módulo CCP no modo PWM.**7.4.2 - Fórmulas para cálculos dos tempos em cada estado.**

O período do PWM (T) é controlado diretamente pelo Timer2 através do registro PR2. Sempre que TMR2 = PR2, o temporizador é resetado e um novo período é iniciado.

a) Período (T) = [(PR2) + 1] x 4 x Tosc x (Prescaler TMR2)

b) Frequência do PWM (PWM_{Freq}) = 1/T

c) Tempo do pulso em nível alto (tp).

$tp = CCPRxL:CCPxCON<5:4> \times Tosc \times (\text{Prescaler TMR2}).$

CCPRxL:CCPxCON<5:4> é uma composição de bits realizada ao concatenar os oito bits do registro CCPRxL com os bits CCPxCON<5:4>.

d) Ciclo ativo (Duty cycle) (tp/T).

É a relação entre o tempo do pulso em nível alto e o período total do PWM.

$tp/T = CCPRxL:CCPxCON<5:4> \times Tosc \times (\text{Prescaler TMR2}) /$

$[(PR2) + 1] \times 4 \times Tosc \times (\text{Prescaler TMR2})$

$tp/T = CCPRxL:CCPxCON<5:4> / [(PR2) + 1] \times 4$

Obs: Apesar do período e do tempo do pulso dependerem do Tosc e do ajuste do prescaler do Timer2, o ciclo ativo depende única e exclusivamente dos valores ajustados nos registros PR2, CCPRxL e CCPxCON<5:4>.

e) A máxima resolução PWM (bits) para uma dada frequência é encontrada pela fórmula:

$$Resolução_{PWM} = \frac{\log\left(\frac{f_{osc}}{f_{PWM}}\right)}{\log(2)} \text{ bits}$$

Obs: Se $(CCPR1L + CCP1X + CCP1Y) > PR2 \times 4 \rightarrow$ ciclo ativo de 100%

Se $(CCPR1L + CCP1X + CCP1Y) = 0 \rightarrow$ ciclo ativo de 0%

Quando se usa uma resolução de 10 bits, também se encadeiam os oito bits do TMR2 com os dois bits Q do relógio interno, fazendo com que conte a cada Tosc, em vez de a cada 4Tosc. Se usando 8 bits, os dois bits de menos peso são colocados a zero.

7.4.3 – Exemplo:

Calcular os valores para um PWM de 78,125 KHz, frequência do oscilador principal de 20MHz e prescaler do Timer2 ajustado em 1:1.

a) Período PWM (T) = (PR2 + 1) x 4 x T_{osc} x Prescaler.

$$T = 1/f = 1/78,125\text{KHz} = 12,8 \text{ us}$$

$$12,8\text{us} = ((\text{PR2}) + 1) \times 4 \times 1/20\text{MHz} \times 1$$

$$12,8\text{us} = ((\text{PR2}) + 1) \times 4 \times 50\text{ns} \times 1 \quad \therefore \text{PR2} = 63$$

b) Cálculo da resolução.

$$12,8\text{us} = 2^{\text{Resolução PWM}} \times (1/20) \times 1$$

$$12,8\text{us} = 2^{\text{Resolução PWM}} \times 50\text{ns} \times 1$$

$$256 = 2^{\text{Resolução PWM}}$$

$$\log 256 = (\text{Resolução PWM}) \times \log 2$$

$$\text{Resolução PWM} = \log 256 / \log 2 = 2,403.$$

$$\text{Resolução PWM} = 8 \text{ bits.}$$

Para esta mesma frequência do oscilador principal do micro damos na **Figura 7.8** os valores para o prescaler do temporizador PR2 e a máxima resolução, em diferentes frequências de saída.

7.4.3 - Organização para operação do PWM.

a) Esta deve ser a ordem a seguir para sua configuração:

I) Colocar em PR2 o período calculado.

II) Colocar nos registros CCPR1L e CCP1CON<5:4> a largura do pulso calculada.

III) Programar como saída o bit TRISC<2>.

IV) Colocar o valor do prescaler do TMR2 e habilitar o temporizador (T2CON).

V) Configurar o CCP1 para uma operação PWM (bits CCPxM3:CCPxM0).

O registro CCPR1H e dois bits do latch interno são usados para duplicar o buffer do ciclo de trabalho no modo PWM. Esta duplicação do buffer é essencial para evitar interferências (“glitch”) durante a operação PWM.

Quando o registro CCPR1H junto com os dois bits do latch se igualam a TMR2, concatenados com os dois bits do clock Q interno ou dois bits do prescaler do TMR2, o pino CCP1 será limpo (resetado).

7.4.4 – PWM durante SLEEP.

PWM não funciona e nem gera interrupção durante o SLEEP, porém a interrupção do Timer2 pode continuar acontecendo.

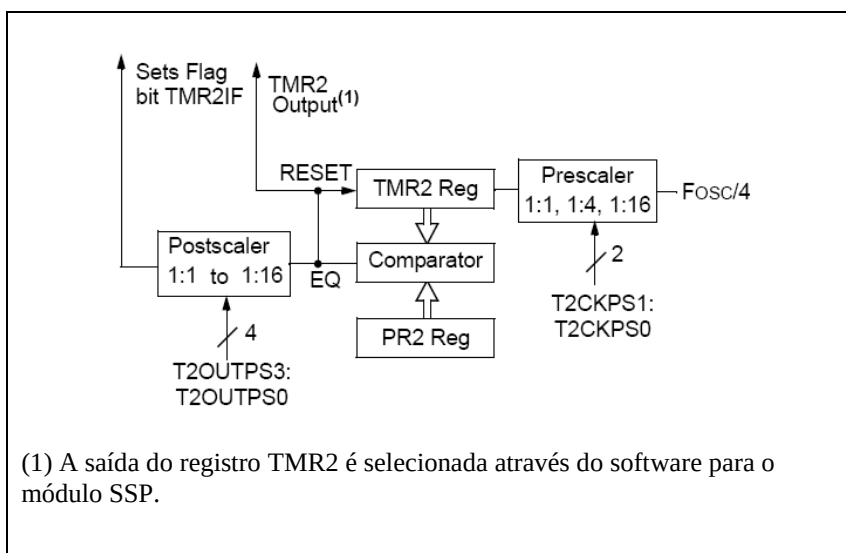


Figura 7.7: Diagrama de blocos do TIMER2.

Frequência do PWM	1,22KHz	4,88KHz	19,53KHz	78,12KHz	156,3KHz	208,3KHz
Prescaler (1, 4, 16)	16	4	1	1	1	1
Valor PR2	0xFFh	0xFFh	0xFFh	0x3Fh	0x1Fh	0x17h
Máx. resolução (bits)	10	10	10	8	7	5,5

Figura 7.8: Frequência de saída do PWM e diferentes resoluções para uma frequência de 20MHz.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
0Dh	PIR2	—	—	—	—	—	—	—	CCP2IF	---- --0	---- --0
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
8Dh	PIE2	—	—	—	—	—	—	—	CCP2IE	---- --0	---- --0
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111
11h	TMR2	Timer2 Module's Register								0000 0000	0000 0000
92h	PR2	Timer2 Module's Period Register								1111 1111	1111 1111
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	-000 0000
15h	CCPR1L	Capture/Compare/PWM Register1 (LSB)								xxxx xxxx	uuuu uuuu
16h	CCPR1H	Capture/Compare/PWM Register1 (MSB)								xxxx xxxx	uuuu uuuu
17h	CCP1CON	—	—	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0	--00 0000	--00 0000
1Bh	CCPR2L	Capture/Compare/PWM Register2 (LSB)								xxxx xxxx	uuuu uuuu
1Ch	CCPR2H	Capture/Compare/PWM Register2 (MSB)								xxxx xxxx	uuuu uuuu
1Dh	CCP2CON	—	—	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000	--00 0000

Figura 7.9: Registros associados com o PWM e Timer2.

CAPÍTULO 8

MÓDULO MSSP (PORTA SÉRIE SÍNCRONA MESTRA).

8.1 – INTRODUÇÃO.

O modulo Master Synchronous Serial Port (MSSP) é uma interface série síncrona usada para a comunicação com outros periféricos ou dispositivos microcontroladores. Dispositivos ou periféricos que podem usar estes barramentos para comunicação com um MCU são: EEPROMs seriais, registradores de deslocamento, display, conversores A/D e D/A, sensores, drives de LCD, MCU, etc.

8.1.1 – Este módulo pode trabalhar de dois modos diferentes:

a) Modo SPI - Serial Peripheral Interface (Interface Série de Periféricos)

b) Modo I²C - Inter-Integrated Circuit (Interface Inter-Circuitos)

Cada uma das interfaces série síncrona pode fazer o MCU funcionar como dispositivo Mestre ou Escravo.

Os dois protocolos (SPI e I²C) fazem parte do MSSP, como eles compartilham os mesmos recursos (pinos, hardware internos e registros), somente um pode ser usado por vez.

SPI	I ² C
Criado pela Motorola	Criado pela Phillips
Opera nos modo Mestre e Escravo	Opera nos modos Mestre e Escravo
Comunicação síncrona (com clock)	Comunicação síncrona (com clock)
Comunicação feita em três vias: - Clock (SCK) - Data in (SDI) - Data out (SDO) - /SS	Comunicação feita em duas vias - Clock (SCL) - Data In/Out (SDA)
Não endereçável	Endereçável de 7 ou 10 bits
8 bits de dados	8 bits de dados

Figura 8.1: Comparação entre os modos SPI e I²C.

8.1.2 - O módulo MSSP possui seis registros usados na configuração e no controle dos barramentos. São eles:

- a) **SSP Status Register (SSPSTAT)** – indica o estado da comunicação.
- b) **SSP Control Register (SSPCON) e SSP Control Register2 (SSPCON2)** – registros de controle das comunicações. SSPCON2 somente para o modo I²C.
- c) **Serial Receive/Transmit Buffer (SSPBUF)** – armazena o dado recebido ou o dado a ser transmitido.
- d) **SSP Shift Register (SSPSR)** – não é diretamente endereçado, é um registro de deslocamento que recebe o byte a ser transmitido do registro SSPBUF, de onde os bits são colocados, um a um, na linha SDA/SDI. Também recebe os bits recém chegados na linha SDA/SDI durante a recepção. Quando um byte é recebido pelo registro SSPSR, ele é imediatamente enviado para o registro SSPBUF.
- e) **SSP Address Register (SSPADD)** – armazena o endereço I²C quando o MCU é configurado como dispositivo escravo. Quando o MCU é configurado como dispositivo mestre, recebe o valor que definirá a taxa de transferência da comunicação I²C.

8.1.3 - SSPSTAT: Registro de Status da Porta Série Síncrona (endereço 94h)

R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SMP	CKE	D/A	P	S	R/W	UA	BF
bit 7							bit 0

bit 7 SMP: Sample bit (Bit de amostragem de dados)

Modo Mestre SPI:

1 = Dado de entrada é amostrado no final do tempo do dado de saída.

0 = Dado de entrada amostrado no meio do tempo do dado de saída.

Modo SPI escravo:

SMP deve ser apagado quando SPI é usado no modo escravo.

Modo I²C Mestre ou Escravo:

1 = Controle de slew rate desabilitado para o modo de velocidade padrão (100 kHz e 1MHz)

0 = Controle de slew rate habilitado para o modo de alta velocidade (400 kHz).

bit 6 CKE: SPI Clock Edge Select (Seleção da borda do clock SPI - Figura 8.4, Figura 8.5 e Figura 8.6)

Modo SPI com CKP = 0:

1 = Dado transmitido na borda de subida de SCK

0 = Dado transmitido na borda de descida de SCK

Modo SPI com CKP = 1:

1 = Dado transmitido na borda de descida de SCK

0 = Dado transmitido na borda de subida de SCK

Modo I²C Mestre ou Escravo:

1 = Níveis de entrada conforme especificação SMBus.

0 = Níveis de entrada conforme especificações I²C.

bit 5 D//A: Data//Address bit (Bit dado/endereço, somente modo I²C)

1 = Indica que o último byte recebido ou transmitido é um dado.

0 = Indica que o último byte recebido ou transmitido é um endereço.

bit 4 P: STOP bit (Bit Stop, somente no modo I²C). Este bit é apagado quando o módulo MSSP está desabilitado ou quando o último bit Start é detetado, SSPEN é apagado.

1 = Indica que um bit de Stop foi detetado (este bit é zero (0) no reset)

0 = Bit STOP não foi detetado.

bit 3 S: START bit (Bit Start, somente modo I²C). Este bit é apagado quando o módulo MSSP está desabilitado ou quando o último bit Start é detetado, SSPEN é apagado

1 = Indica que foi detetado um bit Start (este bit é '0' no reset)

0 = Bit Start não foi detetado.

bit 2 R//W: Read/Write bit Information (Bit de informação de Leitura/Escrita, somente modo I²C).

Modo I²C Escravo:

1 = Leitura

0 = Escrita

Modo I²C Mestre:

1 = Transmissão em progresso. No final, é limpo automaticamente por hardware.

0 = Nenhuma transmissão em progresso.

Logica OR deste bit com SEN, RSEN, PEN, RCEN, ou ACKEN indica se o módulo MSSP está no modo inativo (IDLE mode).

bit 1 UA: Update Address (Atualização de endereço, somente modo I²C de 10 bits)

1 = Indica que o usuário precisa atualizar o endereço no registro SSPADD (parte alta ou baixa), para nova comparação.

0 = Não precisa atualizar endereço.

bit 0 BF: Buffer Full Status bit (Indicação de buffer cheio)

Recepção (modos SPI e I²C):

1 = Recepção completada, registro SSPBUF está cheio.

0 = Recepção não foi completada, registro SSPBUF está vazio.

Transmissão (somente modo I²C):

1 = Transmissão de dado em progresso (não inclui os bits ACK e STOP), registro SSPBUF ainda está cheio.

0 = Terminada a transmissão de dado (não inclui os bits ACK e STOP), registro SSPBUF está vazio.

8.1.4 - SSPCON: Registro de Controle da Porta Série Síncrona (endereço 14h).

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
bit 7							bit 0

bit 7 WCOL: Write Collision Detect bit (Detetor de colisão de escrita do registro SSPBUF)

Modo mestre:

1 = Ocorreu uma escrita no SSPBUF enquanto uma condição I²C não era válida (este bit deve ser apagado pelo software).

0 = Sem colisão

Modo Escravo:

1 = Ocorreu uma escrita no registro SSPBUF enquanto ainda estava transmitindo (deve ser limpo pelo software)

0 = Sem colisão.

bit 6 SSPOV: Receive Overflow Indicator bit (Indicação de estouro na recepção)

Modo SPI:

1 = Um novo byte foi recebido antes que o dado anterior tivesse sido lido no registro SSPBUF.

Dado no registro SSPSR é perdido no estouro.

No modo Escravo, o usuário deve ler o registro SSPBUF, mesmo se somente transmitindo dados, para evitar transbordamento. No modo Mestre, o bit de transbordamento não sobe para 1, já que em cada recepção/transmissão é inicializado pela escrita do registro SSPBUF (deve ser limpo pelo software).

0 = Não há transbordamento

Modo I²C:

1 = Um novo byte foi recebido antes que o dado anterior tivesse sido lido no registro SSPBUF. No modo de transmissão o SSPOV é "don't care" (deve ser apagado pelo software).

0 = Não há transbordamento.

bit 5 SSPEN: Synchronous Serial Port Enable bit (Habilitação da Porta Série Síncrona – módulo SSP)

No modo SPI, quando habilitado, estes pinos devem ser configurados como entrada ou saída.

1 = Habilita a porta série e configura SCK, SDO, SDI e /SS como fonte dos pinos da porta série (controle interno).

0 = Desabilita a porta série e configura estes pinos como pinos de entrada/saída convencionais.

No modo I²C, quando habilitado, estes pinos devem ser configurados como entrada ou saída.

1 = Habilita a porta série e configura os pinos SDA e SCL como fonte da porta série (controle interno).

0 = Desabilita a porta série e configura estes pinos como pinos de entrada/saída convencionais.

bit 4 CKP: Clock Polarity Select bit (Seleção da polaridade do clock).**No modo SPI:**

1 = Estado inativo (Idle state) para o clock é nível alto. Transmissão ocorre na borda de descida e recepção na borda de subida.

0 = Estado inativo (Idle state) para o clock é nível baixo. Transmissão ocorre na borda de subida e recepção na borda de descida.

No modo I²C Escravo:**Controle de desbloqueio ou liberação de SCK:**

1 = Habilita o clock.

0 = Mantém o clock (pino SCL) em nível baixo (pausa).

Não é usado no modo I²C Mestre.**bit 3-0 SSPM3:SSPM0:** Synchronous Serial Port Mode Select bits (Seleção do modo de trabalho da porta SSP)

0000 = modo SPI Mestre, clock = $F_{osc}/4$

0001 = modo SPI Mestre, clock = $F_{osc}/16$

0010 = modo SPI Mestre, clock = $F_{osc}/64$

0011 = modo SPI Mestre, clock = $TMR2 / 2$

0100 = modo SPI Escravo, clock = pino SCK. Pino de controle /SS habilitado.

0101 = modo SPI Escravo, clock = pino SCK. Pino de controle /SS desabilitado (pode ser usado como entrada/saída).

0110 = modo I²C Escravo, endereço de 7 bits

0111 = modo I²C Escravo, endereço de 10 bits

1000 = modo I²C Mestre, clock = $F_{osc}/[4 * (SSPAD+1)]$

1011 = I²C Firmware Controlled Master mode (slave idle)

1110 = modo I2C Firmware Controlled Master mode, endereço de 7 bits com bit de interrupção Start e Stop habilitados.

1111 = modo I²C Firmware Controlled Master mode, endereço de 10 bits com bit de interrupção Start e Stop habilitados.

1001, 1010, 1100 e 101 = Reservados.

8.1.5 - SSPCON2: Registro 2 de Controle da Porta Série Síncrona (endereço 91h)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
bit 7							bit 0

bit 7 GCEN: General Call Enable bit (Habilitação do endereço global, somente modo I²C Escravo)

1 = Habilita interrupção quando uma chamada de endereço global (0000h) é recebida no registro SSPSR.

0 = Desabilita chamada de endereço global.

bit 6 ACKSTAT: Acknowledge Status bit (Valor recebido como ACK (reconhecimento) após a transmissão de um dado, somente no modo I²C Mestre).

Modo Mestre na transmissão:

1 = ACK (Acknowledge) não foi recebido do Escravo.

0 = ACK foi recebido do Escravo.

bit 5 ACKDT: Acknowledge Data bit (Valor a ser respondido como ACK após o término de uma recepção, somente modo I²C Mestre)

Modo Mestre na recepção: Valor que será transmitido quando o usuário inicia uma sequência de reconhecimento no final de uma recepção.

1 = Não reconhecimento, o Escravo deve parar de transmitir dados para o Mestre.

0 = Reconhecimento.

bit 4 ACKEN: Acknowledge Sequence Enable bit (Habilitação da sequência de reconhecimento, somente modo I²C Mestre)

No modo Mestre de recepção:

1 = Inicia uma sequência de reconhecimento nos pinos SDA e SCL e transmite o bit de dado ACKDT. Automaticamente apagado pelo hardware.

0 = Sequência de reconhecimento em repouso (idle).

bit 3 RCEN: Receive Enable bit (Habilita da recepção de dados, somente no modo I²C Mestre)

1 = Habilita recepção, Mestre pronto para receber dados do Escravo.

0 = Recepção em repouso (idle)

bit 2 PEN: STOP Condition Enable bit (Habilitação da condição Stop na linha, modo I²C Mestre)

Controle de desbloqueio ou liberação de SCK:

1 = Inicia a condição Stop nos pinos SDA e SCL. Automaticamente limpo por hardware.

0 = Condição Stop em repouso (idle)

bit 1 RSEN: Repeated START Condition Enable bit (Habilitação da condição de repetição (restart) de Start, somente no modo I²C Mestre)

1 = Inicia a condição nos pinos SDA e SCL. Automaticamente limpo por hardware.

0 = Condição de repetição em repouso (idle)

bit 0 SEN: START Condition Enable bit (Habilitação da condição Start, somente no modo I²C Mestre)

1 = Inicia condição Start nos pinos SDA e SCL. Automaticamente apagado por hardware.

0 = Condição Start em repouso (idle)

Nota: Para os bits ACKEN, RCEN, PEN, RSEN, SEN:

Se o módulo de I²C não estiver no modo inativo, este bit não pode ser setado, e o SSPBUF não pode ser escrito (ou escritas para o SSPBUF estão desabilitadas).

8.2 - OPERAÇÃO DA PORTA MSSP NO MODO SPI.

Neste modo, o protocolo usa três, quatro e em alguns casos, dois condutores de conexão para controle, e são:

a) SDO - Serial Data Out (Saída de dados Série), pino 24 (RC5/SDO).

b) SDI - Serial Data In (Entrada de dados Série), pino 23 (RC4/SDI/SDA).

c) SCK - Serial Clock (Sinal de clock), pino 18 (RC3/SCK/SCL).

d) /SS - Slave Select (Seleção de Escravos), pino 7 (RA5/SS/AN4), opcional, usado no modo Escravo de operação.

Como a comunicação é série síncrona, ela exige uma via de sincronismo entre os dois periféricos envolvidos (receptor/transmissor) para que haja a marcação do momento exato da transmissão/recepção de cada bit. Este sincronismo é feito pelo sinal de clock. Como só pode existir um clock, não podemos deixar que ambos os lados da comunicação sejam responsáveis por ele. Para resolver este problema foi criado o conceito de Mestre (Master) e Escravo (Slave). O dispositivo denominado Mestre será responsável pelo controle da comunicação, gerando o clock para o sincronismo. Do outro lado, o dispositivo denominado Escravo ficará sempre aguardando o clock enviado pelo Mestre.

Como o modo SPI não permite o endereçamento, a comunicação só pode ser feita entre dois pontos, sendo um deles Mestre e o outro Escravo.

O diagrama de blocos do módulo, **Figura 8.2**, mostra os quatro pinos que o micro usa para estas funções, que tem de ser programados como entradas ou saídas, conforme o caso. Os oito bits de dados podem ser transmitidos e recebidos simultaneamente.

As funções da porta SPI são configuradas e controladas por meio de dois registros especiais, SSPSTAT<7:6> e SSPCON<5:0>.

8.2.1 - Quando se inicializa o modo SPI, é necessário especificar no registro SSPCON as diferentes opções de trabalho:

- a) Modo Master** (onde SCK é saída de clock)
- b) Modo Slave** (onde SCK é entrada de clock)
- c) Polaridade do Clock** (Idle state de SCK)
- d) Velocidade do Clock** (só no modo Master)

A **Figura 8.2** mostra o diagrama de blocos do módulo MSSP quando no modo SPI.

Para habilitar a porta série, o MSSP Enable bit, SSPEN (SSPCON<5>) deve ser setado. Para resetar ou reconfigurar o modo SPI, zerar o bit SSPEN, reinicialize os registros SSPCON e então set o bit SSPEN. Isto configura os pinos SDI, SDO, SCK e /SS como pinos da porta série. Para que os pinos se comportem com função de porta série, alguns bits de direção de dados (registro TRIS) devem ser corretamente programados.

8.2.2 - Esses pinos são:

- a) SDI** é automaticamente configurado pelo sistema como entrada.
- b) SDO** deve ser configurado por software como saída (TRISC<5> = 0).
- c) SCK** deve ser configurado no modo Mestre como saída (TRISC<3> = 0).
- d) SCK** deve ser configurado no modo Escravo como entrada TRISC<3> = 1).
- e) /SS** quando habilitado no modo Escravo deve ser configurado por software como entrada TRISA<5> = 1 e o registro ADCON1 deve está setado de modo que o pino RA5 seja configurado como entrada/saída digital.

O pino /SS (RA5), na maioria das vezes é opcional e serve como um sinal de Chip Select. Quando em nível baixo, o sistema de comunicação funciona normalmente, com o pino SDO funcionando como saída de dados a cada pulso de clock. Em nível alto, o sistema de comunicação é desligado e o pino SDO fica flutuando. Assim, com este recurso, podemos montar uma rede de comunicação com um Mestre e vários Escravos,

desde que o Mestre controle individualmente os pinos /SS de cada Escravo. Este controle terá de ser implementado no software e deve garantir que somente um Escravo está ativado de cada vez, evitando conflitos nas saídas SDO.

Qualquer função da porta série que não seja desejada, pode ser anulada pela programação do correspondente registro de direção de dados (TRIS) para o valor oposto.

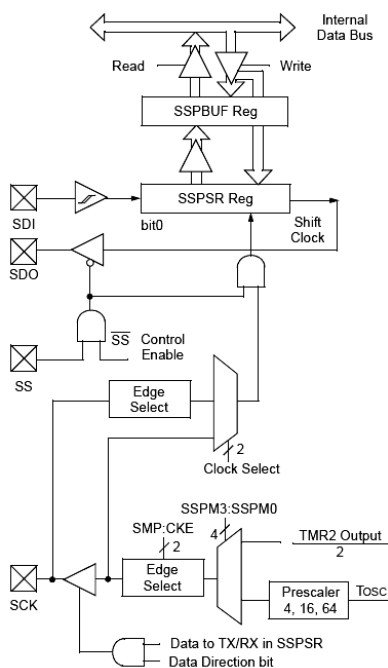


Figura 8.2: Diagrama de blocos do modulo SSP no modo SPI.

Na **Figura 8.3** temos a conexão entre dois MCUs onde só um funciona como mestre. O mestre inicia a transferência de dados com o sinal de clock. Cada bit é deslocado no registro por cada borda do clock e na borda seguinte é armazenado no latch.

Quando o micro for configurado como mestre os dados são transmitidos/recebidos cada vez que se escreve no registro SSPBUF. Se o MCU agir como escravo, o terminal SCK deverá ser configurado como entrada para que o sinal, presente no terminal SDI, se desloque no registro SSPSR para cada pulso de clock recebido. O intervalo programável para o clock com os bits SSPM3:SSPM0 é de $F_{osc}/4$, $F_{osc}/16$, $F_{osc}/64$ e $\text{Timer2 (saída)}/2$.

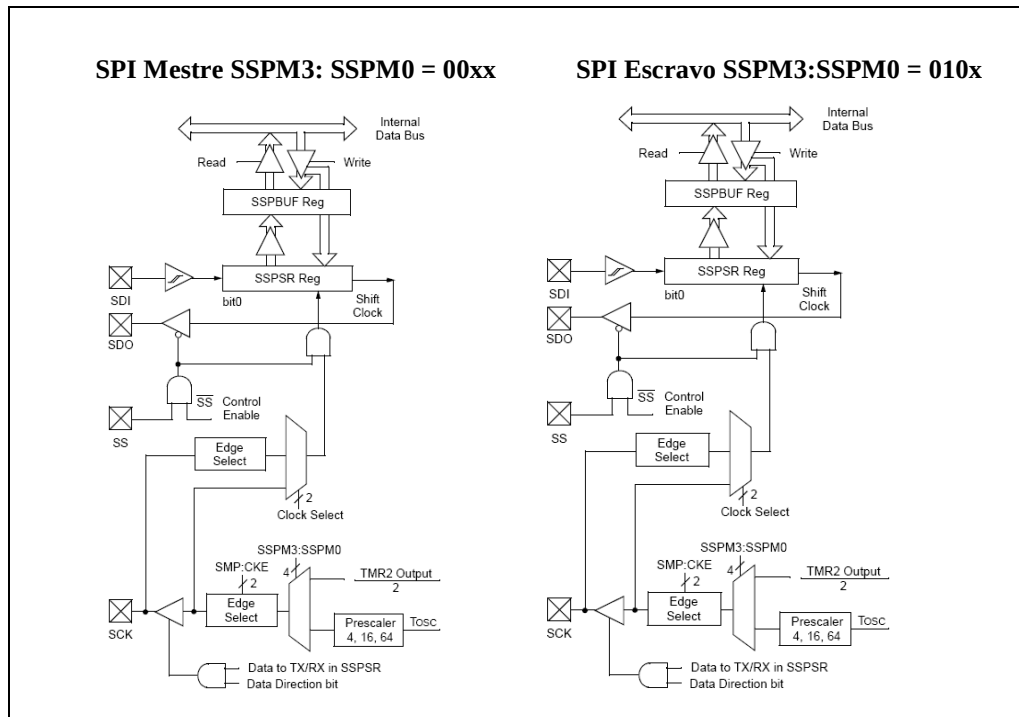


Figura 8.3: Conexão Mestre /Escravo

Como vimos, o PIC mestre é responsável pela geração do clock. Seu pino SCK deve ser configurado como saída. O PIC escravo possui o pino de clock configurado como entrada.

O protocolo de comunicação trabalha com um registro de deslocamento interno, não acessível, denominado SSPSR. A cada ciclo de clock, um bit é lido da porta SDI e escrito nesse registro, começando pelo bit menos significativo e rotacionando todos os bits para a direita. Ao mesmo tempo, o bit mais significativo é enviado para a porta SDO. Por isso, enquanto recebemos um byte também transmitimos outro. O registro SSPBUF é o responsável pelo valor a ser transmitido e também pelo recebido.

8.2.3 - A comunicação funciona da seguinte maneira:

- Quem manda na comunicação é o mestre.** Por isso, um valor qualquer é escrito no registro SSPBUF deste PIC.
- O valor é então movido em paralelo para o registro SSPSR.**
- Em seguida 8 pulsos de clock (SCK) são gerados, cada pulso transmite um bit de dado.

- d) Ao término dos 8 pulsos o valor que estava armazenado no registro SSPSR_(mestre) é trocado com o valor que estava no registro SSPSR_(escravo).
- e) Tanto de um lado quanto do outro o valor do registro SSPSR é então movido novamente para o registro SSPBUF.
- f) Também de ambos os lados, flags são ativados para informar ao sistema o fim da comunicação. Os dados podem então ser acessados novamente em SSPBUF.
- g) Neste nosso exemplo de dois MCUs, vimos que a comunicação é feita sempre em sentido duplo e ao mesmo tempo. Dessa forma, 3 situações diferentes podem acontecer:

I) Mestre envia dado válido e Escravo envia dado não válido.

Mestre faz uma pergunta ao Escravo que terá de ser processada antes da resposta ser transmitida de volta. O dado recebido pelo Mestre ao final da transmissão deve ser descartado.

II) Mestre envia dado não válido e Escravo envia dado válido.

Este caso complementa o caso anterior. Depois que o Mestre fez uma pergunta e o Escravo a processou, uma resposta deverá ser retomada. Acontece que como é Mestre quem manda no clock, ele precisará disparar uma nova transmissão para poder receber o dado do Escravo.

III) Mestre envia dado válido e Escravo envia dado válido.

Usado para aumento da velocidade. O Mestre faz uma pergunta e ao mesmo tempo recebe a resposta da pergunta anterior.

Obs: Caso a comunicação seja em um só sentido, a via de transmissão do Escravo (SDO) pode ser desativada configurando esse pino como entrada, a recepção do Mestre (SDI) fica sem uso.

8.2.4 - Configuração para efetuar uma transmissão:

- a) Configurar pinos de acordo com o **ítem 8.2.2.**
- b) Configurar a comunicação como sendo Mestre ou Escravo com os bits SSPM3:SSPM0 (SSPCON<3:0>). Existe mais de uma opção
- c) Configurar a condição de recebimento através do bit SMP (SSPSTAT<7>). O sistema pode efetuar a leitura da porta SDI em dois pontos distintos.

- d)** Configurar a forma de onda do clock. O padrão SPI disponibiliza 4 opções que podem ser ajustadas em dois bits distintos, CKP (SSPCON<4>) e CKE (SSPSTAT<6>), como na **Figura 8.4**
- CKP = 0**, clock normalmente em nível baixo.
- CKP = 1**, clock normalmente em nível alto.
- CKE = 0**, pulso de clock no começo do período.
- CKE = 1**, pulso de clock no final do período.
- e)** Ligar a comunicação SPI setando o bit SSPEN (SSPCON<5>).
- f)** Por último, escrever o dado que se deseja transmitir no registro SSPBUF. Se isto for feito no Mestre, a transmissão será iniciada imediatamente após a escrita. Para o escravo, o sistema ficará aguardando o clock.
- g)** No término dos oito pulsos do clock, o dado terá sido enviado do Mestre para o Escravo e vice-versa. Os registros SSPBUF serão, então, atualizados e o flag BF (SSPSTAT<0>) será setado indicando SSPBUF cheio. Este bit será limpo automaticamente quando SSPBUF for lido.
- h)** No mesmo instante, o bit SSPIF PIR1<3> será setado e uma interrupção poderá acontecer se ela estiver habilitada.

Obs1: Caso haja tentativa de escrita em SSPBUF antes do término da transmissão/recepção, será ignorado, mas o bit WCOL (SSPCON<7>) será setado indicando colisão de escrita. Este bit deve ser apagado por software.

Obs2: Caso o Escravo receba um novo valor antes do SSPBUF ser lido (BF = 1), este novo valor será perdido e o flag de overflow bit SSPOV (SSPCON<6>) será setado. Este bit deve ser limpo pelo software. Para o Mestre este flag não possui função.

Em uma operação SLEEP, quando configurado como mestre, todos os módulos de clock são desligados e a transmissão/recepção permanecerá congelada até o micro despertar.

No modo escravo, os deslocamentos sobre os registros são assíncronos, o que permite continuar a transmissão/recepção com o micro em SLEEP. O micro despertará do SLEEP quando os 8 bits tiverem sido recebidos e a interrupção correspondente habilitada.

Como já visto, no modo de recepção, os dados recebidos são introduzidos em série no registro de deslocamento SSPSR que, uma vez recebido o oitavo bit, transfere para o registro SSPBUF em paralelo, isto faz com que os bits BF (SSPSTAT<0>) e SSPIF (PIR1<3>) sejam setados. O primeiro para indicar que o buffer (SSPBUF) está cheio e o segundo para indicar que a recepção/transmissão acabou.

Se durante a recepção/transmissão tentar se escrever no registro SSPBUF, a escrita será ignorada, mas o bit WCOL (SSPCON<7>) será setado para advertir que houve “colisão”. O bit BF será apagado (passará para zero) quando se proceder à leitura do registro SSPBUF. É preciso realizar a leitura antes de ter lugar uma nova recepção.

O registro SSPBUF desempenha um papel muito importante, pois os dados têm de passar por ele tanto na transmissão como na recepção.

8.2.5 - O seguinte exemplo mostra o processo de seu carregamento no modo de transmissão:

```
bsf STATUS,RP0    ;banco 1
btfss SSPSTAT,BF   ;transmissão completa?
goto Cheio?        ;não
bcf STATUS,RP0     ;banco 0
movf SSPBUF,W      ;registro W
movwf Rxd          ;salva em RAM
movf Txd,W         ;novo dado
movwf SSPBUF       ;a transmitir
```

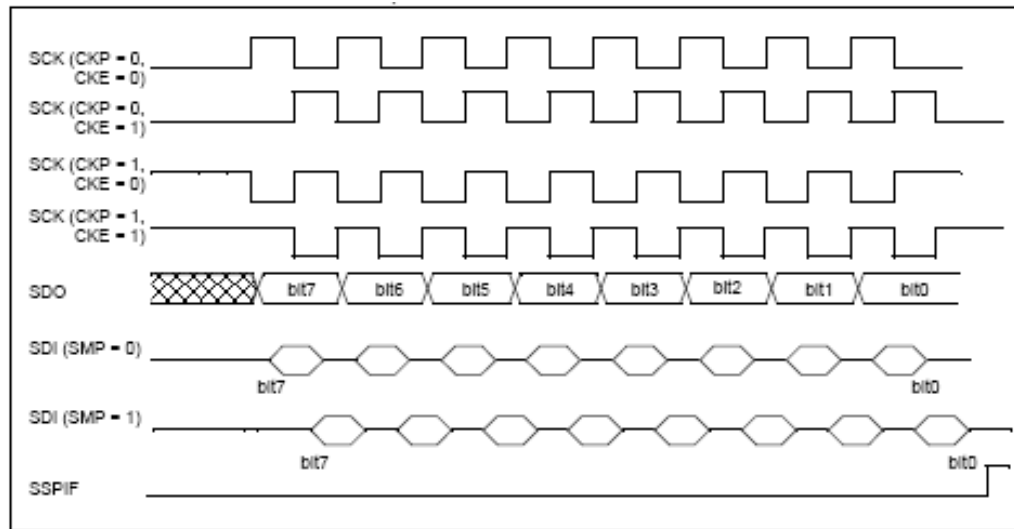


Figura 8.4: Temporização SPI no modo mestre.

8.2.6 - SPI no modo Escravo.

Neste modo, os dados são transmitidos ou recebidos com o controle do sinal externo de clock. Quando o último bit for armazenado, o flag de interrupção SSPIF (PIR1<3>), será setado. Neste caso, como a fonte de clock é externa, o terminal SCK deverá ser programado como entrada.

O pino SS permite sincronizar o modo escravo. Quando este modo é habilitado (SSPCON<3:0> = 0100), o pino SS comporta-se como entrada, pelo que será necessário programá-lo como tal (TRISA<5> = 1). Forçando este pino para nível alto, o módulo SPI é inicializado e o bit SSPEN (SSPCON<5>) apagado (zero).

É possível realizar transmissões e recepções só com dois fios de conexão, neste caso os pinos SDI e SDO podem ser unidos, eliminando assim a terceira conexão. Quando o pino SDI precisar operar como recepção, o pino SDO deverá ser configurado como uma entrada, e deste modo desabilita-se a transmissão e não se cria conflito com os dados da SDI.

8.2.7 - SPI durante o SLEEP.

Quando no modo SLEEP, o escravo pode transmitir/receber dados. Quando um byte é recebido, o dispositivo é despertado da condição de SLEEP.

Obs1: Quando o módulo SPI está no modo Escravo com o pino de controle de habilitação /SS (SSPCON<3:0> = 0100), o módulo SPI resetará se o pino /SS estiver setado para VDD.

Obs2: Se o SPI está sendo usado no modo Escravo com CKE = 1, então o pino de controle /SS precisa estar habilitado.

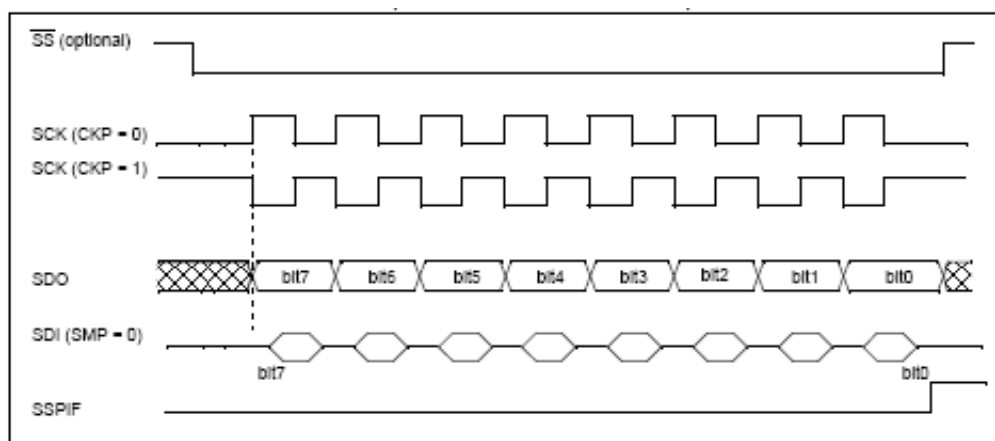


Figura 8.5: Temporização SPI no modo escravo com CKE = 0.

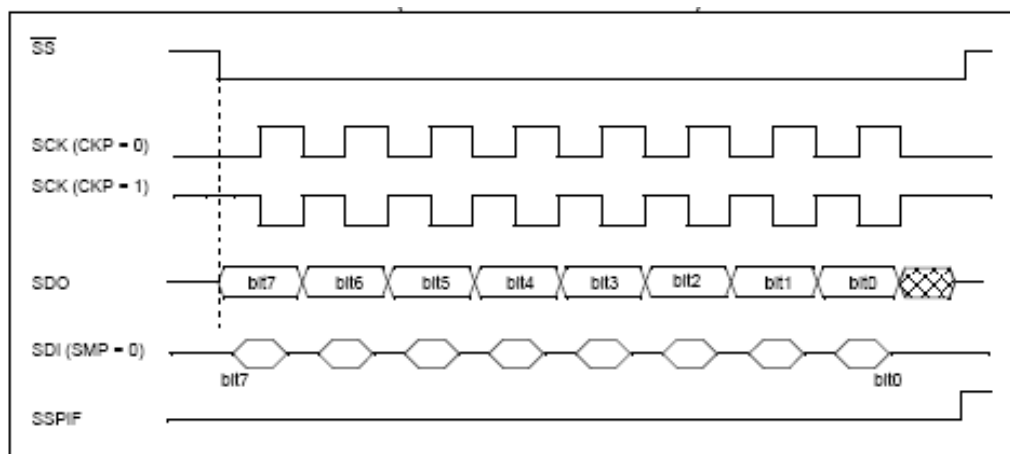


Figura 8.6: Temporização SPI no modo escravo com CKE = 1.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on: MCLR, WDT
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								xxxx xxxx	uuuu uuuu
14h	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000	0000 0000
94h	SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF	0000 0000	0000 0000

Figura 8.7: Registros associados com operação SPI

8.3 - OPERAÇÃO DA PORTA MSSP NO MODO I²C.

O módulo MSSP no modo I²C pode ser configurada como Mestre ou Escravo (incluindo suporte a chamada geral) e é provido de interrupções nos bits Start e Stop em hardware, para determinar um barramento livre (função multi-master). O módulo MSSP implementa especificações para o modo padrão bem como, endereçamento de 7 e 10 bits.

O barramento I²C é um protocolo de comunicação série síncrono desenvolvido pela Philips nos anos 80, tornando-se um padrão mundial bastante conhecido. Sua principal finalidade é permitir a comunicação entre dispositivos internos instalados em uma mesma placa de circuito impresso. Em função da evolução, é possível hoje a comunicação a grandes distâncias com o uso de circuitos integrados Extensores de Barramento I²C.

8.3.1 – Este barramento é composto de duas linhas:

- a) **SDA** – Serial Data/Address, pino 23 (RC4/SDI/SDA).
- b) **SCL** – Serial Clock, pino 18 (RC3/SCK/SCL).

Pela linha SDA trafegam dados e endereços e a linha SCL é o sinal de clock que irá sincronizar a comunicação.

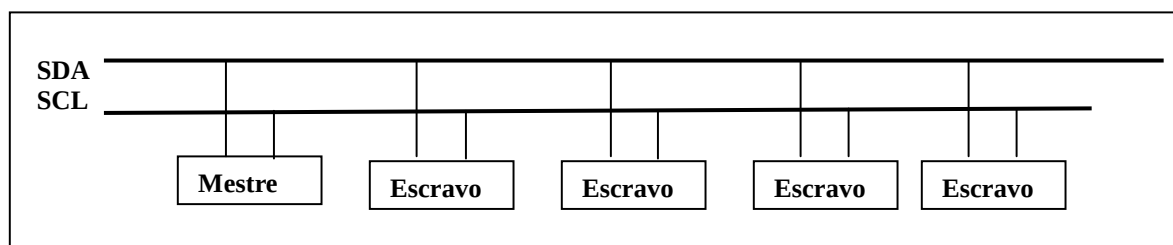


Figura 8.8: Barramento I²C com um mestre e 4 escravos.

Os dispositivos envolvidos na comunicação são denominados de Master (Mestre) e Slave (Escravo). O mestre é sempre o responsável pela geração do clock. Embora o dispositivo mestre gere o clock, além de iniciar e finalizar a transmissão, ele pode tanto transmitir quanto receber dados dos dispositivos escravos. Portanto, um dispositivo conectado em um barramento I²C pode ser identificado em uma transmissão como um dispositivo mestre-transmissor, mestre-receptor, escravo-transmissor ou escravo-receptor.

De uma forma geral, o dispositivo mestre é desempenhado por um MCU, enquanto os dispositivos escravos são periféricos como memórias, driver de LCD, conversores A/D e D/A, chip de relógio/calendário, processador de áudio, etc. MCUs também podem ser configuradas para funcionar como dispositivos escravos.

Para que o protocolo funcione corretamente, evitando conflito na linha, cada um dos dispositivos ligados à rede deve somente impor nível baixo (0), sendo o nível alto (1) imposto pelos resistores de pull-up. O hardware interno do PIC respeita essas condições, mas para isso, é necessário configurar inicialmente os pinos SDA e SCL como entradas (alta Z), possibilitando que o controle do I²C possa forçar nível baixo quando necessário. A imposição de nível baixo na linha é feita por intermédio de um transistor interno canal N. Este fato permite checar colisão de dados na linha SDA.

O número de dispositivos que pode ser conectado a este barramento é limitado pela capacitância máxima do barramento, que não pode ser superior a 400pF.

8.3.2 - Taxa de transferência.

Este barramento sofreu atualizações ao longo dos anos de modo que de 100Kb/s (Standard-mode) inicial, hoje temos 400Kb/s (Fast-mode) e 3,4Mb/s (High-mode).

8.3.3 - Endereçamento.

Uma das principais características deste barramento é o fato de o endereçamento ser feito por software. Sendo assim, a comunicação será sempre iniciada pelo dispositivo mestre, que coloca inicialmente na linha SDA o endereço do dispositivo escravo com o qual irá se comunicar. O endereço pode ser de 7 bits ou de 10 bits.

O dispositivo mestre não diferencia automaticamente o tipo de comunicação em 7 ou 10 bits, ele somente transmite e recebe informações de 8 bits. Por isso, toda a lógica de comunicação deve ser feita por intermédio do programa.

A informação que trafega na linha SDA (dado ou endereço) flui sempre em grupo de 8 bits. Se o endereço for de 7 bits, o mestre dará início a comunicação enviando um byte pela linha SDA, sendo os primeiros 7 bits o endereço do dispositivo escravo com o qual o mestre vai estabelecer a comunicação. O oitavo bit é o bit R/W que define a direção do fluxo dos dados, ou seja, se a comunicação será de escrita ou de leitura. Quando o endereço for de 10 bits, o mestre dará início a comunicação enviando pela linha SDA dois bytes, os quais conterão os 10 bits de endereço mais o bit R/W.

8.3.4 - Iniciar e finalizar uma transmissão.

As linhas SDA e SCL são bidirecionais e devem se manter conectadas à linha de alimentação Vdd por meio de resistores pull-ups para baixas taxas de transferências, ou por meio de fontes de correntes para altas taxas de transferências (High-mode). A **Figura 8.9** mostra dispositivos conectados ao barramento operando no máximo em 400kb/s.

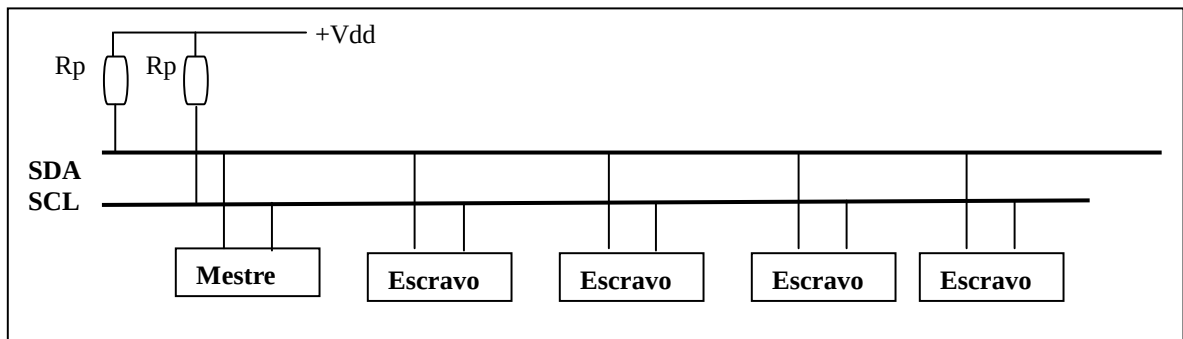


Figura 8.9: Resistores de pull-up usados nas linhas SDA e SCL.

a) Condição START (S).

O barramento está ocioso sempre que as linhas SDA e SCL estiverem em nível lógico alto. Deste modo, uma comunicação terá início quando ocorrer uma transição negativa na linha SDA, isto é, de alto para baixo enquanto a linha SCL estiver em nível lógico alto. Esta é a condição START. Após a condição START o barramento deve ser considerado ocupado, pois uma transmissão de dados está em andamento.

Pelo usuário, a condição START é iniciada quando da habilitação do bit SEN (SSPCON2<0>). Se os pinos SDA e SCL estiverem em nível alto, o gerador de

bauds é recarregado com o conteúdo do registro SSPADD<6:0> e inicia a contagem. Os dois pinos SCL e SDA serão mantidos no nível alto até o decorrer do tempo T_{BRG} (Baud rate generator times out), depois do qual o pino SDA muda para nível baixo.

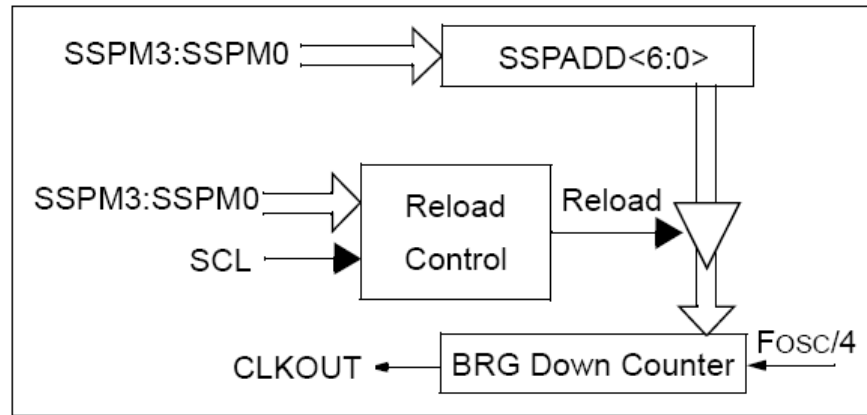


Figura 8.10: Diagrama de blocos do Gerador de Baud Rate.

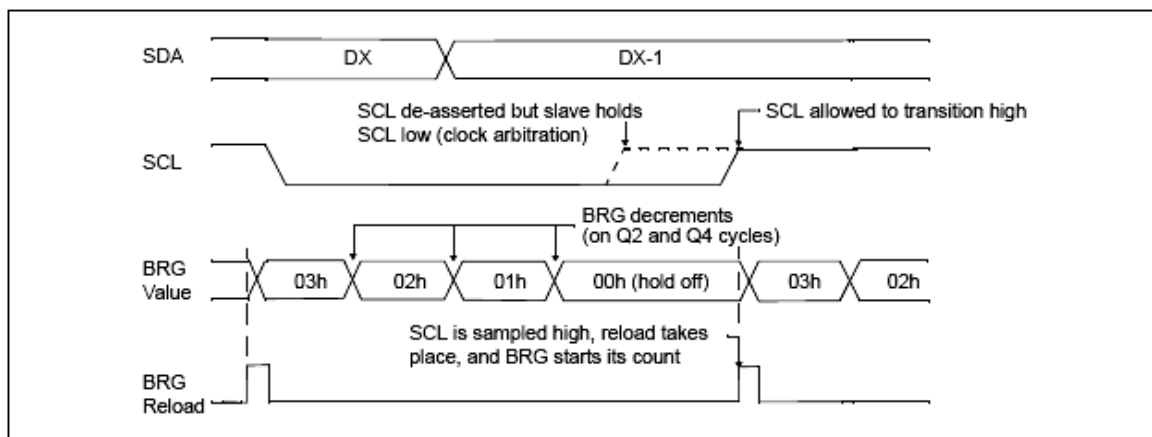


Figura 8.11: Temporização entre o gerador de Baud Rate e o clock.

Esta ação de mudar SDA para nível baixo enquanto SCL (clock) está em nível alto é a condição START e isto faz com que o bit S (SSPSTAT<3>) seja setado. Continuando assim, o gerador volta a ser recarregado com o conteúdo de SSPADD<6:0> e recomeça nova contagem.

Uma vez decorrido o tempo T_{BRG} , o bit SEN (SSPCON2<0>) é automaticamente zerado por hardware e assim, suspende-se a ação destinada a manter congelado no nível baixo o pino SDA. A condição Start está completada. Se o usuário tentar escrever no registro SSPBUF quando uma condição Start estiver em progresso, o bit WCOL (SSPCON<7>) será setado e não será carregado o registro SSPBUF.

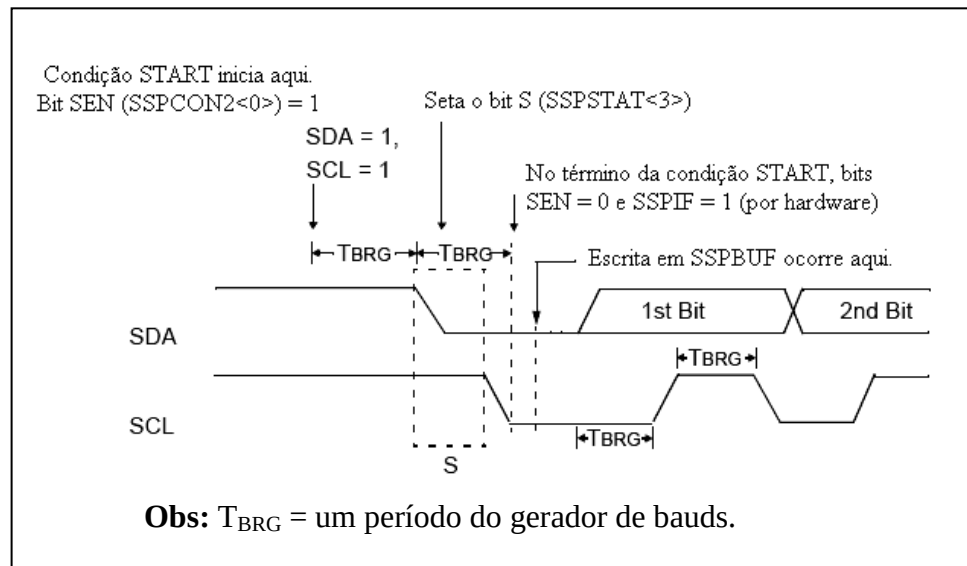


Figura 8.12: Condição START.

Após a condição START o sinal de clock é ativado pelo dispositivo mestre. A partir deste momento os bits vão sendo colocados, um a um, na linha SDA, devendo o bit da vez assumir o nível lógico correto enquanto o sinal de clock estiver em nível baixo. O receptor deve ler os bits enquanto o sinal de clock estiver em nível lógico alto, período em que o bit presente na linha SDA deve se manter obrigatoriamente estável.

Obs: Uma mudança no nível lógico de SDA enquanto SCL estiver em nível lógico alto, durante uma transmissão em andamento, será considerada uma colisão, e imediatamente a transmissão é abortada.

b) Condição STOP (P).

A condição STOP acontece quando ocorre uma transição positiva na linha SDA enquanto a linha SCL estiver em nível lógico alto, ou seja, quando SDA mudar de baixo para alto.

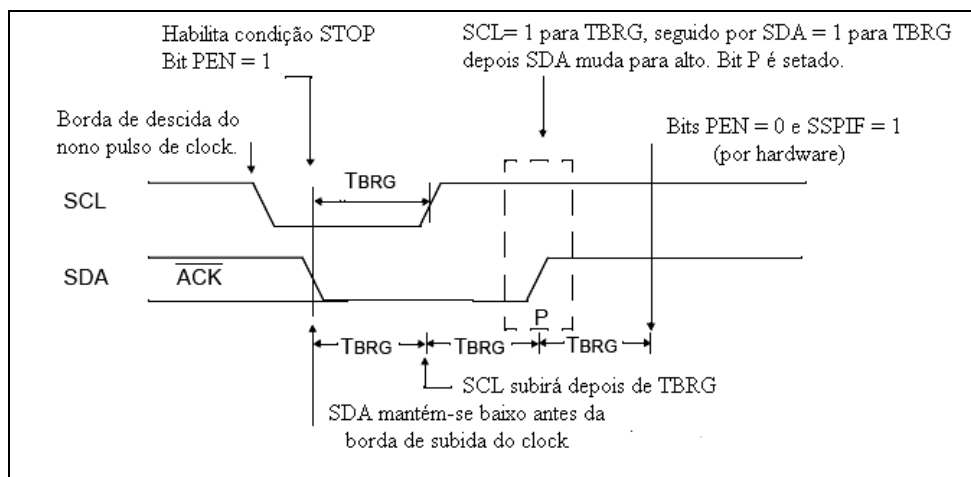


Figura 8.13: Condição STOP.

Uma fração de segundo após a condição de STOP o barramento é considerado novamente ocioso, podendo uma nova transmissão ser iniciada pelo dispositivo mestre se for gerada por ele uma nova condição START.

Durante uma transmissão a mudança no nível lógico da linha SDA ocorre enquanto a linha SCL está em nível lógico baixo. Esta é uma condição inegociável no barramento I²C.

c) Endereçamento de 7 bits (ciclo de leitura, ciclo de escrita e condição ACK).

Como visto, o endereço do dispositivo escravo e a direção do fluxo dos dados são definidos no início da transmissão, embutidos no primeiro byte enviado ao barramento I²C pelo dispositivo mestre quando o endereço for de sete bits. Os bits são colocados na linha SDA a partir do bit mais significativo. Os bytes seguintes são os dados propriamente ditos, não tendo limite para a quantidade de bytes de dados a serem transmitidos. As **Figuras 8.14 e 8.15** mostram o diagrama de tempos de duas transmissões, uma de escrita e outra de leitura, com endereço de sete bits e um byte de dados.

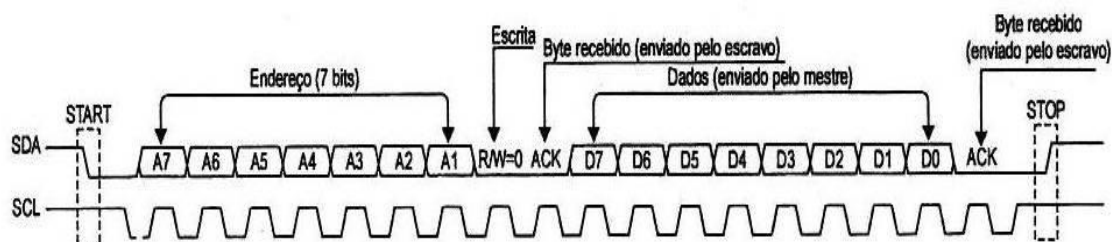


Figura 8.14: Ciclo de escrita.

Depois dos sete bits de endereço o dispositivo mestre coloca na linha SDA o bit R/W, que definirá a direção do fluxo dos dados. O dispositivo mestre fará $R/W = 1$ para leitura ou $R/W = 0$ para escrita.

Ao final de cada byte transmitido o receptor tem de enviar um pulso ACK (Acknowledge), conhecido como pulso de reconhecimento. Isto é realizado fazendo a linha SDA assumir nível lógico baixo. Nas **Figuras 8.14 e 8.15**, este pulso é gerado no nono pulso de clock, logo após a transmissão do bit R/W. O pulso ACK indica para o transmissor que um byte foi recebido com sucesso. Ocorre que no primeiro ciclo de clock colocado na linha SCL depois de cada byte transmitido, o transmissor aplica na linha SDA nível lógico alto. É nesse momento que o receptor deve acenar (indicar) para o transmissor com um pulso ACK, forçando a linha SDA assumir nível baixo se o byte foi recebido corretamente. O nível baixo aplicado na linha SDA pelo receptor predomina sobre o nível alto. A **Figura 8.16** mostra somente uma condição ACK.

Quando o transmissor é o dispositivo mestre, ele deve receber um pulso ACK após cada byte transmitido (ciclo de escrita), podendo finalizar a transmissão após a recepção de um pulso ACK impondo no barramento uma condição STOP. A transmissão também será finalizada com uma condição STOP caso o dispositivo mestre receba um pulso NACK após a transmissão de um byte. O pulso NACK indica um erro na transmissão, pois o dispositivo escravo não recebeu o byte enviado. O dispositivo mestre pode, após gerar uma condição STOP, dar início a uma nova transmissão e reenviar os dados ao dispositivo escravo, veja **Figura 8.15**.

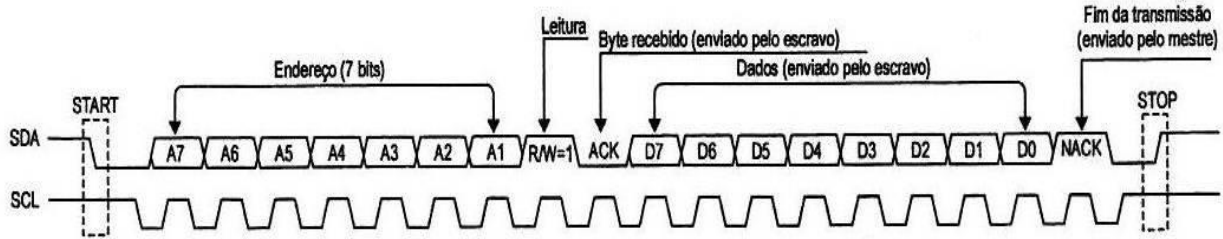


Figura 8.15: Ciclo de leitura.

Quando o dispositivo mestre estiver recebendo dados (ciclo de leitura), ele deve gerar um pulso ACK após cada byte recebido com sucesso. Se não desejar mais receber dados, ele deve enviar um pulso NACK após o último byte recebido, informando ao dispositivo escravo que a transmissão será finalizada. Após o pulso NACK o dispositivo mestre encerra a transmissão impondo uma condição STOP no barramento.

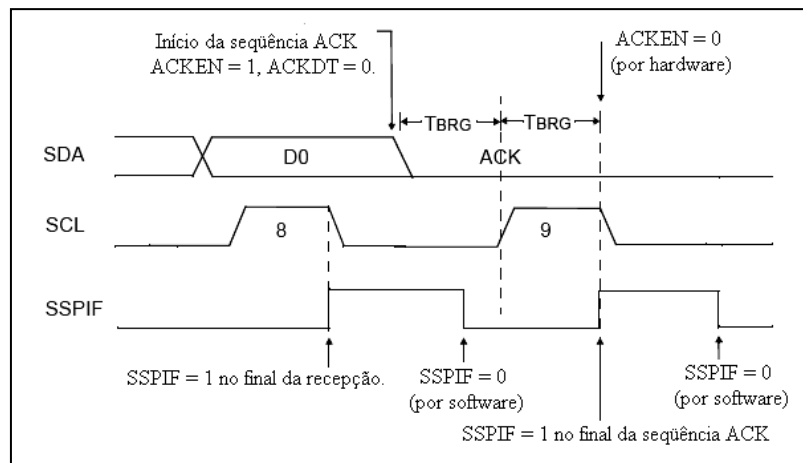


Figura 8.16: Sequência de reconhecimento (ACK).

d) Pausa na transmissão.

Como a quantidade de bytes de dados que pode ser enviada em uma única transmissão é teoricamente ilimitada, o receptor, em um determinado momento da transmissão, pode querer fazer algum tipo de processamento com os dados já recebidos, ou executar uma outra tarefa como atendimento a uma interrupção. Neste caso o receptor pode efetuar uma pausa na transmissão, forçando a linha SCL a assumir nível baixo até quando ele estiver novamente apto a receber novos bytes de

dados. A transmissão será retomada quando o nível baixo for retirado da linha, como na **Figura 8.17**.

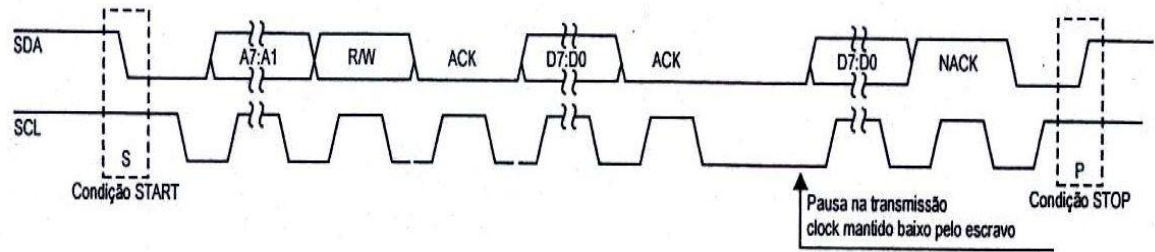


Figura 8.17: Pausa na transmissão.

e) Condição Re-START (Sr).

O barramento I²C permite que seja alterada a direção do fluxo dos dados (alterar de um ciclo de leitura para um ciclo de escrita ou vice-versa) durante uma transmissão em andamento sem que o dispositivo mestre precise antes finalizar a transmissão atual com uma condição STOP. Isto é permitido impondo uma condição Re-START no barramento após o último byte transmitido.

A condição Re-START, é quase idêntica a uma condição START. A diferença reside no fato de que a condição Re-START é gerada pelo dispositivo mestre logo após a transmissão de um byte de dados, ou seja, durante uma transmissão em andamento.

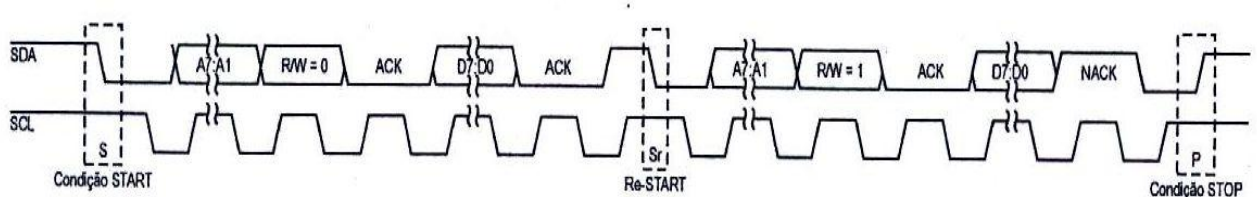


Figura 8.18: Condição Re_START.

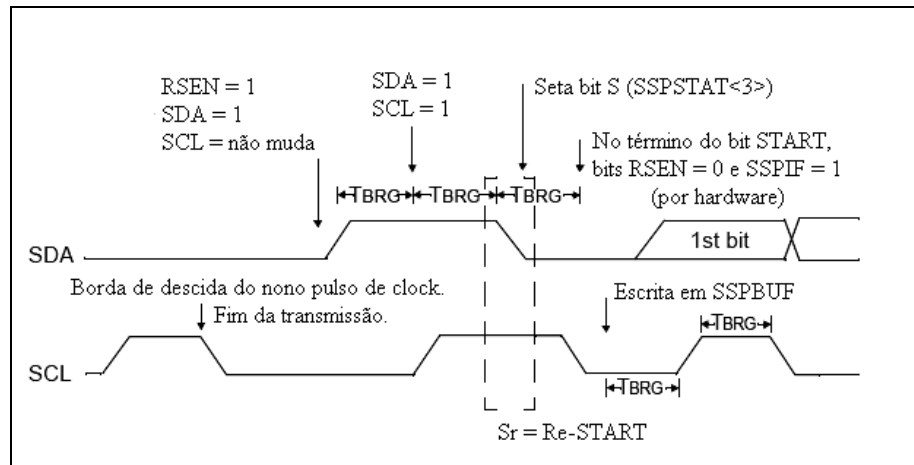


Figura 8.19: Ampliando a condição Re-START.

Veja na **Figura 8.18** que o dispositivo mestre alterou o fluxo dos dados de escrita para leitura em uma única transmissão. Se este recurso for utilizado de forma conveniente, pode contribuir para um aumento da velocidade de transferência dos dados e melhora na performance do sistema.

f) Endereçamento de 10 bits.

Como visto, o barramento I²C permite ao dispositivo mestre se comunicar com dispositivos escravos endereçados com 7 bits ou 10 bits, podendo inclusive o barramento ser compartilhado por dispositivos com 7 bits e 10 bits de endereço. Assim, é correto afirmar que o endereçamento de 10 bits não afeta o endereçamento de 7 bits.

O endereçamento de um dispositivo escravo com 10 bits de endereço é efetuado pelos dois primeiros bytes enviados ao barramento pelo dispositivo mestre. Após a condição START são enviados os bits 11110<A9:A8> seguidos pelo bit R/W (byte mais significativo), que definirá a direção do fluxo de dados. O dispositivo escravo que identificar esse primeiro byte como parte do seu endereço deve gerar um pulso ACK. O byte seguinte contém os 8 bits (byte menos significativo) do endereço (A7:A0). O dispositivo escravo endereçado deve gerar um novo pulso ACK. Os bytes que vão trafegar em seguida no barramento serão de dados.

Quando o transmissor é o dispositivo mestre, ele deve receber um pulso ACK após cada byte transmitido (ciclo de escrita), podendo finalizar a transmissão após a

recepção de um pulso ACK impondo no barramento uma condição STOP. A transmissão também será finalizada com uma condição STOP caso o dispositivo mestre receba um pulso NACK após a transmissão de um byte. O pulso NACK indica um erro na transmissão, pois o dispositivo escravo não recebeu o byte enviado. O dispositivo mestre pode gerar uma condição STOP e após, dar início a uma nova transmissão e reenviar os dados ao dispositivo escravo.

Quando o dispositivo mestre estiver recebendo dados (ciclo de leitura), ele deve gerar um pulso ACK após cada byte recebido com sucesso. Se não desejar mais receber dados, ele deve enviar um pulso NACK após o último byte recebido, informando ao dispositivo escravo que a transmissão será finalizada. Após o pulso NACK o dispositivo mestre encerra a transmissão impondo uma condição STOP no barramento.

Caso o receptor precise de tempo para processar alguma informação durante a transmissão, ele pode gerar uma pausa na transmissão forçando a linha SCL a assumir nível lógico baixo antes da transmissão de um novo byte, como visto para o endereçamento de 7 bits.

S	1 1 1 1 0 A9 A8 R/W	ACK	A7 A6 A5 A4 A3 A2 A1 A0	ACK	DADO	ACK	DADO	ACK	P
	endereço 0		endereço						

Figura 8.20: Transmissão com endereço de 10 bits.

Obs: START, endereços, dados e STOP são gerados pelo mestre enquanto ACK é gerado pelo escravo.

A **Figura 8.20** mostra uma transmissão com endereçamento de 10 bits. A comunicação apresenta um ciclo de escrita ($R/W = 0$). Após o endereçamento foram enviados para o receptor que é o dispositivo escravo, dois bytes de dados.

Quando uma transmissão que envolve um dispositivo escravo com 10 bits de endereço é um ciclo de leitura, é necessário que seja feita uma mudança de direção no fluxo dos dados após o endereçamento. Esta mudança é obtida com uma condição Re-START. A **Figura 8.21** mostra uma comunicação de leitura entre um dispositivo mestre e um escravo. Observe que após a condição Re-START o

dispositivo mestre só precisa reenviar o primeiro byte do endereço com a mudança de direção no fluxo dos dados ($R/W = 1$).

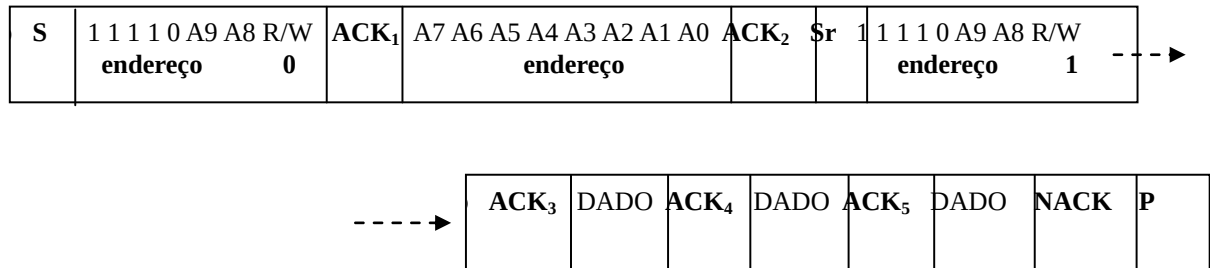


Figura 8.21: Mestre recebe com endereço de 10 bits (condição Re-START).

Obs: START, endereços, Re-START, ACK₄, ACK₅, NACK e STOP são gerados pelo mestre enquanto ACK₁, ACK₂, ACK₃ e DADOS são gerados pelo escravo.

8.3.5 - Endereço de Chamada Geral.

Este barramento possui um recurso que permite ao dispositivo mestre endereçar todos os dispositivos escravos simultaneamente. Isto é feito por meio do Endereço de Chamada Geral que é usado quando o dispositivo mestre quiser enviar dados para todos os dispositivos escravos simultaneamente. É um tipo de comunicação conhecido como Broadcast. Assim, deduzimos que os dispositivos escravos possuem dois endereços. Um deles é o seu endereço no barramento I²C (registro SSPADD) e o outro é o endereço de chamada geral, que é 0000h.

Esta chamada geral tem lugar quando o endereço enviado tem todos os bits a zero, inclusive R/W, e é reconhecida se o bit GCEN (SSPCON2<7>) = 1

Quando o endereço geral é inserido na rede, todos os Escravos irão receber a informação e todos aqueles com GCEN deverão responder com ACK. Quando ACK = 0, o Escravo coloca SDA como saída em nível baixo. Quando ACK = 1, SDA continua como entrada e o nível alto é imposto pelo resistor de pull-up. Isto possibilita que diversos escravos respondam ACKs diferentes, tendo sempre como preferência o erro, ou seja, ACK = 0.

Com a habilitação do endereço geral a interrupção será gerada e o tratamento ou não das informações será escolha do programador.

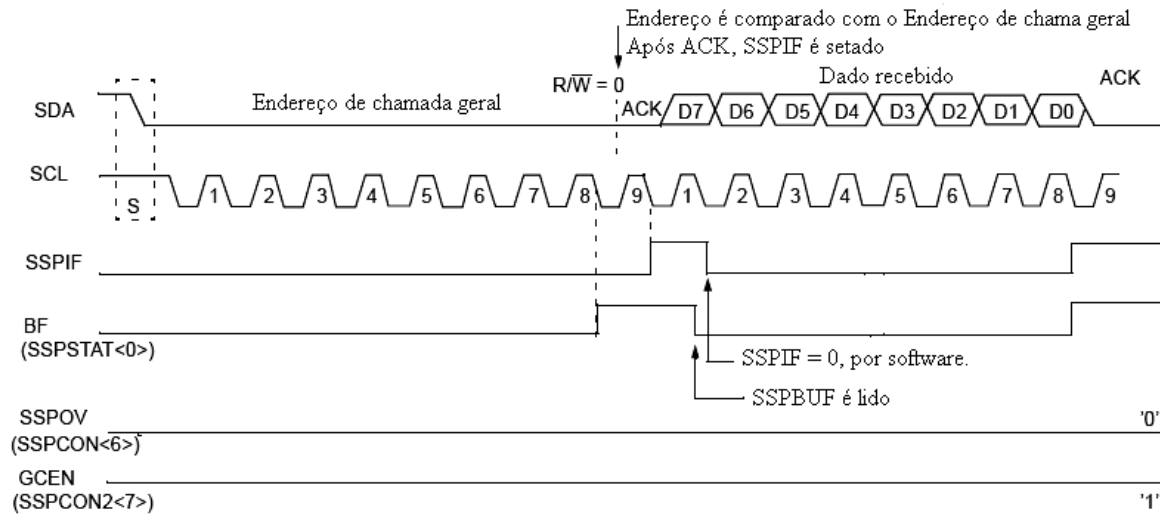


Figura 8.22: Chamada geral (modo escravo 7 ou 10 bits).

8.3.6 - Operação Multi-Master.

O barramento I²C permite a conexão de vários dispositivos mestres no mesmo barramento (múltiplos mestres) de forma que todos possam transmitir e receber informações, porém não simultaneamente. Isto significa que apenas um dos dispositivos mestres pode assumir o controle do barramento de cada vez e gerar seu próprio sinal de clock.

Quando mais de um mestre tenta no mesmo instante assumir o controle do barramento para efetuar uma transmissão, ocorre uma Colisão. Um procedimento denominado “*Arbitration*” assegura que apenas um mestre irá concluir a transmissão e os demais irão abortar a transmissão à medida que vão perdendo a “*arbitration*”.

O procedimento de “*arbitration*” ocorre na linha SDA enquanto a linha SCL estiver em nível lógico alto. Veja na **Figura 8.23**.

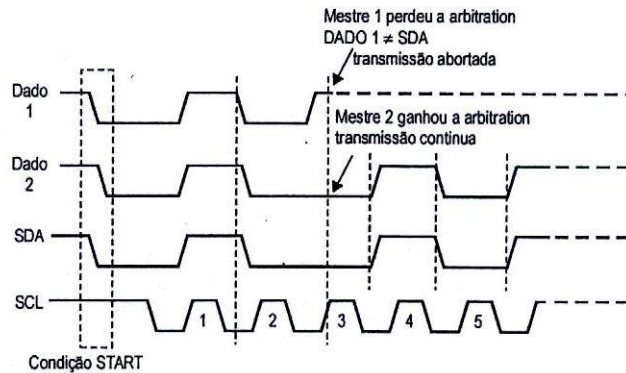


Figura 8.23: Operação com Múltiplos Mestres.

a) Na **Figura 8.21** temos:

- I)** O gráfico DADO1 mostra o sinal que o dispositivo mestre 1 está tentando aplicar na linha SDA.
- II)** O gráfico DADO2 mostra o sinal que o dispositivo mestre2 está tentando aplicar na linha SDA.
- III)** O gráfico SDA exibe o sinal que irá realmente aparecer na linha SDA. Observe que mestre 1 perdeu a “*arbitration*” no terceiro pulso de clock, uma vez que ele aplicou nível lógico 1 na linha SDA, tendo prevalecido o nível lógico baixo aplicado na linha SDA pelo dispositivo mestre2.

O dispositivo mestre que perdeu a arbitration deve tentar uma nova transmissão quando o barramento estiver novamente ocioso. A “*arbitration*” garante que dados não sejam perdidos quando dois ou mais mestres tentam assumir o controle do barramento no mesmo instante.

O procedimento de “*arbitration*” não é permitido durante as condições START, STOP e Re-START.

8.4 - CONFIGURANDO O MCU COMO DISPOSITIVO MESTRE.

8.4.1 - Antes de ativar a interface I²C do módulo MSSP temos:

- a)** Configurar pinos SDA e SCL como entrada.
- b)** O bit CKE (SSPSTAT<6>) configura o tipo de Smith Trigger utilizado nos pinos SCL e SDA.

CKE = 0, entradas conforme especificações I²C.

CKE = 1, entradas conforme especificações SMBUS.

- c) Configurar o modo de funcionamento da interface I²C através dos bits
SSPM3:SSPM0 = 1000 (SSPCON<3:0>).
- d) Habilitar/desabilitar bit SMP (SSPSTAT<7>). Serve para habilitar ou desabilitar um sistema interno de tratamento de bordas, quando operando em frequência de 400KHz.
- e) Habilitar a porta série síncrona setando o bit SSPEN (SSPCON<5>).
- f) A frequência de clock deve ser configurada através do registro SSPADD, respeitando-se a fórmula abaixo.

Calcular, pela fórmula abaixo, o valor para uma dada taxa de transferência que será armazenado no registro SSPADD.

$$\text{SSPADD} = [(F_{\text{osc}} / \text{Bit rate}) / 4] - 1$$

Fosc = frequência do oscilador principal

Bit rate = taxa de transferência

Exemplo: Calcule o valor que será armazenado no registro SSPADD onde Fosc é de 4MHz e Taxa de transferência de 100Kb/s.

$$\text{SSPADD} = [(4000000 / 100000) / 4] - 1$$

$$\text{SSPADD} = (40 / 4) - 1$$

$$\text{SSPADD} = 10 - 1$$

$$\text{SSPADD} = 9.$$

Obs: Todos os periféricos da rede devem estar áptos a responder na frequência escolhida.

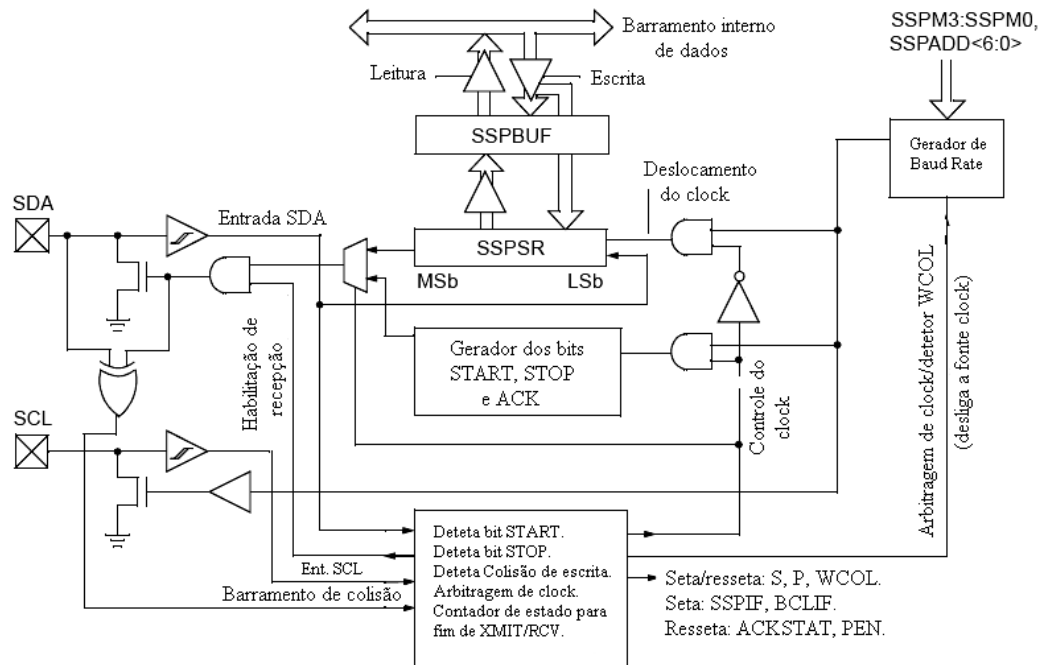


Figura 8.24: SSP configurado como I²C no modo Mestre.

O fragmento de código a seguir mostra como configurar o módulo MSSP para que o MCU funcione como dispositivo mestre, transmitindo a uma taxa de transferência de 100Kb/s. O controle de inclinação (slew rate control) foi desativado, sua ativação só é indicada para taxas de 400Kb/s.

```

bank1
movlw    b'00011000'
iorwf    TRISC,F      ;configura pinos SDA e SCL como entrada
bank0
movlw    b'00101000'
movwf    SSPCON        ;configura dispositivo como mestre <3:0>
                        ;habilita a porta serial e configura pinos SDA e SCL <5>

bank1
movlw    .9
movwf    SSPADD         ;bit rate = 100Kb/s e Fosc = 4MHz
movlw    b'10000000'
movwf    SSPSTAT        ;controle slew rate desabilitado, e
                        ;níveis de entrada conforme especificações I2C

bank0

```

8.4.2 - Verificação se o barramento está livre.

Uma vez que o MCU foi configurado para operar no modo mestre, não podemos esquecer de que ele irá controlar o barramento inicializando e finalizando qualquer transmissão. Para inicializar uma transmissão, o MCU precisa se certificar de que a interface I²C está liberada, ou seja, não está envolvida em alguma transmissão. Se existir conectado no barramento apenas um dispositivo mestre, o barramento estará livre se a interface I²C estiver liberada.

A sub-rotina abaixo, mostra uma forma genérica de verificar se o barramento está livre, considerando que só existe um dispositivo mestre. O teste é efetuado no bit R/W (SSPSTAT<2>) e nos bits ACKEN, RCEN, PEN, RSEN e SEN (SSPCON2<4:0>), que devem estar todos apagados.

I2C_LIVRE		
bank1		
btfsc	SSPSTAT,R_W	;transmissão em andamento?
goto	BUS_OCUPADO	;sim, desvia
movf	SSPCON2,W	;não, W = SSPCON2
andlw	H'1F'	;W = SSPCON2 and 1Fh
btfss	STATUS,Z	;barramento liberado?
goto	BUS_OCUPADO	;não, desvia
bank0		;sim, seleciona banco0
retlw	.1	;retorna com W = 1
BUS_OCUPADO		
bank0		
retlw	.0	;retorna com W = 0

a) Pela sub-rotina I2C livre, temos:

- I)** Barramento ocupado, a sub-rotina retorna ao ponto de chamada com W = 0.
- II)** Barramento livre, a sub-rotina retorna ao ponto de chamada com W = 1. Neste caso, o MCU pode dar início a uma nova transmissão.
- III)** O bit R_W (SSPSTAT<2>) = 0, informa que uma transmissão não está em progresso (primeiro teste).
- IV)** Para um barramento livre é necessário ainda que os bits SSPCON2<4:0> = 0 (apagados). Este é o segundo teste efetuado pela sub-rotina.

8.4.3 - Dispositivo Mestre Transmite com endereçamento de 7 bits.

Uma transmissão pode ser iniciada se o barramento estiver livre, ou seja, se a interface I²C estiver liberada. Ela consiste em uma série de eventos que devem ser executados em seqüência, começando com a condição START e terminando com a condição STOP. Não é permitido iniciar um evento sem que o anterior tenha terminado.

a) Uma MCU configurada como dispositivo mestre pode dar início a um dos eventos descritos a seguir:

- I)** Gerar/monitorar condição START.
- II)** Gerar/monitorar condição Re-START.
- III)** Iniciar a transmissão de um byte escrevendo no registro SSPBUF (mestre-transmissor).
- IV)** Configurar a interface I²C para receber dados (mestre-receptor).
- V)** Gerar/monitorar pulso ACK (ou um NACK) no final de um byte recebido (mestre-receptor).
- VI)** Gerar/monitorar condição STOP e finalizar a transmissão.
- VII)** Gerar/monitorar o clock.
- VIII)** Transmitir ou receber informações de 8 bits.
- IX)** etc.

A condição START é gerada quando o bit SEN (SSPCON2<0>) é setado. Este bit é automaticamente apagado pelo hardware no final da condição START, sendo que neste momento o bit SSPIF (PIR1<3>) é setado. Uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

Sub-rotina para iniciar a condição START e detetar seu fim.

IIC_START

```

bank1
bsf    SSPCON2,SEN ;inicia condição START
btfsc  SSPCON2,SEN ;condição START terminou?
goto   $-1         ;não, testa novamente
bank0  ;sim, seleciona banco0
return ;retorna

```

A condição Re-START é gerada quando o bit RSEN (SSPCON2<1> é setado. Este bit é automaticamente apagado pelo hardware no final da condição Re-START, sendo que neste momento o bit SSPIF (PIR1<3>) é setado. Uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

A sub-rotina abaixo mostra como iniciar a condição Re_START e detetar seu final.

I2C_RESTART

```
bank1
bsf    SSPCON2,RSEN ;inicia condição Re_START
btfsc  SSPCON2,RSEN ;Re-START terminou?
goto   $-1          ;não, testa novamente
bank0
        ;sim, seleciona banco0
return
```

Depois de gerada a condição START ou Re_START, o byte a ser transmitido deve ser armazenado no registro SSPBUF e logo após, transferido para o registro de deslocamento SSPSR de onde os bits são colocados, um a um, na linha SDA. No momento em que o registro SSPBUF recebe o dado a ser transmitido, é acionado o circuito gerador de clock. Para cada bit transmitido é produzido um ciclo de clock, de forma que são necessários oito ciclos de clock para a transmissão de um byte. Nesse instante, o bit BF SSPSTAT<0> é setado (e se mantém assim até o oitavo pulso de clock quando é apagado pelo hardware), o bit R/W também é setado indicando transmissão em progresso (mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware). Ver **Figura 8.25**.

O bit SSPIF (PIR1<3>) é setado no final do nono pulso de clock, e uma interrupção será gerada se ela estiver habilitada.

Se durante a transmissão de um novo byte, enquanto ele estiver sendo transferido do registro SSPSR para o barramento (transmissão em andamento), ocorrer uma escrita no registro SSPBUF, o bit WCOL (SSPCON<7>) será setado para indicar que houve uma tentativa de escrita no registro SSPBUF num momento não permitido. Este bit precisa ser apagado por software.

A sub-rotina abaixo mostra como inicializar a transmissão de um byte e detetar o seu fim. O dado a ser transmitido é carregado no registo SSPBUF a partir do registo DADO_I²C.

```

I2C_TRANSMITE
    bank0
    movf    DADO-I2C,W
    movwf   SSPBUF
    bank1
    btfsc   SSPSTAT,R_W    ;transmissão em progresso?
    goto    $-1             ;sim, testa novamente
    bank0
    return                ;não, seleciona banco0

```

Depois de cada byte transmitido o mestre precisa receber do dispositivo escravo um bit de reconhecimento, pulso ACK (ou bit ACK). O bit ACK é recebido no nono pulso de clock e armazenado no bit ACKSTAT (SSPCON2<5>).

ACKSTAT = 0, dispositivo escravo recebeu o byte transmitido com sucesso.

ACKSTAT = 1, dispositivo mestre deve abortar a transmissão gerando um STOP, o escravo não deseja mais receber dados, pois ocorreu um erro na escrita, como, por exemplo, um estouro no buffer de recepção. Sendo assim, o dispositivo mestre precisa testar o bit ACK para tomar a decisão certa em relação à continuidade ou não da transmissão.

A sub-rotina abaixo testa o bit ACK e retorna o valor lido no registo W.

```

I2C_TESTA_ACK
    bank1
    btfsc   SSPCON2,ACKSTAT ;escravo respondeu com ACK = 0?
    goto    ABORTA_TRANSM   ;não, desvia
    bank0
    retlw   .0              ;sim, seleciona banco0
                                ;retorna com W = 0

ABORTA_TRANSM
    bank0
    retlw   .1              ;retorna com W =1

```

A condição STOP é gerada quando o bit PEN (SSPCON2<2>) é setado. Este bit é automaticamente apagado pelo hardware no final da condição STOP, sendo que

neste momento o bit SSPIF (PIR1<3>) é setado. Uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

Sub-rotina abaixo mostra como iniciar a condição STOP e detetar seu fim.

IIC_STOP		
bank1		
bsf	SSPCON2,PEN	;inicia condição STOP
btfsc	SSPCON2,PEN	;condição STOP terminou?
goto	\$-1	;não, testa novamente
bank0		;sim, seleciona banco0
return		

b) Detalhando uma transmissão com endereço de 7 bits:

I) Transmissão do endereço do escravo.

- A) Configurar SCL e SDA como entradas.
- B) Deve ser imposta uma condição START na linha, setando o bit SEN (SSPCON2<0>).
- C) Ao término da condição START o bit SEN será apagado e o flag de interrupção bit SSPIF (PIR1<3>) será setado, podendo gerar uma interrupção. Este bit deve ser apagado por software.
- D) O byte a ser transmitido, endereço do periférico com o qual desejamos nos comunicar (escravo), deve ser armazenado no registro SSPBUF<7:1> e após transferido para o registro de deslocamento SSPSR para então ser enviado ao barramento através da linha SDA. O bit 0 é R/W = 0. Ver **Figura 8.25**.
- E) No momento em que o registro SSPBUF recebe o dado a ser transmitido, é acionado o circuito gerador de clock, que gera um ciclo de clock para cada bit transmitido (8 ciclos de clock para a transmissão de um byte).
- F) No mesmo instante em que o registro SSPBUF recebe o dado a ser transmitido, o bit BF (SSPSTAT<0>) é setado e se mantém assim até a transmissão do oitavo bit (7 bits de endereço + bit R/W) ou oitavo pulso de clock, quando é automaticamente apagado pelo hardware (BF = 0). Ver **Figura 8.25**.
- G) O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do

nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.

- H)** O byte será transmitido nos oito pulsos de clock em SCL e no final $BF = 0$.
- I)** O nível de ACK, recebido do periférico (escravo) no nono pulso de clock, que deve ser $ACK = 0$, é armazenado no bit ACKSTAT (SSPCON2<6>).
- J)** No final do nono pulso de clock o bit R/W será zerado. O bit SSPIF (PIR1<3>) é setado no final do nono pulso de clock, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

II) Transmissão do dado.

- A)** O byte a ser transmitido (dado de 8 bits) deve ser armazenado no registro SSPBUF e após, transferido para o registro de deslocamento SSPSR para então ser enviado ao barramento através da linha SDA.
- B)** No momento em que o registro SSPBUF recebe o dado a ser transmitido, é acionado o circuito gerador de clock, que gera um ciclo de clock para cada bit transmitido (8 ciclos de clock para a transmissão de um byte).
- C)** No mesmo instante em que o registro SSPBUF recebe o dado a ser transmitido, o bit BF (SSPSTAT<0>) é setado e se mantém assim até a transmissão do oitavo bit de dados (oitavo pulso de clock), quando é automaticamente apagado pelo hardware ($BF = 0$).
- D)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- E)** O byte será transmitido nos oito pulsos de clock em SCL e no final $BF = 0$.
- F)** O nível de ACK, recebido do periférico (escravo) no nono pulso de clock, que deve ser $ACK = 0$, é armazenado no bit ACKSTAT (SSPCON2<6>).
- G)** No final do nono pulso de clock o bit R/W será zerado. O bit SSPIF (PIR1<3>) é setado no final do nono pulso de clock, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

- H)** Se for necessário transmitir um novo byte, o processo é reiniciado no **ítem II) (A)**. Para interromper a transmissão, uma condição de STOP deve ser gerada. Isto é conseguido setando o bit PEN (SSPCON2<2>).
- I)** No final da condição STOP teremos o bit PEN = 0 (por hardware). O bit PSPIF será setado e uma interrupção será gerada se ela estiver habilitada. Este bit deve ser apagado por software.
- A **Figura 8.25** mostra o diagrama de tempo I²C modo Mestre com transmissão de um byte com endereçamento de 7 bits.

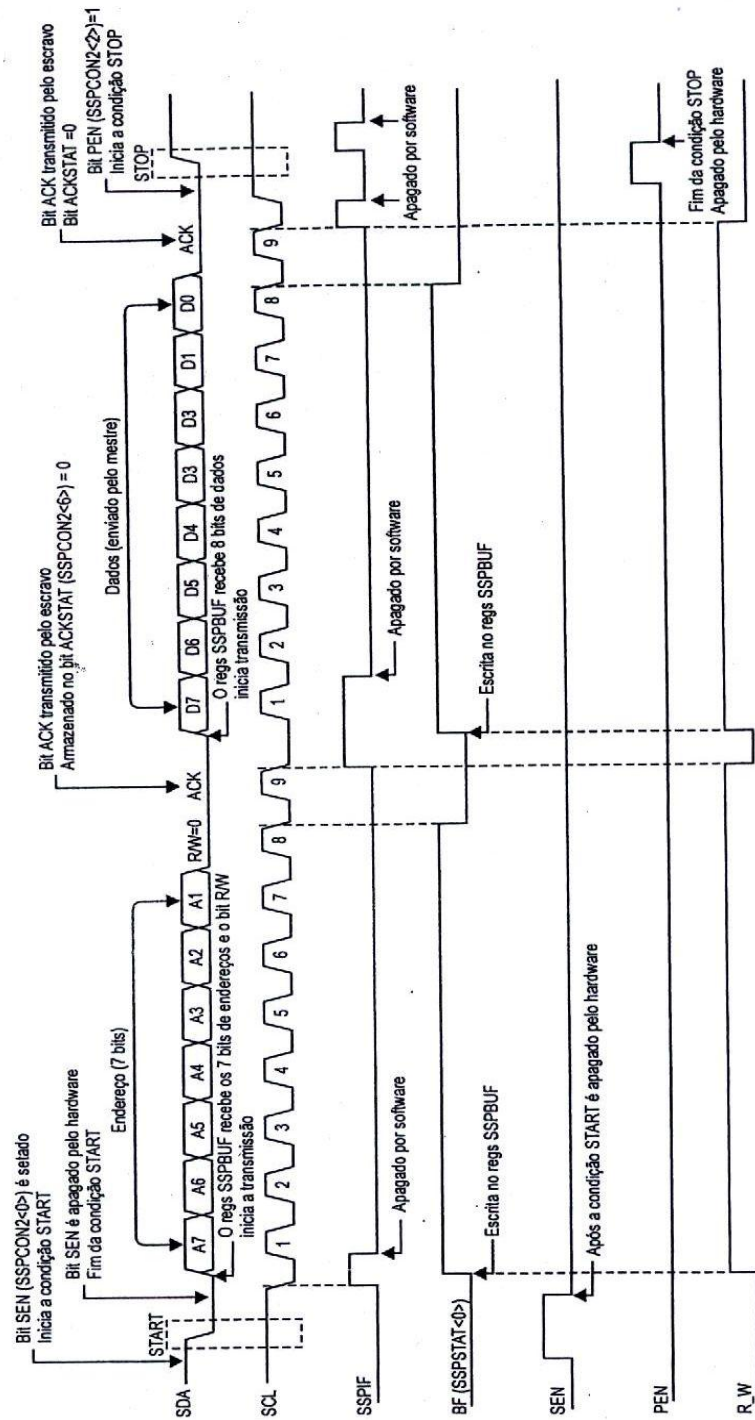


Figura 8.25: Temporização I²C modo Mestre na transmissão e endereço de 7 bits.

A sub-rotina seguinte pode ser usada para transmitir um byte de dados, cuja temporização é vista na **Figura 8.25**. Com adaptações, pode transmitir mais bytes. O byte de dados a ser transmitido deve ser armazenado no registro

DADO_I2C antes de ser chamada a sub-rotina I2C_TRANSMITE, pois é desse registro que será obtido o byte a ser transmitido.

Veja que após a transmissão de cada byte foi feito um teste no bit ACK enviado pelo escravo. Caso ACK = 1, a transmissão será abortada e o bit I2C_ERRO será setado indicando que ocorreu um erro na transmissão.

;Esta sub-rotina transmite um byte com endereçamento de 7 bits utilizando a comunicação série síncrona I2C.

I2C_ESCRITA

```

call    I2C_LIVRE          ;verifica se o barramento está livre, na sub-rotina
xorlw   .0
btfsc   STATUS,Z           ;barramento livre?
return  ;não,
call    IIC_START          ;sim, chama sub-rotina que gera condição START
bcf     STATUS,C           ;apaga bit C
rlf     ENDERECO,F         ;rotaciona registro ENDERECO para a esquerda
movf    ENDERECO,W         ;W recebe ENDERECO
movwf   DADO_I2C           ;registro DADO_I2C recebe endereço do dispositivo escravo
call    I2C_TRANSMITE      ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK      ;chama sub-rotina que testa bit ACK
xorlw   .0
btfss   STATUS,Z           ;escravo endereçado respondeu com bit ACK = 0?
goto    ERRO_I2C           ;não, desvia
movf    DADO,W             ;sim, W recebe registro DADO
movwf   DADO_I2C           ;registro DADO_I2C recebe o byte de dados que será transmitido
call    I2C_TRANSMITE      ;chama sub-rotina, transmite byte de dados
call    I2C_TESTA_ACK      ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z           ;escravo recebeu byte de dados, ACK = 0?
goto    ERRO_I2C           ;não, desvia
goto    FIM_TRANSMISSAO    ;sim, desvia
ERRO_I2C
bsf     I2C_ERRO           ;seta flag de sinalização de erro
FIM_TRANSMISSAO
call    IIC_STOP           ;chama sub-rotina, gera condição de STOP
return  ;retorna

```

8.4.4 - Dispositivo Mestre Recebe com endereçamento de 7 bits.

Para que um MCU operando como dispositivo mestre receba dado, ele precisa ser configurado como mestre-receptor. Isto é feito setando o bit RCEN (SSPCON2<3>). O bit RCEN é apagado pelo hardware na borda de descida do oitavo pulso de clock. O byte recém chegado, que está armazenado no registro SSPSR, é carregado no registro

SSPBUF, o bit BF (SSPSTAT<0>) é setado, o bit SSPIF (PIR1<3>) é setado e a linha SCL é mantida em nível baixo até que a interface receba um novo comando. Ver **Figura 8.26**.

A sub-rotina abaixo, mostra como configurar o dispositivo mestre como mestre-receptor para receber um novo byte e detetar o fim da recepção. O dado recebido é carregado no registro DADO_I2C.

I2C_RECEBE

```

bank1
bsf      SSPCON2,RCEN ;ativa mestre-receptor
btfsc    SSPCON2,RCEN ;recepção em progresso?
goto     $-1          ;sim, testa novamente
bank0    ;não, seleciona banco0
movf     SSPBUF,W      ;W recebe SSPBUF
movwf    DADO_I2C      ;DADO_I2C = SSPBUF
return

```

Após a chegada de um novo byte o dispositivo mestre deve ativar a sequência “*acknowledge*” para que possa ser transmitido o bit ACK. Se desejar receber mais dados ele deve gerar um bit ACK = 0, caso contrário, ele deve gerar um bit NACK (ACK = 1). Para transmitir o bit acknowledge, é preciso setar o bit ACKDT (bit NACK) ou apagar o bit ACKDT (bit ACK). Depois é preciso dar início à sequência ACK setando o bit ACKEN, que será automaticamente apagado pelo hardware na borda de descida do nono pulso de clock. O bit SSPIF (PIR1<3>) também é setado na borda de descida do nono pulso de clock, sendo gerada uma interrupção se ela estiver habilitada. Este bit deve ser apagado por software.

A sub-rotina abaixo mostra como gerar o bit ACK (ACKDT = 0) na sequência *acknowledge* e como detetar seu fim.

I2C_ACK

```

bank1
bcf      SSPCON2,ACKDT ;carrega bit ACK
bsf      SSPCON2,ACKEN ;inicia sequência ACK
btfsc    SSPCON2,ACKEN ;sequência ACK terminou?
goto     $-1          ;não, testa novamente
bank0    ;sim, seleciona banco0
return

```

A sub-rotina mostra como gerar o bit NACK (ACKDT = 1) na sequência *acknowledge* e como detetar seu fim.

I2C_NACK		
bank1		
bsf	SSPCON2,ACKDT	;carrega bit NACK
bsf	SSPCON2,ACKEN	;inicia sequência ACK
btfsc	SSPCON2,ACKEN	;sequência ACK terminou?
goto	\$-1	;não, testa novamente
bank0		;sim, seleciona banco0
return		

Na recepção o bit BF (SSPSTAT<0>) setado, indica que o registro SSPBUF está cheio, sendo esse bit apagado quando o registro SSPBUF for lido. Um novo byte pode chegar sem que o registro SSPBUF tenha sido lido. Porém, se o último bit do byte chegar no registro SSPSR sem que o byte que chegou anteriormente tenha sido lido do registro SSPBUF, o bit SSPOV (SSPCON<6>) será setado indicando que houve estouro. Esta situação deve ser evitada para que não haja perda de dados.

Se após a chegada de um novo byte, enquanto ele estiver sendo transferido do registro SSPSR para o SSPBUF, ocorrer uma escrita via programa no registro SSPBUF, o bit WCOL (SSPCON<7>) será setado indicando que houve uma tentativa de escrita no registro SSPBUF em um momento não permitido (colisão). Este bit precisa ser apagado por software.

a) Detalhando uma recepção com endereço de 7 bits:

I) Transmissão do endereço do escravo:

- A)** Configurar como entrada as linhas SCL e SDA.
- B)** Deve ser imposta uma condição START setando o bit SEN (SSPCON2<0>).
- C)** Ao término da condição START o bit SEN será apagado e o flag de interrupção bit SSPIF (PIR1<3>) será setado, podendo gerar uma interrupção se ela estiver habilitada. Este bit deve ser apagado por software.
- D)** O byte a ser transmitido (endereço do escravo) deve ser armazenado no registro SSPBUF<7:1> e após transferido para o registro de deslocamento SSPSR para então ser enviado ao barramento através da linha SDA. O bit 0 é R/W = 1 indicando leitura. Ver **Figura 8.26**.

- E)** No momento em que o registro SSPBUF recebe o dado a ser transmitido, é acionado o circuito gerador de clock, que gera um ciclo de clock para cada bit transmitido (8 ciclos de clock para a transmissão de um byte).
- F)** No mesmo instante em que o registro SSPBUF recebe o dado a ser transmitido, o bit BF (SSPSTAT<0>) é setado e se mantém assim até a transmissão do oitavo bit (7 bits de endereço + bit R/W) ou oitavo pulso de clock, quando é automaticamente apagado pelo hardware (BF = 0). Ver **Figura 8.26.**
- G)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- H)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- I)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- J)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

II) Recepção do dado:

- A)** O mestre deve, então, ser colocado em modo de recepção (mestre-receptor) setando o bit RCEN (SSPCON2<3>) e tem início a transferência do dado.
- B)** Se a linha de clock não estiver em pausa, será recebido um byte de dados para os próximos oito pulsos do clock. Como o byte recebido é equivalente a um dado, teremos o bit D/A (SSPSTAT<5>) setado. O dado é recebido no registro SSPSR e logo depois armazenado no registro SSPBUF.
- C)** No mesmo instante em que o registro SSPBUF recebe o dado, o bit BF (SSPSTAT<0>) é setado indicando que o registro SSPBUF está cheio, sendo este bit apagado logo assim que o registro SSPBUF tenha sido lido.
- D)** No borda de descida do oitavo pulso de clock o bit RCEN é zerado e o bit SSPIF é setado podendo gerar uma nova interrupção. Este bit deve ser apagado por software.

- E)** Se for necessário receber um novo byte, o mestre deve responder $ACK = 0$, caso contrário, $ACK = 1$ (NACK). Para responder o ACK, deve-se colocar o valor desejado no bit ACKDT (SSPCON2<5>) e habilitar a seqüência ACK setando o bit ACKEN (SSPCON2<4>).
- F)** Ao término da seqüência ACK (borda de descida do nono pulso de clock) teremos $ACKEN = 0$ e $SSPIF = 1$, gerando-se uma nova interrupção se ela estiver habilitada.
- G)** Caso o mestre responda com $ACK = 0$, o processo retorna ao **item (I)**
- (I)** Para que um novo byte seja recebido. Se $ACK = 1$, o processo deve ser finalizado através de uma condição STOP, setando o bit PEN (SSPCON2<2>).
- H)** No final da condição STOP o bit PEN será apagado por hardware e o bit PSPIF será setado, podendo gerar uma nova interrupção. O bit PSPIF deve ser apagado por software.

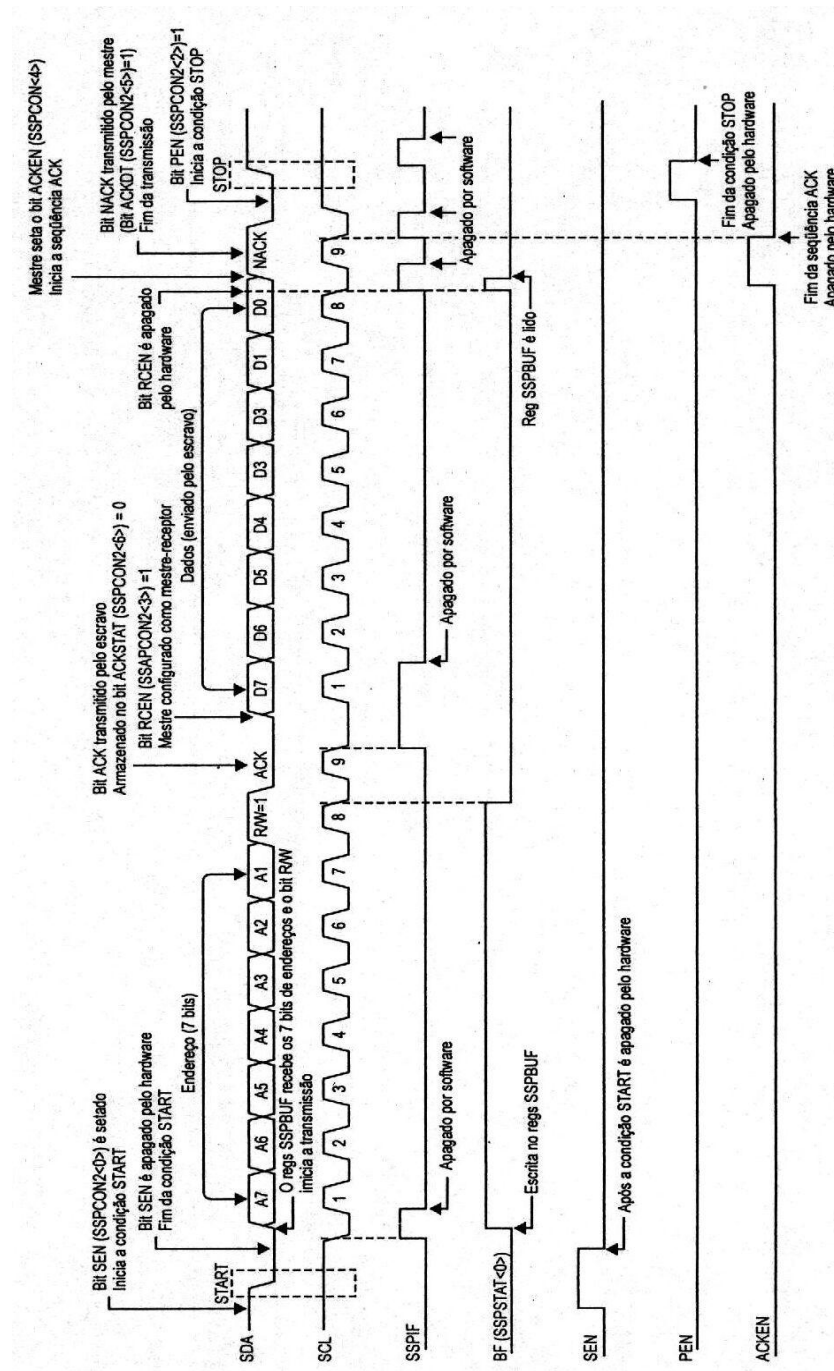


Figura 8.26: Temporização I²C modo Mestre na recepção e endereço de 7 bits.

A sub-rotina abaixo pode ser usada para a recepção de um byte, cuja temporização é vista na **Figura 8.26**. Com algumas modificações ela pode ser utilizada para a recepção de mais bytes de dados. O byte recebido é armazenado no registro

DADO_RECEBIDO depois de ser chamada a sub-rotina I2C_RECEBE, pois nesse registro será armazenado o byte recém chegado.

```

;Esta sub-rotina efetua a leitura de um byte com endereçamento de 7 bits utilizando a comunicação
;serial síncrona I2C
I2C_LEITURA
    call    I2C_LIVRE
    xorlw   .0
    btfsc   STATUS,Z           ;barramento está livre?
    return  ;não, retorna
    call    IIC_START          ;sim, chama sub-rotina que gera condição START
    bsf     STATUS,C           ;seta bit C
    rlf     ENDERECO,F         ;rotaciona registro ENDERECO para a esquerda
    movf    ENDERECO,W         ;W = ENDERECO
    movwf   DADO_I2C           ;registro DADO_I2C recebe endereço do dispositivo escravo
    call    I2C_TRANSMITE      ;chama sub-rotina, transmite endereço
    call    I2C_TESTA_ACK      ;chama sub-rotina, testa bit ACK
    xorlw   .0
    btfss   STATUS,Z           ;escravo endereçado respondeu co bit ACK = 0?
    goto    ERRO_I2C          ;não, desvia
    call    I2C_RECEBE         ;sim, chama sub-rotina, recebe byte
    movf    DADO_I2C,W
    movwf   DADO_RECEBIDO      ;transfere byte recebido para registro DADO_RECEBIDO
    call    I2C_NACK           ;chama sub-rotina, transmite bit NACK
    goto    FIM_TRANSMISSAO
ERRO_I2C
    bsf     I2C_ERRO           ;seta flag sinalizador de erro
FIM_TRANSMISSAO
    call    IIC_STOP           ;chama sub-rotina, gera condição STOP
    return

```

8.4.5 - Dispositivo Mestre Transmite com endereçamento de 10 bits.

A transmissão com 10 bits de endereço difere pouco da transmissão com 7 bits de endereço. A diferença é que o dispositivo mestre deve enviar dois bytes de endereço em vez de um, como no endereçamento de 7 bits.

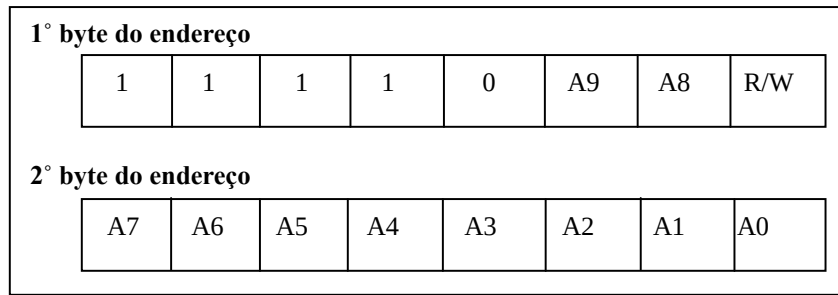


Figura 8.27: Endereço de 10 bits.

O dispositivo mestre inicia a transmissão com uma condição START enviando em seguida o primeiro byte do endereço com 11110:A9:A8:0. O dispositivo escravo que identifica o primeiro byte como parte do seu endereço irá responder com o bit ACK = 0. O dispositivo mestre então envia o segundo byte do endereço contendo os 8 bits menos significativos (A7:A0). O dispositivo escravo deve responder com um novo bit ACK = 0. Os bytes seguintes são de dados, não tendo limite para o número de bytes de dados enviados. Para cada byte recebido o dispositivo escravo deve transmitir um bit ACK = 0.

a) Detalhando uma transmissão com endereço de 10 bits:

I) Transmissão da primeira parte do endereço do escravo:

- A)** Configurar como entrada as linhas SCL e SDA.
- B)** Deve ser imposta uma condição START setando o bit SEN (SSPCON2<0>).
- C)** Ao término da condição START o bit SEN será apagado e o flag de interrupção bit SSPIF (PIR1<3>) será setado, podendo gerar uma interrupção. Este bit deve ser apagado por software.
- D)** O mestre escreve no registro SSPBUF<7:1> a parte alta do endereço do escravo (11110:A9:A8) com o qual deseja se comunicar, veja **Figura 8.28**. Logo após, é transferido para o registro de deslocamento SSPSR para então ser enviado ao barramento através da linha SDA. O bit 0 é R/W = 0 indicando leitura.
- E)** Logo após a escrita no registro SSPBUF, teremos o bit BF (SSPSTAT<0>) setado e se mantém assim até a transmissão do oitavo pulso de clock quando é automaticamente apagado pelo hardware (BF = 0). Ver **Figura 8.28**.

- F)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- G)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- H)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- I)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

II) Transmissão da segunda parte do endereço:

- A)** A segunda parte do endereço (A7:A0) é, então, escrita no registro SSPBUF.
- B)** Logo após a escrita no registro SSPBUF, teremos o bit BF (SSPSTAT<0>) setado e se mantém assim até a transmissão do oitavo bit ou oitavo pulso de clock quando é automaticamente apagado pelo hardware (BF = 0). Ver **Figura 8.28.**
- C)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- D)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- E)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- F)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

III) Transmissão do dado:

- A)** O dado a ser transmitido é, então, colocado no registro SSPBUF. Teremos BF = 1 (até o oitavo pulso de clock) e R/W = 1 (até o nono pulso de clock).

- B)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- C)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- D)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.
- E)** Se for preciso transmitir um novo byte, o processo é reiniciado no **ítem III) (A)**.
- F)** Para interromper a transmissão, uma condição STOP deve ser gerada através do bit PEN (SSPCON2<2>).
- G)** No final da condição STOP o bit PEN = 0 e o bit SSPIF = 1, podendo gerar uma nova interrupção se ela estiver habilitada. O bit SSPIF deve ser desabilitado por software.

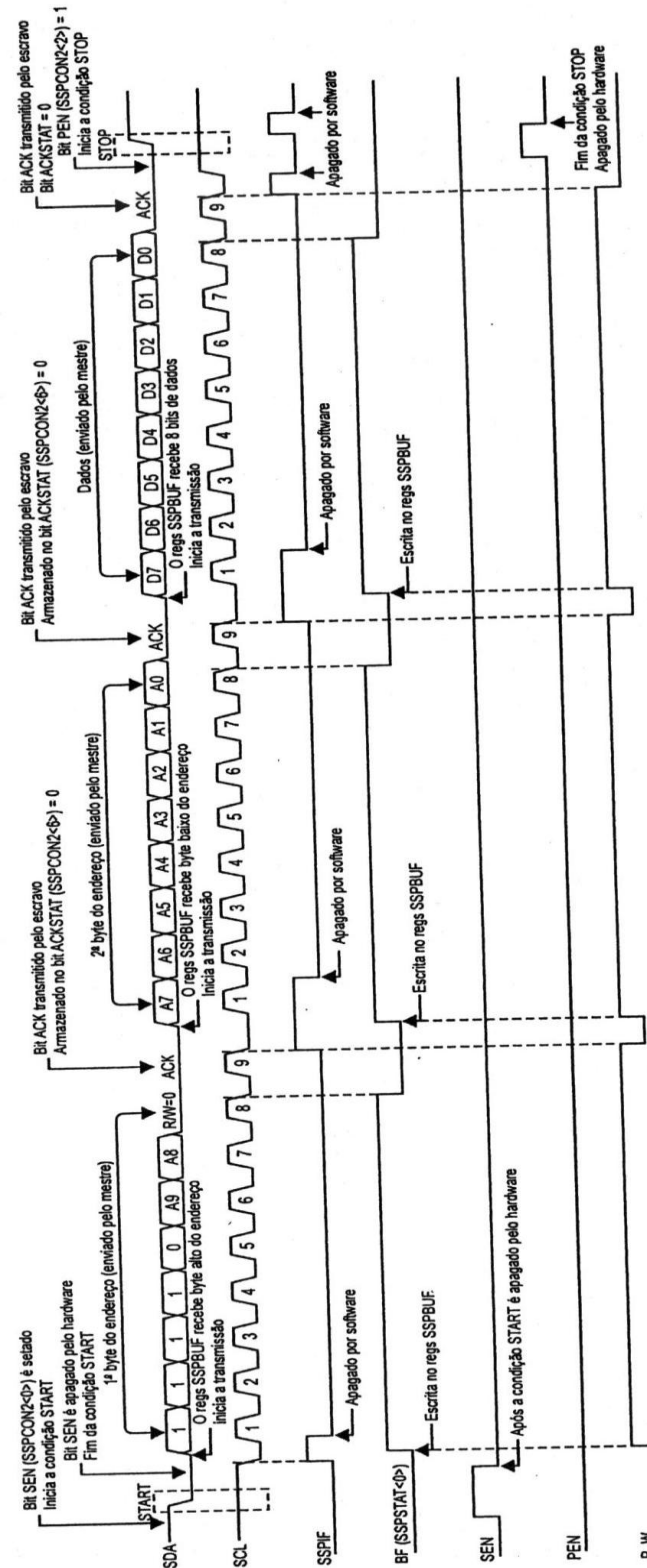


Figura 8.28: Temporização I²C modo Mestre na transmissão e endereço de 10 bits.

A sub-rotina abaixo mostra uma transmissão de um byte de dados com endereçamento de 10 bits e sua temporização é mostrada na **Figura 8.28**.

;Esta sub-rotina transmite um byte com endereçamento de 10 bits utilizando a comunicação serial
;síncrona I2C

I2C_ESCRITA_10BITS

```

call    I2C_LIVRE           ;chama sub-rotina, verifica se barramento está livre
xorlw   .0
btfsc   STATUS,Z           ;barramento livre?
return  ;não, retorna ao ponto de chamada
call    IIC_START          ;sim, chama sub-rotina, gera condição START
bcf     STATUS,C
rlf     ENDERECO_ALTO,F    ;rotaciona registro ENDERECO_ALTO para a esquerda
movf    ENDERECO_ALTO,W    ;W = ENDERECO_ALTO
movwf   DADO_I2C           ;registro DADO_I2C recebe endereço_alto do escravo
call    I2C_TRANSMITE      ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK      ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z           ;escravo endereçado respondeu bit ACK = 0?
goto    ERRO_I2C          ;não, desvia
movf    ENDERECO_BAIXO,W  ;sim, W = ENDERECO_BAIXO
movwf   DADO_I2C          ;registro DADO_I2C recebe endereço_baixo do escravo
call    I2C_TRANSMITE      ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK      ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z           ;escravo endereçado respondeu bit ACK = 0?
goto    ERRO_I2C          ;não, desvia
movf    DADO,W            ;sim, W = DADO
movwf   DADO_I2C          ;registro DADO_I2C recebe byte que será transmitido
call    I2C_TRANSMITE      ;chama sub-rotina, transmite byte
call    I2C_TESTA_ACK      ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z           ;escravo recebeu byte de dados (bit ACK = 0?)
goto    ERRO_I2C          ;não, desvia
goto    FIM_TRANSMISSAO    ;sim, desvia
ERRO_I2C
bsf     I2C_ERRO           ;seta flag de sinalização de erro
FIM_TRANSMISSAO
call    IIC_STOP           ;chama sub-rotina, gera condição STOP
return

```

8.4.6 - Dispositivo Mestre Recebe com endereçamento de 10 bits.

A recepção com 10 bits de endereço segue a mesma lógica da transmissão com 10 bits de endereço. Porém, existe uma diferença significativa entre elas que é a necessidade de gerar uma condição Re_START logo após o envio do segundo byte de endereço. A condição Re-START tem a função de redirecionar o fluxo dos dados para que o dispositivo mestre possa receber os dados que lhe serão enviados. Após a condição Re_START o dispositivo mestre deve reenviar o primeiro byte do endereço com o bit R/W = 1. Não é necessário o dispositivo mestre reenviar o segundo byte do endereço após a condição Re_START.

O dispositivo mestre inicia a transmissão com uma condição START enviando em seguida o primeiro byte do endereço com 11110:A9:A8:0 (bit R/W = 0). O dispositivo escravo que identifica o primeiro byte como parte do seu endereço irá responder com o bit ACK = 0. O dispositivo mestre então envia o segundo byte do endereço contendo os 8 bits menos significativos (A7:A0). O dispositivo escravo endereçado deve responder com um novo bit ACK = 0. A seguir o dispositivo mestre deve gerar uma condição Re_START e então reenviar o primeiro byte de endereço (11110:A9:A8:1) com bit R/W = 1. O dispositivo escravo endereçado deve acenar com mais um bit ACK = 0 e então começar a enviar os dados para o dispositivo mestre. Após cada byte recebido o dispositivo mestre deve enviar um bit ACK = 0. Quando o dispositivo mestre não desejar mais receber dados, ele deve enviar um bit NACK seguido de uma condição STOP.

a) Detalhando uma recepção com endereço de 10 bits:

I) Transmissão da primeira parte do endereço do escravo:

- A)** Configurar como entrada as linhas SCL e SDA.
- B)** Deve ser imposta uma condição START setando o bit SEN (SSPCON2<0>).
- C)** Ao término da condição START o bit SEN será apagado e o flag de interrupção bit SSPIF (PIR1<3>) será setado, podendo gerar uma interrupção. Este bit deve ser apagado por software.
- D)** O mestre escreve no registro SSPBUF<7:1> a parte alta do endereço do escravo (11110:A9:A8) com o qual deseja se comunicar, (veja **Figura 8.29**). Logo após, é transferido para o registro de deslocamento SSPSR para então

ser enviado ao barramento através da linha SDA. O bit 0 é R/W = 0 indicando leitura.

- E)** Logo após a escrita no registro SSPBUF, teremos o bit BF (SSPSTAT<0>) setado e se mantém assim até a transmissão do oitavo pulso de clock quando é automaticamente apagado pelo hardware (BF = 0). Veja **Figura 8.29**.
- F)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- G)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- H)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- I)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

II) Transmissão da segunda parte do endereço:

- A)** A segunda parte do endereço (A7:A0) é, então, escrita no registro SSPBUF.
- B)** Logo após a escrita no registro SSPBUF, teremos o bit BF (SSPSTAT<0>) setado e se mantém assim até a transmissão do oitavo bit ou oitavo pulso de clock quando é automaticamente apagado pelo hardware (BF = 0). Veja na **Figura 8.29**.
- C)** O bit R/W (SSPSTAT<2>) também é setado quando escrevemos no registro SSPBUF (indica transmissão em progresso), mantendo-se assim até o final do nono pulso de clock, ou seja, até a recepção do pulso ACK, sendo automaticamente apagado por hardware.
- D)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- E)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- F)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

III) Geração da condição Re_START:

- A)** Deve ser gerada uma condição Re_START setando o bit RSEN (SSPCON2<1>).
- B)** Ao término da condição Re_START o bit REN será apagado e o flag de interrupção bit SSPIF (PIR1<3>) será setado, podendo gerar uma interrupção. Este bit deve ser apagado por software.
- C)** Mais uma vez devemos colocar no registro SSPBUF a primeira parte do endereço (11110:A9:A8:1) com R/W = 1.
- D)** O bit BF será setado (até o oitavo pulso de clock) e R/W será setado (até o nono pulso de clock).
- E)** O byte será transmitido nos oito pulsos de clock em SCL e no final BF = 0.
- F)** O valor de ACK, recebido do escravo no nono pulso de clock (que deve ser ACK = 0), é armazenado no bit ACKSTAT (SSPCON2<6>).
- G)** No final do nono pulso de clock o bit R/W será zerado e o bit SSPIF (PIR1<3>) é setado, e uma nova interrupção será gerada se ela estiver habilitada. Não esquecer de apagar o bit SSPIF por software.

IV) Recepção do dado:

- A)** O mestre deve, então, ser colocado em modo de recepção (mestre-receptor) setando o bit RCEN (SSPCON2<3>) e tem início a transferência do dado.
- B)** Se a linha de clock não estiver em pausa, será recebido um byte de dados para os próximos oito pulsos do clock. Como o byte recebido é equivalente a um dado, teremos o bit D/A (SSPSTAT<5>) setado. O dado é recebido no registro SSPSR e logo depois armazenado no registro SSPBUF.
- C)** No mesmo instante em que o registro SSPBUF recebe o dado, o bit BF (SSPSTAT<0>) é setado indicando que o registro SSPBUF está cheio, sendo este bit apagado logo assim que o registro SSPBUF tenha sido lido.
- D)** Neste momento, borda de descida do oitavo pulso de clock, o bit RCEN é zerado e o bit SSPIF é setado podendo gerar uma nova interrupção. Este bit deve ser apagado por software.
- E)** Se for necessário receber um novo byte, o mestre deve responder ACK = 0, caso contrário, ACK = 1 (NACK). Para responder o ACK, deve-se colocar o

valor desejado no bit ACKDT (SSPCON2<5>) e habilitar a seqüência ACK setando o bit ACKEN (SSPCON2<4>).

- F)** Ao término da seqüência ACK (borda de descida do nono pulso de clock) teremos ACKEN = 0 e SSPIF = 1, gerando-se uma nova interrupção se ela estiver habilitada.
- G)** Caso o mestre responda com ACK = 0, o processo retorna ao **ítem III) (C)** para que um novo byte seja recebido. Se ACK = 1, o processo deve ser finalizado através de uma condição STOP, setando o bit PEN (SSPCON2<2>).
- H)** No final da condição STOP o bit PEN será apagado por hardware e o bit PSPIF será setado, podendo gerar uma nova interrupção. O bit PSPIF deve ser apagado por software.

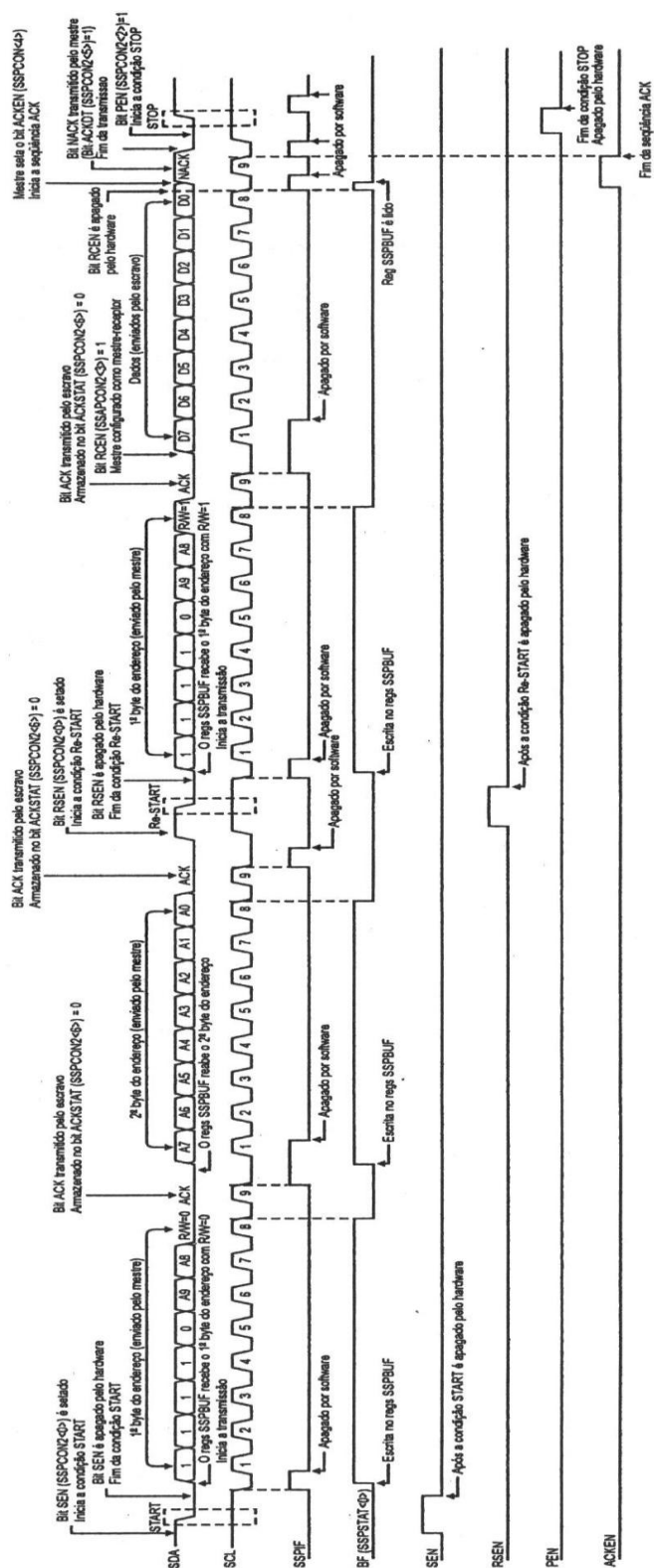


Figura 8.29: Temporização I²C modo Mestre na recepção e endereço de 10 bits.

A sub-rotina abaixo mostra uma recepção de um byte de dados com endereçamento de 10 bits e a temporização é vista na **Figura 8.29**.

;Esta sub-rotina recebe um byte com endereçamento de 10 bits utilizando a comunicação serial síncrona I2C

I²C_LEITURA_10BITS

```

call    I2C_LIVRE                ;chama sub-rotina, verifica se barramento está livre
xorlw   .0
btfsc   STATUS,Z                 ;barramento livre?
return  ;não, retorna ao ponto de chamada
call    IIC_START                ;sim, chama sub-rotina, gera condição START
bcf     STATUS,C
rlf     ENDERECO_ALTO,F          ;rotaciona registro ENDERECO_ALTO para a esquerda
movf    ENDERECO_ALTO,W          ;W = ENDERECO_ALTO
movwf   DADO_I2C                ;registro DADO_I2C recebe endereço_alto do escravo
call    I2C_TRANSMITE            ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK            ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z                 ;escravo endereçado respondeu bit ACK = 0?
goto    ERRO_I2C                ;não, desvia
movf    ENDERECO_BAIXO,W        ;sim, W = ENDERECO_BAIXO
movwf   DADO_I2C                ;registro DADO_I2C recebe endereço_baixo do escravo
call    I2C_TRANSMITE            ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK            ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z                 ;escravo endereçado respondeu bit ACK = 0?
goto    ERRO_I2C                ;não, desvia
call    I2C_RESTART              ;sim, chama subrotina, gera condição Re_START
bsf     STATUS,C
rlf     ENDERECO_ALTO,F          ;rotaciona registro ENDERECO_ALTO para a esquerda
movf    ENDERECO_ALTO,W          ;W = ENDERECO_ALTO
movwf   DADO_I2C                ;registro DADO_I2C recebe byte que será transmitido
call    I2C_TRANSMITE            ;chama sub-rotina, transmite endereço
call    I2C_TESTA_ACK            ;chama sub-rotina, testa bit ACK
xorlw   .0
btfss   STATUS,Z                 ;escravo endereçado respondeu bit ACK = 0?)
goto    ERRO_I2C                ;não, desvia
call    I2C_RECEBE               sim, chama sub-rotina, recebe byte
movf    DADO_I2C,W
movwf   DADO_RECEBIDO            ;transfere byte recebido para registro DADO_RECEBIDO
call    I2C_NACK                 ;chama sub-rotina, transmite bit NACK
goto    FIM_TRANSMISSAO          ;desvia
ERRO_I2C
bsf     I2C_ERRO                 ;seta flag de sinalização de erro
FIM_TRANSMISSAO
call    IIC_STOP                 ;cama sub-rotina, gera condição STOP
return

```

8.4.7 - Considerações.

Existem outros bits e flags envolvidos com o Mestre que não foram comentados até o momento.

Como o sistema pode usar mais de um Mestre, as condições de START e STOP geradas na linha acarretarão na ativação do bit de flag SSPIF, podendo gerar uma interrupção em qualquer Mestre. Os flags S (SSPSTAT<3>) e P (SSPSTAT<4>) informam, respectivamente, as condições de START e STOP. Desta forma, é possível que outros Mestres percebam quando a linha está ou não ocupada.

O flag WCOL (SSPCON<7>) indica uma escrita inválida em SSPBUF e o flag SSPOV (SSPCON<6>) indica o recebimento de um novo byte sem que o byte anterior tenha sido lido de SSPBUF.

As condições START, STOP, Re-START e ACK só podem ser geradas no momento correto, isto é, quando o modo de comunicação está em stand-by. A situação de stand-by acontece sempre que o hardware do I²C acaba o controle de um evento qualquer.

8.4.8 - Detecção de colisão na linha.

Um outro recurso que o Mestre possui está relacionado à colisão de informações na linha de dados (SDA), que funciona da seguinte maneira:

Para que o PIC possa impor nível baixo na linha (condição de saída) existe um transistor interno que conecta este pino ao ponto comum (terra). Quando esta conexão é feita, a linha fica em nível baixo e não há o que monitorar. Entretanto, quando este transistor está aberto, o pino é considerado uma entrada e o nível alto é imposto pelo resistor de pull-up. Nesta situação, o PIC monitora a linha. Caso seja necessário transmitir um bit em 1 (nível alto ou transistor aberto) e a linha esteja em nível baixo (zero), é porque algum outro ponto da rede está em conflito de comunicação.

Quando isto acontece, o bit BCLIF (PIR2<3>) será setado e uma interrupção será gerada se ela estiver habilitada. Teremos ainda o reset do sistema de comunicação, voltando ao estado de stand-by.

As colisões podem acontecer durante a transmissão de dados e as condições de START, STOP, Re-START e ACK.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on: MCLR, WDT
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
0Dh	PIR2	—	(2)	—	EEIF	BCLIF	—	—	CCP2IF	-r-0 0--0	-r-0 0--0
8Dh	PIE2	—	(2)	—	EEIE	BCLIE	—	—	CCP2IE	-r-0 0--0	-r-0 0--0
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								xxxx xxxx	uuuu uuuu
14h	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000	0000 0000
91h	SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN	0000 0000	0000 0000
93h	SSPADD	I ² C Slave Address/Master Baud Rate Register								0000 0000	0000 0000
94h	SSPSTAT	SMP	CKE	D/Ā	P	S	R/W	UA	BF	0000 0000	0000 0000

Legenda: x = desconhecido; u = sem modificações; - = sem implementar, em leitura é zero (0)

SSPCON2 – GCEN só é usado no Escravo e os outros são usados no Mestre

SSPADD – Endereço parte alta ou baixa no Escravo e Frequência de clock no Mestre.

SSPSTAT – Mestre usa todos menos o UA. Escravo usa todos.

Figura 8.30: Registros associados com a operação I²C.

8.5 – CONFIGURANDO O MCU COMO DISPOSITIVO ESCRAVO.

O dispositivo escravo pode enviar ou receber dados do dispositivo mestre. Porém, ele não inicia ou finaliza a transmissão, ficando a cargo do dispositivo mestre.

O dispositivo mestre inicia a transmissão gerando uma condição START no barramento seguida do endereço do dispositivo escravo mais o bit R/W, que definirá se a comunicação será de escrita ou de leitura. Logo após o endereçamento são enviados ao receptor (dispositivos mestre ou escravo) os bytes de dados. Após a transmissão dos dados o dispositivo mestre gera uma condição STOP no barramento e encerra a transmissão.

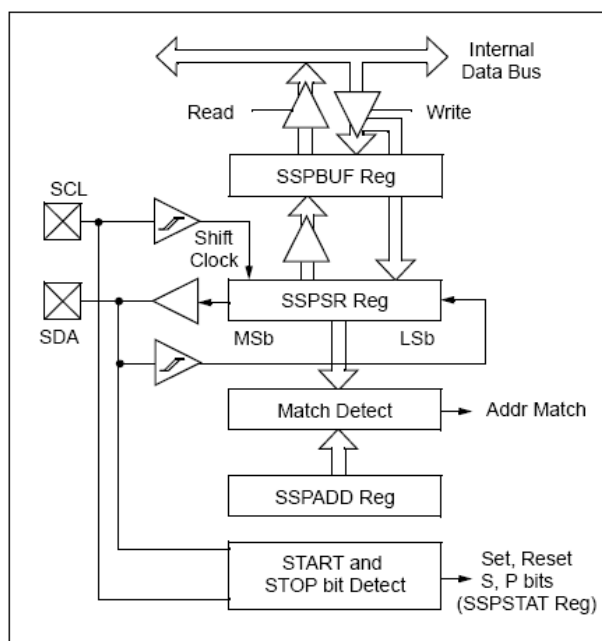


Figura 8.31: Digrama de blocos do Módulo SSP, no modo I²C escravo.

8.5.1 - Para configurar o módulo MSSP do MCU como dispositivo escravo temos:

- a) Configurar os pinos SDA e SCL como entrada através do registro TRISC.
- b) Selecionar o modo de funcionamento da porta série síncrona através dos bits SSPM3:SSPM0 (SSPCON<3:0>), que configuram a interface para operar como dispositivo escravo nos seguintes modos:
 - I) 0110, modo escravo com 7 bits de endereço.
 - II) 0111, modo escravo com 10 bits de endereço.
 - III) 1110, modo escravo com 7 bits de endereço e bits de interrupção de START e STOP habilitados.
 - IV) 1111, modo escravo com 10 bits de endereço e bits de interrupção de START e STOP habilitados.
- c) Habilitar/desabilitar o bit SMP (SSPSTAT<7>) que, neste caso, serve para habilitar/desabilitar um sistema interno de tratamento das bordas quando operando em frequência de 400KHz.
- d) Configurar o bit CKE (SSPSTAT<6>) que serve para configurar o tipo de Smith Trigger usado nos pinos SCL e SDA de acordo com especificações I²C/SMBUS.
- e) Habilitar a porta série síncrona setando o bit SSPEN (SSPCON <5>).

Quando o MCU funciona como dispositivo escravo usa-se os dois primeiros modos de funcionamento da porta série síncrona. Os outros modos são usados quando o MCU funciona ora como mestre ora como escravo. Esta situação é comum em um barramento com mais de um dispositivo mestre. Neste caso, as interrupções de condições START e STOP poderão servir para informar ao MCU se o barramento está ou não ocupado.

O endereço do dispositivo escravo é armazenado nos bits SSPADD<7:1> (endereço de 7 bits). O bit SSPADD<0> deve ser apagado uma vez que ele receberá do dispositivo mestre o bit R/W que define a direção do fluxo dos dados.

O flag de interrupção bit SSPIF (PIR1<3>) será setado sempre que um novo byte de dado for recebido ou transmitido pelo dispositivo escravo. Este bit precisa ser apagado por software dentro da rotina de interrupção.

8.5.2 - No código seguinte temos:

- a) Como configurar o módulo MSSP para funcionar como dispositivo escravo.
- b) O endereço do dispositivo no barramento foi armazenado no registro SSPADD<7:1> por meio da constante ENDERECO.
- c) A interrupção do módulo MSSP foi habilitada, assim o sistema irá detetar se um byte foi enviado ou recebido pelo MCU.
- d) Slew rate foi desabilitado o que significa taxa de transferência de até 100Kb/s.

```

Bank1
Movlw  B'00011000'  ;W = 18h
Iorwf  TRISC,F      ;configura pinos SDA e SCL como entrada
Bank0
Molw   H'36'        ;W = 36h
Movwf  SSPCON       ;configura dispositivo como escravo
                        ;habilita porta serial e configura pinos
                        ;ativa modo escravo com 7 bits

Bank1
Clrf   SSPSTAT      ;limpa registro SSPSTAT
Bsf    SSPSTAT,SMP  ;controle de slew rate desabilitado
                        ;níveis de entrada conforme especificações I2C
Bcf    SSPCON2,SEN  ;clock stretching ativado somente na transmissão
Movlw  ENDERECO     ;W = ENDERECO<7:1>
Movwf  SSPADD       ;endereço é armazenado no registro SSPADD
Bsf    PIE1,SSPIE   ;habilita chave individual de interrupção do módulo MSSP
Bsf    INTCON,PEIE  ;habilita interrupção de periféricos
Bsf    INTCON,GIE   ;habilita chave geral de interrupções
Bank0

```

8.5.3 – Dispositivo Escravo Recebe com endereçamento de 7 bits:

a) Detalhando uma recepção com endereço de 7 bits (Figura 8.32):

- I) Configurar como entrada as linhas SCL e SDA. A linha receberá, então, uma condição START.
- II) A condição START é reconhecida e o bit S (SSPSTAT<3>) é setado. Nenhuma interrupção acontece.
- III) O escravo receberá um byte com 7 bits de endereço e mais o bit R/W = 0, durante os 8 pulsos de clock. O bit R/W (SSPSTAT<2>) = 0, indicando escrita. Como o byte recebido é um endereço, então o bit D/A (SSPSTAT<5>) = 0.
- IV) Neste momento, o endereço recebido pelo registro SSPSR é automaticamente comparado com o armazenado no registro SSPADD. Se diferente, a comunicação do lado escravo é resetada (ACK = 1) e o mesmo ficará aguardando um novo START. Se igual, o bit BF (SSPSTAT<0>) é setado informando que um valor foi transferido para o registro buffer SSPBUF (BF = 0 logo assim que o registro SSPBUF seja lido). O pino SDA se torna saída, pois vai para nível baixo (ACK = 0) até o término do nono pulso de clock. Logo depois, SDA é liberado tornando-se novamente entrada. No mesmo instante (nono pulso de clock) o flag PSPIF (PIR1<3>) é setado e uma interrupção pode acontecer se ela estiver habilitada. É importante que a interrupção aconteça para que seja feita uma leitura do registro SSPBUF, forçando novamente o bit BF = 0.
- V) O escravo receberá um byte de dados durante os 8 pulsos de clock seguintes. Como agora o byte recebido é equivalente a um dado, então o bit D/A (SSPSTAT<5>) = 1. Neste momento (final do oitavo pulso de clock), o bit BF = 1 e o dado serão transferidos do registro SSPSR para o registro SSPBUF (BF = 0 logo assim que o registro SSPBUF seja lido).
- VI) Mais uma vez (nono pulso de clock) o pino SDA impõe um nível baixo para responder ACK = 0 e logo em seguida, volta à condição de entrada (alta Z).
- VII) Também no nono pulso de clock, o bit SSPIF será setado e uma nova interrupção deve acontecer para que o dado recebido possa ser tratado.

- VIII)** Novos bytes de dados podem continuar sendo recebidos até que uma condição de STOP seja imposta na linha. Se isto acontecer, o bit P (SSPSTAT<4>) será setado e a comunicação finalizada. Aqui, não é gerada nenhuma interrupção.
- X)** Caso seja recebido uma condição Re-START, entre START e STOP, o escravo ficará esperando um novo byte de endereço, com o R/W, e não mais um byte de dados.

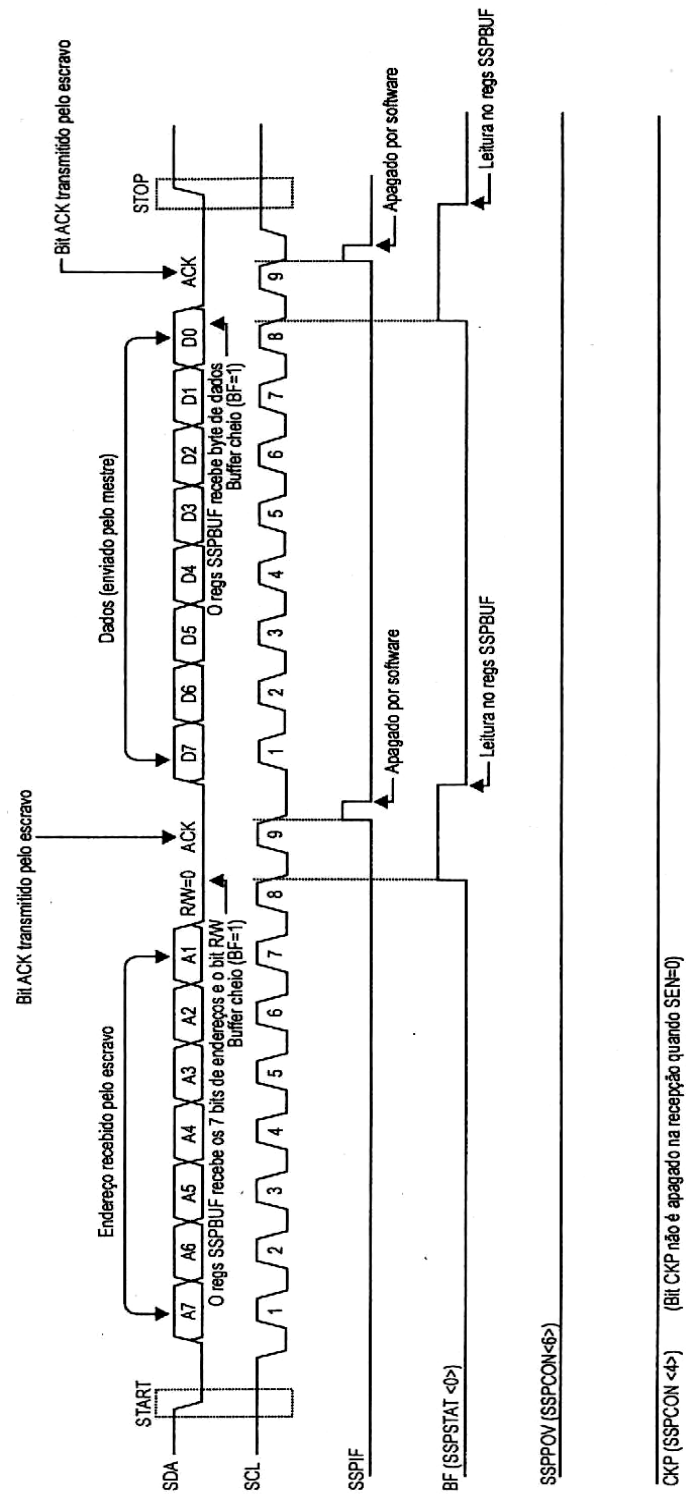


Figura 8.32: Temporização I²C modo Escravo na recepção e endereço de 7 bits (bit SEN = 0).

Devido ao buffer duplo de recepção, isto é, o dado entra no registro SSPSR e depois é colocado no registro SSPBUF, a informação recebida pode então ser lida mesmo durante o recebimento do próximo byte. Após a leitura do registro SSPBUF, o flag BF é limpo automaticamente pelo hardware. Caso o registro SSPBUF não seja lido antes da nova recepção terminar, o dado recém chegado será perdido (SSPBUF não será atualizado) e o bit SSPOV (SSPCON<6>) será setado, indicando um overflow (deve ser apagado por software).

Na **Figura 8.33**, verificamos que dependendo das condições de BF e SSPOV a resposta ACK pode ser negativa (ACK = 1).

Estado dos bits na transferência		SSPSR → SSPBUF (atualiza?)	ACK	Interrupção SSPIF = 1
BF	SSPOV			
0	0	Sim	0 (Ok)	Sim
1	0	Não	1 (Erro)	Sim
1	1	Não	1 (Erro)	Sim
0*	1*	Sim	1 (Erro)	Sim
* Esta condição representa um erro de programação, pois o SSPBUF foi limpo e o bit SSPOV não.				

Figura 8.33: Atuação dos bits BF e SSPOV.

8.5.4 – Dispositivo Escravo Transmite com endereçamento de 7 bits:

a) Detalhando uma transmissão com endereço de 7 bits (Figura 8.34):

- I) Configurar como entrada as linhas SCL e SDA. A linha receberá, então, uma condição START.
- II) A condição START é reconhecida e o bit S (SSPSTAT<3>) é setado. Nenhuma interrupção acontece.
- III) O escravo receberá um byte com 7 bits de endereço e mais o bit R/W = 1, durante os 8 pulsos de clock. O bit R/W (SSPSTAT<2>) = 1, indicando leitura. Como o byte recebido é um endereço, então o bit D/A (SSPSTAT<5>) será apagado (0).
- IV) Neste momento, o endereço recebido pelo registro SSPSR é automaticamente comparado com o armazenado no registro SSPADD. Se diferente, a comunicação do lado escravo é resetada (ACK = 1) e o mesmo ficará aguardando um novo START. Se igual, o pino SDA é transformado automaticamente em saída com

nível baixo (ACK = 0) até o término do nono pulso de clock. Logo depois, SDA é liberado tornando-se novamente entrada.

- V)** O sistema desabilita automaticamente o clock, no término do nono pulso de clock, (pausa) limpando o bit CKP (SSPCON<4>) = 0. Isto é feito impondo-se nível baixo em SCL (saída).
- VI)** No mesmo instante (nono pulso de clock) o flag PSPIF (PIR1<3>) é setado e uma interrupção pode acontecer se ela estiver habilitada. É importante que a interrupção aconteça para que seja escrito o valor a ser transmitido no registro SSPBUF, e setar o bit BF (1).
- VII)** Depois do buffer atualizado, para que a comunicação proceda, basta setar o bit CKP, SCL será novamente liberada (entrada).
- VIII)** O clock será liberado e o byte de dados existente em SSPBUF será enviado durante os próximos oito pulsos de clock. Ao término, teremos o bit BF = 0.
- IX)** Mais uma vez (nono pulso de clock) o pino SDA fica aguardando um ACK. Se ACK = 1 a comunicação será encerrada através da condição STOP, setando o bit P (SSPSTAT<4>). A interrupção não é gerada. Caso ACK = 0, o sistema volta ao item **(V)** para que um novo byte de dados seja transmitido.
- X)** Durante a transferência de dados não pode acontecer uma condição Re_START. Para reiniciar completamente o sistema, é necessário terminar a transferência com ACK = 1 por parte do mestre, seguido de uma condição STOP. Só então uma nova condição START poderá ser gerada.
- XI)** Caso haja tentativa de escrever um valor no registro SSPBUF enquanto uma transmissão esteja em progresso (BF = 1) o valor de SSPBUF não será atualizado e o bit WCOL (SSPCON<7>) será setado, indicando uma colisão na escrita. Este bit deve ser apagado por software.

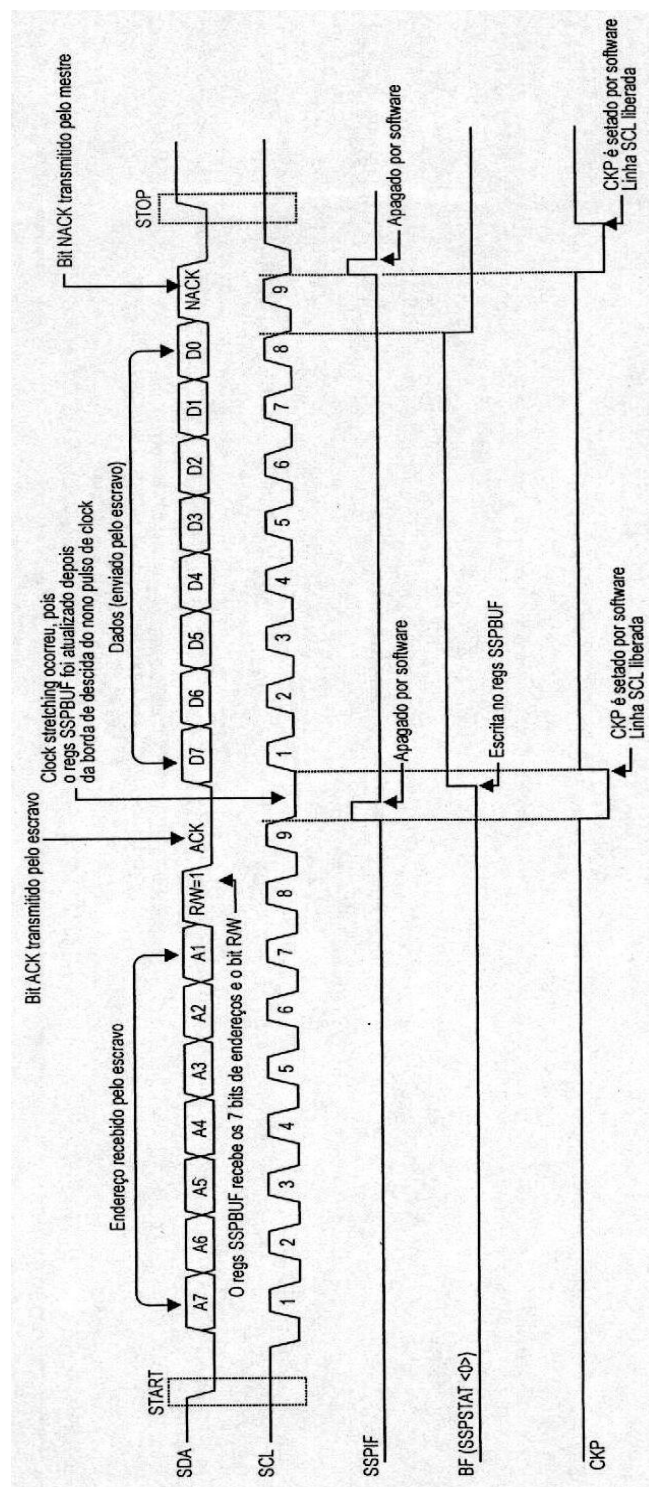


Figura 8.34: Temporização I²C modo Escravo na transmissão e endereço de 7 bits.

8.5.5 – Dispositivo Escravo recebe com endereçamento de 10 bits:

a) Detalhando uma recepção com endereço de 10 bits (Figura 8.35):

- I) Configurar como entrada as linhas SCL e SDA. A linha receberá, então, uma condição START.
- II) A condição START é reconhecida e o bit S (SSPSTAT<3>) é setado. Nenhuma interrupção acontece.
- III) O escravo receberá o primeiro byte do endereço (1110:A9:A8:0), o bit R/W = 0, durante os 8 pulsos de clock. O bit R/W (SSPSTAT<2>) = 0, indicando escrita. Como o byte recebido é um endereço, então o bit D/A (SSPSTAT<5>) = 0.
- IV) Neste momento (oitavo pulso de clock), o endereço recebido pelo registro SSPSR é automaticamente comparado com o armazenado no registro SSPADD. Se diferente, a comunicação do lado escravo é resetada (ACK = 1) e o mesmo ficará aguardando um novo START. Se igual, o bit BF (SSPSTAT<0>) é setado informando que um valor foi transferido para o registro buffer SSPBUF (BF será apagado (0) logo assim que o registro SSPBUF seja lido). O bit UA (SSPSTAT<1>) é setado indicando que o endereço deve ser atualizado.
- V) O pino SDA se torna saída, pois vai para nível baixo (ACK = 0) até o término do nono pulso de clock. Logo depois, SDA é liberado tornando-se novamente entrada. No mesmo instante (nono pulso de clock) o flag PSPIF (PIR1<3>) é setado e uma interrupção pode acontecer se ela estiver habilitada. É importante que a interrupção aconteça para que seja feita uma leitura do registro SSPBUF, forçando novamente o bit BF = 0. O registro SSPADD deve, então, ser atualizado com a parte baixa do endereço (A7:A0), forçando automaticamente o bit UA = 0. Depois do ACK e até o momento em que o registro SSPADD é atualizado, o escravo trava a comunicação (pausa) impondo nível baixo em SCL. Este controle é totalmente automático.
- VI) O escravo receberá, então, o segundo byte do endereço (A7:A0), durante os próximos 8 pulsos de clock. Como o byte recebido é um endereço, continuamos com o bit D/A (SSPSTAT<5>) = 0.

- VII)** Todo o processo descrito nos itens IV e V serão repetidos. No final, o sistema entra em nova pausa até que o registro SSPADD seja novamente atualizado com a parte alta do endereço.
- VIII)** O escravo receberá um byte de dado no registro SSPSR, durante os 8 pulsos de clock seguintes. Como o byte recebido é um dado, o bit D/A (SSPSTAT<5>) será setado. Neste momento também o bit BF será setado e o dado transferido para o registro SSPBUF.
- IX)** O pino SDA é transformado automaticamente em saída com nível baixo (ACK = 0) e o sistema aguarda o nono pulso de clock. Logo depois SDA é transformado novamente em entrada.
- X)** Uma nova interrupção acontece (SSPFIF = 1) para que o dado recebido possa ser tratado.
- XI)** Novos bytes de dados podem continuar sendo recebidos até que uma condição de STOP seja imposta na linha. Neste momento, a condição será reconhecida, setando-se o bit P (SSPSTAT<4>) e finalizando a comunicação. A interrupção não será gerada.
- XII)** Caso seja recebido uma condição Re_START (entre START e STOP), o escravo ficará esperando um novo byte de endereço, com o bit R/W e não mais um byte de dados.
- Obs:** Os comentários feitos a respeito do buffer e dos flags BF e SSPOV para a comunicação de 7 bits também são válidos aqui.

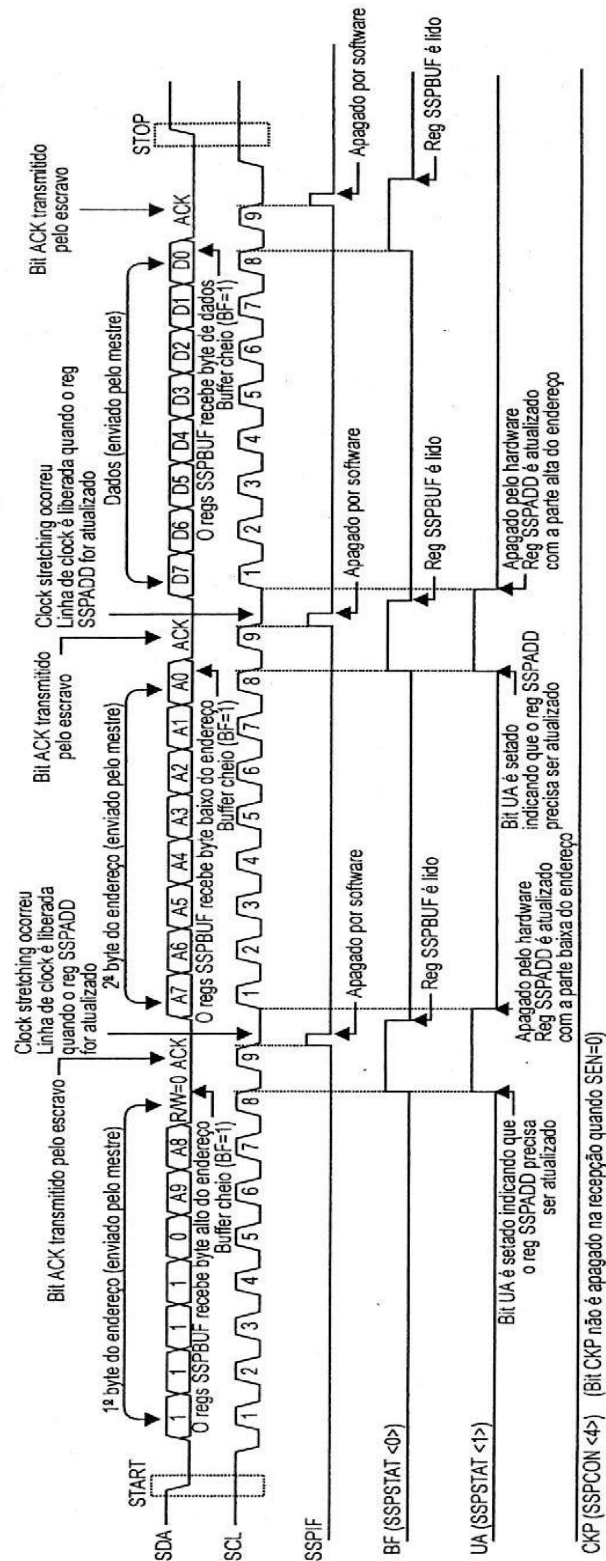


Figura 8.35: Temporização I²C modo Escravo na recepção e endereço de 10 bits
(Bit SEN = 0)

8.5.6 – Dispositivo Escravo transmite com endereçamento de 10 bits:

a) Detalhando uma transmissão com endereço de 10 bits (Figura 8.36):

- I) Configurar como entrada as linhas SCL e SDA. A linha receberá, então, uma condição START.
- II) A condição START é reconhecida e o bit S (SSPSTAT<3>) é setado. Nenhuma interrupção acontece.
- III) O escravo receberá o primeiro byte do endereço (1110:A9:A8:0), o bit R/W = 0, durante os 8 pulsos de clock. O bit R/W (SSPSTAT<2>) = 0. Como o byte recebido é um endereço, então o bit D/A (SSPSTAT<5>) = 0.
- IV) Neste momento (oitavo pulso de clock), o endereço recebido pelo registro SSPSR é automaticamente comparado com o armazenado no registro SSPADD. Se diferente, a comunicação do lado escravo é resetada (ACK = 1) e o mesmo ficará aguardando um novo START. Se igual, o bit BF (SSPSTAT<0>) é setado informando que um valor foi transferido para o registro buffer SSPBUF (BF será apagado (0) logo assim que o registro SSPBUF seja lido). O bit UA (SSPSTAT<1>) é setado indicando que o endereço deve ser atualizado.
- V) O pino SDA se torna saída, pois vai para nível baixo (ACK = 0) até o término do nono pulso de clock. Logo depois, SDA é liberado tornando-se novamente entrada. No mesmo instante (nono pulso de clock) o flag PSPIF (PIR1<3>) é setado e uma interrupção pode acontecer se ela estiver habilitada. É importante que a interrupção aconteça para que seja feita uma leitura do registro SSPBUF, forçando novamente o bit BF = 0. O registro SSPADD deve, então, ser atualizado com a parte baixa do endereço (A7:A0), forçando automaticamente o bit UA = 0. Depois do ACK e até o momento em que o registro SSPADD é atualizado, o escravo trava a comunicação (pausa) impondo nível baixo em SCL. Este controle é totalmente automático.
- VI) O escravo receberá, então, o segundo byte do endereço (A7:A0), durante os próximos 8 pulsos de clock. Como o byte recebido é um endereço, continuamos com o bit D/A (SSPSTAT<5>) = 0.

- VII)** Todo o processo descrito nos itens IV e V serão repetidos. No final, o sistema entra em pausa até que o registro SSPADD seja novamente atualizado com a parte alta do endereço.
- VIII)** A linha fornecerá, então, uma condição de Re-START, informando que o próximo byte é novamente uma informação de controle.
- IX)** O escravo receberá em SSPSR, outro byte com a parte alta do endereço e com $R/W = 1$, nos próximos 8 pulsos de clock. Como o byte recebido ainda é um endereço, continuamos com o bit D/A ($SSPSTAT<5>$) = 0. O valor recebido será transferido para o registro SSPBUF e o bit BF será setado.
- X)** O pino SDA se torna saída, pois vai para nível baixo ($ACK = 0$) até o término do nono pulso de clock. Logo depois, SDA é liberado tornando-se novamente entrada.
- XI)** O sistema automaticamente coloca o bit CKP ($SSPCON<4>$) = 0, desabilitando o clock (pausa). Isto é feito transformando SCL em saída com nível baixo.
- XII)** No mesmo instante (nono pulso de clock) o flag PSPIF ($PIR1<3>$) é setado e uma interrupção pode acontecer se ela estiver habilitada. É importante que a interrupção aconteça para que seja feita uma leitura do registro SSPBUF, forçando novamente o bit BF = 0. O valor a ser transmitido deve ser escrito no registro SSPBUF e o bit BF será setado.
- XIII)** Depois do buffer atualizado, para que a comunicação proceda, basta setar o bit CKP. Neste momento, SCL será novamente liberada.
- XIV)** O clock será liberado e o byte armazenado no registro SSPBUF será enviado nos próximos 8 pulsos de clock, ao término, o bit BF = 0.
- XV)** Mais uma vez o pino SDA fica aguardando um ACK no nono pulso de clock. Caso $ACK = 1$, a comunicação será encerrada, sendo aguardada uma condição STOP. Neste momento, a condição será reconhecida, o bit P $SSPSTAT<4>$ será setado, a comunicação finalizada, e nenhuma interrupção será gerada. Caso $ACK = 0$, o sistema volta ao **item VI** para que um novo byte de dado seja transmitido.
- XVI)** Durante a transferência de dados não pode acontecer uma nova situação de Re-START. Para reiniciar completamente o sistema é necessário terminar a

transferência com um $ACK = 1$ por parte do mestre, seguido de uma condição STOP. Só então uma nova condição de START poderá ser gerada.

Obs: A condição de colisão (WCOL) descrita na comunicação de 7 bits também é válida para a de 10 bits.

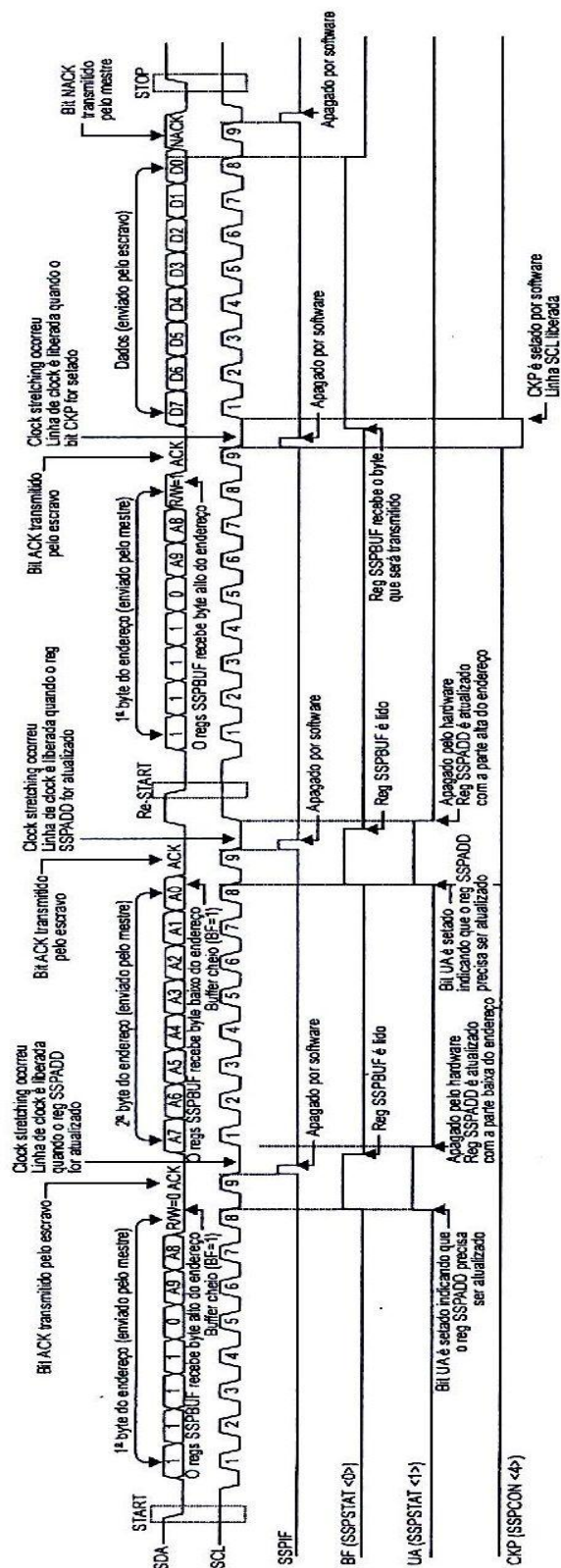


Figura 8.36: Temporização I²C modo Escravo na transmissão e endereço de 10 bits.

8.5.7 - Tratamento da interrupção SSP.

Como só existe uma interrupção da porta série (SSP) e diversas condições, os bits adicionais desta porta devem ser verificados para sabermos exatamente que atitude deve ser tomada. Vejamos algumas condições:

BF	UA	D/A	SSPOV	Situação
1	X	X	X	Sempre que a interrupção acontecer BF = 1, é porque alguma informação foi colocada em SSPBUF. Para transmissão, este flag pode ser checado para sabermos se o processo já terminou, evitando uma colisão (WCOL).
1	X	0	X	Foi recebido um endereço. Como o endereço é checado automaticamente, a interrupção deve somente ler SSPBUF para limpar BF.
1	1	0	X	Somente quando configurada comunicação de 10 bits. A interrupção deverá atualizar o endereço, primeiro com o valor baixo e depois com o valor alto.
1	0	0	X	Esta situação só acontece para 7 bits ou quando recebido o segundo byte de controle para a transmissão quando operando em 10 bits, onde também teremos R/W = 1.
1	X	X	1	Significa que a última informação recebida foi perdida, pois SSPBUF não havia sido limpo.

Figura 8.37: Verificando a situação do sistema através dos bits.

CAPÍTULO 9

UNIVERSAL SYNCHRONOUS ASYNCHRONOUS RECEIVER TRANSMITTER (USART).

9.1 – INTRODUÇÃO.

O módulo USART (Universal Synchronous Asynchronous Receiver Transmitter é outra das portas de comunicação série, também conhecida como SCI (Serial Communications Interface). Para configurar o modo USART basta setar os bits SPEN (RCSTA<7>) e TRISC<7:6>), de modo a configurar os pinos RC6/TX/CK e RC7/RX/DT como USART.

9.1.1 - A USART pode ser configurada de dois modos diferentes:

- a) Assíncrono (full duplex),** – usado para comunicação com terminais, computadores, etc.
- b) Síncrono Mestra (half duplex) ou Síncrono Escrava (half duplex),** - para comunicar o microcontrolador com outros periféricos tais como circuitos integrados conversores A/D ou D/A, memórias EEPROM série, modems, etc.

No modo Síncrono, as duas configurações usam formatos padrão: um bit de START, oito (mais usado) ou nove bits de dados e um bit de STOP.

9.2 – GERADOR DE BAUD RATE DA USART (BRG).

O BRG é um gerador dedicado de baud rate de 8 bits e suporta os modos Assíncrono e Síncrono da USART. Baud Rate (Taxa de transferência) é a quantidade de bits que pode ser transmitida por segundo (bps). O PIC16F877 possui um circuito gerador de baud rate (BRG), cujo valor para a transmissão é calculado a partir das fórmulas mostradas na **Figura 9.1**. O valor do baud rate depende da Fosc, do bit BRGH (TXSTA<2>) e do valor armazenado no registro SPBRG.

O registro SPBRG controla o período do temporizador livre de 8 bits. No modo assíncrono o bit BRGH (TXSTA<2>) controla o baud rate (alta ou baixa velocidade) e no modo síncrono este bit é ignorado.

A **Figura 9.1** mostra a fórmula para cálculo do baud rate para diferentes modos da USART no modo mestre (clock interno).

Para seu cálculo é preciso usar a seguinte formula:

$$\text{Bauds} = \text{Fosc}/[n(\text{SPBRG} + 1)] \therefore \text{SPBRG} = [\text{Fosc}/(n.\text{Baud Rate})] - 1.$$

Obs: $n = 4$ para o modo síncrono, 16 para o modo assíncrono de alta velocidade (BRGH=1) e 64 para o modo assíncrono de baixa velocidade (BRGH=0). O valor que será carregado no registro SPBRG varia entre 0 e 255.

O ideal é que se faça o cálculo tanto para alta como para baixa velocidade e após, fazer a verificação da percentagem do erro para decidir o melhor valor para o registro SPBRG (menor erro).

Exemplo: Qual valor deve ser escrito no registro SPBRG para uma transmissão assíncrona com Baud Rate = 19200 bps e uma Fosc = 4MHz?

$$\text{Bauds} = \text{Fosc}/[n(\text{SPBRG} + 1)]$$

$$\text{SPBRG} = [\text{Fosc}/(n.\text{Baud Rate})] - 1$$

BRGH = 0 $\rightarrow n = 64$	BRGH = 1 $\rightarrow n = 16$
$\text{SPBRG} = [\text{Fosc}/(n.\text{Baud Rate})] - 1$	$\text{SPBRG} = [\text{Fosc}/(n.\text{Baud Rate})] - 1$
$\text{SPBRG} = [4.10^6/(64.19200)] - 1$	$\text{SPBRG} = [4.10^6/(16.19200)] - 1$
SPBRG = 2,25	SPBRG = 12,02

Como o resultado não deu valor inteiro em ambos os casos, teremos que aproximar para o valor inteiro mais próximo. Assim teremos:

BRGH = 0	BRGH = 1
SPBRG = 2	SPBRG = 12

A aproximação fará com que o baud rate seja diferente de 19200 bps. Essa diferença é chamada de Erro de Baud Rate (EBR) e será calculada em percentagem:

BRGH = 0	BRGH = 1
Baud rate = $4 \times 10^6 / [64 \times (2 + 1)]$	Baud rate = $4 \times 10^6 / [16 \times (12 + 1)]$
Baud rate = 20833,33 bps	Baud rate = 19230,76 bps.
$\text{EBR} = [(20833,33 - 19200)/19200] \times 100$	$\text{EBR} = [(19230,76 - 19200)/19200] \times 100$
EBR = 8,5%	EBR = 0,16%

Observe que o Erro de baud rate é muito menor para BRGH = 1, sendo que a pequena diferença não causará problemas na transmissão.

SYNC (TXSTA<4>)	BRGH = 0 (baixa velocidade)	BRGH = 1 (alta velocidade)
0 (Assíncrono)	Baud Rate = $F_{osc}/[64x(SPBRG + 1)]$	Baud Rate = $F_{osc}/[16x(SPBRG + 1)]$
1 (Síncrono)	Baud Rate = $F_{osc}/4x(SPBRG + 1)]$	Não usado.

Figura 9.1: Fórmula para Baud Rate.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 - 010	0000 - 010
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.2: Registro associado com o gerador de baud rate.

Os dados do pino RC7/RX/DT são mostrados em três tempos para detetar e determinar melhor se o nível presente no pino RX é alto ou baixo.

Se o bit BRGH = 0 (TXSTA<2>) a amostragem é realizada na sétima, oitava e nona borda abaixo de x16 de clock. Se, pelo contrário, o bit BRGH = 1, a amostragem é realizada na terceira borda de clock, precedido da segunda subida do clock a da primeira descida de x4 de clock.

9.2.1 - TXSTA: Registro de Controle e Estado da Transmissão (endereço 98h).

R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R-1	R/W-0
CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D
bit 7							bit 0

bit 7 CSRC: Clock Source Select bit (Bit de seleção da fonte de clock para o modo síncrono mestre/escravo)

Synchronous mode:

1 = modo Mestre (clock gerado internamente de BRG)

0 = modo Escravo (clock de uma fonte externa)

bit 6 TX9: 9-bit Transmit Enable bit (Bit de habitação da transmissão de 8 ou 9 bits)

1 = Transmissão em 9 bits

0 = Transmissão em 8 bits

bit 5 TXEN: Transmit Enable bit (Habilitação da transmissão)

1 = Transmissão habilitada

0 = Transmissão desabilitada

Nota: SREN/CREN anulam (overrides) TXEN no modo SYNC.

bit 4 SYNC: USART Mode Select bit (Seleção do modo da USART)

1 = Modo Síncrono

0 = Modo Assíncrono

bit 3 sem implementar: Em leitura é zero (0)

bit 2 BRGH: High Baud Rate Select bit (Bit de seleção da velocidade de Baud Rate – somente para o modo assíncrono)

Modo assíncrono:

1 = Alta velocidade

0 = Baixa velocidade

bit 1 TRMT: Transmit Shift Register Status bit (Bit de estado do Registro de Deslocamento de Transmissão (TSR))

1 = TSR vazio

0 = TSR cheio

bit 0 TX9D: 9th bit of Transmit Data, can be parity bit (Nono bit de dado na transmissão, pode ser usado como bit de paridade ou endereçamento)

9.2.2 - RCSTA: Registro de Controle e Estado da Recepção (endereço 18h).

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-x
SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
bit 7							bit 0

bit 7 SPEN: Serial Port Enable bit (Habilitação da porta série – habilita a USART)

1 = Porta série habilitada, habilita a USART (configura os pinos RC7/RX/DT e RC6/TX/CK para porta série)

0 = Porta série desabilitada – desabilita a USART

bit 6 RX9: 9-bit Receive Enable bit (Habilita recepção de 9 ou 8 bits).

1 = Seleciona 9 bits da recepção

0 = Seleciona 8 bits da recepção

bit 5 SREN: Single Receive Enable bit (Habilitação da recepção única – somente no modo síncrono mestre)

Modo síncrono - mestre:

1 = Habilita recepção única

0 = Desabilita

Nota: Este bit é apagado após completar a recepção.

bit 4 CREN: Continuous Receive Enable bit (Habilitação da recepção contínua)

Modo Assíncrono:

1 = Habilita recepção contínua

0 = Desabilita

Modo síncrono:

1 = Habilita recepção contínua até que o bit CREN seja apagado (CREN anula SREN)

0 = Desabilita recepção contínua

bit 3 ADDEN: Address Detect Enable bit (Bit de habilitação de detecção de endereço)

Modo Assíncrono de 9 bits (RX9 = 1):

1 = Habilita a detecção de endereço, habilita a interrupção e carrega o buffer do receptor quando RSR<8> = 1.

0 = Desabilita a detecção de endereço, todos os bytes são recebidos e o nono bit pode ser usado como bit de paridade.

bit 2 FERR: Framing Error bit (Erro de Stop bit; somente modo assíncrono)

1 = Ocorreu erro. Stop bit = 0 (deve atualizado lendo o registro RCREG e recebe o próximo dado válido).

0 = Não ocorreu erro. Stop bit = 1.

bit 1 OERR: Overrun Error bit (Erro de muitos bytes recebidos sem nenhuma leitura ou erro de transbordamento)

1 = Erro de transbordamento (estouro do limite de 3 bytes recebidos antes da leitura de RCREG), pode ser limpo pelo bit CREN = '0'.

0 = Não houve erro

bit 0 RX9D: 9th bit of Received Data (Nono bit de dados recebido, pode ser bit de paridade)

9.3 – USART NO MODO ASSÍNCRONO.

O módulo USART assíncrono consiste dos seguintes blocos:

a) Gerador de Baud Rate.

b) Circuito de amostragem.

c) Transmissor assíncrono.

d) Receptor assíncrono.

Neste modo de funcionamento, a USART pode receber e transmitir dados simultaneamente, de 8 ou 9 bits e um bit STOP. O transmissor e o receptor funcionam de modo independente, porém precisam utilizar o mesmo formato de dados e o mesmo Baud rate. O BRG produz sempre os sinais de clock x16 ou x64 (dependendo do bit BRGH, TXSTA<2>) para o deslocamento dos bits.

O bit de paridade, embora não suportado pelo hardware, pode ser implementado por software, sendo transmitido como nono bit. O modo assíncrono é selecionado apagando o bit SYNC (TXSTA<4>).

9.3.1 - USART com Transmissão Assíncrona.

A transmissão é habilitada quando setamos o bit TXEN (TXSTA<5>), e é iniciada quando o dado a ser transmitido é carregado no registro TXREG. Uma vez no TXREG, o dado é automaticamente transferido para o registro TSR (Transmitter Shift Register), sendo que no mesmo instante o bit TXIF (PIR1<4>) setado, indicando que o TXREG está vazio. Depois de transferido para o registro TSR, o dado a ser transmitido vai sendo colocado, bit a bit, no pino RC6/TX/CK durante o tempo que chamamos de Período do bit ($P_b = 1/\text{Baud Rate}$). O bit TRMT (TXSTA<4>) indica o estado do registro TSR (setado indica TSR vazio). O registro TSR não é mapeado na RAM e, portanto o usuário não tem acesso a ele.

Quando a transmissão for de 9 bits, devemos setar o bit TX9 (TXSTA<6>) e escrever o nono bit em TX9D (TXSTA<0>) antes de carregarmos os outros 8 bits no registro TXREG. Isto evita que o nono bit seja transmitido erradamente, caso TXREG seja transferido imediatamente para o TSR.

Se o bit TXEN for apagado antes da transmissão ter sido completada, ela será abortada e o módulo transmissor será resetado, sendo o pino TX colocado em alta Z.

A **Figura 9.3** mostra o diagrama de blocos do módulo transmissor da USART.

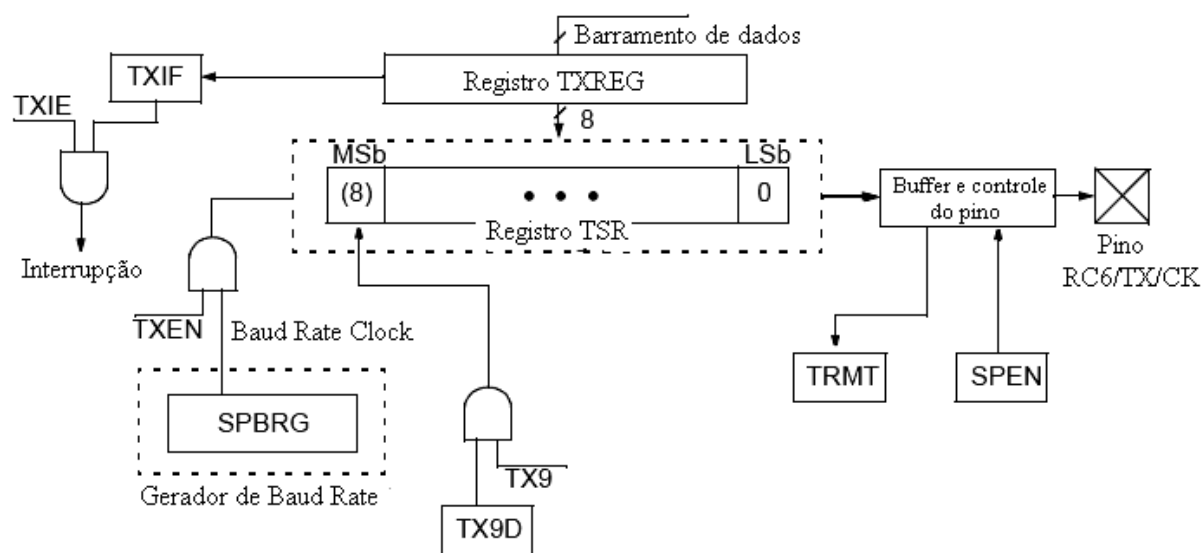


Figura 9.3: Diagrama de blocos do transmissor da USART.

9.3.2 - Interrupção do módulo transmissor.

Caso a interrupção esteja habilitada pelo bit TXIE (PIE1<4>), ela ocorrerá no momento em que o dado for transferido para o registro TSR, o que acontece um ciclo de instrução após o dado ter sido carregado no registro TXREG. Ao ocorrer a interrupção, o programa será desviado para o endereço 0x04 no qual a interrupção deve ser tratada. O bit TXIF (PIR1<4>) setado indica que a interrupção ocorreu, porém ele não se apagado por software e sim por hardware quando um novo dado for carregado no registro TXREG.

Obs: Para que a interrupção ocorra e o micro tome conhecimento todos os bits chaves de interrupções deverão estar habilitados.

9.3.3 - Os passos necessários para uma Transmissão Série Assíncrona são:

- a) Inicializar o registro SPBRG com o valor apropriado de baud rate de acordo com os cálculos realizados. Se desejar baud rate de alta velocidade, setar o bit BRGH.
- b) Configurar pinos RC7:RC6 como entradas.
- c) Habilitar a porta série assíncrona apagando o bit SYNC e setando o bit SPEN.
- d) Se desejar, habilite a interrupção.
- e) Setar o bit TX9 se a transmissão for de 9 bits.
- f) Habilitar o transmissor setando o bit TXEN, o qual setará também o bit TXIF.

- g) Se a transmissão for de 9 bits, escrever o nono bit no bit TX9D (TXSTA<0>).
- h) Carregar o dado a ser transmitido no registro TXREG (inicia a transmissão).
- i) Se usar interrupções, assegure que os bits GIE e PEIE estejam habilitados.
- j) A transmissão estará finalizada quando bit TRMT for setado pelo hardware.

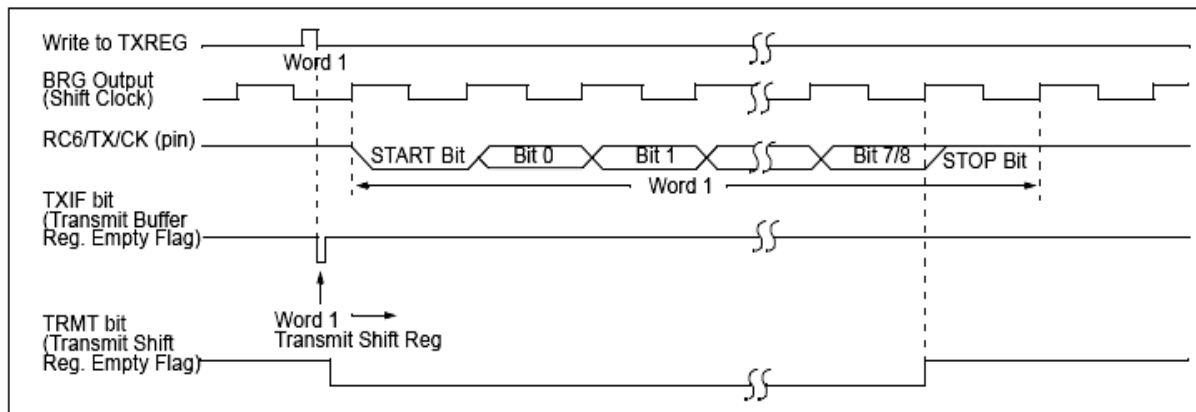


Figura 9.4: Temporização para uma transmissão assíncrona.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 - 00x	0000 - 00x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 - 010	0000 - 010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.5: Registros associados com a transmissão assíncrona.

O código seguinte mostra como inicializar a USART e transmitir a letra “A” com uma taxa de transferência de 19200 bps com $F_{osc} = 4\text{MHz}$. Usaremos a código ASCII onde a letra A = 41h.

```
;Transmissao_Assincrona
bank1
bsf    TXSTA,BRGH ;alta velocidade de transferência
movlw  . 12
movwf  SPBRG      ;configura baud rate para 19200 bps
bcf    TXSTA,SYNC ;ativa comunicação assíncrona
bsf    TRISC,7
bsf    TRISC,6
bank0.
bsf    RCSTA,SPEN ;habilita porta serial
bank1
bsf    TXSTA,TXEN ;habilita transmissão
bank0
movlw  A'A'
movwf  TXREG      ;inicia transmissão do caracter 'A' (41h)
bank1
btfss  TXSTA,TRMT ;transmissão completada?
goto   $-1        ;não, testa novamente
bank0
        ;sim, ativa banco0
return          ;retorna
```

9.3.4 - Recepção Assíncrona da USART.

O diagrama de blocos do receptor assíncrono da USART é mostrado na **Figura 9.6**.

No modo de recepção, os dados são recebidos pelo pino RC7/RX/DT e introduzidos em série no registro RSR. Depois de receber o bit de STOP, o dado acumulados neste registro é transferido para o registro de dados RCREG (se estiver vazio). Quando a transferência estiver completada, o bit RCIF (PIR1<5>) será setado. A interrupção pode ser habilitada ou desabilitada pelo bit RCIE (PIE1<5>). Este bit será apagado quando o registro RCREG tiver sido lido e estiver vazio.

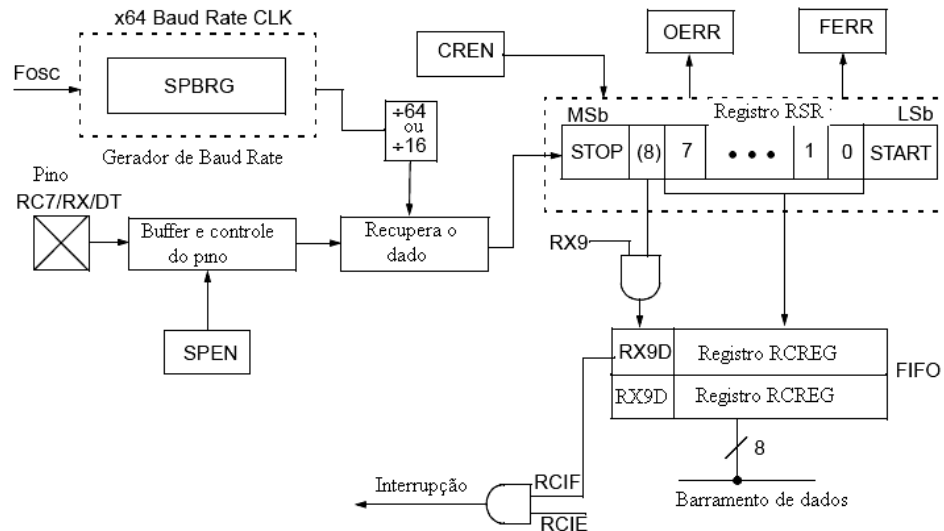


Figura 9.6: Diagrama de blocos do receptor da USART.

A USART pode receber até dois bytes sem necessidade de leitura do registro RCREG. Cada vez que um novo byte chega pela USART, ele é armazenado primeiramente em um registro não acessível pelo software chamado RSR. Do registro RSR o dado é transferido automaticamente para o registro RCREG, que possui dois níveis de pilha do tipo FIFO. Isto significa que nós podemos ter dois bytes armazenados no registro RCREG e um terceiro byte pode começar a chegar ao registro RSR. Porém, se o STOP bit do terceiro byte chegar antes da leitura do registro RCREG, o byte que acabou de chegar será perdido e o bit OERR (RCSTA<1>) será setado, informando que houve um transbordo.

O registro RCREG deve ser lido logo após a chegada de um novo byte para evitar o risco de ocorrência de transbordo. Isto pode ser facilmente conseguido, se a interrupção de recepção da USART estiver habilitada. Uma vez que tenha ocorrido a interrupção, deve ser efetuada a leitura do registro RCREG dentro da rotina de interrupção.

Uma vez que tenha ocorrido um transbordo, bit OERR setado, a transferência de dados do registro RSR para o registro RCREG estará inibida, tendo o usuário que apagar o bit OERR para que a USART possa voltar a receber dados. Isto não pode ser feito diretamente, tendo o usuário que apagar o bit CREN (RCSTA<4>).

Um outro tipo de erro que pode ser detectado pelo módulo receptor da USART é o STOP bit recebido como zero (0). Se isto acontecer, o bit FERR (RCSTA<2>) será

setado, indicando que ocorreu um erro de transmissão. Como uma leitura no registro RCREG atualiza os bits FERR e o RX9D, recomenda-se efetuar a leitura desses dois bits antes de ler o registro RCREG, para que eles não sejam perdidos.

A **Figura 9.7**, diagrama de temporização, mostra três palavras na entrada do Rx. O registro RCREG (buffer recepção) é lido após a terceira palavra causando um overflow (bit OERR está setado).

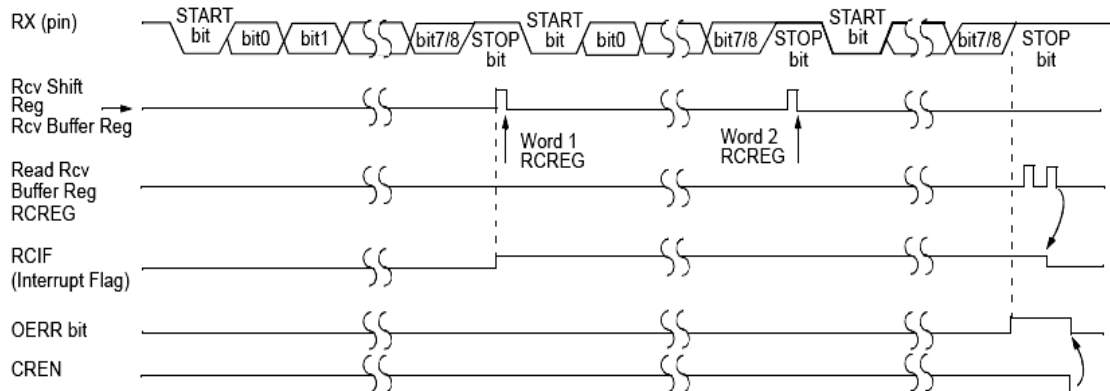


Figura 9.7: Temporização para a recepção assíncrona.

9.3.5 - Interrupção de módulo receptor.

A interrupção é sinalizada pelo bit RCIF (PIR1<5>), não podendo ser apagado por software. Uma vez que a interrupção esteja habilitada e o dado recém chegado pela USART for carregado no registro RCREG, a interrupção acontecerá e o programa será desviado para o endereço 0x04, onde ela deverá ser tratada. O bit RCIF é automaticamente apagado pelo hardware quando o registro RCREG estiver vazio. Se a pilha do registro RCREG estiver cheia, ele terá de ser lido duas vezes para que o bit RCIF seja apagado.

A interrupção do módulo receptor da USART é habilitada setando o bit RCIE (PIE1<5>). Não esquecer também de habilitar os bits GIE e PEIE.

9.3.6 – Os passos necessários para uma Recepção Série Assíncrona são:

- a) Inicializar o registro SPBRG com o valor apropriado de baud rate de acordo com os cálculos realizados. Se desejar baud rate de alta velocidade, setar o bit BRGH.
- b) Configurar os pinos RC7:RC6 como entradas.

- c) Habilitar a porta série assíncrona apagando o bit SYNC (TXSTA<4>) e setando o bit SPEN (RCSTA<7>).
- d) Se desejar, habilite a interrupção setando o RCIE.
- e) Se a recepção for de 9 bits, setar o bit RX9.
- f) Habilite a recepção setando o bit CREN (RCSTA<4>).
- g) O bit de flag de interrupção RCIF será setado quando uma recepção estiver completa e uma interrupção será gerada se ela estiver habilitada.
- h) Ler o registro RCSTA para obter o nono bit (RX9D) ou para verificar se ocorreu um erro durante a recepção (FERR e OERR).
- i) Ler os 8 bits de dados recebidos, que se encontram no registro RCREG.
- j) Se tiver ocorrido um erro, apague o bit CREN para que novos dados possam ser recebidos.
- k) Se usando interrupções, assegure que os bits GIE e PEIE (NTCON<7:6>), estejam habilitados.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	R0IF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.8: Registros associados com a recepção assíncrona.

9.3.7 – Selecionando o modo de 9 bits com detecção de endereço.

Para selecionar Recepção assíncrona com detecção de endereço habilitada, devemos:

- a) Inicializar o registro SPBRG com o valor apropriado de baud rate de acordo com os cálculos realizados. Se desejar baud rate de alta velocidade, setar o bit BRGH.
- b) Configurar os pinos RC7:RC6 como entradas.
- c) Habilitar a porta série assíncrona apagando o bit SYNC (TXSTA<4>) e setando o bit SPEN (RCSTA<7>).
- d) Se desejar, habilite a interrupção setando RCIE.

- e) Setar o bit RX9 pra habilitar recepção de nove bits.
- f) Setar o bit ADDEN para habilitar a detecção de endereço.
- g) Habilite a recepção setando o bit CREN.
- h) O bit de flag de interrupção RCIF será setado quando uma recepção estiver completa e uma interrupção será gerada se ela estiver habilitada.
- i) Ler o registro RCSTA para obter o nono bit (RX9D) ou para verificar se ocorreu um erro durante a recepção (FERR e OERR).
- j) Ler os 8 bits de dados recebidos, que se encontram no registro RCREG para determinar se o dispositivo está endereçado.
- k) Se tiver ocorrido um erro, apague o bit CREN para que novos dados possam ser recebidos.
- l) Se o dispositivo tiver sido endereçado, apagar o bit ADDEN para permitir que bytes de dados e bytes de endereços sejam lidos no buffer receptor e interrompa a CPU.

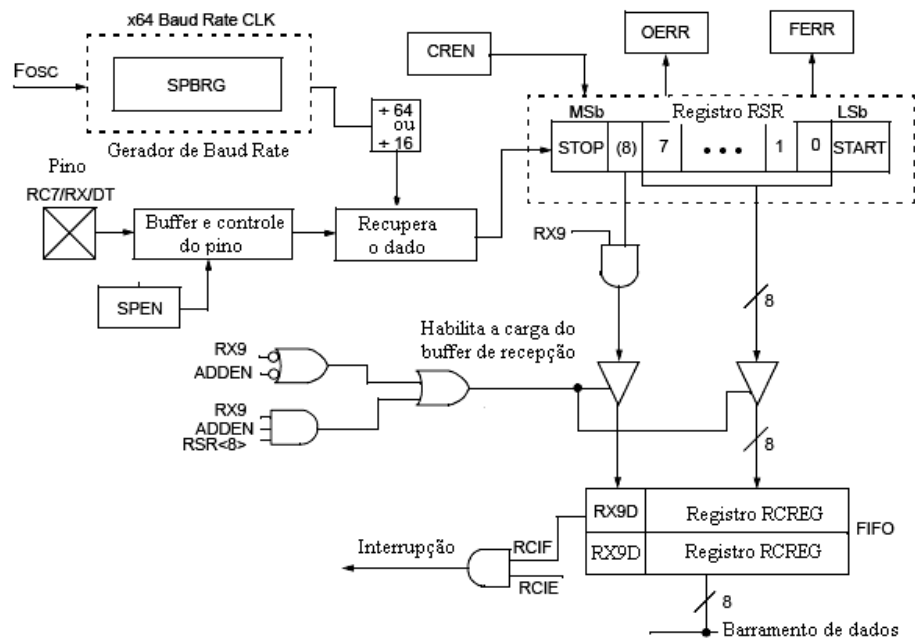


Figura 9.9: Diagrama de blocos do receptor da USART.

A **Figura 9.10**, diagrama de temporização, mostra um byte de dado seguido por um byte de endereço. O byte de dado não é lido no registro RCREG (buffer de recepção) por causa do bit ADDEN = 1.

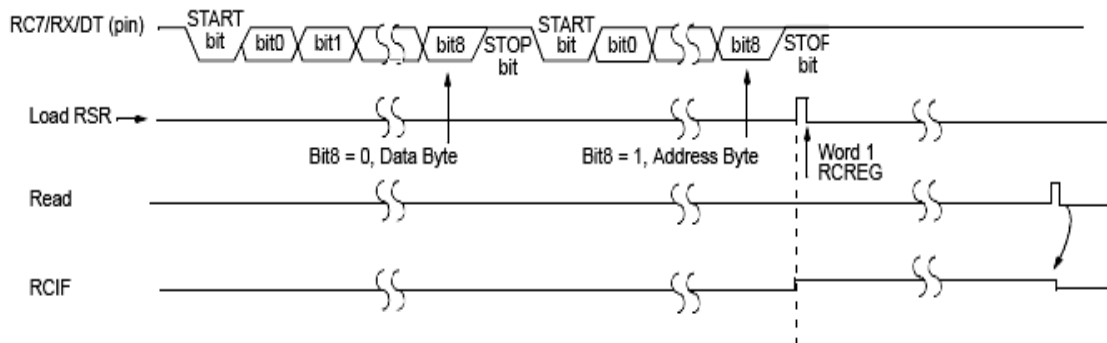


Figura 9.10: Temporização para recepção assíncrona com detecção de endereço.

A **Figura 9.11**, diagrama de temporização, mostra um byte de endereço seguido por um byte de dado. O byte de dado não foi lido no registro RCREG (buffer de recepção) por causa do bit ADDEN que não foi atualizado (continua em zero).

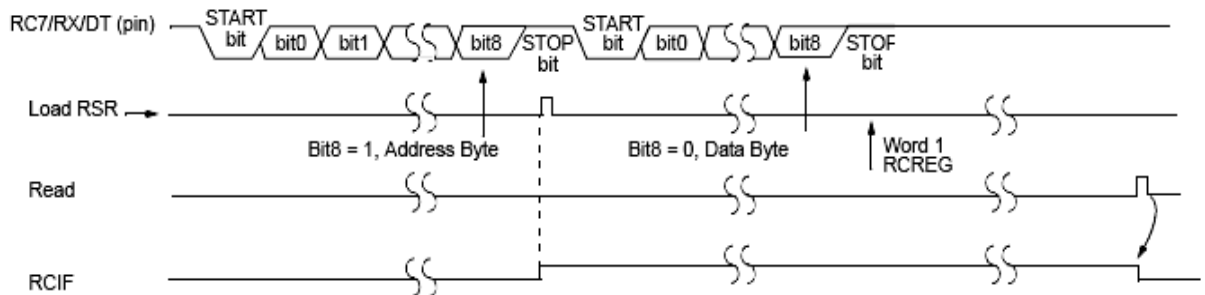


Figura 9.11: Temporização para recepção assíncrona com o byte de endereço primeiro

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	R0IF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.12: Registros associados com a recepção assíncrona

O código seguinte mostra como configurar a USART para receber 8 bits de dados no modo assíncrono com uma taxa de transferência de 19200 bps e Fosc = 4MHz.

```

RECEPÇÃO_ASSINCRONA
    bank1
    bsf    TXSTA,BRGH    ;alta velocidade de transferência
    movlw  .12
    movwf  SPBRG        ;configura baud rate para 19200 bps
    bcf    TXSTA,SYNC    ;ativa comunicação assíncrona
    bsf    TRISC,7
    bsf    TRISC,6
    bank0.
    bsf    RCSTA,SPEN    ;habilita porta serial
AGUARDA_DADO
    bsf    RCSTA,CREN    ;habilita recepção
    btfss  PIR1,RCIF     ;recepção completada?
    goto   $-1           ;não, testa novamente
    btfsc  RCSTA,FERR     ;bit Stop = 1?
    goto   ERRO_RECEPCAO ;não, houve um erro de recepção
    movf   RCREG,W       ;sim, recepção correta
    return
ERRO_RECEPCAO
    bcf    RCSTA,CREN    ;desabilita recepção
    bsf    SERIAL_ERRO   ;seta flag
    return

```

9.4 - USART NO MODO SÍNCRONO MESTRE.

Como já vimos, a USART no modo síncrono e semi-duplex ou half duplex e pode funcionar como Mestra ou como Escravo.

A seleção entre Síncrona Mestra ou Escravo é feita pelo bit CSRC (TXSTA<7>).

No modo Síncrono Mestra, a transmissão e a recepção não acontecem ao mesmo tempo, quando se transmite um dado a recepção é inibida e vice-versa.

Na transmissão, o registro de deslocamento, TSR, obtém os dados do registro de leitura/escrita TXREG, mas este não é carregado até o último bit do carregamento anterior ter sido transmitido. Quando isto suceder, o TSR volta a ser carregado com o novo dado do registro TXREG (se estiver disponível), este é esvaziado e é gerada uma interrupção (se estiver habilitada pelo bit TXIE, PIE1<4>).

9.4.1 - Transmissão Síncrona Mestra da USART.

A transmissão é habilitada pelo bit TXEN (TXSTA<5>) e se for apagado durante a transmissão, esta será abortada.

Para iniciar uma Transmissão Síncrona Mestra, devemos:

- a) Inicializar o registro SPBRG com o valor apropriado (baud rate) de acordo com os cálculos realizados.
- b) Habilitar a porta série síncrona mestra setando os bits SYNC, SPEN e CSRC.
- c) Se for usar interrupção, setar o bit TXIE.
- d) Se desejar transmissão de 9 bits, setar o bit TX9.
- e) Habilite a transmissão setando o bit TXEN.
- f) Se tiver habilitado transmissão de 9 bits, o nono será carregado no bit TX9D.
- g) A transmissão será iniciada ao carregar o dado no registro TXREG.
- h) Se usar interrupções assegure que os bits GIE e PEIE (INTCON<7:6>) estejam habilitados.

Esta seleção configura os pinos de entradas/saídas RC6/TX/CK e RC7/RX/DT para CK (clock) e DT (dados), respectivamente.

Na **Figura 9.13** temos o modo síncro mestre, SPBRG = 0 e transmissão contínua de duas palavras de 8 bits cada.

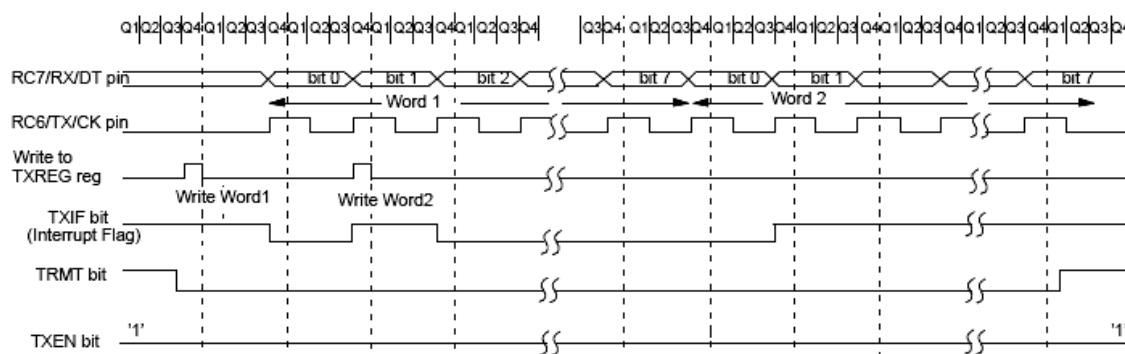


Figura 9.13: Temporização para uma transmissão síncrona mestra

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	R0IF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.14: Registros associados com transmissão síncrona mestra.

9.4.2 - Recepção Síncrona Mestra da USART.

Para selecionar este modo, é preciso setar o bit de habilitação SREN (RCSTA<5>) ou o bit de habilitação CREN (RCSTA<4>). O dado é amostrado no pino RC7/RX/DT na borda de descida do clock.

Se o bit de habilitação SREN estiver setado, só se receberá uma palavra.

Se o bit de habilitação CREN estiver setado a recepção é contínua até que CREN ser apagado.

Se ambos os bits estiverem a setados, CREN tem precedência.

Depois do último pulso de clock, o dado recebido no registro de deslocamento RSR (Receive Shift Register), será transferido para o registro RCREG (se estiver vazio).

Quando terminada a recepção, o flag de interrupção, bit RCIF (PIR1<5>) será setado, gerando uma interrupção (se habilitada). A interrupção pode ser habilitada/desabilitada

pelo bit RCIE (PIE1<5>). O bit de flag é um bit somente de leitura e que é resetado pelo hardware.

Neste caso, é resetado quando o registro RCREG tenha sido lido e esteja vazio. O RCREG é um registro buffer duplo (dois FIFO). É possível receber dois bytes de dados e transferir para o RCREG FIFO e um terceiro byte para iniciar o deslocamento para o registro RSR. No clock do último bit do terceiro byte, se o registro RCREG estiver ainda cheio, o “*overflow error bit*” OERR (RCSTA<1>) é setado. A palavra no RSR será perdida.

O registro RCREG pode ser lido duas vezes para recuperar os dois bytes no FIFO. O bit OERR tem que ser limpo por software (limpando o bit CREN). Se o bit OERR estiver setado, transferências do RSR para o RCREG estarão inibidas, então é essencial limpar o bit OERR se ele estiver setado.

O nono bit recebido é carregado da mesma maneira que o dado recebido. Ler o registro RCREG carregará o bit RX9D com um novo valor, conseqüentemente, é essencial para o usuário ler o registro de RCSTA antes da leitura de RCREG de modo a não perder a informação anterior de RX9D.

A sequência para iniciar uma Recepção Síncrona Mestra, é:

- a)** Inicializar o registro SPBRG com o valor apropriado de bauds.
- b)** Habilitar a Porta Série Síncrona Mestre setando os bits SYNC, SPEN e CSRC.
- c)** Assegurar-se que os bits CREN e SREN estão apagados.
- d)** Se desejar empregar interrupções, setar o bit RCIE.
- e)** Se desejar recepção de nove bits, então setar o bit RX9.
- f)** Se a recepção for de palavra única, setar o bit SREN. Para recepção contínua, setar o bit CREN.
- g)** O flag de interrupção bit RCIF será setado quando a recepção estiver completa e uma interrupção será gerada caso o bit RCIE tenha sido habilitado.
- h)** Ler o registro RCSTA para conseguir o nono bit (se habilitado) e verificar se ocorreu um erro durante a recepção.
- i)** Ler os oito bits de dados recebidos pela leitura do registro RCREG.
- j)** Se ocorreu qualquer erro, apague o erro, apagando o bit CREN.

k) Se estiver usando interrupção, assegure que os bits GIE e PEIE (INTCON<7:6>) estejam setados.

A **Figura 9.15**, diagrama de temporização, demonstra o modo síncrono mestre com o bit SREN setado e o bit BRG apagado (0).

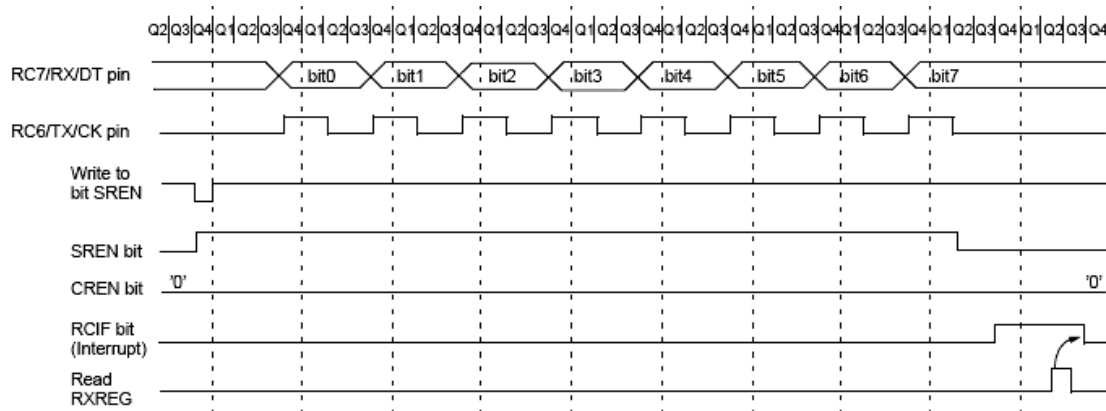


Figura 9.15: Recepção síncrona modo mestre (SREN)

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	R0IF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.16: Registros associados com a recepção síncrona mestra.

9.5 - USART NO MODO SÍNCRONO ESCRAVO.

O modo Escravo Síncrono difere do modo Mestre pelo fato do sinal de clock de deslocamento ser fornecido externamente através do pino RC6/TX/CK (em vez de internamente como no modo Mestre). Isto permite que seja possível transferir ou receber dados quando o dispositivo está no modo SLEEP. O modo escravo é selecionado quando setamos o bit CSRC (TXSTA<7>).

9.5.1 - Transmissão Síncrona Escrava da USART.

As operações de transmissão são idênticas ao modo Mestre, exceto no caso do modo SLEEP.

Se duas palavras forem escritas no registro TXREG e após se executar uma instrução SLEEP, pode ocorrer o seguinte:

- a) A primeira palavra será imediatamente transferida para o registro TSR e transmitida.
- b) A segunda palavra permanecerá no registro TXREG.
- c) O flag bit TXIF não será setado.
- d) Quando a primeira palavra tiver saído do registro TSR, a segunda palavra, que permanece no registro TXREG, será enviada para o registro TSR e o flag, bit TXIF, será setado.
- e) Se o bit de habilitação TXIE estiver setado, a interrupção despertará o micro de SLEEP e se a interrupção global estiver habilitada, o programa saltará para o endereço do vetor de interrupções (0004h).

9.5.2 - Para iniciar uma Transmissão Síncrona Escrava, deve-se:

- a) Habilitar a Porta Série Síncrona Escrava setando os bits SYNC e SPEN e apagando o bit CSRC.
- b) Apagar os bits CREN e SREN.
- c) Se usar interrupções, setar o bit TXIE.
- d) Se desejar transmissão de 9 bits, setar o bit TX9.
- e) Habilite a transmissão setando o bit TXEN.
- f) Se selecionada transmissão de 9 bits, o nono bit será carregado no bit TX9D.
- g) Inicie a transmissão carregando os dados no registro TXREG.
- h) Se usando interrupções, assegure que os bits GIE e PEIE (INTCON<7:6>), estão setados.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.17: Registros associados com a transmissão síncrona escrava.

9.5.3 - Recepção Síncrona Escrava da USART.

As operações de recepção são idênticas ao modo Mestre, exceto no caso do modo SLEEP. O bit SREN é “*don't care*” no modo Escravo.

Se a recepção foi habilitada setando o bit CREN, antes da instrução SLEEP, então a palavra pode ser recebida durante o SLEEP. Após o completo recebimento da palavra, o registro RSR transfere os dados para o registro RCREG e se o bit RCIE estiver setado, a interrupção gerada despertará o micro do SLEEP. Se a interrupção geral estiver habilitada, o programa muda para o vetor de interrupção (0004h).

9.5.4 - Para a Recepção Síncrona Escrava devemos:

- a) Habilitar a Porta Serial Síncrona Mestra setando os bits SYNC e SPEN e limpando o bit CSRC.
- b) Se interrupções são desejadas, setar o bit RCIE.
- c) Se desejar recepção de 9 bits, setar o bit RX9.
- d) Para habilitar a recepção, setar o bit CREN.
- e) O flag, bit RCIF, será setado quando a recepção for completada e uma interrupção será gerada, se setado previamente o bit RCIE.
- f) Ler o registro RCSTA para conseguir o nono bit (se habilitado) e verificar se ocorreu erro durante a recepção.
- g) Ler os 8 bits de dados recebidos pela leitura do registro RCREG.
- h) Se ocorreu erro, limpe o erro apagando o bit CREN.
- i) Se usando interrupções, assegure que os bits GIE e PEIE estejam setados.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	R0IF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Figura 9.18: Registros associados com a recepção síncrona escrava.

CAPÍTULO 10

MÓDULO CONVERSOR ANALÓGICO-DIGITAL (ADC).

10.1 – INTRODUÇÃO.

Esta série de microcontroladores possui somente um módulo Conversor Analógico/Digital de 10 bits. Os dispositivos de 28 pinos possuem cinco canais de entrada multiplexadas e os demais, oito canais multiplexados. A cada momento é realizada a conversão para digital de um dos canais.

A entrada analógica carrega um capacitor de amostragem e retenção. A saída do capacitor é para a entrada do conversor. O conversor gera um sinal digital a partir de aproximações sucessivas do nível. A conversão de analógico para digital do sinal analógico de entrada, resulta em um número digital correspondente, de 10 bits. O módulo tem níveis alto e baixo de voltagem de referência de entrada que são selecionáveis por software para algumas combinações de VDD, VSS, RA2, ou RA3.

O Conversor possui uma única característica, ser capaz de operar enquanto o dispositivo está no modo SLEEP. Para operar no modo SLEEP, o clock do conversor A/D é derivado de um oscilador RC interno, configurado através dos bits ADCS1:ADCS0 (ADCON0<7:6>)

10.1.1 - O módulo possui quatro registros:

- a) **ADRESH** – guarda os dois MSb ou os oito MSb
- b) **ADRESL** – guarda os oito LSb ou os dois LSb.
- c) **ADCON0** - registro de controle 0
- d) **ADCON1** - Registro de controle 1

O registro ADCON0, controla a operação do módulo A/D e o registro ADCON1, configura as funções dos pinos da porta. Os pinos da porta podem ser configurados como entradas analógicas (RA3 pode também ser voltagem de referência), ou como entrada/saída digital.

10.1.2 - Tensão de referência.

A tensão de referência define a faixa de tensão analógica que o conversor A/D pode receber na sua entrada. A faixa de tensão é definida pelas tensões aplicadas nas entradas Vref+ (pino RA3/AN3) e Vref- (pino RA2/AN2). A tensão aplicada em

V_{ref+} define o valor máximo da tensão que pode ser aplicada na entrada do conversor enquanto V_{ref-} define o valor mínimo.

Para que a faixa de tensão a ser convertida esteja entre 0V a 5V, é necessário aplicar nas entradas de referência $V_{ref+} = 5V$ e $V_{ref-} = 0V$. Portanto, podemos concluir que a diferença entre as tensões define a faixa de tensão que será convertida. Por outro lado, o valor absoluto de tensão aplicado em cada entrada de referência definirá os limites máximos e mínimos da tensão que será convertida.

As tensões de referências podem ser as internas VDD e VSS ou tensões externas aplicadas nos pinos RA3/AN3 (V_{ref+}) e a RA2/AN2 (V_{ref-}), a escolha é realizada por software. Na escolha de externa, é preciso ter em conta que existem algumas limitações, como:

- a) Máxima voltagem aplicada ao pino V_{ref+} (RA3/AN3) será $VDD + 0,3V$ e a mínima, $VDD - 2,5V$.
- b) Máxima voltagem aplicada ao pino V_{ref-} (RA2/AN2) será $V_{ref+} - 2,0V$ e a mínima, $VSS - 0,3V$.
- c) Máxima voltagem para a diferença entre V_{ref+} e V_{ref-} ($V_{ref+} - V_{ref-}$) será $Vdd + 0,3V$ e a mínima, $2,0V$.

Com alimentação de 5,0V, a tensão em RA3/AN3 não poderá exceder 5,0V, pelo que $V_{ref-} = 3,0V$. Temos que cumprir que a tensão de referência seja $V_{ref+} - V_{ref-} = 2,0V$.

As chaves digitais abrem ou fecham de acordo com os bits dos diferentes registros.

Exemplo1: $V_{ref+} = 5V$ e $V_{ref-} = 0$

$$V_{in} = 5 - 0 = 5V$$

Portanto, faixa de tensão a ser convertida é de 5V enquanto, os valores mínimos e máximos são 0V (mínimo) a 5V (máximo). A resolução será $5V/256 = 19,53mV$

Exemplo2: $V_{ref+} = 4V$ e $V_{ref-} = 1V$

$$V_{in} = 4 - 1 = 3V$$

Portanto, faixa de tensão a ser convertida é de 3V, enquanto os valores mínimos e máximos são 1V (mínimo) e 4V (máximo). A resolução será $3V/256 = 11,71mV$

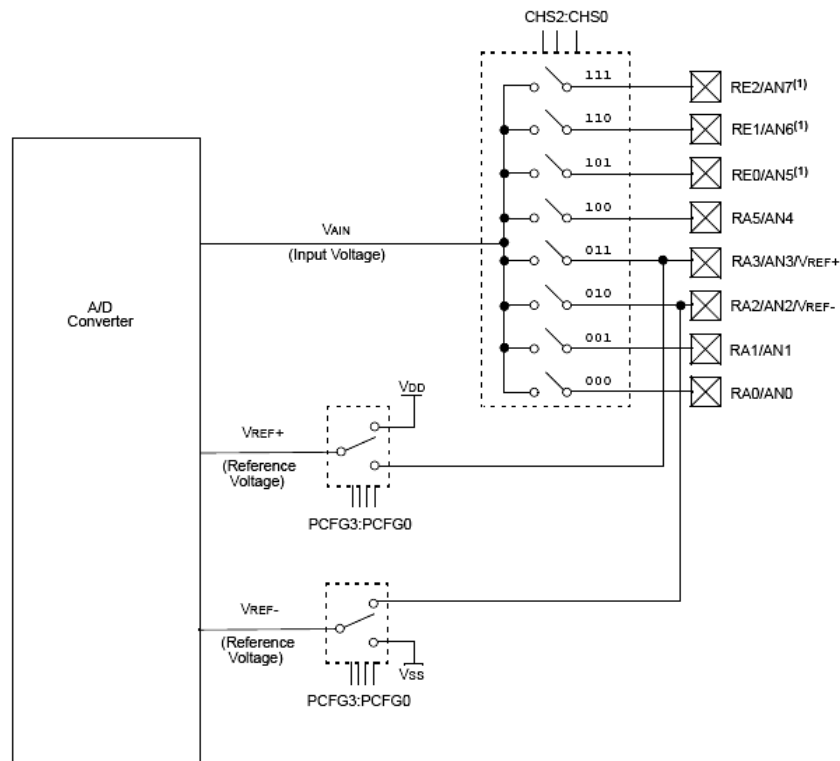


Figura 10.1: Diagrama de blocos do módulo A/D.

10.2 – REGISTROS ADCON0 E ADCON1

10.2.1 - Registro ADCON0 (endereço 1Fh).

Com os bits deste registro configuramos diferentes funções do módulo conversor. Com ADCS1:ADCS0 (ADCON<7:6>), escolhe-se a frequência do clock para o módulo conversor. Com 11₂, o clock é baseado em uma rede RC interna. O conversor pode trabalhar durante o SLEEP, mas para isso, o clock tem que ser interno (rede RC).

O canal ou pino de entrada que se conecta internamente ao módulo é escolhido com os bits CHS2:CHS0 (ADCON0<5:3>).

Antes de iniciar uma conversão é necessário setar o bit GO/DONE (ADCON0<2>).

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON
bit 7							bit 0

bit 7-6 ADCS1:ADCS0: A/D Conversion Clock Select bits (Especificação do clock para o conversor A/D).

00 = $F_{osc}/2$

01 = $F_{osc}/8$

10 = $F_{osc}/32$

11 = F_{RC} (clock de um oscilador RC interno específico para o conversor A/D).

bit 5-3 CHS2:CHS0: Analog Channel Select bits (Bits de seleção do canal de entrada do conversor).

000 = canal 0, (RA0/AN0)

001 = canal 1, (RA1/AN1)

010 = canal 2, (RA2/AN2)

011 = canal 3, (RA3/AN3)

100 = canal 4, (RA5/AN4)

101 = canal 5, (RE0/AN5)

110 = canal 6, (RE1/AN6)

111 = canal 7, (RE2/AN7)

bit 2 GO/DONE: A/D Conversion Status bit (Bit de estado do conversor A/D – depende do bit ADON)

Se ADON = 1:

1 = Conversão em andamento (em '1' lógico, inicia a conversão)

0 = Não efetua a conversão (este bit é automaticamente limpo por hardware quando termina uma conversão)

bit 1 Sem implementar: Em leitura é zero (0).

bit 0 ADON: A/D On bit (Ativa/Desativa o conversor)

1 = Conversor A/D ativado

0 = Conversor desligado para economizar energia.

10.2.2 - Registro ADCON1 (endereço 9Fh).

Anteriormente, mostramos que os 10 bits da conversão, 8 são armazenados no registro ADRESL e os outros 2 no registro ADRESH, mas isto só é verdade em parte. Para que aconteça o bit ADFM (ADCON1<7>) deve estar setado, assim os 10 bits da conversão são alinhados à direita, ficando 8 LSb no registro ADRESL<7:0> e 2 MSb nos dois LSb do registro ADRESH<1:0>. Com o bit ADFM = 0, os 10 bits são

alinhados à esquerda, ficando 8 MSb no registro ADRESH e 2 LSb no MSb do registro ADRESL.<7:6>. Em qualquer caso, sempre ficarão 6 bits livres.

O resultado da conversão é armazenado nos registros ADRESH e ADRESL com os bits justificados à direita ou à esquerda.

Por meio dos bits PCFG3:PCFG0 (ADCON1<3:0>), obtém-se diferentes combinações entre os pinos que serão configurados como entradas analógicas ou entradas/saídas digitais. Como exemplo, com 0110 ou 0111, todos os pinos estão configurados como entradas/saídas digitais e com tensões de referência internas para a conversão (alimentações).

Os registros ADRESH:ADRESL contém os 10 bits resultantes da conversão. Quando a conversão é completada, o resultado é carregado nesses registros, o bit GO/DONE (ADCON0<2>) é apagado e o flag de interrupção, bit ADIF será setado. O diagrama de blocos do módulo ADC é visto na **Figura 10.1**.

Após o módulo ter sido configurado como desejado, o canal selecionado deve ser obtido da conversão a ser iniciada. Os canais de entradas analógicas devem ter seus correspondentes bits do registro TRIS selecionados como entradas.

10.2.3 - Passos a serem seguidos para fazer a conversão A/D:

a) Configure o módulo A/D:

- I)** Configure os pinos para analógico/voltagem de referência e digital I/O (ADCON1)
- II)** Selecione o canal de entrada A/D (ADCON0)
- III)** Selecione o clock para a conversão A/D (ADCON0)
- IV)** Ative o módulo A/D (ADCON0)

b) Configure interrupção A/D (se desejada):

- I)** Limpe o bit ADIF
- II)** Setar o bit ADIE
- III)** Setar o bit PEIE
- IV)** Setar o bit GIE

c) Espere o tempo necessário para a aquisição.

d) Inicie a conversão:

- I)** Setar o bit GO/DONE (ADCON0)

e) Espere a conversão terminar, por um dos dois modos:**I)** Pela seleção do bit GO/DONE ser limpo (com interrupções habilitadas)**II)** Espere pela interrupção A/D**f) Ler o par de registros (ADRESH:ADRESL), limpe o bit ADIF se necessário.****g) Para a próxima conversão, vá para step 1 ou step 2, se necessário.** O tempo de conversão A/D por bit é definido como T_{AD} . Uma espera mínima de $2T_{AD}$ é necessária antes do início da próxima aquisição.

U-0	U-0	R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0
ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0
bit 7							bit 0

bit 7 ADFM: A/D Result Format Select bit (Justificação para o resultado da conversão A/D de 10 bits)

1 = Justificação à direita. Os seis MSb do ADRESH são lidos como zero (0).

0 = Justificação à esquerda. Os seis LSb do ADRESL são lidos como zero (0).

bit 6-4 Sem implementar: Em leitura é zero (0).**bit 3-0 PCFG3:PCFG0:** A/D Port Configuration Control bits (Configuração dos pinos da porta analógicos/digitais/tensões de referência).

PCFG3: PCFG0	AN7 ⁽¹⁾ RE2	AN6 ⁽¹⁾ RE1	AN5 ⁽¹⁾ RE0	AN4 RA5	AN3 RA3	AN2 RA2	AN1 RA1	AN0 RA0	VREF+	VREF-	CHAN/ Refs ⁽²⁾
0000	A	A	A	A	A	A	A	A	VDD	VSS	8/0
0001	A	A	A	A	VREF+	A	A	A	RA3	VSS	7/1
0010	D	D	D	A	A	A	A	A	VDD	VSS	5/0
0011	D	D	D	A	VREF+	A	A	A	RA3	VSS	4/1
0100	D	D	D	D	A	D	A	A	VDD	VSS	3/0
0101	D	D	D	D	VREF+	D	A	A	RA3	VSS	2/1
011x	D	D	D	D	D	D	D	D	VDD	VSS	0/0
1000	A	A	A	A	VREF+	VREF-	A	A	RA3	RA2	6/2
1001	D	D	A	A	A	A	A	A	VDD	VSS	6/0
1010	D	D	A	A	VREF+	A	A	A	RA3	VSS	5/1
1011	D	D	A	A	VREF+	VREF-	A	A	RA3	RA2	4/2
1100	D	D	D	A	VREF+	VREF-	A	A	RA3	RA2	3/2
1101	D	D	D	D	VREF+	VREF-	A	A	RA3	RA2	2/2
1110	D	D	D	D	D	D	D	A	VDD	VSS	1/0
1111	D	D	D	D	VREF+	VREF-	D	A	RA3	RA2	1/2

A = Analog input D = Digital I/O

Obs1: Estes canais não estão disponíveis no dispositivo PIC16F873/876.

Obs2: Esta coluna indica o número de canais analógicos disponíveis como entradas A/D e o número de canais analógicos usados como entradas das voltagens de referências.

Figura 10.2: Os bits PCFG3:PCFG0 configuram os pinos como entradas analógicas ou entradas/saídas digitais.

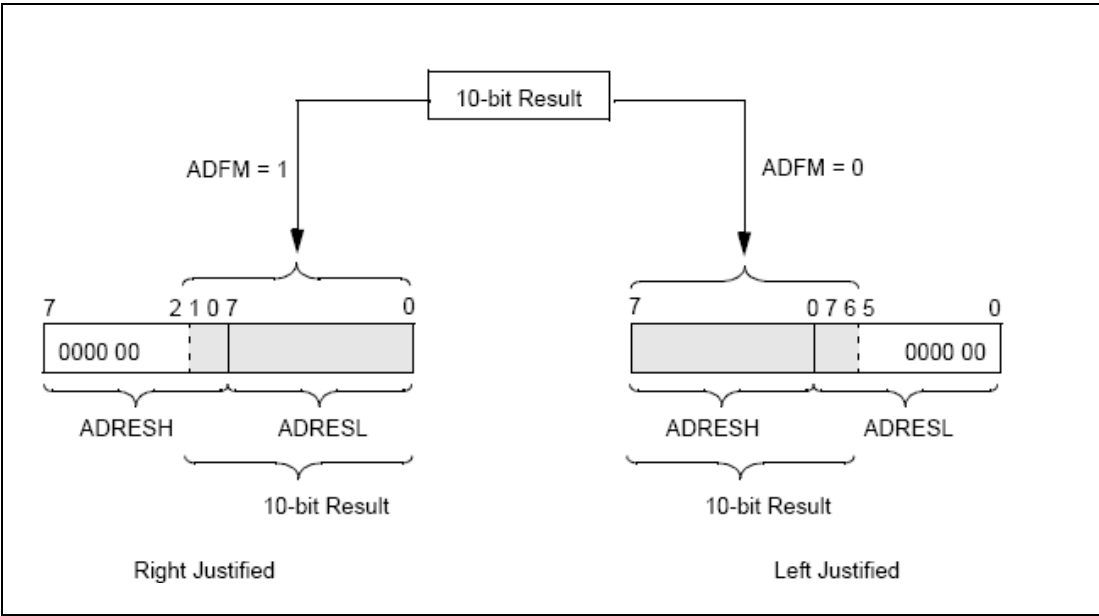


Figura 10.3: Justificação do resultado.

10.2.4 - Registros ADRESH (endereço 1Eh) e ADRESL (endereço 9Eh).

Quando finaliza uma conversão, além de serem carregados os registros ADRESH e ADRESL com o resultado, o bit GO/DONE (ADCON0<2>) será apagado pelo hardware e o bit ADIF (PIR1<6>) será setado indicando fim da conversão (este bit deve ser apagado por software).

É preciso ter em mente que os bits PCFG3:PCFG0 (ADCON1<3:0>) só configuram o comportamento analógico ou digital dos pinos que podem ser conectados ao módulo conversor, mas não os configuram como entradas, o que é feito com os bits do registro TRIS.

a) Exemplo para realizar uma conversão A/D.

```
bcf  STATUS,RP1
bsf  STATUS,RP0    ;seleciona banco 1
clrf ADCON1        ;configura entradas analógicas
bsf  PIE1,ADIE     ;habilita interrupção
bcf  STATUS,RP0    ;seleciona banco 0
movlw b'11000001'
movwf ADCON0       ;clock RC, A/D operativo, Canal 0
bcf  PIR1,ADIF     ;apaga flag de interrupção
bsf  INTCON,PEIE   ;habilita interrupções
bsf  INTCON,GIE    ;habilita todas as interrupções
bsf  ADCON0,GO     ;começa a conversão
```

Obs: Ao término da conversão o bit ADIF será setado e GO/DONE apagado.

10.3 - OPERAÇÃO DURANTE O SLEEP DO CONVERSOR A/D.

Para que o módulo conversor possa funcionar quando a CPU estiver em repouso, é necessário que o clock esteja configurado no modo RC através dos bits ADCS1:ADCS0 = 11 (ADCON0<7:6>). Nesta forma de trabalho, o módulo espera um ciclo de instrução antes de iniciar a conversão. É então quando a instrução SLEEP é executada (o MCU entra no modo Sleep) sem interromper a conversão. Quando finalizar a conversão, se as interrupções estiverem habilitadas, o micro acordará. Se o modo do clock for diferente do RC, a instrução SLEEP fará com que o módulo se desligue e a conversão em andamento é abortada.

10.4 - EFEITOS DE UM RESET.

Um Reset força todos os registros para a condição de Reset. Desliga o módulo A/D e qualquer conversão é abortada. Todos os pinos de entradas A/D são configurados como entradas analógicas. O valor que está nos registros ADRESH:ADRESL, não serão modificados no Power-on Reset (POR). Os registros ADRESH:ADRESL possuirão dados desconhecidos após o Power-on Reset.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on MCLR, WDT
0Bh,8Bh,10Bh,18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
1Eh	ADRESH	A/D Result Register High Byte								xxxx xxxx	uuuu uuuu
9Eh	ADRESL	A/D Result Register Low Byte								xxxx xxxx	uuuu uuuu
1Fh	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON	0000 00-0	0000 00-0
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000
85h	TRISA	—	—	PORTA Data Direction Register						--11 1111	--11 1111
05h	PORTA	—	—	PORTA Data Latch when written: PORTA pins when read						--0x 0000	--0u 0000
89h ⁽¹⁾	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction bits			0000 -111	0000 -111
09h ⁽¹⁾	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu

Figura 10.4: Registros associados com o conversor A/D

CAPÍTULO 11.

CARACTERÍSTICAS ESPECIAIS DA CPU.

11.1 – INTRODUÇÃO.

Todos os dispositivos PIC16F87X possuem internamente características especiais destinadas a maximizar a confiabilidade do sistema, minimizar custos com a eliminação de componentes externos, fornecer modos de operação com baixo consumo e oferecer código de proteção. Algumas dessas características são configuradas através de um registro especial chamado de Registro CONFIG.

11.1.2 - Características especiais são:

- a)** Seleção do oscilador
- b)** RESET
 - I)** Power-on Reset (POR)
 - II)** Power-up Timer (PWRT)
 - III)** Oscillator Start-up Timer (OST)
 - IV)** Brown-out Reset (BOR)
- c)** Interrupções
- d)** Temporizador Watchdog (WDT)
 - I)** Reset do WDT
- e)** SLEEP
 - I)** Reset durante sleep
- f)** Código de proteção
- g)** ID Locations
- h)** In-Circuit Serial Programming
- i)** Low Voltage In-Circuit Serial Programming
- j)** In-Circuit Debugger

O PIC16F877 possui um temporizador Watchdog que pode ser desligado somente através dos bits de configuração. Ele funciona com seu próprio oscilador RC.

Há dois temporizadores que fornecem retardos necessários no Power-up. Um deles é o Oscillator Start-up Timer (OST), destinado a manter o micro em Reset até que o oscilador a cristal se estabilize. O outro é o Power-up Timer (PWRT), o qual produz

um retardo fixo de 72 ms (nominal), somente no Power-up. Sua função é manter o sistema em Reset enquanto a alimentação se estabiliza. Com estes dois temporizadores no micro, a maioria das aplicações não precisam de circuito externo de Reset.

O modo SLEEP é usado para oferecer uma corrente muito baixa no modo Power-down. O despertar de um SLEEP pode ser através de um Reset externo, do Watchdog Timer Wake-up, ou por uma interrupção.

Várias opções de oscilador estão disponíveis para permitir o uso de acordo com a aplicação. O oscilador RC reduz custos enquanto o LP a cristal reduz o consumo de energia. Os bits de configuração são usados para selecionar várias opções.

11.2 - BITS DE CONFIGURAÇÃO DO REGISTRO CONFIG DO PIC16F877.

Depois de analisar todas as funções deste micro, veremos alguns dos procedimentos típicos de funcionamento e configuração. Em primeiro lugar falaremos da Palavra de Configuração do registro CONFIG, pois o estado de seus 13 bits determinará, na hora de programar este micro, o comportamento esperado do dispositivo. Quando empregamos o sistema de desenvolvimento MPLAB no modo MPLAB-ICD Debugger ou se conectado ao PICSTAR Plus, os bits de configuração podem ser modificados nas janelas correspondentes antes de iniciar a fase de gravação propriamente dita do dispositivo.

O Registro CONFIG (usado para armazenar configurações do hardware interno) fica no endereço 2007h da memória de programa e somente é acessível durante a programação do dispositivo. Os bits de configuração podem ser vistos se editarmos os arquivos da pasta MPLAB que possuam a extensão “INC”.

Os bits de configuração podem ser programados (lidos como 0) ou deixados sem programar (lido como 1) para selecionar várias configurações do dispositivo. O valor apagado ou sem programar, da palavra de configuração, é 3FFFh. Estes bits estão mapeados na memória de programa, localização 2007h. É importante observar que o endereço 2007h é após o espaço usado pela memória de programa o qual pode ser acessado somente durante a programação.

11.2.1 - Registro da Palavra de Configuração (endereço 2007h)(1)

CP1	CP0	DEBUG	—	WRT	CPD	LVP	BODEN	CP1	CP0	PWRTÉ	WDTE	F0SC1	F0SC0
bit13										bit0			

bit 13-12 e 5-4 CP1:CP0: FLASH Program Memory Code Protection bits (2) (Bits do código de proteção contra a leitura para a memória Flash de programa)

11 = Sem código de proteção.

10 = endereço protegido (1F00h a 1FFFh, PIC16F877, 876)

10 = endereço protegido (0F00h a 0FFFh, PIC16F874, 873)

01 = endereço protegido (1000h a 1FFFh, PIC16F877, 876)

01 = endereço protegido (0800h a 0FFFh, PIC16F874, 873)

00 = endereço protegido (0000h a 1FFFh, PIC16F877, 876)

00 = endereço protegido (0000h a 0FFFh, PIC16F874, 873)

bit 11 DEBUG: In-Circuit Debugger Mode (Modo In-Circuit Debugger)

1 = In-Circuit Debugger desabilitado, RB6 e RB7 são pinos de entrada/saída normais.

0 = In-Circuit Debugger habilitado, RB6 e RB7 são dedicados ao debugger.

bit 10 Sem implementar: Em leitura estará setado.

bit 9 WRT: FLASH Program Memory Write Enable (Habilita escrita em memória Flash de programa).

1 = Sem proteção, é possível escrever na memória de programa com o controle EECON.

0 = Sem proteção, mas não se pode escrever com o controle EECON.

bit 8 CPD: Data EE Memory Code Protection (Código de proteção da memória de dados EEPROM).

1 = Sem proteção.

0 = Memória EEPROM de dados protegida.

bit 7 LVP: Low Voltage In-Circuit Serial Programming Enable bit (Habilitação da programação em baixa voltagem)

1 = Pino RB3/PGM têm função PGM, baixa voltagem de programação habilitada.

0 = RB3 é entrada/saída digital, alta voltagem em /MCLR deve ser usada para programação.

bit 6 BODEN: Brown-out Reset Enable bit (3) (Habilitação de Brown-out Reset)

1 = BOR habilitado.

0 = BOR desabilitado.

bit 3 /PWRTÉ: Power-up Timer Enable bit (3) (Habilitação do temporizador Power-up)

1 = PWRT desabilitado

0 = PWRT habilitado.

bit 2 WDTE: Watchdog Timer Enable bit (Habilitação do temporizador Watchdog)

1 = WDT habilitado

0 = WDT desabilitado.

bit 1-0 FOSC1:FOSC0: Oscillator Selection bits (Seleção do modo do oscilador)

11 = Oscilador RC.

10 = Oscilador HS.

01 = Oscilador XT.

00 = Oscilador LP.

Nota1: Valor apagado (não programado) da palavra de configuração é 3FFFh.

Nota2: Todos pares CP1:CP0 tem que estar com o mesmo valor para habilitar o esquema do código de proteção listado.

Nota3: Ao habilitar o Brown-out Reset automaticamente é habilitado o temporizador Power-up Timer (PWRT), independente do valor do bit /PWRT. Habilitando o Power-up Timer é habilitado ao mesmo tempo a Brown-out Reset.

11.2.2 - Inclusão dos bits de configuração no Código-Fonte.

Em vez de usarmos o Registro CONFIG para configurar o hardware interno, podemos usar a diretiva `_CONFIG` no início do programa para configurar este registro de forma automática de modo que, quando abrirmos a janela de gravação, já saia com os bits na configuração desejada.

Essa diretiva `_CONFIG`, seguida de diferentes parâmetros, gera, de forma automática a palavra de configuração.

O arquivo de inclusão (.INC) de cada chip fornecido pela Microchip, já contém as definições a serem utilizadas por esta diretiva.

a) Os bits de configuração para este micro usados na diretiva `_CONFIG`, são:

`_CP_ALL` (EQU H'1FFF): protege toda a memória de programa contra leitura.

`_CP_OFF` (EQU H'3FFF): memória de programa desprotegida contra leitura.

`_CP_HALF` (EQU H'1FDF): ?

`_CP_UPPER_256` (EQU H'2FEF): ?

`_DEBUG_OFF` (EQU H'3FFF): In-Circuit Debugger desabilitado, pinos RB6 e RB7 são I/O.

`_DEBUG_ON` (EQU H'37FF): In-Circuit Debugger habilitado, RB6 e RB7 dedicados ao debugger.

`_WRT_OFF` (EQU H'3FFF): memória de programa desprotegida contra escrita.

`_WRT_256` (EQU H'3DFF): proteção contra escrita dos endereços 0000h – 00FFh da memória de programa.

`_WRT_1FOURTH` (EQU `H'3BFF`): proteção contra escrita dos endereços 0000h – 07FFh da memória de programa.

`_WRT_HALF` (EQU `H'39FF`): proteção contra escrita dos endereços 0000h – 0FFFh da memória de programa.

`_WRT_ENABLE_ON` (EQU `H'3FFF`): ?

`_WRT_ENABLE_OFF` (EQU `H'3DFF`): ?

`_CPD_OFF` (EQU `H'3FFF`): memória EEPROM desprotegida contra leitura.

`_CPD_ON` (EQU `H'3EFF`): memória EEPROM protegida contra leitura.

`_LVP_ON` (EQU `H'3FFF`): ativa a programação de baixa voltagem.

`_LVP_OFF` (EQU `H'3F7F`): desativa a programação de baixa voltagem.

`_BODEN_ON` (EQU `H'3FFF`): ativa Brown-out Detect.

`_BODEN_OFF` (EQU `H'3FBF`): desativa Brown-out Detect.

`_PWRTT_OFF` (EQU `H'3FFF`): desativa Power-Up Timer.

`_PWRTT_ON` (EQU `H'3FF7`): ativa Power-Up Timer.

`_WDT_ON` (EQU `H'3FFF`): habilita Watchdog Timer.

`_WDT_OFF` (EQU `H'3FFB`): desativa Watchdog Timer.

`_RC_OSC` (EQU `H'3FFF`): RC externo com saída de clock.

`_LP_OSC` (EQU `H'3FFC`): cristal de baixa potência (até 200 KHz).

`_XT_OSC` (EQU `H'3FFD`): cristal/ressonador (até 4MHz).

`_HS_OSC` (EQU `H'3FFE`): cristal/ressonador de alta frequência (até 20MHz).

Obs: Não existe `_WRT_ON`, pois a proteção é por endereços.

b) Exemplo de configuração do registro CONFIG que pode ser incluído no código fonte:

```
_CONFIG_WDT_OFF & _CP_OFF & _XT_OSC & _PWRTT_OFF
```

Podemos verificar que o Watchdog está desabilitado, memória de programa desprotegida contra leitura, oscilador é a cristal e Power Up Timer está desativado.

11.3 - CONFIGURAÇÕES DO OSCILADOR.

11.3.1 - Tipos de osciladores.

O PIC16F87X pode ser operado com quatro modos diferentes de oscilador. O usuário pode programar o tipo de oscilador através dos bits de configuração FOSC (CONFIG<1:0) para selecionar um destes quatro modos:

a) LP - Cristal de baixa potência (até 200KHz).

b) XT - Cristal/Reessonador (até 4 MHz).

c) **HS** - Cristal/Ressonador cerâmico de alta frequência (até 20MHz).

d) **RC** - Resistor/Capacitor externo com saída de clock.

11.3.2 - Oscilador a Cristal/Ressonadores de cerâmica.

Nos modos XT, LP ou HS, um cristal ou ressonador de cerâmica é conectado entre os pinos OSC1/CLKIN (pino 13) e OSC2/CLKOUT (pino 14) para estabilizar as oscilações, como na **Figura 11.1**. Permite também uma fonte de clock externa para excitar o pino OSC1/CLKIN, como na **Figura 11.2**.

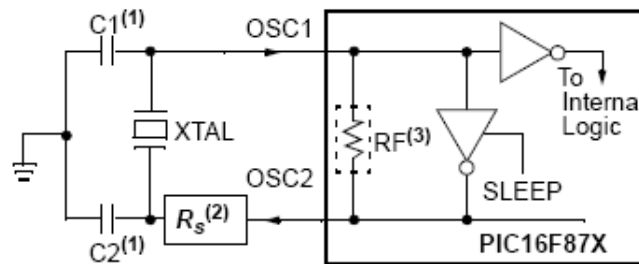


Figura 11.1: Conexão do cristal ou ressonador nos modos HS, XT ou LP.

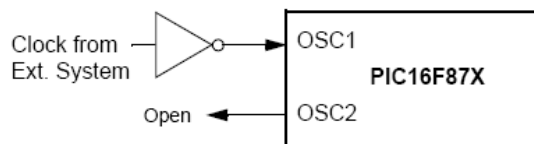


Figura 11.2: Conexão do oscilador externo nos modos HS, XT ou LP.

11.3.3 - Oscilador RC.

Para aplicações onde a precisão não é crítica usamos o oscilador RC que oferece ainda redução de custos. A frequência do oscilador RC é função dos valores da voltagem de alimentação, do resistor (R_{EXT}), do capacitor (C_{EXT}), e da temperatura de operação. Em adição a isto, a frequência do oscilador varia de unidade a unidade devido ao processo normal de variação dos parâmetros. O usuário também precisa levar em conta a tolerância dos componentes externos R e C usados. A **Figura 11.3** mostra como a combinação RC é conectada ao PIC16F87X. Esta opção fornece no

pino 14 (OSC2/CLKOUT) um sinal com frequência quatro vezes menor que a do oscilador principal.

Com $C = 100\text{pF}$, $R = 4,7\text{K}$ e temperatura de 25°C o PIC trabalha próximo de 1MHz .

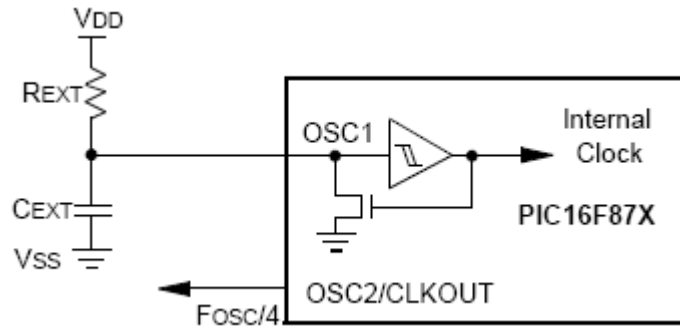


Figura 11.3: Oscilador no modo RC

11.4 – TIPOS DE RESET.

No PIC16F877, existem diferentes formas de se produzir um Reset, **Figura 11.4:**

- a) **Power-on Reset (POR)** – quando se liga a alimentação (Power-up) temos o PWRT e o OST
- b) **/MCLR reset durante operação normal** - aplicação de um valor lógico baixo no pino 1 do micro durante o funcionamento normal. É usado muitas vezes, durante o desenvolvimento de um programa.
- c) **/MCLR reset durante o SLEEP.**
- d) **WDT reset durante operação normal.**
- e) **WDT Wake-up durante o SLEEP.**
- f) **Brown-out Reset (BOR)**

Alguns registros não são afetados por qualquer condição de Reset. Seus estados são desconhecidos no POR e não mudam em quaisquer outros resets.

Muitos outros registros são resetados para o estado de Reset durante o Power-on Reset (POR), no /MCLR e WDT Reset, no /MCLR Reset durante o SLEEP e no Brown-out Reset (BOR). Eles não são afetados pelo WDT Wake-up, pois este é visto como um retorno normal de operação. Os bits /TO e /PD são setados ou zerados,

diferentemente, em diferentes situações de reset como visto na **Figura 11.5**. Esses bits são usados no software para determinar a natureza do reset.

O diagrama de blocos simplificado do circuito de Reset é mostrado na **Figura 11.4**.

Nele, podemos ver os diferentes circuitos que podem ocasionar um reset. O dispositivo possui um filtro de ruído /MCLR no caminho do /MCLR Reset. O filtro detecta e ignora pequenos pulsos.

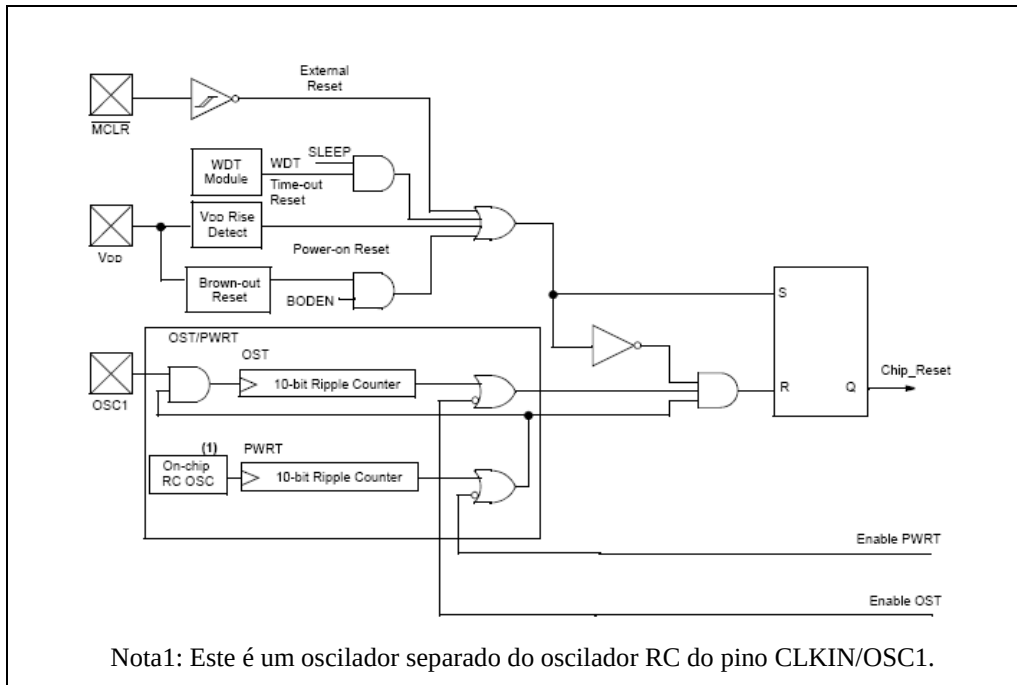


Figura 11.4: Diagrama de blocos simplificado do circuito de Reset interno.

Estado dos bits /POR, /BOR, /TO e /PD depois da inicialização darão uma idéia clara de como foi ocasionado um reset.

/POR	/BOR	/TO	/PD	
0	x	1	1	Ocorreu um Power-on Reset
0	x	0	x	Não pode ocorrer. /TO é setado em /POR.
0	x	x	0	Não pode ocorrer. /PD é setado em /POR
1	0	1	1	Ocorreu um Brown-out Reset (BOR)
1	1	0	1	Ocorreu um WDT Reset.
1	1	0	0	Ocorreu um WDT Wake-up (acordou)
1	1	u	u	Ocorreu um /MCRL Reset durante operação normal.
1	1	1	0	Ocorreu um /MCRL Reset durante SLEEP ou acordou de SLEEP por interrupção.

Figura 11.5 - Estado dos bits /POR, /BOR, /TO e /PD depois da inicialização.

Com a leitura dos bits /TO e /PD do registro de Status e /POR e /BOR do registro PCON, saberemos durante todo o tempo quem e como se produziu um reset.

Levando para zero o pino /MCLR, é possível gerar um reset. Caso não se queira um reset, ligue este pino direto à alimentação ou através de um resistor.

11.4.1 – Reset quando se liga a alimentação - Power-On Reset (POR).

O pulso Power-on Reset (POR) é gerado internamente sempre que uma subida de Vdd variar entre 1,2V a 1,7V, ou seja, somente na energização do micro. Isto garante que o microcontrolador opere corretamente e não sofra problemas durante a energização. Caso não se queira um reset, ligue este pino MCLR diretamente ou através de um resistor a Vdd. Isto elimina os componentes RC externos necessários para gerar o POR. O pulso dispara o temporizador Power-up timer, caso este esteja habilitado. Quando ocorre um POR, o bit /POR (PCON<1>) será resetado.

11.4.2 - Power-up Timer (PWRT).

É um temporizador utilizado para garantir que o micro somente comece a operar depois que a alimentação estiver estabilizada. O PWRT é formado por um contador de 10 bits que se encarrega de contar 1024 pulsos de clock interno formado por um oscilador RC (habilitado no momento da gravação do PIC), que é diferente do oscilador principal. Proporciona um atraso de 72ms e após seu término, habilita-se o OST, também de 10 bits que conta 1024 pulsos de um oscilador externo.

Esse tempo é contabilizado após o pulso de POR e pode variar em função da pastilha e da temperatura.

Deve obrigatoriamente estar habilitado caso se deseje utilizar o Brown-out Reset (BOR). Um bit de configuração é usado para habilitar ou desabilitar o PWRT.

O PWRT é habilitado/desabilitado durante a gravação do microcontrolador pelo bit /PWRT (CONFIG<3>).

11.4.3 - Oscillator Start-up Timer (OST).

O Oscillator Start-up Timer (OST) fornece um retardo de 1024 ciclos do cristal (ou do oscilador ligado ao pino OSC1) após o retardo do PWRT ter terminado (se o PWRT estiver habilitado), para iniciar a execução do software. Isto assegura que o oscilador a cristal ou ressonador inicie estabilizado. É habilitado automaticamente quando o micro estiver utilizando osciladores do tipo XT, LP ou HS e seu retardo (1024 ciclos do oscilador) será gerado sempre em três diferentes situações:

- a) Na energização (POR)
- b) Após um Brown-out Reset
- c) Após um despertar de um SLEEP.

11.4.4 - Brown-out Reset - BOR (Queda de energia).

O BOR força um reset do micro sempre que a tensão de alimentação cair abaixo de 4V (típico) durante um intervalo de tempo maior que TBOR (100us típico).

Caso a queda de tensão seja durante um intervalo de tempo menor que 100us, o reset não será gerado. A duração do reset é de 72ms e, enquanto a alimentação não for superior a 4V, esse temporizador não será inicializado.

Quando ocorre BOR, o bit /BOR (PCON<0>) será resetado.

Para o PIC16F877A o valor do BOR é fixo e não pode ser alterado (alguns modelos permitem valores programáveis).

O bit de configuração, BODEN (CONFIG<6>), pode habilitar ou desabilitar o circuito do Brown-out Reset durante a gravação do microcontrolador. Se VDD cair abaixo de VBOR (cerca de 4V) por um tempo maior que TBOR (cerca de 100us), a situação de Brown-out, reseta o dispositivo. Se VDD cair abaixo de VBOR por um tempo menor que TBOR (100us), o reset pode não ocorrer.

Uma vez que ocorra um Brown-out, o dispositivo permanece no Brown-out Reset até que VDD aumente acima de VBOR (4V). O Power-up Timer então mantém o dispositivo em reset por TPWRT (cerca de 72ms). Se VDD cair abaixo de VBOR durante TPWRT, o processo Brown-out Reset reiniciará quando VDD subir para um valor acima de VBOR com o Power-up Timer Reset. O Power-up Timer é sempre habilitado quando o circuito Brown-out Reset estiver habilitado independentemente do estado do bit de configuração do PWRT

11.4.5 – Sequência de Time-out.

No power-up, a sequência do time-out é a seguinte:

- a) O PWRT retarda a inicialização (se habilitado) quando ocorrer um reset POR. Então o OST inicia contando 1024 ciclos do oscilador após o PWRT (LP, XT ou HS). Quando o OST termina, o dispositivo sai do reset.
- b) Se o /MCLR for mantido em nível baixo por longo tempo, “o time-outs” expira e se em nível alto, inicia execução imediata. Isto é útil para propósito de teste ou na sincronização de mais de um PIC operando em paralelo.

A **Figura 11.6** mostra os diferentes atrasos (time-out total) ocasionados durante a inicialização do microcontrolador

A **Figura 11.10** mostra a condição de Reset para os registros STATUS, PCON e PC, enquanto a Figura 11.11 mostra as condições de Reset para todos os outros registros.

Oscillator Configuration	Power-up		Brown-out	Wake-up from SLEEP
	PWRT = 0	PWRT = 1		
XT, HS, LP	72 ms + 1024TOSC	1024TOSC	72 ms + 1024TOSC	1024TOSC
RC	72 ms	—	72 ms	—

Figura 11.6: Diferentes atrasos ocasionados na inicialização (time-out).

11.4.6 - Power Control/Status Register (PCON).

Como visto, o registro PCON (Power Control/Status Register), possui apenas dois bits.

- a) O bit 0 é o bit /BOR (Brown-out Reset Status bit), é desconhecido no Power-on Reset (POR). Deve ser setado pelo usuário e verificado nos resets subsequentes para ver se foi resetado, indicando que ocorreu um BOR. Quando o Brown-out

Reset está desabilitado o estado do bit /BOR é imprevisível (unpredictable) e é, portanto, inválido em qualquer tempo.

- b)** O bit 1 é /POR (Power-on Reset Status bit), é resetado no Power-on Reset e não é afetado de outra maneira. O usuário deve setar este bit seguindo o Power-on Reset.

Registro PCON

bit 1 /POR: Power-on Reset Status bit (Estado do bit Power-on Reset)

1 = Não ocorreu um Power-On Reset

0 = Ocorreu um Power-On Reset (deve ser setado pelo software).

bit 0 /BOR: Brown-out Reset Status bit (Bit de estado se houver queda de energia)

1 = Não ocorreu um Brown-out Reset.

0 = Ocorreu um Brown-out Reset (deve ser setado pelo software).

Condition	Program Counter	STATUS Register	PCON Register
Power-on Reset	000h	0001 1xxx	---- --0x
MCLR Reset during normal operation	000h	000u uuuu	---- --uu
MCLR Reset during SLEEP	000h	0001 0uuu	---- --uu
WDT Reset	000h	0000 1uuu	---- --uu
WDT Wake-up	PC + 1	uuu0 0uuu	---- --uu
Brown-out Reset	000h	0001 1uuu	---- --u0
Interrupt wake-up from SLEEP	PC + 1 ⁽¹⁾	uuu1 0uuu	---- --uu

Legenda: u = não muda; x = desconhecido; - = não implementado, em leitura é zero (0).

Obs: Quando desperta por interrupção e o bit GIE está setado, o PC é carregado com o vetor de interrupção (0004h).

Figura 11.10: Estado dos bits dos registros especiais depois de um reset.

Register	Devices				Power-on Reset, Brown-out Reset	MCLR Resets, WDT Reset	Wake-up via WDT or Interrupt
W	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
INDF	873	874	876	877	N/A	N/A	N/A
TMR0	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
PCL	873	874	876	877	0000h	0000h	PC + 1 ⁽²⁾
STATUS	873	874	876	877	0001 1xxx	000q quuu ⁽³⁾	uuuq quuu ⁽³⁾
FSR	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTA	873	874	876	877	--0x 0000	--0u 0000	--uu uuuu
PORTB	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTC	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTD	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
PORTE	873	874	876	877	---- -xxx	---- -uuu	---- -uuu
PCLATH	873	874	876	877	--0 0000	--0 0000	--uu uuuu
INTCON	873	874	876	877	0000 000x	0000 000u	uuuu uuuu ⁽⁴⁾
PIR1	873	874	876	877	x000 0000	x000 0000	uuuu uuuu ⁽⁴⁾
	873	874	876	877	0000 0000	0000 0000	uuuu uuuu ⁽⁴⁾
PIR2	873	874	876	877	-x-0 0--0	-x-0 0--0	-x-u u--u ⁽⁴⁾
TMR1L	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
TMR1H	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
T1CON	873	874	876	877	--00 0000	--uu uuuu	--uu uuuu
TMR2	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
T2CON	873	874	876	877	-000 0000	-000 0000	-uuu uuuu
SSPBUF	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
SSPCON	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
CCPR1L	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCPR1H	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCP1CON	873	874	876	877	--00 0000	--00 0000	--uu uuuu
RSTA	873	874	876	877	0000 000x	0000 000x	uuuu uuuu
TXREG	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
RCREG	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
CCPR2L	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCPR2H	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
CCP2CON	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
ADRESH	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
ADCON0	873	874	876	877	0000 00-0	0000 00-0	uuuu uu-u
OPTION_REG	873	874	876	877	1111 1111	1111 1111	uuuu uuuu
TRISA	873	874	876	877	--11 1111	--11 1111	--uu uuuu
TRISB	873	874	876	877	1111 1111	1111 1111	uuuu uuuu
TRISC	873	874	876	877	1111 1111	1111 1111	uuuu uuuu
TRISD	873	874	876	877	1111 1111	1111 1111	uuuu uuuu
TRISE	873	874	876	877	0000 -111	0000 -111	uuuu -uuu
PIE1	873	874	876	877	x000 0000	x000 0000	uuuu uuuu
	873	874	876	877	0000 0000	0000 0000	uuuu uuuu

Register	Devices				Power-on Reset, Brown-out Reset	MCLR Resets, WDT Reset	Wake-up via WDT or Interrupt
PIE2	873	874	876	877	-x-0 0--0	-x-0 0--0	-x-u u--u
PCON	873	874	876	877	---- --qq	---- --uu	---- --uu
PR2	873	874	876	877	1111 1111	1111 1111	1111 1111
SSPAD	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
SSPSTAT	873	874	876	877	--00 0000	--00 0000	--uu uuuu
TXSTA	873	874	876	877	0000 -010	0000 -010	uuuu -uuu
SPBRG	873	874	876	877	0000 0000	0000 0000	uuuu uuuu
ADRESL	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
ADCON1	873	874	876	877	0--- 0000	0--- 0000	u--- uuuu
EEDATA	873	874	876	877	0--- 0000	0--- 0000	u--- uuuu
EEADR	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
EEDATH	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
EEADRH	873	874	876	877	xxxx xxxx	uuuu uuuu	uuuu uuuu
EECON1	873	874	876	877	x--- x000	u--- u000	u--- uuuu
EECON2	873	874	876	877	---- ----	---- ----	---- ----

Legenda: u = não muda; x = desconhecido; - = não implementado, em leitura é zero (0); q = valor depende das condições; r = reservado, manter zerado.

Nota 1: Um ou mais bits de INTCON, PIR1 e/ou PIR2 serão afetados (por causa do wake-up).

2: Quando wake-up por uma interrupção e o bit GIE estiver setado, o PC é carregado com o vetor de interrupção (0004h).

3: Veja registros especiais na **Figura 11.10**.

Figura 11.11: Condições de inicialização para todos os registros.

11.5 – INTERRUPÇÕES.

É um evento externo ao programa que provoca a parada de sua execução, a verificação e tratamento do referido evento pelo micro e em seguida o retorno do programa ao ponto em que havia sido interrompido.

11.5.1 - As interrupções são controladas pelos seguintes registros:

- Registro geral das interrupções (INTCON)
- Registros de controles individuais de periféricos (PIE1 habilita e PIR1 são flags)
- Registros de controles individuais de periféricos (PIE2 habilita e PIR2 são flags).

Este PIC possui um total de 14 interrupções diferentes. Entretanto, como nos demais modelos, todas gerarão o desvio do programa para o mesmo vetor de interrupção (endereço 0004h), desde que devidamente configuradas para isso.

11.5.2 - As interrupções podem ser divididas em dois grupos:

- Convencionais (mesmas usadas em PICs menores)
- De periféricos.

Para que as interrupções ocorram e o micro tome conhecimento, é necessário que, além do evento acontecer, os bits global, de periféricos e individuais, estejam habilitados.

As interrupções são identificadas pelos bits de flag de cada uma delas.

Se a interrupção ocorrer, o bit de flag será setado independentemente dos bits geral, individual e periférico.

Quando ocorre um evento de interrupção, a instrução em execução é completada, o conteúdo do PC é armazenado na Pilha (endereço de retorno), o bit GIE é desabilitado, para desabilitar qualquer posterior interrupção, e o programa é desviado para o vetor de interrupção, endereço 0x04h. No vetor de interrupção deve existir uma sub-rotina de tratamento da interrupção. Após o tratamento da interrupção, a instrução RETFIE irá habilitar novamente o bit GIE, carregar o PC com o endereço que está na pilha e o programa retorna ao ponto em que estava quando correu a interrupção, reiniciando seu fluxo normal. O bit de flag de interrupção deve ser zerado por software antes de habilitar novamente a interrupção para evitar erro de interrupção.

Para evento de interrupção externa, tais como pino RB0/INT ou PORTB Change Interrupt, a latência de interrupção é de três ou quatro ciclos de instrução. A latência exata depende de quando o evento de interrupção ocorreu. Individualmente, os bits flags de interrupções são setados, independentemente do estado dos correspondentes bits de habilitação, bit PEIE ou bit GIE.

O bit GIE é sempre limpo no RESET.

11.5.3 - A lógica de tratamento de uma interrupção é a seguinte:

- a) Como a chave geral GIE é desligada antes de desviar para o vetor de interrupção, duas interrupções não serão tratadas ao mesmo tempo, isto é, uma interrupção não gerará um desvio caso outra interrupção já esteja sendo tratada. Entretanto, o flag da segunda será marcado, e quando o tratamento da primeira terminar, o GIE será novamente ligado (através da instrução RETFIE) e o sistema voltará a ser desviado para o vetor de interrupção.
- b) Como todas as interrupções desviam para o mesmo ponto, é necessário testar os flags de todas as interrupções ligadas para saber qual realmente ocorreu. Como

mais de uma ação vinculada às interrupções podem acontecer ao mesmo tempo, a ordem dos testes é que determina a prioridade de tratamento.

- c) Os flags das interrupções não são limpos automaticamente pelo sistema (exceto ADIF, TXIF e RCIF). Cabe ao usuário efetuar esta operação na rotina de tratamento da interrupção.

11.5.4 - Salvar registros durante uma Interrupção.

Durante uma interrupção somente o valor do PC é salvo na pilha. Usuários podem desejar salvar registros chaves durante uma interrupção (Registro W e Registro STATUS) o que pode ser feito por software. Para o PIC16F873/874, o registro W_TEMP deve ser definido em ambos os bancos 0 e 1 e a definição do endereço deve coincidir nos dois bancos (exemplo: se W_TEMP é definido em 0x20 do banco 0 deve ser definido também 0xA0 no banco 1). O registro PCLATH_TEMP e STATUS_TEMP, são somente definidos no banco 0. Como 16 bytes de cada banco são comuns, registros temporários, como W_TEMP, STATUS_TEMP, e PCLATH_TEMP serão usados aqui. Estas 16 localizações não precisam de banco e, por isso, fica fácil salvar e restaurar o contexto. O código usado aqui pode ser usado normalmente.

Salvando na RAM os registros STATUS, W, e PCLATH.

```
movwf  W_TEMP      ;copia o registro W para o registro TEMP
swapf  STATUS,W    ;troca os nibbles do registro STATUS e salva em W
clrf   STATUS      ;seleção de Banco 0, limpa IRP, RP1 e RP2
movwf  STATUS_TEMP ;salva o registro Status no registro STATUS_TEMP
movf   PCLATH,W    ;só é necessário se usando páginas 1, 2 ou 3.
movwf  PCLATH_TEMP ;salva o registro PCLATH no registro PCLATH_TEMP
clrf   PCLATH      ;seleciona página 0
:
:(ISR)              ;(inserir código)
:
movf   PCLATH_TEMP,W ;restaura PCLATH
movwf  PCLATH        ;move W para PCLATH
swapf  STATUS_TEMP,W ;swap STATUS_TEMP e salva em W
                        ;(volta com o banco ao estado original.
movwf  STATUS        ;move W para o registro STATUS
swapf  W_TEMP,F      ;swap W_TEMP e salva em W_TEMP
```

```
swapf    W_TEMP,W      ;Swap W_TEMP e salva em W
```

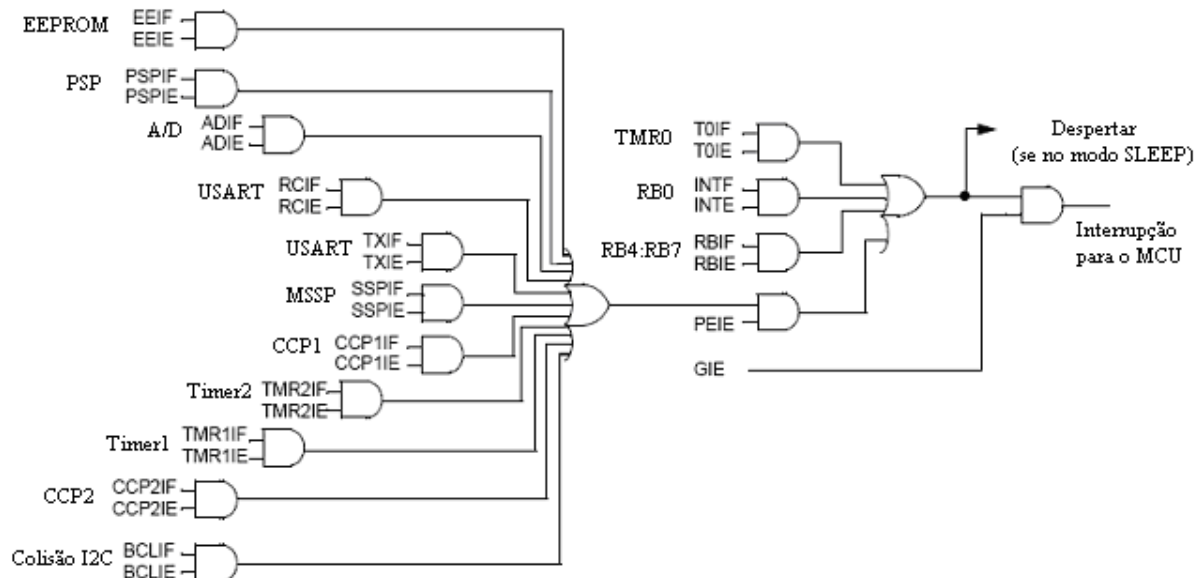


Figura 11.12: Lógica da interrupção.

11.5.1 - Interrupção do Timer0.

Acontece sempre que o contador do TMR0 (Timer0) estoura, ou seja, como ele é um contador de 8 bits, sempre que ele passa de 0xFF para 0x00. É habilitada/desabilitada pelo bit T0IE (INTCON<5>) e identificada pelo bit de flag T0IF (INTCON<2>). Esta interrupção é usada normalmente para a contagem de tempo.

11.5.2 - Interrupção externa no pino RB0/INT.

É gerada por um sinal externo ligado ao pino RB0/INT, caso ele esteja configurado como entrada. Deste modo, podemos identificar e processar imediatamente um sinal externo. É usada para diversas finalidades, como por exemplo, para a comunicação entre micros, garantindo o sincronismo para reconhecimento de botão ou outro sinal do sistema que necessite de uma ação imediata. Esta interrupção pode ser configurada para a borda de subida ou descida do clock, dependendo do bit INTEDG (OPTION_REG<6>). É habilitada pelo bit INTE (INTCON<4>) e identificada pelo

bit de flag INTF (INTCON<1>). Esta interrupção pode despertar o micro de um SLEEP.

11.5.3 - Interrupção por mudança de estado (PORTB INTCON Change).

Ao contrário da interrupção externa, esta interrupção acontece em ambas as bordas. Na verdade, esta interrupção é sensível à diferença de nível existente entre o pino e o latch interno. É relacionada aos pinos da PortaB (RB7:RB4), simultaneamente. Por isso, os pinos devem ser configurados como entradas e a mudança de estado em qualquer deles irá gerar a interrupção. Como o latch só é atualizado quando a porta é lida, a leitura é obrigatória para que o evento da interrupção pare de ocorrer. Este tipo de interrupção pode ser utilizado, por exemplo, para criar um sincronismo com a rede de 60 Hz, para controle de um triac ou outro sistema semelhante.

É habilitada/desabilitada pelo bit RBIE (INTCON<3>) e identificada pelo bit de flag RBIF (INTCON<0>).

11.5.4 - Interrupção da porta paralela (PSP).

Esta interrupção está ligada à porta paralela do tipo escravo (Parallel Slave Port) e acontece sempre que uma operação de escrita ou leitura dessa porta é completada. É habilitada/desabilitada pelo bit PSPIE (PIE1<7>) e identificada pelo bit de flag PSPIF (PIR1<7>). Este recurso foi estudado no Capítulo 4.

11.5.5 - Interrupção do conversor A/D.

Esta interrupção acontece quando uma conversão A/D é completada. É habilitada/desabilitada pelo bit ADIE (PIE1<6>) e identificada pelo bit de flag ADIF (PIR1<6>). O estudo completo deste recurso está no Capítulo 10.

11.5.6 - Interrupção de recepção da USART.

Esta interrupção indica o término da recepção de um dado pela USART (Universal Synchronous Asynchronous Receiver Transmitter). É habilitada/desabilitada pelo bit RCIE (PIE1<5>) e identificada pelo bit de flag RCIF (PIR1<5>). O estudo completo deste recurso está no Capítulo 9.

11.5.7 - Interrupção de transmissão da USART.

Esta interrupção indica o esvaziamento do buffer relacionado à transmissão de um dado pela USART. É habilitada/desabilitada pelo bit TXIE (PIE1<4>) e identificada pelo bit de flag TXIF (PIR1<4>). O estudo completo está no Capítulo 9.

11.5.8 - Interrupção de comunicação série (SPI e I²C).

É o segundo sistema de comunicação série além da USART, é denominado MSSP (Master Synchronous Serial Port), possui dois modos de comunicação: SPI e I²C. Sempre que um dado é transmitido ou recebido por esta porta série, esta interrupção acontecerá. Ela também acontecerá em muitas outras situações. É habilitada/desabilitada pelo bit SSPIE (PIE1<3>) e identificada pelo bit de flag SSPIF (PIR1<3>). O estudo completo está no Capítulo 8.

11.5.9 - Interrupção do CCP1 (Capture/Compare/PWM).

Esta interrupção está vinculada ao primeiro sistema CCP, que engloba os seguintes recursos: Capture1, Compare1 e PWM. É habilitada/desabilitada pelo bit CCP1IE (PIE1<2>) e identificada pelo bit de flag CCP1IF (PIR1<2>). Estudo completo deste recurso foi realizado no Capítulo 7.

11.5.10 - Interrupção do Timer2.

Acontece sempre que o Timer 2 (TMR2) acrescido do seu Postscaler estourar. Este é um contador de 8 bits que possui um Prescaler para incrementar o registrador e um Postscaler para gerar a interrupção. É habilitada/desabilitada pelo bit TMR2IE (PIE1<1>) e identificada pelo bit de flag TMR2IF (PIR1<1>). Esta interrupção foi estudada no Capítulo 6.

11.5.11 - Interrupção do Timer1.

Acontece sempre que o Timer 1 (TMR1) estourar. Este é um contador de 16 bits. É habilitada/desabilitada pelo bit TMR1IE (PIE1<0>) e identificada pelo bit de flag TMR1IF (PIR1<0>). Timers foram estudados no Capítulo 6.

11.5.12 - Interrupção por fim de escrita na EEPROM/FLASH.

Esta interrupção serve para detectarmos o fim de escrita nessas memórias. A utilização da interrupção não é obrigatória para que a escrita funcione, mas, como a EEPROM é lenta na hora de escrever, em alguns sistemas sua utilização pode ser necessária para evitar que o programa pare durante a escrita na EEPROM. A interrupção também é válida para a memória FLASH (memória de programa). É habilitada/desabilitada pelo bit EEIE (PIE2<4>) e identificada pelo bit de flag EEIF (PIR2<4>).

11.5.13 - Interrupção de colisão de dados (Bus Collision).

Esta interrupção serve para informar o sistema sobre colisões de dados na comunicação I²C. É habilitada/desabilitada pelo bit BCLIE (PIE2<3>) e identificada pelo bit de flag BCLIF (PIR2<3>). O estudo completo deste recurso está no Capítulo 8.

11.5.14 - Interrupção de CCP2 (Capture/Compare/PWM).

Esta interrupção é vinculada ao segundo sistema CCP, que engloba os recursos Capture2, Compare2 e PWM2. É habilitada/desabilitada pelo bit CCP2IE (PIE2<0>) e identificada pelo bit de flag CCP2IF (PIR2<0>). O estudo completo está no Capítulo 7.

11.5.15 - Interrupções dos comparadores.

É diretamente ligada a mudança de estado dos comparadores internos do PIC.

Chaves	SFR	Bit	Chaves	SFR	Flag
GIE (geral)	INTCON	7	-		
PEIE (periféricos)	INTCON	6	-		
T0IE	INTCON	5	T0IF	INTCON	2
INTE	INTCON	4	INTF	INTCON	1
RBIE	INTCON	3	RBIF	INTCON	0
PSPIE	PIE1	7	PSPIF	PIR1	7
ADIE	PIE1	6	ADIF	PIR1	6
RCIE	PIE1	5	RCIF	PIR1	5
TXIE	PIE1	4	TXIF	PIR1	4
SSPIE	PIE1	3	SSPIF	PIR1	3
CCP1IE	PIE1	2	CCP1IF	PIR1	2
TMR2IE	PIE1	1	TMR2IF	PIR1	1
TMR1IE	PIE1	0	TMR1IF	PIR1	0
EEIE	PIE2	4	EEIF	PIR2	4
BCLIE	PIE2	3	BCLIF	PIR2	3
CCP2IE	PIE2	0	CCP2IF	PIR2	0

Figura 11.13: Registros com seus bits de habilitação e identificação de interrupções.

11.6 – SLEEP (POWER-DOWN MODE).

O micro entra em SLEEP (Power-down Mode) sempre que a instrução SLEEP for executada. Neste modo de operação, o oscilador é desligado, os pinos de I/O mantêm o estado anterior à instrução SLEEP, bit /PD (STATUS<3>) e resetado, bit /TO (STATUS<4>) é setado e o WDT (se habilitado) é resetado embora continue operando. Para minimizar consumo, devemos desligar o conversor A/D, desabilitar clocks externos, colocar todos os pinos como entradas (tri-state) e desligar pull-ups da porta B. Para casos extremos, devemos levar em conta também o uso do WDT, PWRT e BOR (WDT e PWRT usam osciladores RC).

O pino /MCLR deve ser mantido em nível lógico alto (V_{IHMC}).

11.6.1 - Despertar de um SLEEP.

Existem três formas para que o micro volte a operar e despertar da situação de SLEEP:

- a) Reset externo na entrada /MCLR (pino 1 do micro);
- b) Estouro do WDT, se habilitado;
- c) Evento de alguma interrupção (como visto em Interrupções).

Um reset externo na entrada /MCLR causará o reset do micro (PC = 0x00). Todas as outras situações são consideradas uma continuação do programa e causarão um despertar na linha seguinte à instrução SLEEP. Os bits /TO e /PD (STATUS<4:3>) poderão ser testados para descobrir a causa do reset. O bit /PD, o qual é setado no Power-up, é limpo na instrução SLEEP. O bit /TO é limpo se ocorreu estouro do WDT e causou um despertar.

11.6.2 - Interrupção durante SLEEP.

As seguintes interrupções de periféricos podem acordar o dispositivo de um SLEEP:

- a) Leitura ou escrita na PSP.
- b) Interrupção do TMR1 (Timer1 deve estar operando como contador assíncrono).
- c) Interrupção do CCP no modo Capture.
- d) Evento especial de disparo (Timer1 no modo assíncrono usando clock externo).
- e) SSP (START/STOP) bit de detecção de interrupção.
- f) Transmissão/recepção do SSP no modo Escravo (SPI/I²C).
- g) USART RX ou TX (Modo síncrono Escravo).
- h) Conversão A/D (com oscilador RC).

i) Operação de escrita na EEPROM completada.

11.6.3 - Wake-up pelo uso de interrupção.

Quando a interrupção global está desabilitada ($GIE = 0$) e qualquer outra fonte de interrupção esteja com o bit de interrupção e o flag indicador setados, ocorre o seguinte:

- a) Se uma interrupção ocorrer **antes** da execução de uma instrução SLEEP, a instrução será completada como um NOP. Por isso, o WDT e seu postscaler não serão limpos, o bit /TO não será setado e o bit /PD não será limpo.
- b) Se a interrupção ocorrer **durante ou após** a execução da instrução SLEEP, o dispositivo despertará imediatamente do SLEEP. A instrução SLEEP será totalmente executada antes do despertar. Por isso, o WDT e seu postscaler serão limpos, o bit /TO será setado e o bit /PD será limpo.

Para determinar quando a instrução SLEEP foi executada, teste o bit PD. Se o bit PD estiver setado a instrução SLEEP foi executada como um NOP. Para assegurar que o WDT está limpo (zerado), a instrução CLRWDT deverá ser executada antes da instrução SLEEP.

Para assegurar que o WDT esteja resetado, a instrução CLRWDT deve ser executada antes de uma instrução SLEEP.

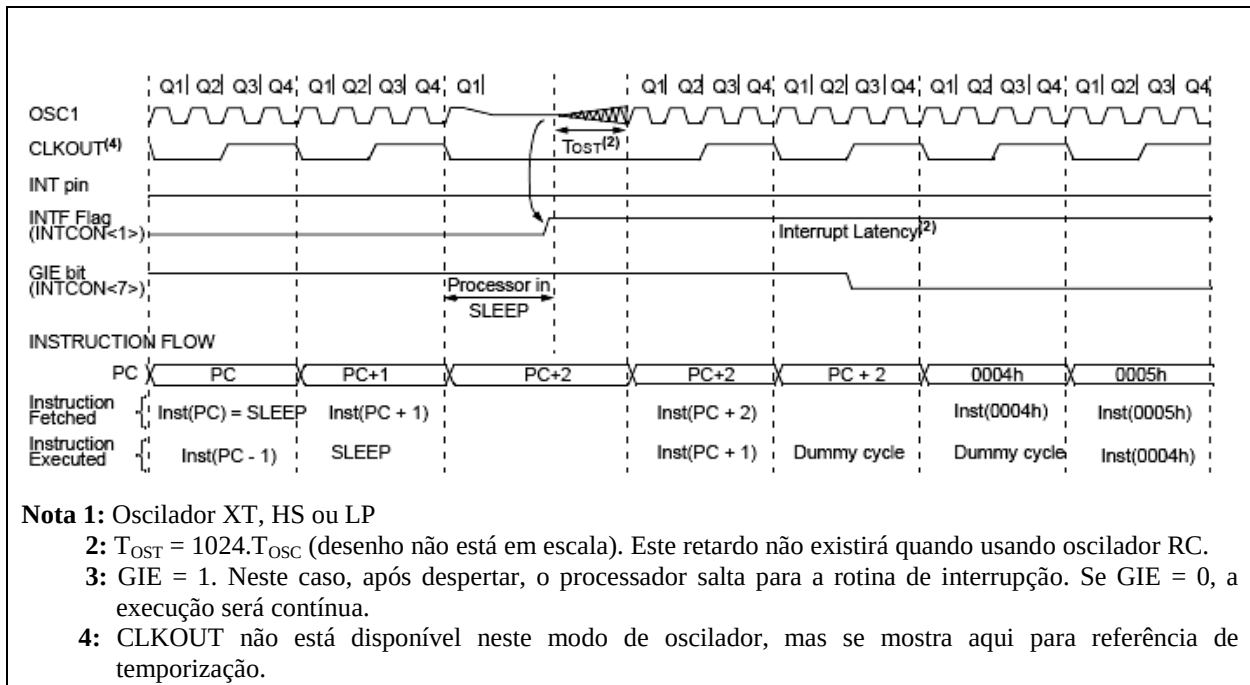


Figura 11.14: Dispertar de um Sleep por interrupção.

Parte do Registro STATUS.

bit 4 /T0: Time-out bit (Indicação de estouro de tempo) indica o estado em que se encontra a contagem do Watchdog.

1 = Ocorreu Power-up ou foram executadas instruções CLRWDWT ou SLEEP.

0 = Ocorrer um estouro do Watchdog Timer (chegou ao final da contagem e provocou um reset)

bit 3 /PD: Power-down bit (indicação Power-down (desligamento)). Indica o estado do controle de energia do micro.

1 = Ocorreu Power-up (alimentação) ou foi executada a instrução CLRWDWT.

0 = Indica que a instrução SLEEP foi executada e entrou no modo de baixo consumo.

11.6.4 - Execução de uma instrução SLEEP.

Quando a instrução Sleep é executada, a próxima instrução (PC+1) já estará sendo processada (conceito Pipeline). Para que o micro “ acorde ” por um evento de interrupção, a chave individual da interrupção deve estar ligada. O despertar independe do estado do bit GIE.

a) Ao “ acordar ” nesta situação, existem duas possibilidades:

- I) Se o bit GIE estiver habilitado, a instrução seguinte à instrução SLEEP será processada e, em seguida, o PC será desviado para o endereço 0x0004 (vetor de interrupção). Por esse motivo é recomendado que logo após a instrução SLEEP seja adicionado um NOP.
- II) Se o bit GIE estiver desabilitado, o programa continuará a ser executado no ponto seguinte à instrução SLEEP.
- b) Se o bit GIE estiver desabilitado e caso nenhuma interrupção habilitada individualmente tenha ocorrido, ao ser executada a instrução SLEEP duas situações podem ocorrer:
 - I) Se o evento da interrupção ocorrer antes da execução da instrução SLEEP, a instrução SLEEP será executada como NOP. Entretanto, o WDT não será resetado, o bit /TO não será setado e o bit /PD não será limpo.
 - II) Se o evento da interrupção ocorrer durante ou após a execução da instrução SLEEP, a instrução será executada normalmente, porém o micro imediatamente “acordará” do SLEEP. Neste caso, o WDT será resetado, o bit/TO será setado e o bit /PD será limpo.

Desta maneira, não adianta testar esses bits antes da execução da instrução SLEEP, pois os mesmos podem ser alterados enquanto a instrução está sendo executada. Para saber se a instrução foi corretamente executada, deve-se testar o bit /PD. Se ele estiver setado, a instrução foi executada como um NOP. Quanto ao WDT, recomenda-se executar um CLRWDT antes da instrução SLEEP.

11.7 – TEMPORIZADOR WATCHDOG (WDT).

Como visto, o Watchdog é um recurso disponível no PIC que parte do princípio que todo sistema é passível de falha. Se todo sistema pode falhar, cabe ao mesmo ter recursos para que, em ocorrendo uma falha, algo seja feito de modo a tornar o sistema novamente operacional.

Dentro do PIC existe um contador incrementado por um sinal oscilador (RC) independente. Isso significa que o WDT continua a operando mesmo em modo SLEEP.

Todas as vezes em que este contador extrapola o seu valor máximo retornando a zero, é provocado a reinicialização ou reset (endereço 0000h) do sistema.

Se o sistema estiver funcionado da maneira correta, de tempos em tempos uma instrução denominada Clear Watchdog Timer (CLRWDT) zera o valor deste contador, impedindo que o mesmo chegue ao valor máximo. Desta maneira, o Watchdog somente irá "estourar" quando algo de errado acontecer.

O período normal de estouro do Watchdog Timer é de aproximadamente 18 ms. No entanto, algumas vezes este tempo é insuficiente para que o programa seja normalmente executado. A saída neste caso é alocar o recurso de um postscaler (divisor de frequência) de modo a aumentar este período. Se sem o prescaler o período é de 18ms, quando atribuímos ao Watchdog Timer um prescaler de 1:2 nós dobramos este período de modo que o processamento possa ser executado sem que seja feita uma reinicialização.

O aumento do tempo é feito ajustando-se o valor do postscaler através dos bits PS2:PS0 (OPTION_REG<2:0>) quando o postscaler estiver atribuído ao Watchdog pelo bit PSA (OPTION_REG<3>).

Os bits /TO e /PD (STATUS<4:3>) serão setados sempre que a instrução CLRWDT for executada. O bit /TO será zerado sempre que ocorrer o estouro do Watchdog Timer. O ideal é que apenas uma instrução CLRWDT seja utilizada no software todo. Em geral, essa instrução é encontrada apenas no loop principal do programa. Dependendo da aplicação, em algumas rotinas de delay também se faz necessário o uso da instrução CLRWDT, apesar disso não ser muito aconselhável.

Não existe controle por software para ligar ou desligar o contador do WDT. Ele somente pode ser habilitado no momento da gravação do micro. Quando ele está ligado, o consumo é maior devido ao oscilador RC.

11.7.1 - Watchdog durante o SLEEP.

Com o WDT habilitado pelo bit WDTE (CONFIG<2>), o despertar de um SLEEP se dá na linha seguinte á instrução SLEEP e não no endereço 0000h. O bit /TO será limpo após transbordo WDT.

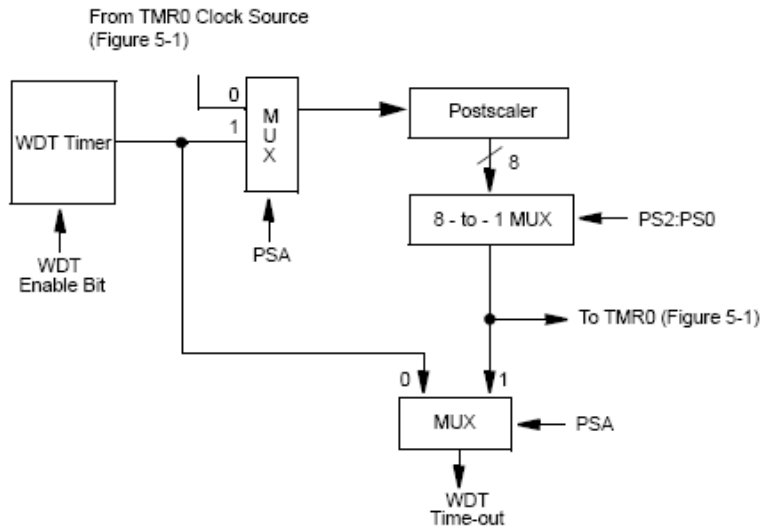


Figura 11.7: Bloco do WDT.

As instruções CLRWDT e SLEEP limpam o WDT e seu postscaler, se estiver atribuído ao WDT.

Quando uma instrução CLRWDT é executada e o prescaler está atribuído ao WDT, a contagem do postscaler será zerada, mas a atribuição do postscaler não é mudada.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
2007h	Config. bits	(1)	BODEN ⁽¹⁾	CP1	CP0	PWRTE ⁽¹⁾	WDTE	FOSC1	FOSC0
81h,181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

Figura 11.8: Registros associados ao WDT.

11.8 - SISTEMA DE EMULAÇÃO IN-CIRCUIT (DEBUGGER MODE)

Quando o bit DEBUG na palavra de configuração é programado com zero (0), a funcionalidade do In-Circuit Debugger é habilitada. É um poderoso recurso que permite visualizar o conteúdo dos registros internos do micro em tempo real. Uma das ferramentas usadas para este fim é o programador MPLAB ICD2 fornecido pela Microchip.

Quando esta função está habilitada alguns recursos do PIC não estão disponíveis, como visto na Figura 11.14.

Pinos de I/O	RB6 e RB7 não operam mais como I/O convencionais.
Pilha	Perde-se 1 nível da pilha para o sistema de emulação.
Memória de programa	O endereço 0000h deve possuir um NOP.
	Não podem ser utilizadas as últimas 100h words.
Memória de dados	Não podem ser utilizados os endereços 0x070 (0x0F0, 0x170 e 0x1F0) e de 0x1EB a 0x1EF.

Figura 11.14: Características perdidas quando habilitamos o In-Circuit Debugger.

Além dessas precauções, o sistema deve ainda disponibilizar um modo de interligação (conector) para um emulador externo. A Microchip possui uma família de emuladores para operar com o sistema In-Circuit Debugger Mode chamado MpLab ICD. A conexão entre o ICD e o PIC será feita por cinco vias (MCLR/Vpp, Vdd, Vss, RB6 e RB7).

11.9 – SISTEMA DE CÓDIGO DE PROTEÇÃO (CODE PROTECTION).

Depois que seu sistema estiver pronto e um protótipo for entregue a terceiros, é muito importante que essa opção seja ativada, pois ela impedirá que qualquer pessoa consiga ter acesso ao programa gravado no PIC. Para o PIC16F877A, que é do tipo regravável (Flash), o sistema de proteção impedirá a leitura, mas não a regravação. Entretanto, toda vez que uma nova gravação for feita quando o “*Code protection*” está ativado, primeiramente toda a memória de programa será limpa, evitando assim que qualquer tipo de alteração indevida seja efetuada no software. Esse sistema de proteção é interno ao PIC, não dependendo dos programas externos utilizados para gravação/leitura.

11.10 - REGISTROS DE IDENTIFICAÇÃO (IDS).

O PIC possui quatro registros especiais que se encontram nos endereços de 0x2000 a 0x2003 e não podem ser acessados pelo programa. Entretanto, esses registros podem ser lidos e escritos por meio do sistema de gravação. A diretiva `_IDLOCS` é usada para este fim.

Podemos utilizá-los, por exemplo, para a identificação da versão do programa gravado. A grande vantagem dos IDs é que podem ser lidos mesmo quando o “*Code*

Protection” está ativado, possibilitando a rastreamento do software em casos de problemas futuros.

de gravação. O sistema MPLAB possui incrementos para a geração de número. No entanto, esses 4 bytes são mais limitados do que parecem, pois só devem ser utilizados os 4 bits menos significativos de cada um. Dessa forma, apesar de 4 bytes, só possuímos 16 bits disponíveis.

O valor a ser gravado nos IDs pode ser escolhido diretamente durante as configurações de série.

Exemplo de código: `_IDLOCS H'5BC2'`

11.11 - GRAVAÇÃO EM SÉRIE NO PRÓPRIO CIRCUITO – ICSP (In-circuit Serial Programming).

O PIC16F87X e alguns outros modelos da Microchip possuem a possibilidade de serem programados diretamente nos circuitos, isto é, já montados na placa final. Essa gravação é conhecida como **In-Circuit** e pode possibilitar um ganho de produtividade.

Como a gravação é feita de forma série, utilizando-se somente cinco pinos, é possível a instalação de um soquete na placa para introduzir os sinais necessários à gravação. Obviamente que uma série de cuidados especiais no projeto do hardware devem ser tomados, a fim de que as condições de gravação não prejudiquem os demais circuitos da placa.

Não vamos tratar aqui do protocolo de gravação, mas um exemplo de aplicação é a própria placa McLab2, que possui um conector para gravação **In-Circuit** por meio dos gravadores McFlash ou McPlus (Mosaico).

a) No esquema elétrico dessa placa, os pinos necessários à gravação são:

I) V_{DD} : Alimentação convencional do PIC (5V).

II) V_{SS} : GND

III) MCRL/ V_{pp} (Tensão de programação): Para que o PIC entre em modo de programação, a tensão nesse pino deve ser elevada para aproximadamente 13V.

Por isso, o resto do circuito ligado a ele deve ser protegido.

IV) RB6: Clock da comunicação série imposto pelo gravador.

- V) RB7: Dados da comunicação série, que podem ser impostos pelo gravador (escrita) ou pelo próprio PIC (leitura).

11.12 – GRAVAÇÃO EM BAIXA TENSÃO - LVP (Low Voltage Programming)

Além do sistema de gravação **In-Circuit** visto anteriormente, o PIC 16F8777 possui um segundo modo de gravação. No sistema convencional pudemos observar que a gravação só pode ser efetuada por intermédio da presença de uma tensão especial em torno de 13V. Isso impossibilita a gravação em nível TTL. Com o sistema de gravação em baixa tensão (LVP), o PIC pode ser colocado em modo de programação com apenas 5V. Isso torna possível, por exemplo, que o PIC seja gravado diretamente por outro PIC.

O único problema é que, para poder utilizar o LVP, o pino RB3 é perdido como I/O convencional. Por isso, a habilitação desse sistema é feita por meio das configurações de gravação.

a) Configurações:

- I) O modo de Programação em Alta Voltagem está sempre disponível, independente do estado do bit LVP, pela aplicação de V_{IH} no pino /MCLR.
- II) Enquanto no modo ICSP de Baixa Voltagem, o pino RB3 não pode ser usado como pino I/O de propósito geral.
- III) Quando usando programação ICSP de baixa voltagem (LVP) e o pull-ups no PORTB está habilitado, o bit 3 no registro TRISB deve ser limpo para desabilitar o pull-up em RB3 e assegurar a operação adequada do dispositivo.
- IV) Não é permitida variações em RB3 com LVP habilitado. Um dispositivo de pull-down externo será usado como padrão para o modo de operação normal do dispositivo. Se RB3 variar para alto o PIC entra no modo de programação.
- V) O modo LVP mode é habilitado por padrão em todos os dispositivos da Microchip. Pode ser desabilitado limpando o bit LVP do registro CONFIG.

CAPÍTULO 12

INSTRUÇÕES DO PIC16F877

12.1 – INTRODUÇÃO.

Cada instrução de PIC16F877 é uma palavra de 14 bits, dividida em um OPCODE que especifica o tipo da instrução e um ou mais operandos que posteriormente especificam a operação da instrução. O conjunto de instruções do PIC16F877 é mostrado na **Figura 12.3** que lista Operações orientadas a byte, Orientadas a bit, e de Literal e de controle. A **Figura 12.1** mostra as descrições do Campo do opcode.

12.1.1 - Para instruções orientadas a Byte, **f** representa um designador do registro de arquivo e **d** representa um designador do destino. O designador do registro de arquivo especifica qual registro de arquivo será usado pela instrução. O designador do destino especifica onde o resultado da operação deve ser colocado. Se **d** for 0 ou W, o resultado é colocado no registro W. Se **d** for 1 ou F, o resultado está colocado no registro de arquivo especificado na instrução.

12.1.2 - Para instruções orientadas a bit, **b** representa o bit designador do campo que seleciona o número do bit afetado pela operação, enquanto **f** representa o endereço do arquivo no qual o bit ficado localizado.

12.1.3 - Para literal e operações de controle, **k** representa uma constante de 8 ou 11 bits ou um valor literal.

Campo	Descrição
F	Endereço do arquivo do registro f especificado (0x00 a 0x7F)
W	Registro de trabalho (acumulador)
B	Bit de endereço dentro dos 8 bits do arquivo de registro f
K	Campo literal, dado constante ou etiqueta
X	Irrelevante (0 ou 1), o assembler gera um código com x = 0. É ecomendado para compatibilidade com todas as ferramentas de software da Microchip.
D	Seleção de destino; d = 0 ou W: armazena o resultado em W d = 1 ou F: armazena o resultado no arquivo de registro f
PC	Contador de programa
TO	Time-out bit
PD	Power-down bit
Label	Nome da etiqueta
TOS	Topo da pilha
PCLATH	Latch de maior peso do PC
GIE	Bit de habilitação global de interrupções
WDT	Temporizador/contador Watchdog
→	Atribuído a
< >	Bit (s) de um registro
()	Conteúdo
[]	Opcional

Figura 12.1: Descrição do campo do Opcode.

Todas as instruções são executadas dentro de um ciclo de instrução, a menos que um teste condicional seja verdadeiro ou o PC seja mudado como resultado de uma instrução. Neste caso, a execução demora dois ciclos de instrução, com o segundo ciclo executado como NOP.

Um ciclo de instrução consiste de quatro períodos do oscilador. Assim, para uma frequência do oscilador de 4 MHz, o tempo normal de execução de uma instrução será de 1us. Se um teste condicional for verdadeiro, ou mudar o PC como resultado de uma instrução, o tempo da execução da instrução será de 2us.

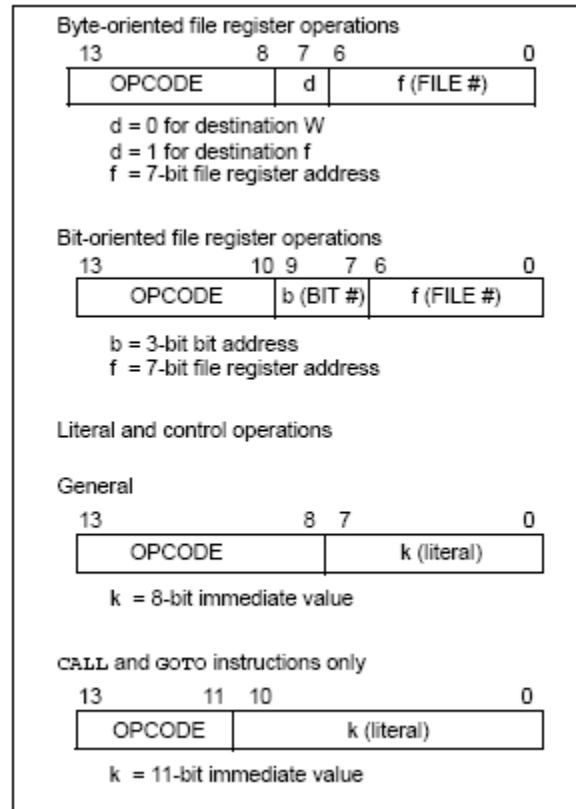


Figura 12.2: Formato geral para as instruções.

12.2 – CONJUNTO DE INSTRUÇÕES DO PIC16F877.

Utiliza a arquitetura Harvard e conjunto de instruções reduzidas (RISC), com 35 instruções. Possui dois barramentos um de dados e outro de instruções. O barramento de dados é de 8 bits e o de instrução de 14 bits, assim o OPCODE da instrução já possui o dado e o endereço onde ela vai operar, se for o caso, utilizando uma posição de memória por instrução.

Mnemônicos e Operandos (4)	Descrição	Ciclo	Código OP 14-bits	Status afetados	Notas
INSTRUÇÕES QUE USAM REGISTROS E SÃO ORIENTADAS A BYTE (guarda resultado em d)					
ADDWF f, d	Soma W e f	1	MSB 00 0111 dfff ffff	LSB C,DC,Z	1, 2
ANDWF f, d	AND entre W e f	1	00 0101 dfff ffff	Z	1, 2
CLRF f	Apaga f	1	00 0001 1fff ffff	Z	2
CLRW -	Apaga W	1	00 0001 0xxx xxxx	Z	
COMF f, d	Complementa f	1	00 1001 dfff ffff	Z	1, 2
DECF f, d	Decrementa f	1	00 0011 dfff ffff	Z	1, 2
DECFSZ f, d	Decrementa f, salta se 0	1 (2)	00 1011 dfff ffff		
INCF f, d	Incrementa f	1	00 1010 dfff ffff	Z	1, 2, 3
INCFSZ f, d	Incrementa f, salta se 0	1 (2)	00 1111 dfff ffff		
IORWF f, d	OU entre W e f	1	00 0100 dfff ffff	Z	1, 2
MOVF f, d	Move f	1	00 1000 dfff ffff	Z	1, 2
MOVWF f	Move (copia) W para f	1	00 0000 1fff ffff		
NOP -	Não opera (gasta 1 ciclo)	1	00 0000 0xx0 0000		
RLF f, d	Rotaciona f um bit para a esquerda através do carry.	1	00 1101 dfff ffff	C	1, 2
RRF f, d	Rotaciona f um bit para a direita através do carry.	1	00 1100 dfff ffff	C	1,2
SUBWF f, d	Subtrai W de f	1	00 0010 dfff ffff	C,DC, Z	1,2
SWAPF f, d	Inversão entre nibbles de f.	1	00 1110 dfff ffff		1,2
XORWF f, d	OU exclusivo entre W e f	1	00 0110 dfff ffff	Z	1,2
INSTRUÇÕES QUE UTILIZAM BITS E SÃO ORIENTADAS A BIT (b é o bit afetado pela instrução)					
BCF f, b	Apaga o bit b de f	1	01 00bb bfff ffff		1, 2
BSF f, b	Seta o bit b de f	1	01 01bb bfff ffff		1, 2
BTFSC f, b	Testa o bit b de f, salta se 0	1 (2)	01 10bb bfff ffff		3
BTFSS f, b	Testa o bit b de f, salta se 1	1 (2)	01 11bb bfff ffff		3
INSTRUÇÕES DE CONTROLE E DE OPERANDOS IMEDIATOS (guarda resultado em W)					
ADDLW k	Soma a constante k com W	1	11 111x kkkk kkkk	C,DC,Z	
ANDLW k	AND entre a constante k e W	1	11 1001 kkkk kkkk	Z	
CALL k	Executa subrotina	2	10 0kkk kkkk kkkk		
CLRWDT -	Apaga Watchdog Timer	1	00 0000 0110 0100	/TO,/PD	
GOTO k	Salto incondicional	2	10 1kkk kkkk kkkk		
IORLW k	OR entre constante k e W	1	11 1000 kkkk kkkk	Z	
MOVLW k	Move (copia) literal k para W	1	11 00xx kkkk kkkk		
RETFIE -	Retorno de uma interrupção	2	00 0000 0000 1001		
RETLW k	Retorno de subrotina com k em W	2	11 01xx kkkk kkkk		
RETURN -	Retorno de subrotina	2	00 0000 0000 1000		
SLEEP -	Micro no modo de repouso	1	00 0000 0110 0011	/TO,/PD	
SUBLW k	Subtrai W da constante (literal) k	1	11 110x kkkk kkkk	C,DC,Z	
XORLW k	OU exclusivo entre literal k e W	1	11 1010 kkkk kkkk	Z	
Nota 1: Quando um registro do I/O for modificado em função de si mesmo (exemplo, MOVF PORTB, 1), o valor usado será o valor presente nos próprios pinos. Por exemplo, se o latch de dados for '1' para um pino configurado como entrada e for mudado para baixo por um dispositivo externo, o dado será escrito novamente com um zero (0). 2: Se esta instrução for executada no registro TMR0 (e, onde aplicável, d = 1), o preescaler será apagado se designado ao módulo do Timer0. 3: Se o PC for modificado ou um teste condicional for verdadeiro, a instrução requer dois ciclos. O segundo ciclo é executado como um NOP. 4: Mnemônico = Nome da instrução e Operando = Argumento 5: Instruções de controle, são: NOP, CALL, CLRWDT, GOTO, RETFIE, RETLW, RETURN e SLEEP.					

Figura 12.3: Conjunto de instruções.

12.2.1 – Descrição das instruções.

Antes de estudarmos as instruções, vejamos como elas são construídas.

a) Instruções do tipo MNEMÔNICO k .

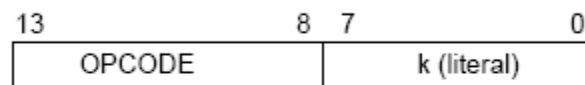
Neste tipo de instrução, encontramos um operando k cujo valor será usado pela instrução de uma das duas formas:

I) Armazenado no registro W.

II) Usado como parte do endereço de destino em instruções de desvio.

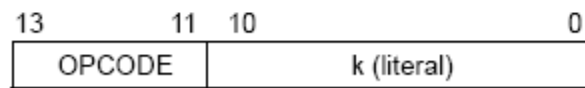
Literal and control operations

General



k = 8-bit immediate value

CALL and GOTO instructions only



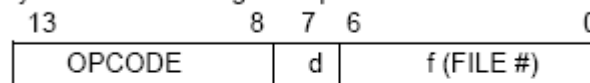
k = 11-bit immediate value

b) Instruções do tipo MNEMÔNICO REG (MOVWF f).

I) Neste tipo de instrução, encontramos um operando REG que representa um dos registros SFR ou GPR disponíveis no PIC.

II) Observe que REG é representado por um código binário de 7 bits, o que nos limita a um máximo de 128 registros. Daí a necessidade de implementar os bancos de memória no PIC.

Byte-oriented file register operations



d = 0 for destination W

d = 1 for destination f

f = 7-bit file register address

c) Instruções do tipo MNEMÔNICO REG, DES (ADDWF f, d):

Neste tipo de instrução, encontramos dois operandos REG e DES:

I) REG, representa um dos registros SFR ou GPR disponíveis no PIC (W não é um registro SFR ou GPR).

II) DES, consiste em um bit indicador do destino da instrução, onde podemos ter duas possibilidades:

A) Se no lugar de DES tiver a letra W ou o número 0, o destino da instrução será o registro W.

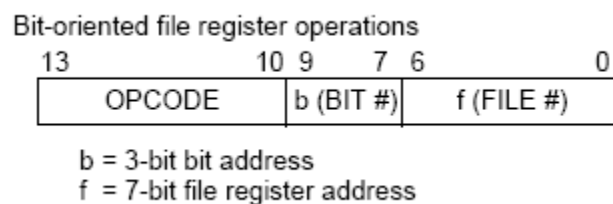
B) Se no lugar de DES tiver a letra f ou o número 1, o destino da instrução será o próprio registro representado por REG.

d) Instruções do tipo MNEMÔNICO REG, BIT (BCF f, b):

Neste tipo de instrução, encontramos dois operandos: REG e BIT

I) REG, representa um dos registros SFR ou GPR disponíveis no PIC (W não é um registro SFR ou GPR).

II) BIT, representa qual dos 8 bits que compõem o registro REG será o alvo da instrução. Ex: “MNEMÔNICO PORTB,1”, o bit 1 da porta B será o alvo da instrução.



Existem instruções que alteram indiretamente os flags no registro STATUS, ou seja, devido à natureza de sua operação, certas instruções, após a realização de sua operação, alterarem um ou mais flags de acordo com o resultado. Dessa forma, além de alterar diretamente o registro de destino da operação, essas instruções alteram também indiretamente o registro STATUS.

12.2.2 - Para a base numérica podemos usar qualquer uma delas desde que esteja de acordo com as regras estabelecidas.

- a) Para hexadecimal: 0xXX ou H'XX'
- b) Para binário: B'XXXXXXXX'
- c) Para decimal: D'XX' ou .XX
- d) Para ASCII: A'X'

12.2.3 – Observando o Registro de STATUS em algumas operações.

- a) Quando o resultado alterar os flags C, DC e Z, em uma soma temos:

- I) C = 1, se o resultado ultrapassar o valor 255;
- II) C = 0, se o resultado não ultrapassar o valor 255;
- III) DC = 1, se houver transbordo do bit 3 para o bit 4 do registro;
- IV) DC = 0, se não houver transbordo do bit 3 para o bit 4 do registro;
- V) Z = 1, se o valor armazenado em W for igual a zero;
- VI) Z = 0, se o valor armazenado em W for diferente de zero.

Obs: Para determinar o valor real armazenado no destino (W) quando superior a 255, basta subtrair 256 do resultado. O valor resultante será o efetivamente armazenado no destino. Neste caso, o flag C estará setado, indicando que houve a condição de estouro do registro.

- b) Quando o resultado alterar os flags C, DC e Z, em uma subtração temos:

- I) C = 1, indica que o resultado é positivo ou igual a zero (não houve empréstimo);
- II) C = 0, indica que o resultado é negativo (houve empréstimo);
- III) DC = 1, não houve empréstimo do bit 4 para o bit 3;
- IV) DC = 0, houve empréstimo do bit 4 o bit 3;
- V) Z = 1, se o valor armazenado em W for igual a zero;
- VI) Z = 0, se o valor armazenado em W for diferente de zero.

Obs1: Para determinar o valor real armazenado no destino (W) quando negativo, basta adicionar 256 decimal ao resultado. O novo valor resultante será o efetivamente armazenado no registro de destino (W). Neste caso, o flag C estará apagado (zero), indicando que houve a condição de empréstimo na subtração.

Obs2: Em algumas instruções podemos usar o número ou o nome do bit. Ex: BCF STATUS,0 ou BCF STATUS,Z (zera o bit Z ou bit 0).

12.2.4 - Instruções de Desvios.

Dentre as instruções do PIC, algumas são consideradas de desvio. Instruções de desvio são usadas para desviar o fluxo do programa para outro endereço.

a) Podem ser classificadas em:

- I) Desvio com retorno – também chamadas de sub-rotinas, acontecem quando o fluxo do programa é desviado para outro ponto (sub-rotina) e após a execução dela, o fluxo retorna ao ponto inicial de desvio e segue adiante. As subrotinas fazem uso da pilha. Ex: CALL
- II) Desvios sem retorno – o fluxo do programa é desviado para outro endereço, seguindo então a partir de lá. Os desvios sem retorno podem ser dos tipos condicionais (IF) e incondicionais (GOTO k). Nos desvios condicionais, ocorre o desvio do programa se a condição for verdadeira e nos incondicionais, o desvio ocorre independente de qualquer condição.

b) São consideradas instruções de desvio:

- I) Instruções de controle de desvio (GOTO e CALL)
- II) Instruções de retorno de interrupções (RETFIE)
- III) Instruções de retorno de subrotinas (RETURN e RETLW)
- IV) BTFSS, BTFSC, DECFSZ e INCFSZ.

12.2.5 - Termos usados:

- a) **Work** (registro W): Registro temporário para as operações da ULA
- b) **File**: Referência a um registro, SFR ou GPR (endereço de memória). F no nome da instrução e f no argumento
- c) **Literal**: Um número qualquer no formato binário, decimal ou hexadecimal. L no nome da instrução e k no argumento.
- d) **Destino**: Local onde será armazenado o resultado da operação. Com (f ou 1), será armazenado no próprio registro, com (w ou 0) será armazenado no registro Work (W).
- e) **Bit**: Refere-se a um bit dentro do byte. B no nome da instrução e b no argumento.

- f) Teste:** Quando queremos testar um bit para saber se ele é 0 ou 1. T no nome da instrução.
- g) Skip:** Usado para criar um desvio, pulando a próxima linha. S no nome da instrução.
- h) Set:** Setar um bit. S no nome da instrução.
- i) Clear:** Limpar um bit (zerar). C no nome da instrução.
- j) Zero:** Algumas instruções podem gerar desvios se o resultado da operação for zero, neste caso, usa-se Z para indicar tal condição.

ADD: soma	AND: lógica “E”	IOR: lógica “OU”	COM: complemento
DEC: Decremento	INC: incremento	MOV: mover, copiar	XOR: lógica “OU exclusivo”.
RL: rotacionar um bit para a esquerda		RR: rotacionar um bit para a direita	
SUB: subtração		SWAP: inversão entre nibbles	
Exemplo: DECFSZ = Decrementa (DEC) o registro (F) e pula (S) se resultado for zero (Z).			

12.2.6 – Comentando cada instrução.

ADDLW Soma uma literal com o registro W.	ADDWF Soma entre os registros W e f.
Sintaxe: [<i>label</i>] ADDLW <i>k</i> Descrição: O valor da constante ou literal passada no argumento <i>k</i> é somado ao valor de W e o resultado é armazenado no próprio W. Limites: $0 \leq k \leq 255$ Operação: $(W) + (k) \rightarrow (W)$ Status afetados: C, DC, Z Código: 11 111x kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo1: ADDLW 0x15 Antes da instrução: W = 0x20 Após a instrução: W = 0x35 Z = 0; DC = 0; C = 0 Exemplo2: ADDLW 0x15 Antes da instrução: W = 0xF1 Após a instrução: W = 0x06 Z = 0; DC = 0; C = 1 Exemplo3: ADDLW 0x01 Antes da instrução: W = 0xFF Após a instrução: W = 0x00 Z = 1; DC = 1; C = 1	Sintaxe: [<i>label</i>] ADDWF <i>f</i>,<i>d</i> Descrição: O valor de W é somado ao do registro <i>f</i> e o resultado é armazenado no lugar definido por <i>d</i>. Limites: $0 \leq f \leq 127$ <i>d</i> = 0 (W) ou <i>d</i> = 1 (<i>f</i>) Operação: $(W) + (f) \rightarrow (d)$ Status afetados: C, DC, Z Código: 00 0111 dfff ffff Palavra: 1 Ciclo: 1 Exemplo1: ADDWF INTCON, F Antes da instrução: W = 0x20 e INTCON = 0x07 Após a instrução: W = 0x20 e INTCON = 0x27 Z = 0; DC = 0; C = 0 Exemplo2: ADDWF PORTA, W Antes da instrução: W = 0xF1 e PORTA = 0x10 Após a instrução: W = 0x01 Z = 0; DC = 0; C = 1 Exemplo3: ADDWF FSR, 1 Antes da instrução: W = 0xFF e FSR = 0x01 Após a instrução: W = 0xFF e FSR = 0x00 Z = 1; DC = 1; C = 1

COMF Complemento de f.	DECF Decrementa o registro f.
Sintaxe: <i>[label]</i> COMF f,d Descrição: Complemento do registro f (inverte os bits). O resultado é armazenado em d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (f) → (d) Status afetados: Z Código: 00 1001 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: COMF PORTA,F Antes da instrução: PORTA = 0x22 Após a instrução: PORTA = 0xDD e Z = 0	Sintaxe: <i>[label]</i> DECF f,d Descrição: Decrementa de uma unidade o valor do registro f. O resultado é armazenado no lugar definido por d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (f) - 1 → (d) Status afetados: Z Código: 00 0011 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: DECF CNT,1 Antes da instrução: CNT = 0x01 e Z = 0 Após a instrução: CNT = 0x00 e Z = 1

DECFSZ Decrementa f, pulando se zero.	INCF Incrementa o registro f.
Sintaxe: <i>[label]</i> DECFSZ f,d Descrição: Decrementa uma unidade do valor do registro f. O resultado é armazenado no lugar definido por d. Se o resultado da operação for zero, a linha seguinte é pulada. Quando isto acontece, a instrução demora dois ciclos. É um desvio condicional. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (f) - 1 → (d) Status afetados: nenhum Código: 00 1011 dfff ffff Palavra: 1 Ciclo: 1 ou 2 Exemplo: DECFSZ CNT, F GOTO continua GOTO acabou Antes da instrução: PC = Linha 1 e CNT = 0x10 Após a instrução: Se CNT - 1 ≠ 0 PC = GOTO continua Se CNT - 1 = 0 PC = GOTO acabou CNT = CNT - 1	Sintaxe: <i>[label]</i> INCF f,d Descrição: Incrementa o valor do registro f em uma unidade. O resultado é armazenado no lugar definido por d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (f) + 1 → (d) Status afetados: Z Código: 00 1010 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: INCF CNT,1 Antes da instrução: CNT = 0xFF e Z = 0 Após a instrução: CNT = 0x00 e Z = 1

INCFSZ Incrementa f, pulando se zero. Sintaxe: <i>[label]</i> INCFSZ f,d Descrição: Incrementa de uma unidade o valor do registro f. O resultado é armazenado no lugar definido por d. Se o resultado da operação for zero, a linha seguinte é pulada. Quando isto acontece, a instrução demora dois ciclos. É um desvio condicional. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: $(f) + 1 \rightarrow (d)$ Status afetados: nenhum Código: 00 1111 dfff ffff Palavra: 1 Ciclo: 1 ou 2 Exemplo: INCFSZ CNT,F GOTO continua GOTO acabou Antes da instrução: PC = Linha 1 e CNT = 0x00 Após a instrução: Se CNT + 1 $\neq$ 0 PC = GOTO continua Se CNT + 1 = 0 PC = GOTO acabou CNT = CNT + 1	SUBLW Subtrai W da literal ou constante. Sintaxe: <i>[label]</i> SUBLW k Descrição: Subtrai o conteúdo do registro W da constante k. O resultado é armazenado no próprio W. Limites: $0 \leq k \leq 255$ Operação: $(k) - (W) \rightarrow (W)$ Status afetados: C, DC, Z Código: 11 110x kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo1: SUBLW 0x09 Antes da instrução: W = 0x01 Após a instrução: W = 0x08 Z = 0; DC = 1; C = 1 (positivo) Exemplo2: SUBLW 0x00 Antes da instrução: W = 0x02 Após a instrução: W = 0x02 (negativo) ou 0xFE Z = 0; DC = 0; C = 0 (negativo) Exemplo 3: SUBLW 0x10 Antes da instrução: W = 0x10 Após a instrução: W = 0x00 Z = 1; DC = 1; C = 1 (positivo ou zero)
--	---

SUBWF Subtração entre W e f Sintaxe: <i>[label]</i> SUBWF f,d Descrição: Subtrai do registro f o valor de W. O resultado é armazenado no lugar definido pelo operando d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: $(f) - (W) \rightarrow (d)$ Status afetados: C, DC, Z Código: 00 0010 dfff ffff Palavra: 1 Ciclo: 1 Exemplo1: SUBWF REG,F Antes da instrução: W = 0x20 e REG = 0x07 Após a instrução: W = 0x20 e REG = 0xE7 (7 - 32 = -25 decimal ou -25 + 256 = 231 decimal) Z = 0; DC = 1; C = 0 (negativo) Exemplo2: SUBWF PORTA,W Antes da instrução: W = 0x09 e PORTA = 0x10 Após a instrução: W = 0x07 e PORTA = 0x10 Z = 0; DC = 0; C = 1 (positivo ou zero) Exemplo 3: SUBWF REG,0 Antes da instrução: W 0x10 e REG = 0x10 Após a instrução: W 0x00 e REG = 0x10 Z = 1; DC = 1; C = 1 (positivo ou zero)

ANDLW Operação “E” entre uma literal e W.	ANDWF Operação “E” entre W e f.
Sintaxe: <i>[label]</i> ANDLW k Descrição: Executa um “E” lógico entre o valor da constante passado no argumento k e o valor de W. O resultado é armazenado em W. Limites: $0 \leq k \leq 255$ Operação: (W) AND (k) → (W) Status afetados: Z Código: 11 1001 kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo: ANDLW 0x0F Antes da instrução: W = 0x95 Após a instrução: W = 0x05 e Z = 0	Sintaxe: <i>[label]</i> ANDWF f,d Descrição: Executa um “E” lógico entre o valor de W e o valor do registro f. O resultado é armazenado no lugar definido por d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (W) AND (f) → (d) Status afetados: Z Código: 00 0101 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: ANDWF FSR,W Antes da instrução: W = 0x77 e FSR = 0xFF Após a instrução: W = 0x77; FSR = 0xFF e Z = 0

XORLW Operação “OU exclusivo” entre k e W.	XORWF Operação “OU exclusivo” entre W e f.
Sintaxe: <i>[label]</i> XORLW k Descrição: Executa um “OU” exclusivo lógico entre o valor da constante passado no argumento k e o valor de W. O resultado é armazenado em W. Limites: $0 \leq k \leq 255$ Operação: (W) XOR (k) → (W) Status afetados: Z Código: 11 1010 kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo: XORLW 0x12 Antes da instrução: W = 0x04 Após a instrução: W = 0x16 e Z = 0	Sintaxe: <i>[label]</i> XORWF f,d Descrição: Executa um “OU” lógico exclusivo entre o valor de W e o valor do registro f. O resultado é armazenado no lugar definido por d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (W) XOR (f) → (d) Status afetados: Z Código: 00 0110 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: XORWF REG,F Antes da instrução: W = 0x22 e REG = 0x22 Após a instrução: W = 0x22, REG = 0x00 e Z = 1

BCF Limpa (clear) um bit do registro f.	BTFSC Testa o bit de f, pula se zero.
Sintaxe: <i>[label]</i> BCF f,b Descrição: Põe a 0 (zero) o bit b do registro f. Limites: $0 \leq f \leq 127$ $0 \leq b \leq 7$ Operação: $0 \rightarrow (f,b)$ Status afetados: nenhum Código: 01 00bb bfff ffff Palavra: 1 Ciclo: 1 Exemplo: BCF STATUS,0 ou BCF STATUS,C Antes da instrução: STATUS = B'11111111' Após a instrução: STATUS = B'11111110' Obs: Apesar da instrução não afetar indiretamente nenhum dos flags, neste caso, foi determinado que ela alterasse o flag de carry do registro de Status (C = 0).	Sintaxe: <i>[label]</i> BTFSC f,b Descrição: Se o bit d do registro f for 1, então a próxima linha será executada. Caso seja 0, a próxima linha será pulada. Neste caso, a instrução leva 2 ciclos. É um desvio condicional. Limites: $0 \leq f \leq 127$ $0 \leq b \leq 7$ Operação: pula se $(f,b) = 0$ Status afetados: nenhum Código: 01 10bb bfff ffff Palavra: 1 Ciclo: 1 ou 2 Exemplo: BTFSC STATUS,2 CLRW ADDLW 0x02 Antes da instrução: PC = Linha 1 e STATUS = 0x18 Após a instrução: Se bit2 = 0, pula para a instrução ADDLW 0x02, caso seja = 1, CLRW.

BSF Seta um bit do registro f.	BTFSS Testa um bit de f, pula se 1.
Sintaxe: <i>[label]</i> BSF f,b Descrição: Seta o bit b do registro f. Limites: $0 \leq f \leq 127$ $0 \leq b \leq 7$ Operação: $1 \rightarrow (f,b)$ Status afetados: nenhum Código: 01 01bb bfff ffff Palavra: 1 Ciclo: 1 Exemplo: BSF TRISB,1 Antes da instrução: TRISB = B'00000000' Após a instrução: TRISB = B'00000010' Obs: Observe que o registro TRISB encontra-se nos bancos 1 e 3. Esta instrução configura o pino RB1 da porta B como entrada.	Sintaxe: <i>[label]</i> BTFSS f,b Descrição: Se o bit b do registro f for 0, então a próxima linha será executada. Caso seja 1, a próxima linha será pulada. Neste caso, a instrução leva 2 ciclos. É um desvio condicional. Limites: $0 \leq f \leq 127$ $0 \leq b \leq 7$ Operação: pula se $(f,b) = 1$ Status afetados: nenhum Código: 01 11bb bfff ffff Palavra: 1 Ciclo: 1 ou 2 Exemplo: BTFSS STATUS,2 CLRW ADDLW 0x02 Antes da instrução: PC = Linha 1 e STATUS = 0x18 Após a instrução: Se bit2 = 0, não pula a próxima instrução e caso seja 1, pula para ADDLW 0x02.

CALL Chama uma sub-rotina. Sintaxe: [label] CALL k Descrição: Chama a sub-rotina especificada por k, que é um endereço da memória de programação mas normalmente é representado por um nome (label). Antes do desvio, o endereço de retorno (PC + 1) é armazenado na pilha e o endereço da sub-rotina é colocado no PC. Call é um desvio com retorno. Limites: $0 \leq k \leq 2047$ Operação: (PC) + 1 → (TOS) (k) → (PC, 10:0) (PCLATH 4:3) → (PC, 12:11) Status afetados: nenhum Código: 10 0kkk kkkk kkkk Palavra: 1 Ciclo: 2 Exemplo: CALL DELAY Antes da instrução: PC = Linha 1 Após a instrução: PC = DELAY	CLRF Limpa (zera) o registro f. Sintaxe: [label] CLRF f Descrição: Apaga o valor do registro f e seta o flag Z. Limites: $0 \leq f \leq 127$ Operação: 0 → (f) 1 → Z Status afetados: Z Código: 00 0001 1fff ffff Palavra: 1 Ciclo: 1 Exemplo1: CLRF TMR0 Antes da instrução: TMR0 = B'11111111' Após a instrução: TMR0 = B'00000000' Z = 1 Exemplo2: CLRF STATUS Antes da instrução: STATUS = B'00011111' Após a instrução: STATUS = B'00011111' Z = 1 Obs: IRP, RP1 e RP0 = 000 TO e PD (somente leitura) DC e C (não mudam) Z = 1
---	--

CLRWDT Limpa o Watchdog Timer. Sintaxe: [label] CLRWDT Descrição: Reseta o contador do WDT e seu postscaler, impedindo que o WDT reset a CPU. Bits /TO e /PD são setados. Limites: nenhum. Operação: 0 → (WDT) e WDT postscaler 1 → /TO e /PD Status afetados: /TO, /PD Código: 00 0000 0110 0100 Palavra: 1 Ciclo: 1 Exemplo: CLRWDT Antes da instrução: WDT contador = ? WDT prescaler = ? Após a instrução: WDT contador = 0 WDT prescaler = 0 /TO = 1 e /PD = 1	GOTO Desvia para um outro endereço. Sintaxe: [label] GOTO k Descrição: Desvia o programa para um outro endereço especificado por k, que é um endereço da memória de programa, normalmente representado por um nome (label). Este é um desvio incondicional e sem retorno. Limites: $0 \leq k \leq 2047$ Operação: (k) → (PC, 10:0) (PCLATH,4:3) → (PC,12:11) Status afetados: nenhum Código: 10 1kkk kkkk kkkk Palavra: 1 Ciclo: 2 Exemplo: GOTO CONTINUA Antes da instrução: PC = Linha 1 Após a instrução: PC = CONTINUA
---	---

IORLW Operação “OU” entre uma literal e W.	IORWF Operação “OU” entre W e f.
Sintaxe: <i>[label]</i> IORLW k Descrição: Executa um “OU” lógico entre o valor da constante passado no argumento k e o valor de W . O resultado é armazenado no próprio W . Limites: $0 \leq k \leq 255$ Operação: $(W).OR.(k) \rightarrow (W)$ Status afetados: Z Código: 11 1000 kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo: IORLW 0x0F Antes da instrução: W = 0x95 Após a instrução: W = 0x9F e Z = 0	Sintaxe: <i>[label]</i> IORWF f,d Descrição: Executa um “OU” lógico entre o valor de W e o valor de f . O resultado é armazenado no lugar definido por d . Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: $(W).OR.(f) \rightarrow (d)$ Status afetados: Z Código: 00 0100 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: IORWF REG,F Antes da instrução: W = 0x77 e REG = 0xFF Após a instrução: W = 0x77 e REG = 0xFF e Z = 0

MOVF Move o valor de f para o destino d.	MOVWF Move o valor de W para f.
Sintaxe: <i>[label]</i> MOVF f,d Descrição: Copia o valor do registro f para o local determinado pelo destino d . Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: $(f) \rightarrow (d)$ Status afetados: Z Código: 00 1000 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: MOVF TRISB,W Antes da instrução: TRISB = 0x00 e W = 0x10 Após a instrução: W = 0x00, TRISB = 0x00 e Z = 1	Sintaxe: <i>[label]</i> MOVWF f Descrição: Copia o valor do registro W para o registro f . Limites: $0 \leq f \leq 127$ Operação: $(W) \rightarrow (f)$ Status afetados: nenhum Código: 00 0000 1fff ffff Palavra: 1 Ciclo: 1 Exemplo: MOVWF OPTION Antes da instrução: OPTION = 0xFF e W = 0x4F Após a instrução: OPTION = 0x4F e W = 0x4F

NOP Não executa nada.	RETFIE Retorno de uma interrupção.
Sintaxe: <i>[label]</i> NOP Descrição: Esta instrução é usada somente para gastar tempo, pois produz retardo de 1 ciclo de instrução. Limites: nenhum Operação: nenhuma Status afetados: nenhum Código: 00 0000 0xx0 0000 Palavra: 1 Ciclo: 1 Exemplo: NOP	Sintaxe: <i>[label]</i> RETFIE Descrição: Retorna da interrupção, recuperando o último endereço da pilha e setando o bit GIE. Limites: nenhum Operação: $TOS \rightarrow (PC)$ 1 $\rightarrow$ GIE Status afetados: nenhum Código: 00 0000 0000 1001 Palavra: 1 Ciclo: 2 Exemplo: RETFIE Após a instrução: PC = TOS e GIE = 1

RETLW Retorna com uma literal em W.	RETURN Retorno de uma sub-rotina.
Sintaxe: <i>[label]</i> RETLW <i>k</i> Descrição: Retorna de uma sub-rotina, recuperando o último endereço da pilha, e colocando o valor passado por <i>k</i> em W. Limites: $0 \leq k \leq 255$ Operação: $(k) \rightarrow (W)$ TOS $\rightarrow$ (PC) Status afetados: nenhum Código: 11 01xx kkkk kkkk Palavra: 1 Ciclo: 2 Exemplo: RETLW 0x50 Após a instrução: PC = TOS e W = 0x50	Sintaxe: <i>[label]</i> RETURN Descrição: Retorna de uma sub-rotina, recuperando o último endereço da pilha. Limites: nenhum Operação: TOS $\rightarrow$ (PC) Status afetados: nenhum Código: 00 0000 0000 1000 Palavra: 1 Ciclo: 2 Exemplo: RETURN Após a instrução: PC = TOS

RLF Desloca f um bit para a esquerda.	RRF Desloca f um bit para a direita.
Sintaxe: <i>[label]</i> RLF <i>f,d</i> Descrição: Desloca o registro <i>f</i> um bit para a esquerda. O valor do carry é colocado no bit 0, e o valor do bit 7 é colocado em carry. O resultado é armazenado no lugar definido por <i>d</i>. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: conforme descrito Status afetados: C Código: 00 1101 dfff ffff Palavra: 1 Ciclo: 1 Exemplo1: RLF TRISB,W Antes da instrução: W = 0x22, TRISB = 0x42 e C = 1 Após a instrução: W = 0x85, TRISB = 0x42 e C = 0 (bit excedente da rotação foi zero) Exemplo2: RLF TRISB,F Antes da instrução: W = 0x22, TRISB = 0x8F e C = 0 Após a instrução: W = 0x22, TRISB = 0x1E e C = 1 (bit excedente da rotação foi um) Obs: Rotação à esquerda consiste em uma multiplicação por 2	Sintaxe: <i>[label]</i> RRF <i>f,d</i> Descrição: Desloca o registro <i>f</i> um bit para a direita. O valor do carry é colocado no bit 7 e o valor do bit 0 é colocado em carry. O resultado é armazenado no lugar definido por <i>d</i>. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: Conforme descrito. Status afetados: C Código: 00 1100 dfff ffff Palavra: 1 Ciclo: 1 Exemplo1: RRF TRISB,0 Antes da instrução: W = 0x22, TRISB = 0x42 e C = 1 Após a instrução: W = 0xA1, TRISB = 0x42 e C = 0 (bit excedente de rotação foi zero). Exemplo2: RRF TRISB,F Antes da instrução: W = 0x22, TRISB = 0x8F e C = 0 Após a instrução: W = 0x22, TRISB = 0x47 e C = 1 (bit excedente da rotação foi um) Obs: Rotação à direita consiste em uma divisão por 2

SLEEP Entra em modo Sleep.	SWAPF Inverte os nibbles do registro f.
Sintaxe: [<i>label</i>] SLEEP Descrição: Coloca o MCU em modo Power-Down ou Sleep (baixo consumo). O processamento é parado e o conteúdo da memória é preservado. Limites: nenhum Operação: 0 → WDT 0 → WDT postscaler 1 → /TO 0 → /PD Status afetados: /TO, /PD Código: 00 0000 0110 0011 Palavra: 1 Ciclo: 1 Exemplo: SLEEP Após a instrução: /TO = 1 e /PD = 0	Sintaxe: [<i>label</i>] SWAPF f,d Descrição: Troca os 4 bits MSB com os 4 bits LSB do registro f. O resultado é armazenado no lugar definido por d. Limites: $0 \leq f \leq 127$ d = 0 (W) ou d = 1 (f) Operação: (f,3:0) → (d,7:4) (f,7:4) → (d,3:0) Status afetados: nenhum Código: 00 1110 dfff ffff Palavra: 1 Ciclo: 1 Exemplo: SWAPF REG,F ou SWAPF REG,1 Antes da instrução: REG = 0x45 Após a instrução: REG = 0x54 W = ?

CLRW Limpa (zera) o registro W.	MOVLW Move uma literal para W.
Sintaxe: [<i>label</i>] CLRW Descrição: Apaga o valor do registro W e seta o flag Z. Limites: nenhum Operação: 0 → (W) 1 → Z Status afetados: Z Código: 00 0001 0000 0011 Palavra: 1 Ciclo: 1 Exemplo: CLRW Antes da instrução: W = B'11111111' Após a instrução: W = B'00000000' Z = 1	Sintaxe: [<i>label</i>] MOVLW k Descrição: Copia o valor da constante k (literal) para o registro W. Limites: $0 \leq k \leq 255$ Operação: k → (W) Status afetados: nenhum Código: 11 00xx kkkk kkkk Palavra: 1 Ciclo: 1 Exemplo: MOVLW 0x15 Antes da instrução: W = ? Após a instrução: W = 0x15

ANEXO A
BIBLIOGRAFIA

- a) BRASIL. **Microcontroladores e microprocessadores.** _Revista Eletrônica. Editora Multipress. Rio de Janeiro, 2000;
- b) EUA. **PIC16F87x Data Sheet da Microchip – 28/40 Pin 8-Bit CMOS FLASH – Microcontrollers,** 2001;
- c) PEREIRA, Fabio - **Microcontrolador PIC – Técnicas Avançadas.** 4^a ed, São Paulo, Érica, 2002;
- d) SOUZA, Daniel José e LAVINIA, Nicolas Cezar - **Conectando o PIC – Recursos Avançados.** 3^a ed, São Paulo, Erica, 2003;
- e) TORRES, Gabriel – **Hardware – Curso Completo** – 2^a ed, Rio de Janeiro, Axcel Books do Brasil Editora, 1998.
- f) ZANCO, Wagner da Silva – **Microcontroladores PIC – Técnicas de Software e Hardware para Projetos de Circuitos Eletrônicos.** 1^a ed, São Paulo, Erica, 2006.